

册07 · 图论与可计算性

《数学基础》· 创世游戏系列丛书基础卷

连接关系与算的边界

7 册 · 53 章 · 苏格拉底对话体自学教材

造物主（读者） $\rightleftharpoons$ 老谟（毒舌导师）—— 知识是推导出来的，不是背出来的

开源教材 · CC BY 4.0 · 在线版: lwq200.github.io/math-foundation

目录

1. 第 01 章 · 图的基础——一堆点和线，凭什么是"数学"？
2. 第 02 章 · 树与图的遍历——怎么保证走遍一座"图城"不迷路？
3. 第 03 章 · 最短路径与 Dijkstra——导航凭什么知道"最短"？
4. 第 04 章 · 图着色——地图染色，最少几种颜色？
5. 第 05 章 · 欧拉与哈密顿——一笔画的问题，和找旅行线路的问题一样吗？
6. 第 06 章 · 计算模型——图灵机
7. 第 07 章 · 可判定性与停机问题——算不出来的东西
8. 第 08 章 · P 与 NP——算得快不快，与算不算得出来
9. 《图论与可计算性》· 计算答案速查页

第 01 章 · 图的基础——一堆点和线，凭什么叫"数学"？

本章从哪个问题出发：一座城市有七座桥，把河中的两个岛和两岸连起来。有人问：能不能一次走过七座桥，每座桥恰好走一次？这个问题听起来像脑筋急转弯，可它在 1736 年被一个人写成了论文，从此开启了一门全新的数学。**你要先回答的，是比"能不能走通"更根本的问题：一堆点和线，凭什么配叫"数学"？**推导到哪个结论：你会发现，"点"和"线"不是图画，是**抽象**——把现实的"陆地""桥"剥成只剩连接关系的骨架，就是**图 (graph)**。沿着这条抽象走到底，你会亲手得到一个威力大得惊人的小引理——**握手引理**（所有顶点的度数之和 = $2 \times$ 边数），并靠它证出"一笔画"的判定条件，回头解开七桥问题。**而这一切的起点，是一个至今仍统治着计算机科学的想法：把关系画成图，把图算成数学。**

核心认知：**图 = 顶点集 + 边集，是"关系"的数学化身。**你画的不是点线，是"谁和谁相连"。这一章是全册图论与可计算性的地基：**握手引理**教会你"数一下就知道全局"，**连通**教会你"从能不能走过去"，****一笔画（欧拉迹）****是第一个"给一个目标，判断它到底成不成立"的定理。你可别小看它——第 7 章要证的停机问题反证法，依赖的正是本章练出来的两种能力：**把一句话翻成严格定义，以及"反证"这把刀。**

引子

我把一张泛黄的地图摊在老谟面前——那是柯尼斯堡城的旧图，普雷格尔河穿城而过，把陆地分成两块岸和两个岛，河上架着七座桥。

"老谟，有个传说，"我指着图上的桥说，"有人说，能不能一次走过这七座桥，每座桥只走一次，最后回到出发点？我试了一下午，没走通，可也不知道是'真的没走通'还是'我没找对路'。"

老谟瞟了一眼地图，没直接答话，反而问我："你刚才这句话里，哪些词是'有用的'，哪些词是'多余的'？"

我一愣：".....桥。还有'一次走一座'？"

"那'河'呢？'岛'呢？'岸'呢？"

"河.....河就是用来架桥的？岛和岸....."我忽然觉得哪不对劲，"等等，走不走得通，好像跟河有多宽、岛有多大、岸有多长，一点关系都没有？"

老谟把茶杯放下："你刚亲手把一块'脑筋急转弯'，剥成了一道'数学问题'。你想不想看看，剥完之后它长什么样？"

对话引擎

第 1 轮：七桥凭什么是个数学问题——"剥"掉没用的，留下连接

老谟："你说走不走得通，跟河宽、岛大没关系。那到底跟什么有关系？"

造物主：".....跟'哪块地和哪块地之间有桥'有关系？只要知道：这块地和那块地通不通，就够了。"

老谟："对。你刚才做了一件了不起的事——**你下意识地做了抽象**。把每块陆地'捏'成一个点，把每座桥'拉'成一条线，剩下的就是一张'谁和谁连着'的图。你告诉我，这张图上有几个点？"

造物主："四块陆地.....两个岛加两岸，是四个点。桥有七座，所以是七条线。哎，可是我把'点'和'线'画出来之后，这问题是不是就变简单了？"

老谟："变简单了，而且变'能算'了。你想：**在你没有把它抽象成图之前，'走不走得通'是一个要靠运气去试的谜；抽象成图之后，它变成了一个可以'判断'的命题**。这一步'从具体到抽象'，就是数学的看家本领。你今天亲手做了一次。可你先别急着高兴——我得问你：凭什么'点'和'线'就能代表'陆地'和'桥'？"

造物主：".....因为走不走得通，只取决于'相连关系'？我把河宽岛大都扔了，可那个'关系'我一点没丢。只要关系保住了，答案就不会变？"

老谟："这话说到点子上了。**抽象不是'丢掉信息'，是'丢掉对这个问题没用的信息'**。关系保住了，问题的本质就保住了。现在，把这个想法正式一点——我们下一轮，给'图'立个名字和定义。"

知识锚点：从七桥到图——“关系”的数学化身

【概念深挖】柯尼斯堡七桥问题，剥掉所有无关信息后，剩下一张图：**四块陆地 → 四个点，七座桥 → 七条线**。真正重要的是“哪两个点之间连了一条线”，即**连接关系（connection）**。这也是**图（graph）**的第一直觉：**图 = 一组“对象” + 一组“对象之间的连接”**。

【历史溯源】**1736 年，欧拉（Leonhard Euler）** 发表论文《关于位置几何问题的解答》（*Solutio problematis ad geometriam situs pertinentis*）——这被公认为**图论的诞生标志**。欧拉的核心洞察正是抽象：他证明“能否走通”与河宽岛大无关，只与“连接关系”有关。值得一提的是，欧拉把这类问题归为“**位置几何（geometry of position）**”——一门研究“位置与相对位置”而非“度量与形状”的几何，这几乎是“图论”最早的雏形。“**把现实问题抽象成图再求解**”，正是图论给计算机科学留下的最深刻的遗产：**邻接关系、网络、搜索，全是这张图的子孙**。

【工程实践】你今天做的“把问题抽象成图”，在工程里每天都在发生：**社交网络**（人是点、好友是边）、**地图导航**（路口是点、道路是边）、**互联网**（主机是点、链路是边）、**程序依赖**（模块是点、引用是边）。**凡是关心“谁和谁相连”而“具体形状无所谓”的地方，都可以/应该抽象成图**。这一句“应该”，就是你在后面所有章节（树、最短路径、着色、P vs NP）里反复用到的起手式。

第 2 轮：图是什么——顶点、边，和它的“规矩”

老谟：“你说‘四块地四个点、七座桥七条线’。那我问你：**‘边’和‘边’之间能随便画吗？一个点能不能画两条线到同一个点？**”

造物主：“.....能啊？桥是一座一座的，岛和岸之间可以有好几座桥，所以两个点之间可以有好几条线？那这样画，还叫图吗？”

老谟：“叫。我们把‘两个点之间不止一条线’这种图，专门叫**多重图（multigraph）**。再问你：**一座桥能不能从一块地出发，又回到同一块地？**”

造物主：“.....那得先有座‘自己绕自己一圈’的桥？这现实里有吗？”

老谟：“现实里少见，可数学里不能堵死它。我们把‘一个点连到它自己’的边，叫**自环（loop）**。你手上有两种‘加戏’：多重边、自环。那问题来了——**最‘干净’的图，长什么样？**”

造物主：“.....没有多重边、没有自环？每两个点之间最多一条边，边也不连到自己？”

老谟：“对。那种图叫**简单图（simple graph）**——它最省事，也最常用。可我们为什么要跟你掰扯多重图和自环？因为**如果不先把‘干净’和‘加戏’的界限划清楚，后面所有定理都会出岔子**。举个例子：一笔画的判定，在有多重边的图里，跟简单图里的说法就得调一调。所以先把图的家底分清楚，后面才敢下结论。现在，我们说‘图’，默认是哪种？”

造物主：“.....默认简单图，除非特别说有多重边或自环？”

老谟："就这个规矩。**约定优先，例外明说。**记住这句话，你后面八章都用得上。"

知识锚点：图的三种形态与"默认约定"

【概念深挖】图 (graph) 由一个**顶点集 (vertex set)** V 和一个**边集 (edge set)** E 组成。边的形态决定图的种类：

- **简单图 (simple graph)**：任意两个顶点之间**至多一条边**，且**没有自环**（边不连接自己）。这是默认形态。
- **多重图 (multigraph)**：允许两个顶点之间有**多条边**（多重边）。
- **带自环的图**：允许边的一个端点就是另一个端点（自环）。

约定观元说明：默认"图=简单图"，这是数学界的**约定选择**——不是因为简单图更"本质"，而是因为它最省事、最常用、定理表述最干净。**正如前几册里"约定是地基"的主线：先定好'默认是什么'，后面的证明才有稳定的立足点。**当需要处理多重边或自环时，一定要**显式声明**，否则定理的适用范围会含糊。这一个"默认+例外明说"的习惯，贯穿全册。

【工程实践】简单图/多重图/自环在工程里不是学术分类，是**数据建模的选择**：社交网络"关注"关系默认是简单图（两人之间只有一种关系）；而"某乘客在某条航线上飞过多次"就是多重图（同一对城市之间多条边）。**选哪类图，取决于你关心的关系是否允许重复/自指。**图建模的第一步，往往就是问自己：这里会有多重边吗？会有自环吗？——想清楚了再决定用什么结构存。

第 3 轮：度数——"一个点连了几条边"，和它藏着的惊天规律

老谟："图有了。现在问个'数数'的问题：**看一个点，它能告诉你什么最朴素的信息？**"

造物主：".....它连了几条边？"

老谟："对，这叫这个顶点的**度 (degree)**，记作 $d(v)$ 。你数一数七桥图里，每个点的度是多少。"

造物主："我数数.....这个岸连了 3 座桥，那个岸 3 座，这个岛 5 座，那个岛 3 座？那就是 d 分别是 3, 3, 5, 3？"

老谟："先记着。现在我要你干一件更怪的事——**把四个点的度加起来，再数数总共有几条边。**"

造物主："加起来： $3 + 3 + 5 + 3 = 14$ 。边嘛.....七座桥，七条边。14 是 7 的两倍？"

老谟："你数出了一个**不是巧合的巧合**。你再随手画一张别的图，数一数，是不是也是'**度数之和 = 2 × 边数**'？"

造物主：".....我再画一张：三个点，两条边连成一条线。中间那个点度是 2，两头各是 1。加起来 $1 + 2 + 1 = 4$ ，边数是 2， $2 \times 2 = 4$ ！又是两倍！这.....为什么？"

老谟："你先别说'为什么'，先想'凭什么'。**'度数之和是边数的两倍'**——这句话你信吗？你觉得它是**'碰巧'**，还是**'必然'**？如果是必然，它凭什么是必然？"

造物主：".....必然？因为每条边有两个端点？一条边数度数的时候，它贡献了两次——一次数到左端点头上，一次数到右端点头上？"

老谟："你这就是在往'证明'的路上走了。这个**'度数之和 = $2 \times$ 边数'**，名字叫**握手引理 (handshake lemma)**。它简单得像废话，可威力大得吓人——下一轮你就知道了。先把'为什么'钉死：完整证明在走廊引理 1，你翻过去对照着读。"

知识锚点：度数定义与"为什么是两倍"

【概念深挖】**度数 (degree)**：顶点 v 的度 $d(v)$ = 与 v 相关联的**边的条数**（在简单图中，就是 v 的邻居个数）。**握手引理 (handshake lemma)**：对任意图（顶点集 V 、边集 E ），

$$\sum_{v \in V} d(v) = 2|E|.$$

为什么想到"两倍"：把"数所有顶点的度"这件事换个角度——**等价于"数每条边的两个端点"**。每条边恰好贡献 2（一个端点被算一次）。于是"所有顶点的度加起来"，其实就是"所有边被端点记了两次账"。这就是" $\times 2$ "的来源。**严格证明在走廊引理 1。**

注：**简单图**里自环另算（自环贡献 2）；本册默认简单图，自环的口径在走廊"边界"节说明。

【工程实践】**度数 = 连接数**，在工程里是顶点的"热度/重要度"的直接度量：**社交网络里一个人的粉丝数 (度数)、网页被其他网页链接的次数 (入度)、网络里一台路由器的连接数**。握手引理说"度数之和必为偶数"——这在工程里是个**绝妙的"合理性检查"**：如果某份数据算出来度数之和是奇数，那一定是你数错了，或数据有缺漏。一个**"数出来的全局性质"**，能反过来揪出局部错误——这就是**全局约束的威力**。

第 4 轮：握手引理的威力——"五个人的手都握了奇数次"为什么不可能

老谟："给你个场景：一场聚会，有 5 个人。有人统计：第一个人握手 1 次，第二个人 2 次，第三个人 3 次，第四个人 4 次，第五个人 5 次。你说，这份统计，可信吗？"

造物主：".....5 个人的度数分别是 1, 2, 3, 4, 5，加起来是 15，是奇数！可握手引理说度数之和必须是 $2|E|$ ，是偶数。所以这统计是假的！"

老谏："对。可我想让你再多看出一步。你刚才只是把它代进公式。要是我不告诉你这个公式，让你数，你也能发现不对劲——可你发现的是'哪个性质'？你想想，度数之和是奇数，意味着什么？"

造物主：".....意味着奇数度的顶点，个数是奇数？1, 3, 5 三个奇数度，三个是奇数个.....那假如我说'这 5 个人握手次数是 1, 2, 3, 4, 6'，和是 16 是偶数，就信了？"

老谏："你差点又掉坑里。1, 2, 3, 4, 6 的度数和是 16，偶数是满足了，可它真的能对应一场真实的聚会吗？——我提醒你：握手是双向的，甲握到乙，乙也握到甲。你回头再数一遍，这串度数里有没有一个更隐蔽的破绽？"

造物主：".....等等，我数一下奇度顶点的个数：1, 3 是奇数度，2, 4, 6 是偶数度，奇数度的顶点有 2 个——2 是偶数个。哎，可这不是又满足了？到底是哪不对？"

老谏："哈哈，这串数啊，**度数和是偶数** 和 **奇数度顶点有偶数个** 两条都满足，可它照样不代表真实聚会——因为 1, 2, 3, 4, 6 里最大度数是 6，可一共只有 5 个人，一个人最多只能跟其他 4 个人握手，度数上限是 4——这串数连'每个人最多握 4 次手'这条常识都违反了，除非那图允许一对人握好几次（多重图）。这一轮我不给你完整答案，只给你一个问题：**握手是双向的，这个事实，除了'每边两度'，还逼出了什么？**真正要紧的，是你刚才自己摸出的那条：**不管怎样，奇数度的顶点，个数必须是偶数。**这条叫'奇偶性结论'，我们下一轮用反证法给它钉死。"

知识锚点：奇数度顶点必为偶数个——握手引理的第一个杀手锏

【概念深挖】**推论（奇度顶点个数）：**任意图中，**度数为奇数的顶点个数必为偶数。**为什么想到这一步：由握手引理， $\sum d(v) = 2|E|$ 是偶数。把和拆成"偶度顶点的和 + 奇度顶点的和"：偶度那部分是偶数（偶数个偶数相加仍为偶数），要让总数为偶数，**奇度那部分（奇数相加）也必须是偶数**，而奇数相加得偶数的唯一办法，就是**有偶数个奇数**。于是奇度顶点个数为偶数。**完整证明在走廊推论 1（反证法）。**

【历史溯源】这条"奇偶性"的用法，正是欧拉 1736 年论文的核心手段之一。欧拉发现：**在一个图形中，若每个顶点被经过的次数是"进出各一次"，则奇数度的顶点会给"一笔画"添麻烦。**他正是用这种"数度数奇偶"的手法，证明了七桥无解。**所以你今天学的不是旁枝末节，是欧拉当年那把开山刀的刀锋。**

【工程实践】奇偶性检验是工程里最廉价的**错误探测器**：**电路里的端口数、网络里的连接数、甚至一份"关系表"里各行的出现次数**，凡是你知道它本质是"图"的数据，就能用"奇度顶点必为偶数个"来反查有没有数错。一个 0 成本的性质，能拦下一堆低级错误。

第 5 轮：连通——从“能不能走过去”到“图分了几块”

老谟：“数数讲完了，该讲‘走’了。你从七桥图的一个点出发，沿着边走，能走到所有点吗？”

造物主：“.....我试试。从岸 A 出发，能到岛甲、岛乙、岸 B.....哎，好像四个点都能到？七座桥连得挺密。”

老谟：“那你再画一张图：两个孤岛，各自一个点、各自一条边，谁也不连谁。从左边那个点出发，能走到右边那个点吗？”

造物主：“走不到！中间隔着河，没桥。所以这图‘断’成两块了？”

老谟：“对，这就是**连通 (connected)**。一张图，如果任意两个点之间都能沿着边走通，就叫连通图；否则它就‘分块’了——那些能互相走通的点，抱成一团，叫一个**连通分量 (connected component)**。你先记住直觉：**连通 = 有没有‘一座跨不过去的断桥’把图隔开**。可我还要问你：‘能走通’这三个字，能不能说严谨一点？什么算‘走通’？”

造物主：“.....从一个点出发，沿着边一步步走，走几步都行，只要最后能到另一个点？这样走出来的‘路线’，叫什么？”

老谟：“你问的‘路线’，就是下一轮的核心：**路径 (path)**。你先憋着，这轮的落点是——**连通，是这张图有几个整体’的判据**。七桥图四个点全连在一起，是一个连通分量。你记住这个‘分块’的直觉，下一轮我们把它和‘能不能走通’能不能一笔画‘拧’在一起。”

知识锚点：连通与连通分量

【概念深挖】路径 (path)：从点 v_0 出发，沿边依次经过 $v_1, v_2, \dots, v_k$ ，每条相邻边都是图中的边，这样一串点叫一条**路径**；起点终点相同的路径叫**回路 (circuit/cycle)**。**连通 (connected)：**图中任意两个顶点之间都存在一条路径。**连通分量 (connected component)：**把“互相能走通”的点们聚成的一团；一张图被分成若干个连通分量。一个点自身也算一个连通分量。

为什么想到“连通分量”：连通性是个“要么全通、要么断块”的全局性质。当图不连通时，“分了几块、每块多大”比“它连不连通”更有信息量——这就是连通分量的用处：它把一张“断开的图”拆成若干张“各自连通的子图”。

【工程实践】连通分量是工程里“分组”的利器：社交网络里的“小圈子”、网络里“彼此可达的网段”、代码依赖图里“相互引用的模块群”，本质都是连通分量。判断一张图“断没断”、断成几块，是许多算法（如后续 BFS/DFS）的第一步——这也是为什么下一章（树与遍历）要先从“怎么走遍一张图”开始。

第 6 轮：回到七桥——“一笔画”的判据，和 4 个奇度顶点的死刑

老谟：“现在把你这章所有的线索，拧到七桥这个终极问题上。**问题：能不能一笔走过七座桥，每座桥恰好一次？**先别管‘一笔’这个词，你想：‘每座桥恰好走一次’，在图上是什么意思？”

造物主：“.....就是每条边恰好走一次？从某个点出发，沿着边走，每条边只用一次，看能不能走完所有边？”

老谟：“对。这样一个‘把每条边恰好走一遍’的路线，叫**欧拉迹 (Eulerian trail)**；如果还要求回到起点，就叫**欧拉回路**。你直觉一下：**什么样的情况下，这种路线存在？**换句话说，‘每座桥恰好走一次’对每个点的度，提了什么要求？”

造物主：“.....我琢磨琢磨。我在中途经过一个点，进来一次、出去一次，就耗掉这个点的 2 度。所以中间的点，度得是.....2 的倍数？偶数度？可起点终点呢？起点第一次是‘出去’，终点最后一次是‘进来’，这俩可以各剩 1 度？”

老谟：“你这句话，就是一笔画判据的雏形。我们把它说全：一条能走遍所有边的路线，**除了起点终点，其余每个点被‘进一次出一次’地消耗，度必须是偶数**；而起点终点如果不同，各自可以‘多出 1 度’，所以允许这两个点是奇数度。合起来：**如果存在一笔画，那么奇数度顶点的个数只能是 0 或 2**。现在——你去数数七桥图里，有几个奇数度顶点？”

造物主：“.....七桥图四个点的度是 3, 3, 5, 3，**四个全是奇数度！**那就是说.....奇数度顶点有 4 个，既不是 0 也不是 2，所以**根本没有一笔画！**我下午走不通，不是因为我不够努力，是因为**这条路根本不存在！**”

老谟：“对。这就是为什么欧拉敢下结论——**他不用傻试，他用证明**。但你要注意，我们刚才这个推理里，还缺一个方向：我们只说了‘如果存在一笔画，那么奇数度顶点是 0 或 2’——这是**必要条件**。它足够判七桥的死刑了，因为七桥连必要条件都不满足。至于‘奇数度是 0 或 2’是不是**充分条件（够不够保证存在）**，那个更强的问题，我们留到第 5 章去彻底解决。这一章，你先把‘必要条件’这半边，在走廊里用反证法钉死。”

知识锚点：一笔画（欧拉迹）判定的直觉

【概念深挖】**欧拉迹（Eulerian trail）**：经过图中**每条边恰好一次**的路径（不必回到起点）；若回到起点，叫**欧拉回路**。**一笔画存在的必要条件**：

- 若存在一条欧拉迹，则图中**奇数度顶点的个数只能是 0 或 2**（0 = 能回到起点的欧拉回路；2 = 从其中一个奇度点出发、到另一个奇度点结束的一笔画）。**为什么想到这步：一条笔迹里，除了起点和终点，任何被“路过”的中间点，都是“进来一次、出去一次”，成对消耗 2 度，所以中间点度必为偶数；起点终点若不同，各多出 1 度，于是这俩是奇度。这就是“0 或 2 个奇度顶点”的来历。完整反证证明在走廊定理 1。**

【历史溯源】**七桥问题正是欧拉迹判定最著名的发源地**。欧拉 1736 年的论文不仅提出“位置几何”，还给出了一笔画/欧拉迹的核心判据，直接判定柯尼斯堡七桥无解。**后人把这个判据形式化为“连通且 0 或 2 个奇度顶点”**（充分性由后续数学家补全），而“欧拉迹”之名也纪念他。**你今天走过的，正是数学史上“图论第一道定理”的路。**

【工程实践】一笔画判据在工程里看似无用，实则不然：**网络布线设计（尽量少断点）、管道巡检路线（一笔走遍所有管线）、甚至“中国邮递员问题”的雏形**，都跟“每边恰走一次”有关。**更普适的是它的方法论：当你面对“一个目标到底存不存在”时，先找它的‘必要条件’——必要条件都不满足，就能直接否定，连试都不用试。**这种“先找必要条件卡掉一半可能”的思路，是算法设计的核心习惯。

第 7 轮：落点——图论是“关系的语言”，也是“可计算性”的入口

老谟：“绕了一圈，回到最开始。你最初问‘一堆点和线，凭什么配叫数学’，现在你怎么答？”

造物主：“……点线不是图画，是‘关系’的化身。我扔掉河宽岛大，留下连接关系，七桥就从谜变成了能判定的命题。图，是‘谁和谁相连’这种关系的数学语言。”

老谟：“你手里现在有哪几件趁手的兵器？”

造物主：“**图（顶点+边）、度数 $d(v)$ 、握手引理 $\sum d(v) = 2|E|$ 、奇度顶点个数为偶数、连通/连通分量**，还有一笔画的必要条件——奇度顶点只能是 0 或 2。”

老谟：“够了。我再给你一句话收尾：**你今天学的不是‘画图’，是‘把关系变成可算的’**。这一句‘可算’，就是整册书的魂——从图到树、到最短路径、到着色，再到图灵机、停机问题、P vs NP，全在问同一件事：**‘这个目标，到底能不能算出来？’**而你这一章，第一次亲手体验了‘判断一个目标存不存在’——不是靠试，是靠证明。你准备好了吗？”

造物主：“……准备好了。我隐约觉得，‘证明一个东西不存在’，比‘证明它存在’更难，也更有意思。”

老谟：“那你去走廊，把‘七桥无解’这件‘不存在’的事，亲手证一遍。”

严格化走廊

位置说明：对话负责"直觉"——为什么度数之和是两倍、为什么一笔画要求奇度顶点是 0 或 2。本走廊负责"严谨"——把直觉落成精确的定义和可复写的证明。本章有**三座可复写证明：引理 1（握手引理）、推论 1（奇度顶点个数为偶数）、定理 1（一笔画必要条件，反证法）**。每步标注"证明类型 + 依据"，绝不用"显然"。

定义：图、简单图、度数

设 V 是非空有限集合（**顶点集**）， E 是 V 中无序二元组（两两不同）构成的集合（**边集**）。称 $G = (V, E)$ 为一个**简单图**。

- 若一条边 $e \in E$ 的两个端点相同，称 e 为**自环 (loop)**；简单图**不含自环**。
- 若 E 允许同一对端点出现多条边，称 (V, E) 为**多重图 (multigraph)**。
- 顶点 $v \in V$ 的**度数 (degree)** $d(v)$ ：与 v 关联的边的条数。**在简单图中**， $d(v) = v$ 的邻居数。

证明类型标注：本章的证明方法开始"分招式"了——**引理 1 用"双计数"（换个角度数两遍），推论 1 用"反证法"（先假设结论不成立，推矛盾），定理 1 也用"反证法"**。反证法正是第 7 章停机问题要反复用的核心招式，本章就从这里开始给你喂第一口。

关于对话：本章第 3 轮、第 4 轮、第 6 轮都做了"引导式推导"（为什么想到"每边数两度"、"中间点进出成对"这些关键一步）。**正式的可复写证明（每步标依据）全部在本走廊**。对话负责"为什么会想到这一步"，走廊负责"怎么把它写标准"——与前几册口径完全一致。

引理 1（握手引理）： $\sum_{v \in V} d(v) = 2|E|$

证明类型：直接证明（双计数——把"数所有点的度"换成"数每条边的两个端点"）。

证明：

- 第 1 步：**改写计数对象**： $\sum_{v \in V} d(v)$ 的含义是"把每个顶点所关联的边的条数加起来"。（依据：度数定义 $d(v) =$ 与 v 关联的边数。）
- 第 2 步：**换个角度数**：等价地，对每条边 $e \in E$ ，它有两个端点（在简单图中两个端点不同），因此在"数所有顶点的度"的过程中，**边 e 恰好被数到 2 次**——一次计入其一个端点，一次计入其另一个端点。（依据：简单图定义——每条边恰有两个端点，且被两端的度数各计入一次。）
- 第 3 步：**汇总**：把"每条边被数 2 次"累加起来，正好是 $2 \times |E|$ 。（依据：第 2 步对每条边都成立，故对所有边求和即得。）

- 第 4 步：由第 1 步（左边是 $\sum d(v)$ ）与第 3 步（右边是 $2|E|$ ），得 $\sum_{v \in V} d(v) = 2|E|$ 。■

依据标注：步 1 用"度数定义"；步 2 是**本引理的发动机**——"双计数"：把"数点"换成"数边"，每条边贡献 2；步 3、4 用加法求和与等式传递收束。**读法：**这个证明教你的，是分析/离散数学里最常用的**"换一个角度数两遍"**——同一件事，数法不同，却能导出关系式。

推论 1（奇度顶点个数为偶数）：任意图中，度数为奇数的顶点个数必为偶数

证明类型：反证法（假设奇数个奇度顶点，推出 $\sum d(v)$ 为奇数，与握手引理矛盾）。

证明：

- 第 1 步：设奇度顶点的个数为 k 。假设**反证前提**： k 是奇数。
- 第 2 步：把 $\sum_{v \in V} d(v)$ 拆成两块：偶度顶点的度之和 + 奇度顶点的度之和。

$$\sum_{v \in V} d(v) = \sum_{v: d(v) \text{ 偶}} d(v) + \sum_{v: d(v) \text{ 奇}} d(v).$$

（依据：把 V 按"偶度/奇度"分成两个不相交子集，和可拆为两部分——加法可分组。）

- 第 3 步：第一块（偶度顶点之和）是**偶数**（偶数个/若干个偶数相加仍为偶数）。（依据：偶数的定义，偶数之和仍为偶数。）
- 第 4 步：第二块（奇度顶点之和）： k 个奇数相加。由于 k 是奇数（反证前提）， k 个奇数相加得**奇数**。（依据：奇偶性算术——奇数个奇数相加为奇数。）
- 第 5 步：于是 $\sum_{v \in V} d(v) = \text{偶数} + \text{奇数} = \text{奇数}$ 。（依据：奇偶性算术——偶数加奇数为奇数。）
- 第 6 步：但由**握手引理**（引理 1）， $\sum_{v \in V} d(v) = 2|E|$ ，是**偶数**。这与第 5 步"为奇数"**矛盾**。（依据：握手引理； $2|E|$ 含因子 2，故为偶数——偶数的定义。）
- 第 7 步：矛盾说明反证前提（ k 为奇数）不成立，故 k 必为**偶数**。■

依据标注：步 1 设反证前提；步 2-5 用加法分组与奇偶性算术（奇数个奇数之和为奇数）；步 6 用握手引理制造矛盾。**这是你本册第一次完整走完一次反证法**——先假设"结论的反面"成立，推出矛盾，从而证明结论成立。这个套路第 7 章会反复出现，先在这里扎下根。

定理 1（一笔画必要条件）：若图 G 存在一条欧拉迹，则 G 中奇度顶点的个数只能是 0 或 2

证明类型：反证法（假设"存在欧拉迹，但奇度顶点个数 $\notin \{0, 2\}$ "，沿这条迹考察每个点的度，推出"除起终点外全是偶度、奇度顶点至多起终点这 2 个"，与假设矛盾）。

证明：

- 第 1 步：设 G 存在一条欧拉迹 T ，它从 s 出发，到 t 结束（可能 $s = t$ ），且经过每条边**恰好一次**。（依据：欧拉迹的定义。）
- 第 2 步：设**反证前提**：奇度顶点的个数 $k \notin \{0, 2\}$ （即 $k \geq 3$ ，因为 k 是非负整数，不属于 $\{0, 2\}$ 就意味着至少是 3）。
- 第 3 步：**沿迹考察每个顶点被“进出”的情况**。对迹 T 上任意一个**既非起点 s 、也非终点 t 的中间顶点 v** ：迹每“路过”它一次，就是“进来一次 + 出去一次”，恰好消耗 v 的 **2 度**。（依据：路径的定义——沿边依次经过；每进入 v 用一条边、离开 v 用另一条边，两条边各计入 v 的度 1 次，合计 2。）
- 第 4 步：由于迹 T 把每条边都用掉了恰好一次，且 v 被路过的总次数就是“与 v 关联的边被使用的次数”，而这些边全被 T 用完。故 $d(v) = (v \text{ 被进出的次数}) \times 2$ ，是**偶数**。（依据：第 3 步 + “每条边恰好用一次”的定义；每进出一轮消耗 2 度，累加仍为 2 的倍数。）
- 第 5 步：**讨论起点终点**（即所有顶点中，除第 3-4 步已证为偶度的中间点外，只剩 s 、 t 两个候选）：
 - 若 $s = t$ （欧拉回路）：起点也视为“既进且出成对”，同样被成对消耗，故 s 也是偶度。此时**所有顶点都是偶度，奇度顶点个数为 0**。
 - 若 $s \neq t$ （一笔画，非回路）： s 比“纯中间点”**多一次出发**（多 1 度）， t **多一次到达**（多 1 度），故 s 、 t 各为奇度；其余点皆偶度。此时**奇度顶点个数为 2**（且正是 s 与 t ）。（依据：第 3、4 步的“进出成对”分析，加上起点多一次出发、终点多一次到达的事实。）
- 第 6 步：综合第 3-5 步：所有“非起终点”的点都是偶度，奇度顶点至多只可能是 s 、 t 这 2 个；更精确地，奇度顶点个数 k 只能是 **0**（当 $s = t$ ）或 **2**（当 $s \neq t$ ）。于是 $k \in \{0, 2\}$ 。（依据：第 4、5 步的分类穷尽所有情况。）
- 第 7 步：**矛盾**：第 6 步推出 $k \in \{0, 2\}$ ，与第 2 步的反证前提 $k \notin \{0, 2\}$ **矛盾**。（依据：矛盾律——同一命题 k 不能同时属于又不属于 $\{0, 2\}$ 。）
- 第 8 步：反证前提不成立，故**若 G 存在欧拉迹，则奇度顶点个数只能是 0 或 2**。■

依据标注：步 1 用欧拉迹定义；步 2 设反证前提（否定结论）；步 3-4 是**本定理的发动机**——“中间点进出成对、消耗 2 度”；步 5 分起终点是否重合，穷尽所有顶点；步 6 汇总出“至多 2 个、且只能是 0 或 2”；步 7 用矛盾律收束；步 8 结论。**读法：**这个证明的骨架是反证法——**先假设“存在一笔画，但奇度顶点却不是 0 或 2”**，然后沿笔迹数度：每个中间点“进一次出一次”消耗 2 度（步 3-4），所以只有起点终点可能“不平衡”（步 5），于是奇度顶点最多只能是起点终点这 2 个（步 6），与“不是 0 或 2”的假设矛盾（步 7）。“**中间点进出成对**”这条，是证明里真正让反证前提破产的那把刀。

反例/边界：一笔画判定的适用边界

1. **必要条件 $\neq$ 充分条件**：本章只证了“存在欧拉迹 $\Rightarrow$ 奇度顶点 0 或 2”。反过来（0 或 2 个奇度顶点 $\Rightarrow$ 存在欧拉迹）还需要图连通，充分性的完整证明留到第 5 章。本章只负责“判死刑”（必要条件不满足就一定不行），不负责“保成功”。
2. **七桥图的判定**：七桥图中四点的度分别是 $d = 3, 3, 5, 3$ ，四个全是奇度，奇度顶点个数是 4，既不满足 0 也不满足 2，故**不存在一笔画**。这是定理 1 的直接应用。
3. **自环的度**：在允许多重图的场合，一条自环对端点贡献 **2 度**（它“出去又回到自己”）。本册默认简单图，自环不出现；若显式出现多重图，需按此口径计算。
4. **孤立点的度**：度为 0 的顶点（孤立点）是偶度，不影响奇度计数；但它若孤立，一笔画永远不可能覆盖到它——这也说明定理 1 只是必要条件，连通性必须单独检查。

知识点总结

知识点（本章推出的核心概念）

- **图 (graph)**：顶点集 V + 边集 E ，是“谁和谁相连”关系的数学化身；默认简单图（无多重边、无自环）。
- **度数 (degree)**： $d(v)$ = 与 v 关联的边数（简单图中即邻居数）。
- **握手引理**： $\sum_{v \in V} d(v) = 2|E|$ ——“每条边被端点记两次账”。
- **奇度顶点个数为偶数**：由握手引理推出，是廉价而强力的全局约束。
- **路径/回路/连通**：路径是“沿边走通”；连通 = 任意两点可走通；连通分量 = 互相走通的点团。
- **欧拉迹判定 (必要条件)**：若存在一笔画（欧拉迹），则奇度顶点个数只能是 **0 或 2**；七桥图有 4 个奇度顶点，故无解。

历史结论（本章涉及的史料）

- **1736 年**，欧拉发表《关于位置几何问题的解答》，用抽象+奇偶性判定柯尼斯堡七桥无解——**图论的诞生标志**。
- 欧拉把这类问题归为“位置几何”（geometry of position）——研究“相对位置”而非“度量形状”，是图论的雏形。
- **欧拉迹/欧拉回路**之名为纪念欧拉；充分性（连通 + 0/2 奇度顶点 $\Rightarrow$ 存在）由后续数学家补全，本册第 5 章解决。

工程实践结论（本章知识在真实工程里怎么用）

- **抽象成图**：社交/导航/网络/依赖，凡关心“谁和谁相连”就能建模成图。
- **度数 = 热度/连接数**：粉丝数、网页入度、路由器连接数；握手引理“度数之和为偶”是**零成本合理性检查**。
- **奇度必偶**：电路端口、关系表出现次数，用奇偶性反查低级错误。
- **先找必要条件**：判断“目标存不存在”，先看必要条件，不满足直接否定，省掉盲目尝试——算法设计的核心习惯。

计算训练场

位置说明：对话负责“为什么”，这里负责“怎么算”。每题动笔 10 分钟以上，先自己写再对答案。**难度递进：易→中→难。**

题 1（易→中）：画出使 $\sum d = 10$ 的 3 顶点图，穷举验证握手引理

题干：老谟让造物主画出若干个 3 个顶点的简单图，使得度数之和恰好为 10，并用穷举验证握手引理 $\sum d(v) = 2|E|$ 。

完整分步解答：

- 第 1 步（定边数）：由握手引理， $\sum d(v) = 10 = 2|E|$ ，故 $|E| = 5$ 。所以任务是画一张**3 个顶点、5 条边**的简单图。
- 第 2 步（可画性检查）：3 个顶点的简单图，最多能有多少条边？每对顶点最多 1 条边，3 个顶点有 $\binom{3}{2} = 3$ 对，所以最多 3 条边。**5 条边超过了上限 3，画不出来！**因此“3 顶点 $\sum d = 10$ 的简单图”**不存在**。
- 第 3 步（验证握手引理的反向约束）：为什么画不出来？因为 $|E|$ 最大只能是 3，所以 $\sum d(v) = 2|E| \leq 6 < 10$ 。**握手引理不仅告诉你“度数之和是多少”，还通过 $|E|$ 的上限，反过来卡死了“度数之和能做到多大”。**这正是握手引理的威力——它是**双向约束**。
- 第 4 步（更正/收尾）：若要 $\sum d = 10$ 且顶点足够多，边数就得是 5。例如 6 个顶点的简单图，可以画 5 条边，比如一条“链” $v_1-v_2-v_3-v_4-v_5$ 再加一条 v_5-v_6 。数一下度数： $d(v_1) = 1, d(v_2) = 2, d(v_3) = 2, d(v_4) = 2, d(v_5) = 3, d(v_6) = 1$ ，和为 $1 + 2 + 2 + 2 + 3 + 1 = 10$ ，边数 5， $2 \times 5 = 10 \checkmark$ 。

答：3 顶点简单图最多 3 条边、 $\sum d \leq 6$ ，故不存在 $\sum d = 10$ 的 3 顶点简单图；这是握手引理通过“边数上限”给出的反向约束。若换成 6 顶点，可画 5 条边实现 $\sum d = 10$ 。

常见错误：① 只记得" $\sum d = 2|E|$ "却不检查 $|E|$ 是否可行（忘了 3 顶点最多 3 条边）；② 以为" $\sum d$ 想要多大都行"，忽略了简单图边数的上限 $\binom{n}{2}$ ；③ 画边时不小心画成多重边或自环，破坏了"简单图"前提。

题 2 (中)：度序列判定——1, 1, 2, 2, 3, 3 能不能是某个简单图的度序列？

题干：给定一串度数 1, 1, 2, 2, 3, 3 (6 个顶点)，老谟问：(a) 它能是一个简单图的度序列吗？(b) 如果是，画出其中一个例子；如果不是，说明为什么。

完整分步解答：

- (a) 第 1 步（握手引理检查）： $\sum d = 1 + 1 + 2 + 2 + 3 + 3 = 12$ ，是偶数 $= 2|E|$ ，故 $|E| = 6$ 。6 个顶点的简单图最多 $\binom{6}{2} = 15$ 条边，6 条可行。**握手引理不拦。**
- (a) 第 2 步（奇偶性检查）：奇度顶点是度数为 1, 1, 3, 3 的那 4 个，**4 是偶数**，奇偶性也满足。**两个初步检查都过。**
- (a) 第 3 步（尝试构造）：试着画：两个度 3 的顶点 a, b ，两个度 2 的 c, d ，两个度 1 的 e, f 。让 a, b 都连向 c, d, e, f （给 a, b 各贡献 4 度，太多）。调整：让 a 连 b, c, d ($d(a) = 3$)， b 连 a, c, e ($d(b) = 3$)， c 连 a, b, f ($d(c) = 3$ ，不对， c 应为 2)。再调整：
 - 令 a 连 $b, c, d \rightarrow d(a) = 3$ 。
 - 令 b 连 $a, e, f \rightarrow d(b) = 3$ 。
 - 令 c 连 $a, f \rightarrow d(c) = 2$ 。
 - 令 d 连 $a, e \rightarrow d(d) = 2$ 。
 - 令 e 连 $b, d \rightarrow d(e) = 2$ ，但 e 应为 1，**不对**。
- (a) 第 4 步（系统修正）：数一下已经连出的度数： $a = 3, b = 3$ 对了； c 连了 a, f ， f 目前只有 1 度（连了 c ）， f 应为 1，可以。 d 连了 a, e ， e 目前连了 b, d 共 2，但 e 应为 1。**冲突：** e 只应连 1 条，却连了 b, d 两条。重新分配：
 - a 连 b, c, d ($d(a) = 3$)
 - b 连 a, e, f ($d(b) = 3$)
 - c 连 a, e ($d(c) = 2$)
 - d 连 a, f ($d(d) = 2$)
 - e 连 b, c ($d(e) = 2$) —— e 应为 1，仍不对。

试着让 e, f 都是度 1：它们各只连一条边。两条"叶" e, f 各连向谁？让 e 连 a ， f 连 b 。则 a 已连 e (1)，还需再连 2 条（到 b, c 或 d ）； b 已连 f (1)，还需 2 条。设 a 连 b, c, d ($a = 3$ 含 e 了吗？ e 连 a 就 1 条， a 要 3 条，可 a 连 b, c, d 就是 3 条，那 e 连 a 就冲突)。重新设

定： a 连 c, d, e ($a = 3$) , b 连 c, d, f ($b = 3$) , 则 c 连 a, b ($c = 2 \checkmark$) , d 连 a, b ($d = 2 \checkmark$) , e 连 a ($e = 1 \checkmark$) , f 连 b ($f = 1 \checkmark$) 。

- (a) 第 5 步 (核对) : $d(a) = 3, d(b) = 3, d(c) = 2, d(d) = 2, d(e) = 1, d(f) = 1$, 正好是 3, 3, 2, 2, 1, 1, 即题给的 1, 1, 2, 2, 3, 3 $\checkmark$ 。边数: $a-c, a-d, a-e, b-c, b-d, b-f$ 共 6 条, $\sum d = 12 = 2 \times 6 \checkmark$ 。

答： (a) 可以, 握手引理与奇偶性都满足; (b) 一个例子: a 连 c, d, e , b 连 c, d, f (其余不连) , 得到度序列 1, 1, 2, 2, 3, 3。

常见错误： ① 只验握手引理 (偶数) 就下结论“能”, 忘了还要**真的画出来** (度序列还需满足“简单图可实现”的更强条件); ② 构造时两个“叶”顶点度数分配不当, 导致某点度数超了; ③ 构造完不回代验证度数是否逐一吻合。

题 3 (难→中) : 七桥图的欧拉迹判定 + 用握手引理找全局矛盾

题干： 柯尼斯堡七桥图 (四个顶点, 度数 3, 3, 5, 3)。(a) 用定理 1 判定是否存在一笔画, 并给出判断依据。(b) 验证 $\sum d = 2|E|$ (数一遍边, 反向核算度数之和)。(c) 如果在这座城再修一座桥, 把某两个岛之间加一条边, 能否让“一笔画”变成可能? 说说你的推理。

完整分步解答：

- (a) 第 1 步 (列度数) : 四顶点度分别为 3, 3, 5, 3。
- (a) 第 2 步 (数奇度) : 3 是奇度, 3 奇度, 5 奇度, 3 奇度——**四个全是奇度顶点**, 奇度个数 = 4。
- (a) 第 3 步 (套定理 1) : 一笔画必要条件要求奇度顶点个数为 **0 或 2**; 这里是 4, **必要条件不满足**, 故**不存在一笔画**。答案: 无解。
- (b) 第 1 步 (求和) : $\sum d = 3 + 3 + 5 + 3 = 14$ 。
- (b) 第 2 步 (数边) : 图有 7 条边 (七座桥) , $2|E| = 2 \times 7 = 14 \checkmark$ 。**反向核算：** 度数之和 14 与 $2 \times$ 边数 14 一致, 验证握手引理成立, 也反向印证“七座桥=7 条边、四个陆地=4 个顶点”的抽象没有数漏。
- (c) 第 1 步 (修一座桥后度数变化) : 在某两个顶点间加一条边, 会使这两个顶点的度各 +1 (都从奇度变偶度, 或偶度变奇度, 取决于原来奇偶)。
- (c) 第 2 步 (能否变 0/2 个奇度) : 现在 4 个奇度。加一条边会让两个顶点的度各 +1, **各改变一次奇偶性：**
 - 若加边连接的是**两个原本就奇度**的顶点: 这两个奇度变偶度, 奇度个数 $4 - 2 = 2$ 。此时存在一笔画 (必要条件满足, 连通性由桥网保证, 故可行)。
 - 若加边连接一奇一偶: 奇度个数仍为 4 (一个变偶、一个变奇)。

- 若连接两个偶度：奇度个数变 6。
- (c) 第 3 步（结论）：**只要在两个原本奇度的顶点之间加一条桥，就能把奇度个数从 4 降到 2，使一笔画成为可能。**（这符合直觉：把"多余的奇度"用一条新桥"配对"掉。）

答：(a) 4 个奇度顶点，不满足"0 或 2"，不存在一笔画；(b) $\sum d = 14 = 2 \times 7 \checkmark$ ；(c) 在两个原奇度顶点间加一条桥，奇度变 2，即可实现一笔画。

常见错误：① 数奇度时漏数 5 或把 5 当偶度（5 是奇数）；② (b) 忘反向核算" $2|E|$ 是否真的等于 $\sum d$ "（这正是红线 4 要求的反向验证）；③ (c) 以为"加一条边"只改变一个顶点（实际上一条边有两个端点，两个顶点的度都变）；④ 忘了奇偶性判断要看"加边前两顶点的原奇偶"。

向导便签

这一章咱们干了什么：从一张柯尼斯堡旧地图，一路剥到"图=顶点+边"这个抽象，然后亲手推翻了七桥传说——不是靠傻试，是靠**证明**。你拿到了四件兵器：**图、度数、握手引理** $\sum d(v) = 2|E|$ ，以及它的两个杀手锏——**奇度顶点必为偶数个、一笔画必要条件（奇度顶点 0 或 2）**。

钉死一句话："一堆点和线"配叫**数学**，因为它们把"关系"变成了"可算的"。度数之和必为偶数、奇度顶点必为偶数个——这些"不用试就知道"的结论，靠的是证明，不是枚举。

记住：你第一次完整走了一次**反证法**（先假设反面，推矛盾）——这是第 7 章停机问题那场硬仗的核心兵器，今天种下了。**"先找必要条件卡掉一半可能"**这个思路，也会跟你到 Dijkstra、到 P vs NP。下一章，我们把"能不能走遍一张图"变成真正的算法：BFS 与 DFS。

造物主手记

我曾踩的坑：

我以为"一笔画走不通"是因为我"没找对路"，被老谟点醒才明白：**那不是'没找对'，是'根本不存在'**——必要条件（奇度顶点 0 或 2）就已经判了死刑。画计算训练场题 1 时，我一开始"想要 $\sum d = 10$ 就画 3 顶点"，怎么画都画不出来，才意识到 3 顶点简单图最多 3 条边、 $\sum d \leq 6$ ——**握手引理是双向的， $|E|$ 有上限，度数之和就有上限**。最险的是题 3 的 (c)：我差点以为"加一条边只改一个点"，被提醒"一条边有两个端点，两个顶点的度都要 +1"。还有数奇度时，我一度把 5 记成偶度——**5 是奇数**，这是最不该犯的错。

顿悟时刻：

真正开窍是"握手引理"落地那一刻——"度数之和等于两倍边数"，居然只要换个角度数：**每条边被两个端点各记一次账**。原来数学里很多"神奇结论"，都藏在一个"换角度数两遍"的动作里。而"一笔画必要

条件"那个证明，让我第一次看见"走笔"的图景：**中间点进出成对、消耗 2 度，只有起点终点能各多 1 度**——所以奇度顶点最多 2 个。更震撼的是，我发现自己真的能**证明一件"不存在"的事**（七桥无解）。原来"证明不存在"不是靠把所有的路都试一遍，而是靠一个绕不开的性质。这让我对"可计算性"这个概念，第一次有了实感。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A（计算，易→中）：握手引理验证 + 度序列

(a) 某图有 8 个顶点、12 条边，求所有顶点度数之和，并判断该图奇度顶点个数的奇偶。(b) 度序列 1, 2, 2, 3, 3, 4（6 个顶点）能满足握手引理吗？能是一个简单图的度序列吗？说明理由。

完整解答：(a) $\sum d(v) = 2|E| = 2 \times 12 = 24$ 。24 是偶数，由推论 1，奇度顶点个数必为偶数。(b) $\sum d = 1 + 2 + 2 + 3 + 3 + 4 = 15$ ，是**奇数**，不满足握手引理（必须为偶数），故**不可能是任何图的度序列**。既然握手引理都不满足，自然也不是简单图。**这就是"先查握手引理，一票否决"的用法。**

题 B（计算，中→难）：一笔画判定 + 构造

给一张简单图，其度序列为 2, 2, 2, 4, 4（5 个顶点）。(a) 判断它是否存在一笔画（欧拉迹）？是欧拉回路还是普通一笔画？(b) 画出满足该度序列的一个具体图，并给出一个欧拉迹/回路的走法。

完整解答：(a) 奇度顶点：度数为 2, 2, 2, 4, 4，全为偶数，奇度个数 = 0。故存在一笔画，且因 0 个奇度顶点，它实际是**欧拉回路**（能回到起点）。 $\sum d = 2 + 2 + 2 + 4 + 4 = 14 = 2 \times 7$ ，边数 7，偶数验证通过。(b) 一个例子：顶点 a, b, c, d, e ，令 d, e 各连 a, b, c （ d 连 $a, b, c \rightarrow d(d) = 3$ ？不对，应 4）。调整：令 d 连 a, b, c, e ， e 连 a, b, c, d ，则 $d(e) = 4, d(d) = 4$ ； a 连 d, e （ $a = 2$ ）， b 连 d, e （ $b = 2$ ）， c 连 d, e （ $c = 2$ ）。度数： $a = 2, b = 2, c = 2, d = 4, e = 4 \checkmark$ 。欧拉回路走法： $a \rightarrow d \rightarrow b \rightarrow e \rightarrow c \rightarrow d \rightarrow e \rightarrow a$ （每条边恰好一次，回到起点 a ）。验证每条边都被用到且只一次。

题 C（计算，中）：用握手引理找矛盾（奇偶性反查）

一场聚会会有 6 个人，有人统计各人握手次数为 0, 1, 1, 2, 2, 5。这份统计可信吗？说明理由。

完整解答：度数（握手次数）之和 = $0 + 1 + 1 + 2 + 2 + 5 = 11$ ，是**奇数**，违反握手引理（必须为 $2|E|$ 偶数），故统计**不可信**。另外注意：5 是奇度、1, 1 是奇度，奇度顶点有 3 个（1, 1, 5），是奇数个，也违反"奇度顶点必为偶数个"——两条都违反，铁定造假。

题 D（概念，检查理解）

(a) "七桥问题"为什么是个数学问题，而不只是脑筋急转弯？(b) 握手引理为什么成立？一句话说明"为什么是两倍"。(c) 一笔画（欧拉迹）存在，为什么要求奇度顶点只能是 0 或 2？

完整解答：(a) 因为它能**抽象成图**：陆地→点、桥→边，问题变成"每条边恰好走一次的路线是否存在"，从而可用度数、奇偶性等数学工具判定，而不再依赖运气试探。(b) 因为每条边恰有两个端点，数所有顶点的度时，每条边被两个端点各记一次账，所以"度数之和"等于" $2 \times$ 边数"。(c) 因为一笔画迹里，每个中间点都是"进一次、出一次"，成对消耗 2 度，所以必为偶度；只有起点（多一次出发）和终点（多一次到达）可以各多 1 度成为奇度，故奇度顶点最多且只能有起点终点这 2 个（若起终点重合则为 0 个）。

预告

你从一张柯尼斯堡旧地图出发，亲手把"七桥"剥成了图，用**握手引理**和**一笔画必要条件**，靠证明——而不是傻试——判了七桥"无解"的死刑。你第一次尝到了反证法的味道，也第一次看见"证明一件不存在的事"长什么样。

可你现在还只会"看"图：你能不能走通、有几块连通分量，你是**肉眼扫出来的**。可图一变大，肉眼的极限立刻就到了——**一张一百万顶点的"图城"**，你怎么保证一条路都不漏地走遍它？

下一章，**树与图的遍历（BFS/DFS）**——我们把"走遍一张图"变成一套**可以照做的算法**，保证你从任意点出发，能**不重不漏**地摸到每一个顶点，还顺手数出它有几块连通分量。而"怎么保证不漏"这件事，会引出本册最重要的一个词：**算法正确性**。你准备好从"看"图，升级到"算"图了吗？

第 02 章 · 树与图的遍历——怎么保证走遍一座“图城”不迷路？

本章从哪个问题出发：上一章你学会了“看图”——数度数、判连通、查一笔画。可图一大，肉眼立刻失效：一张十万顶点的“图城”，你要从一个路口出发，怎么保证**一条路都不漏**地摸到每一个路口？你甚至没法把整张图一次看全，只能一个路口一个路口地问“从这里能走到哪”。推导到哪个结论：你会发现，有一类特殊的图——**树**——虽然“又瘦又疏”，却处处是图的骨架： **n 个顶点的树，恰好 $n-1$ 条边**（这一条会被你亲手用归纳法证明）。而“怎么保证走遍不迷路”，会逼出两个标准答案：**BFS**（一层一层地摸）和 **DFS**（一条道走到黑再回头）——它们不只是“走法”，是**可以照做的算法**，你会在下一章亲眼看到 BFS 怎样为“Dijkstra 最短路径”铺路。

核心认知：**树是“连接得最省、却不散架”的图；BFS/DFS 是“保证不漏”的两种走法**。树告诉你“关系最少也要多少条边才能连成一个整体”，遍历告诉你“怎么把这些关系一条条走完”。这一章是算法思维的正式入场：**“保证不漏”这件事，必须用一个可复写的证明来背书**——这正是你从第 1 章“证明不存在”走到“证明算法正确”的第一次。

引子

我把老谟书房里那张巨大的城市地铁图铺开——几百个站点，几十条线路，密密麻麻缠成一团。我盯着它发呆，老谟走过来，也盯着看。

“在看什么？”

“我在想，”我指着图说，“要是把我随便丢在某个站，我怎样才能保证，把图上每一个站都走一遍，一个不漏？”

老谟没答，反而问我：“你先回答我——‘每一个站都走一遍’，这个目标，你光靠‘随便走走’能保证吗？”

“.....不能保证。我可能绕来绕去，总有角落没去过，可我自己不知道。”

“那你怎么知道‘走完’了？”

“我不知道.....除非有个办法，能让我确定地说：‘我已经走遍了，没有漏的。’”

老谟把茶杯端起来，慢悠悠地吹了口气：“你问出了一个比‘怎么走’深得多的问题——‘**怎么保证**’。而‘保证’这两个字，在数学里，只能由一个东西来给——证明。这一章，我们就要给‘走遍不迷路’，立一座证明。”

对话引擎

第 1 轮：树是什么——“连接得最省，却不散架”

老谟：“在回答‘怎么走遍’之前，你先看一类特殊的图。我给你一个要求：**连接所有顶点，用的边越少越好**。你会画成什么样？”

造物主：“.....越少越好？那就是尽可能少地连，只要还‘不断’就行？比如 5 个点，我用 4 条边把它们串成一条链？能不能更少？”

老谟：“你试试 5 个点，只用 3 条边，能不能让它们‘互相都连得通’。”

造物主：“3 条边只能连 4 个‘档’.....我画：1-2, 2-3, 3-4，那 5 就孤立了，连不通。再加一条 4-5，就是 4 条边了。**好像 5 个点最少要 4 条边？**”

老谟：“你猜对了。 **n 个点要连成一个整体，最少要 $n-1$ 条边**。那种“刚刚够连、一根边也不多”的连通图，就叫**树 (tree)**。它有两个等价的说法：**连通，且没有回路 (环)**；或者**连通，且边数 = 顶点数 - 1**。可你先别急着背定义——我问你：**为什么树‘必须’是这个样子？为什么不能再省一条边？**”

造物主：“.....因为省一条，就有一个点连不上了？或者，会冒出一个环来？”

老谟：“这两个直觉，一个是‘连通’的约束，一个是‘无环’的约束。**树的全部秘密，就藏在这两个约束怎么互相咬合上**。下一轮，我们给“ n 个点 $n-1$ 条边”立一座证明。”

知识锚点：树的两副面孔——“无环”与“最省”

【概念深挖】**树 (tree)** 是**连通**且**无回路 (cycle)** 的图。它的两个等价刻画：

1. **无环 + 连通**：任意两个顶点之间有且仅有一条路径（正因为没有环，“多走一条道”的路不存在）。
2. **最省连通**： n 个顶点的树恰好有 $n-1$ 条边——“连接成一个整体所需的最少边数”。

为什么想到“最省”：树常常被当作一张图的“骨架”——它用最少的边，把所有的点连成一个不散架的整体。“刚刚够、不多一根”正是树的直觉形象。

【历史溯源】**凯莱 (Arthur Cayley, 19 世纪)** 对树的计数做过经典工作——他证明了 n 个不同顶点的**标号树 (labeled tree)** 有 n^{n-2} 棵（即凯莱公式）。这个结果在 19 世纪已出现，后来在工程（尤其网络与化学中“同分异构体的树结构计数”）里反复用到。**提醒你**：‘树的计数’有确凿的史料可引；而 BFS/DFS 这两种遍历算法，虽然常被追溯到家喻户晓的算法前驱，但不宜给具体个人‘强安发明人名’——它们更多是‘自然地长出来的标准做法’，我们后面讲遍历时不硬扣人名。

【工程实践】树是工程里出现频率极高的结构：**文件系统的目录树、组织架构图、语法解析树、数据库索引 (B 树)**，全都长成树。因为它们“**连接得最省、查找不会绕圈**”——树里任意两点路径唯一，正适合做“不迷路的搜索”。这就是为什么“树”会在你学算法时反复登场。

第 2 轮： n 个顶点的树为什么恰好 $n-1$ 条边——剥下一片“叶子”

老谟：“我们要证：**一棵 n 个顶点的树，恰好有 $n-1$ 条边**。你先想想，从这棵树出发，你能不能‘拆掉’一个顶点，让剩下的还是一棵树？”

造物主：“拆掉一个顶点？我得先拆掉连着它的边……拆掉之后剩下的图，还会连通、还没环吗？”

老谟：“你拆哪个顶点，是有讲究的。我提醒你：**树里有没有‘度数为 1’的顶点？**”

造物主：“有啊，端点就是度 1 的——它只连一条边。我把这种顶点叫‘叶子’。”

老谟：“好。**叶子就是你要拆的那个顶点**。你拆掉一片叶子 v 和它唯一的那条边，剩下的图，还是连通、还没环吗？”

造物主：“……剩下的图，肯定没环（环本来就没有）；连通呢？ v 是叶子，只有它一个孤零零地挂在树梢，拆掉它，其他点之间本来怎么连还怎么连，**不会断**。所以剩下的还是一棵树，而且顶点数少了 1、边数也少了 1！”

老谟：“你找到了‘递推’的入口。 **n 个顶点的树，总能找到一片叶子拆掉，变成 $n-1$ 个顶点的树，且边数跟着减 1**。那要是能一直拆下去呢？”

造物主：".....一直拆，最后拆到只剩 1 个顶点、0 条边？那往回推：每拆一次边数比顶点数少 1，最后 $0 = 1 - 1$好像原树的边数就是 $n - 1$ ？可这推理，怎么写成严谨的证明？"

老谟："你刚才的'一直拆下去'，就是**数学归纳法**——它专门对付'从小的成立，推到大的成立'这种链。正式写法在走廊定理 1，你翻过去对照着补全。先把'为什么想到拆叶子'记住：**'拆一片叶子，树还是树，规模还变小'**——这是归纳法想得通的关键。"

▮ 知识锚点：为什么是 $n - 1$ ——"拆叶子"的归纳直觉

【概念深挖】" **n 个顶点 $n - 1$ 条边**"的归纳直觉：任取一棵树的一片叶子 v （度为 1），拆掉 v 和它唯一的边，剩下的仍是一棵树，但顶点数、边数各减 1。如此反复，**规模不断变小**——这是归纳法最顺手的起手式。**严格证明在走廊定理 1（数学归纳法）。**

为什么想到"叶子"：叶子是树里"最孤零零"的顶点，拆它不会破坏任何其他点的连通——"**拆掉一个'无关痛痒'的东西，剩下的还是一样的东西**"，这正是归纳法想要的那种"降规模"结构。

【工程实践】"拆叶子"是很多树算法的基础动作：**文件目录的删除、语法树的剪枝 (pruning)、决策树的剪枝**，本质都是"识别一片叶子、拆掉它、保持剩余结构仍合法"。**"先找叶子，再动手"**——这是你在处理树结构时最该养成的第一反应。

第 3 轮：生成树——一棵树，怎么"藏"在一张连通图里

老谟："树讲完了。现在反过来问：**给一张连通的图，它里面'藏'不藏得下一棵树？**也就是说，能不能从图里删掉一些边，删到只剩下 $n - 1$ 条边，但还能把所有顶点连成一个整体？"

造物主：".....删边？不能把图删散了。那我要删到'刚刚够连'，就是一棵树？这样的一棵树，叫什么？"

老谟："叫**生成树 (spanning tree)**——它'生成'（跨越）了原图的全部顶点，是一棵连通的树。**问题：任何连通图，都一定藏得下一棵生成树吗？**你先直觉——怎么个删法，能保证删完还是连通的？"

造物主：".....我能不能这样：只要删了这条边图还连通，就删；删到不能删为止？"

老谟："你这个'只要不破坏连通就删'的思路，方向对了。可你得想清楚两件事：**第一，这样一直删，会不会把某条'必要'的边也删了导致断掉？第二，删到什么时候能停，停的时候为什么一定是一棵树？**这两个问题的答案，就是'连通图必有生成树'的证明。你下去把细节填一填，走廊定理 2 给了完整构造性证明。"

造物主：".....我大概摸到了：删的时候我**每次只保证不破坏连通**，那删完一定还连通；而'删到不能删'意味着——再删任何一条边都会不连通，也就是说每条剩下的边都是'必经之路'，没有一条是多余的，那就是一棵树？"

老谟："你摸到的是'无环'的直觉：**没有一条边是多余的，就等于没有环。**把'连通 + 无环'对上去，就是树。很好，走廊里你会看到它被写成一步不错的构造。记住这件事：**你不需要发明新的结构，你只需要从一张连通图里，把多余的边一条条拿掉。**"

知识锚点：生成树——删边不破连通

【概念深挖】**生成树 (spanning tree)**：一张连通图 $G = (V, E)$ 的生成树，是包含 G 的全部顶点 V 、且边数恰为 $|V| - 1$ 的树（连通且无环）。**结论：任何连通图都至少有一棵生成树。**

为什么想到"删边"：连通图里可能有"多余的环"，而树没有环。从"有很多环的连通图"走到"无环的连通树"，最自然的动作就是"删边"——只要删了还连通，这条边就不是必要的，可以拿掉。一直删到"再删一条就不连通"，就得到一棵生成树。**构造性证明在走廊定理 2。**

【工程实践】生成树是"用最省的连接覆盖全部点"的答案：**网络布线里，最少的光缆把全部机房连成互通的整体；供电网络的骨干拓扑；广播的"生成树协议 (STP)"**——都直接在找/维护一棵生成树。而"删到不能删"或"从最小边开始一棵棵挑"的思路，是"贪心"这类算法的雏形——你会看到，后面几章里"每次挑当前最有利的"这种贪心，会以不同面貌反复登场。

第 4 轮：BFS——"一层一层地摸"，和它的队列

老谟："树和生成树是'骨架'。现在回到你最开始的问题：**怎么保证走遍一张图不迷路。**我们给两种标准走法。第一种叫 **BFS (Breadth-First Search, 广度优先搜索)**。你先听这名字——'广度优先'，你觉得它想表达什么？"

造物主：".....广度？就是先把'近的'都摸完，再去摸'远一点的'？像水波一圈一圈往外扩？"

老谟："对，就是这个水波。**BFS 从起点出发，先摸到所有'走 1 步能到'的点，再摸所有'走 2 步能到'的，一层一层往外扩。**可这里有个工程问题：**你手里得有个东西，帮你记住'下一层该摸谁'。**你说说，用什么东西记住这个'待办队列'最合适？"

造物主：".....得有个'排队'的地方？我把当前这层摸到的点，按顺序排进一个队里，先排的先摸？"

老谟："对，这就是**队列 (queue)**——**先进先出 (FIFO)**。BFS 的规矩是：从队头取出一个点，把它的邻居里'还没摸过'的都塞进队尾；循环到队列空。你想想，这样为什么能保证'一层一层'？"

造物主："因为先入队的先被摸，而最早入队的就是起点那层.....起点入队，取出起点，它的邻居全入队；然后取队头，是起点的第一个邻居，它的邻居入队.....**好像确实先处理完'走 1 步'的，才轮到'走 2 步'的？**"

老谟："你直觉对了。**队列的 FIFO，正好让 BFS'逐层'的性质落地。**下一轮我们给 DFS，然后走廊里你会看到 BFS 那条'不漏'的证明，以及它和 Dijkstra 的姻缘。"

知识锚点：BFS 与队列——逐层扩展

【概念深挖】**BFS（广度优先搜索）**：从一个起点 s 出发，按“与 s 的距离”逐层扩展——先把所有距离 1 的点摸完，再摸距离 2 的……用**队列**（先进先出 FIFO）维护“待处理”的顶点：

1. s 入队；
2. 从队头取一个顶点 v ，把 v 的所有**尚未访问过**的邻居依次入队尾；
3. 重复直到队列空。**关键性质**：BFS 第一次“摸到”某个顶点 u 时，走过的层数恰是 s 到 u 的**最短距离**（最少边数）。这就是下一章 Dijkstra 的直觉地基。

为什么想到“队列”：要保证“先摸近的”，就得让“先被发现的点”先被处理——**队列的先进先出恰好维持了这个顺序**。换栈（后进先出）就会变成 DFS（一条道走到底），这就是下一轮要讲的区别。

【工程实践】BFS 是**“找最短/最邻近”类问题的标配：社交网络的“几度人脉”、地图的“就近搜索”、网络爬虫的“逐层抓取”、八皇后/迷宫的最少步数求解。“逐层外扩”这个直觉，让 BFS 成为一切“最短路”想法的起点。**

第 5 轮：DFS——“一条道走到黑”，和它的栈/递归

老谏：“第二种走法叫 **DFS（Depth-First Search，深度优先搜索）**。‘深度优先’，你猜它跟 BFS 的区别在哪？”

造物主：“……深度？那就是先往一个方向一条道走到黑，走不动了再回头，换一条岔路继续走？”

老谏：“对。DFS 是‘一条道走到黑，撞了墙再回头（回溯）’。它的数据结构，跟 BFS 的队列正好相反——用**栈（stack，后进先出 LIFO）**。你说说，为什么栈适合‘深度优先’？”

造物主：“……因为栈是‘后进先出’？我走到底发现走不通，得先回到**最近**的分岔口去试另一条路——那正好是‘最后进去的分岔’，用栈刚好取回来？”

老谏：“你答到点子上了。DFS 在岔路口‘一头扎进去’，把回退的路记在栈里（或靠递归自动回溯）；走到底发现没路，就弹出栈顶，回退到上一个岔口，再试没走过的岔路。和 BFS 不同，DFS 常常不是按‘距离’走的，而是按‘一条路走到底再回头’的探险式走法。”

造物主：“那 DFS 和图论的哪些东西搭？我记得你上一章讲过‘连通分量’——DFS 是不是很适合‘数一张图分成几块’？”

老谏：“你又答对了。DFS 一趟下去，恰好能把一个连通分量完整摸完——所以你只要对每个‘还没摸过’的顶点再启动一次 DFS，数一数启动了几次，就是连通分量的个数。这正好回扣你第 1 章学的**连通分量**。BFS/DFS 这对兄弟，一个按层摸、一个按路摸，是后面几乎所有图算法的两块砖。走廊里你会看到它们的标准伪代码，以及 BFS 那条‘不漏’的证明。”

知识锚点：DFS 与栈/递归——回溯探险

【概念深挖】DFS（深度优先搜索）：从一个顶点出发，沿一条边一路深入，直到走不动，再回溯到最近的分岔口尝试未走过的路。可用栈显式实现，或用递归（函数调用栈天然承担回溯）。DFS 一趟恰好访问完一个连通分量——因此“对每个未访问顶点启动 DFS，启动次数 = 连通分量个数”是 DFS 的一大用途。

为什么想到“栈/递归”：DFS 的“深入”和“回溯”本质是“先进后出”——你最后进入的分岔口，最先被回退尝试，这正是栈（或递归调用栈）的语义。*“一头扎进去，走不动再退回来试别的岔路”是 DFS 的核心直觉。

【工程实践】DFS 与“迷宫/搜索/回溯”天然契合：迷宫的走迷宫算法、图的拓扑排序、找出“环”、回溯搜索（八皇后、数独）、递归的目录遍历（列出所有文件）。DFS 用递归写特别自然，但要注意深图里递归可能爆栈——这是工程上从递归改用显式栈的原因之一。

第 6 轮：树的遍历序——先序/后序，和“排队”的不同口味

老谟：“BFS 和 DFS 是‘两张通用地图上的走法’。现在把它们用在树上，会多出一层味道：遍历的顺序，本身成了信息。比如一棵树的 DFS，先访问自己、再深入孩子，和先深入孩子、最后才访问自己，是两种不同的顺序——前者叫先序（preorder），后者叫后序（postorder）。你说说，这两种顺序分别有什么用？”

造物主：“……先访问自己再访问孩子，像是在‘进入’一个节点时先记录它？后序先访问孩子，像是在‘离开’一个节点时才处理它？那……一个适合‘进来就干活’，一个适合‘把孩子的活都干完再收尾’？”

老谟：“你总结得非常准。先序适合‘从上往下’的结构性输出（比如打印目录结构）；后序适合‘自底向上’的归并（比如先算孩子的值，再汇总给父节点）。你看，同样是 DFS，换个访问的时机，就得到不同用途的遍历。树的遍历序，是你在后面（尤其解析树、表达式求值）会反复用到的小宝藏。”

造物主：“……那 BFS 用在树上，就是按层扫一遍，像‘逐层打印目录’那样？”

老谟：“对，树的 BFS 就是层序遍历（level-order）。你看，你手里现在有 BFS、DFS、先序、后序、层序——‘走法’本身，已经是一套可以精确指定的语言了。下一轮我们把这一章收一收，再预告下一章。”

知识锚点：树的遍历序——访问时机的选择

【概念深挖】对树做 DFS，按"访问自己"的时机可分成：

- **先序 (preorder)**：访问自己 → 递归访问孩子。适合"自顶向下"的构建/输出（如打印目录结构、复制树）。
- **后序 (postorder)**：递归访问孩子 → 访问自己。适合"自底向上"的归并（如先算子树结果再汇总给父节点）。
- **（中序 (inorder) 是二叉树的特例：先左子树 → 自己 → 右子树，本册到二叉树再展开。）** 树的 **BFS 叫层序遍历 (level-order)**：按层从上到下、每层从左到右访问。

为什么想到"访问时机"：同样的 DFS 骨架，"在深入之前访问"和"在深入之后访问"是两种不同的活法——一个"先登记再走"，一个"走完再登记"。顺序本身携带语义，这正是"遍历序"能当工具用的原因。

【工程实践】遍历序在工程里直接对应具体场景：**目录树用先序打印**（先父后子）、**文件系统算文件夹总大小时用后序**（先累计子项再汇总）、**语法树/表达式求值用后序或中序**（先算子表达式）。选哪种序，取决于你的数据是"要自上而下构建"还是"要自下而上汇总"。

第 7 轮：落点——"保证不漏"靠证明，算法思维正式入场

老谟："绕了一圈。你最初问'怎么保证走遍不迷路'，现在怎么答？"

造物主：".....我不能靠'随便走'，我要靠**算法**——BFS 逐层摸、DFS 沿路摸，并且我要**证明**它们'一个不漏'。而树是'最省又不断'的骨架， n 个点 $n-1$ 条边，任何连通图都藏得下一棵生成树。"

老谟："你手里多了几件兵器？"

造物主："树（连通无环 / 最省连通）、 **n 顶点树 $n-1$ 条边**（归纳法证的）、**生成树**（删边不破连通）、**BFS**（队列、逐层）、**DFS**（栈/递归、回溯）、**树的遍历序**（先序/后序/层序）。"

老谟："够了。再给你一句话收尾：**你今天做的，是把'怎么走'从'经验'变成了'算法'，又把'算法'钉成了'可证明'的东西。**这条路——从直觉到算法再到证明——就是你后面所有章节要走的路：最短路径、着色、可计算性，全在重复这一遍。而 BFS 那个'逐层最短'的性质，正是下一章 Dijkstra 的引子。"

造物主：".....我隐约觉得，BFS 那个'第一次摸到的就是最短距离'，不是巧合，是能被证明的。"

老谟："那你去走廊，把它亲手证一遍。"

严格化走廊

位置说明：对话负责"直觉"——树为什么最省、为什么用队列做 BFS、为什么用栈做 DFS。本走廊负责"严谨"——给出形式定义、可复写的证明与算法伪代码。本章有**三座证明**：**定理 1 (n 顶点树 n-1 边, 数学归纳法)**、**定理 2 (连通图必有生成树, 构造法)**、**定理 3 (BFS 不漏且给出最短距离, 直接法)**。每步标"证明类型 + 依据", 绝不用"显然/易证/显见" (包括反话)。

定义：树、生成树、遍历序

- **树 (tree)**：连通且无回路的图。
- **生成树 (spanning tree)**：连通图 $G = (V, E)$ 的生成树是包含全部顶点 V 、且边数为 $|V| - 1$ 的树。
- **叶 (leaf)**：度为 1 的顶点。
- **BFS/DFS**：两种图遍历算法 (伪代码见下文)；BFS 用队列 (FIFO)，DFS 用栈 (LIFO) / 递归。
- **先序/后序/层序**：树的三种遍历序 (见知识锚点)。

证明类型标注：本章的证明方法分三类——**定理 1 用"数学归纳法"** (从小的树成立，推大的树成立)，**定理 2 用"构造法"** (直接构造出一棵生成树，再验证它确实是树)，**定理 3 用"反证 + 归纳"** ((a)"不漏"用反证法，(b)"最短距离"用归纳法)。**数学归纳法是你第一次正式登场**——它是"拆叶子降规模"那种直觉的严格化。

关于对话：本章第 2 轮 (拆叶子)、第 3 轮 (删边)、第 4 轮 (队列逐层)、第 5 轮 (栈回溯) 都做了"引导式推导"。**正式的可复写证明与算法全部在本走廊**。对话负责"为什么会想到这步"，走廊负责"怎么把它写标准"——与前几章口径完全一致。

定理 1 (n 个顶点的树恰有 n-1 条边)：若 T 是 n 个顶点的树，则 T 有 $n - 1$ 条边

证明类型：数学归纳法 (对顶点数 n 归纳)。

证明：

- 第 1 步 (**归纳起点**)：当 $n = 1$ 时，一棵只有一个顶点的树，边数为 $0 = 1 - 1$ 。命题成立。
(依据：树连通且无环；单个顶点无边的图既连通 (只有它一个点) 又无环，边数为 0，且 $n - 1 = 0$ 。)
- 第 2 步 (**归纳假设**)：假设对任意 $k \geq 1$ 个顶点的树，边数均为 $k - 1$ 。(依据：归纳法的假设——假设小的规模成立。)

- 第 3 步 (**归纳递推**)：取一棵 n 个顶点的树 T ($n \geq 2$)。先断言 T 存在一片叶子 v (度 = 1 的顶点)。用"最长路径端点必为叶子"证之：
 - 3a：取 T 中一条**最长的简单路径** $P = v_1 v_2 \cdots v_k$ ($k \geq 2$)。(依据： T 连通且 $n \geq 2$ ，故至少存在一条长度 ≥ 1 的简单路径；从中取最长的一条，最长一定存在，因为顶点数有限。)
 - 3b：断言端点 v_1 的度只能是 1。若 $d(v_1) \geq 2$ ，则 v_1 有异于 v_2 的邻居 w 。若 $w \notin P$ ，则 $w v_1 v_2 \cdots v_k$ 是**更长**的简单路径，与" P 最长"矛盾；若 $w \in P$ (设 $w = v_j, j \geq 3$)，则 $v_1 v_2 \cdots v_j v_1$ 构成一个**回路**，与"树无环"矛盾。(依据：度 ≥ 2 的定义；最长路径定义；树无环定义。)
 - 3c：两种情形都矛盾，故 $d(v_1) = 1$ ， v_1 是叶子。(依据：反证法——"度 ≥ 2 "的所有可能都被排除。)(以上依据：最长路径定义 + 树无环 + 矛盾律。注：此处不依赖"连通图边数上界"——那是循环论证的陷阱；最长路径论证直接且无循环。)
- 第 4 步：拆掉叶子 v 和它唯一关联的边 e ，得到图 $T' = T - \{v\}$ 。**断言 T' 仍是树**： T' 去掉一个孤立的叶子不影响其他点的连通 (原 T 中 v 只连一条边，删除它不切断任何两点的路径)，且无环性质不变 (删边不会制造环)，故 T' 连通且无环，是一棵树。(依据：叶子度为 1 的性质 + 连通/无环定义 + 删边不破坏连通与无环。)
- 第 5 步： T' 有 $n - 1$ 个顶点，由归纳假设， T' 有 $(n - 1) - 1 = n - 2$ 条边。(依据：归纳假设代入 $k = n - 1$ 。)
- 第 6 步： T 比 T' 多一个顶点 v 和一条边 e ，故 T 的边数 $= (n - 2) + 1 = n - 1$ 。(依据：第 4 步"只拆了 v 和 e "，边数差恰好 1。)
- 第 7 步：由归纳法 (起点 $n = 1$ 成立 + 递推 $n - 1$ 成立 $\Rightarrow n$ 成立)，**对所有 $n \geq 1$ ， n 顶点树恰有 $n - 1$ 条边。** ■

依据标注：步 1 归纳起点 ($n=1$ 直接验)；步 2 归纳假设；步 3 是关键**存在性**——证"树必有叶子" (用"最长路径端点必为叶子"：端点若有第二邻居，路径要么更长、要么成环，均矛盾)；步 4 拆叶子后 T' 仍是树 (叶子无关痛痒)；步 5 套归纳假设；步 6 数边；步 7 归纳收束。**读法：**归纳法的灵魂是"降规模"——找到一个"拆掉后仍是同类问题且更小"的动作 (拆叶子)，把大问题规约到小问题。**步 3 的"必有叶子"是整个递推能启动的前提，别跳过它。**

定理 2 (连通图必有生成树)：任意连通图 $G = (V, E)$ 都含一棵生成树

证明类型：构造法 (从 G 出发，反复删除"删后仍连通"的边，直至不能再删)。

证明：

- 第 1 步 (**构造起点**)：令 $H = G$ (原连通图)。 H 连通。(依据： G 连通，初值 $H = G$ 。)

- 第 2 步 (**循环删边**)：只要 H 中存在一条边 e ，使得 $H - e$ (删掉 e 后) **仍然连通**，就删掉这条边。重复此步。(依据：连通性的定义——“删掉这条边仍连通”意味着这条边不是“必经之路”，可安全删除。)
- 第 3 步 (**终止性**)： H 的边数有限 ($|E|$ 有限)，每删一条边边数减 1，故上述删边过程必然在有限步内终止。(依据：有限集合的单调递减必终止。)
- 第 4 步 (**终止时的性质**)：设删边停止时得到 H' 。此时 H' 中**不存在**“删了仍连通”的边，即 H' 的每条边都是“必经之路”——删掉任意一条都会使 H' 不连通。**断言 H' 连通**：删除边不会让本来连通的图“从断开变连通”，故从连通的 G 出发删边， H' 仍连通。(依据：删边只能减少连接、不能增加连接。)
- 第 5 步 (**断言 H' 无环**)：若 H' 含一个环 C ，则删掉环 C 上任意一条边 e ，由于环上除 e 外的其余边仍连通 C 上各点， $H' - e$ 仍连通——这与第 4 步“每条边都是必经之路”矛盾。故 H' 无环。(依据：反证法——环上删一边不破坏连通；矛盾的来源是第 4 步。)
- 第 6 步： H' 连通 (第 4 步) 且无环 (第 5 步)，故 H' 是一棵树；又 H' 由 G 删边得到，顶点集仍是 V ，故 H' 是 G 的**生成树**。■

依据标注：步 1 初值连通；步 2 定义“安全删边”；步 3 用有限性保证终止；步 4 用“删边不增加连通”保证终态连通；步 5 用反证法（环上删边不破坏连通）保证终态无环；步 6 由“连通+无环=树”下结论。**读法**：这是“构造法”的典范——**你不证‘存在’用抽象推理，而是真的动手‘删’出一棵树来，再回头验证删完的确实是树**。步 5 那个“环上删一边仍连通”是关键洞察：环是“多余的边”，正是要删掉的。

定理 3 (BFS 不漏，且给出最短距离)：对连通图 G 从起点 s 做 BFS，则 (a) 每个顶点都会被访问到；(b) 顶点 u 首次被访问时走过的层数恰为 s 到 u 的最短距离 $\delta(s, u)$

证明类型：反证 + 归纳 ((a)“不漏”用反证法——路径上找第一个未访问点；(b)“最短距离”用归纳法——沿 BFS 逐层过程归纳)。本定理 (b) 同时为下一章 Dijkstra 埋下性质。

证明：

- 第 1 步 (**(a) 不漏**)：反证。假设存在某顶点 u 从未被 BFS 访问。因 G 连通， s 到 u 之间存在一条路径 $s = v_0, v_1, \dots, v_k = u$ 。沿这条路径找**第一个未被访问**的顶点 v_j ($j \geq 1$ ，因为 s 初始即入队被访问)。则 v_{j-1} 已被访问。(依据：连通定义给出路径； v_j 是“第一个未访问”意味着 v_{j-1} 必已访问。)
- 第 2 步：BFS 每次取出一个已访问顶点时，会把它所有“尚未访问”的邻居入队并标记访问。 v_{j-1} 被取出 (或被访问) 时，其邻居 v_j 尚未访问，故 v_j 必被入队标记为已访问——与“ v_j 从未被访问”矛盾。(依据：BFS 的邻居入队规则 + 矛盾律。)

- 第 3 步：故假设不成立，**每个顶点都被 BFS 访问**，即“不漏”。■（本步证毕 (a)。）
- 第 4 步（**(b) 最短距离**，归纳）：用归纳证明：**在 BFS 中，顶点 u 首次入队（即首次被访问）时，其“层号” $L(u)$ 恰等于 $\delta(s, u)$** 。归纳起点：起点 s 首次入队时 $L(s) = 0 = \delta(s, s)$ 。（依据：BFS 起点层号 0； $\delta(s, s) = 0$ 。）
- 第 5 步（**归纳递推**）：假设对“层号 $\leq L$ ”的顶点，层号即最短距离。取首次入队时层号为 $L + 1$ 的顶点 u ，它必是被某个层号 L 的顶点 v 发现并入队的（BFS 逐层规则）。（依据：BFS 中“下一层顶点由上一层顶点发现”。）
- 第 6 步：一方面 $\delta(s, u) \leq \delta(s, v) + 1 = L + 1$ （由 v 到 u 有一条边，最短距离满足三角不等式：从 s 到 v 走 $\delta(s, v)$ ，再走一条边到 u ）。（依据：最短距离的三角不等式 $\delta(s, u) \leq \delta(s, v) + 1$ 。）
- 第 7 步：另一方面，若 $\delta(s, u) < L + 1$ ，即 $\delta(s, u) \leq L$ ，则 u 会与“层号 $\leq L$ ”的顶点一样在更早的层被 BFS 发现，其首次层号 $L(u) \leq \delta(s, u) \leq L$ ，与“ u 首次层号是 $L + 1$ ”矛盾。故 $\delta(s, u) \geq L + 1$ 。（依据：反证法；其中“若 u 距 s 为 d ，则 BFS 在不超过 d 的层必发现 u ”由 **BFS 逐层扩展的定义直接得出**——第 i 层恰是“与 s 距离为 i 的所有顶点”（距离为 d 的点在第 d 层被发现），这是定义给出的结论，不依赖本定理的结论，故不构成循环依赖。）
- 第 8 步：综合第 6、7 步： $L + 1 \leq \delta(s, u) \leq L + 1$ ，故 $\delta(s, u) = L + 1 = L(u)$ 。由归纳法，(b) 对所有顶点成立。■

依据标注：步 1-3 证 (a) 用反证+路径上“第一个未访问”；步 4 归纳起点；步 5 用 BFS 逐层发现规则；步 6 用最短距离三角不等式得上界；步 7 用反证得下界；步 8 夹逼得等号。**读法：**(a) 的直觉是“连通图里没有 BFS 够不着的死角”——因为只要有路，沿路的第一个死角就会被它的前一个点发现。(b) 的直觉是“BFS 一层层扩，第一次摸到 u 时，没有更短的路能绕过这一层”——**这正是下一章 Dijkstra ‘贪心’合法性的种子：**逐层最短，本身就是贪心能成功的理由。

算法伪代码：BFS 与 DFS（可复写卡片）

BFS(s):

```
Q 初始为空队列;
标记  $s$  已访问,  $s$  入队 Q;
while Q 非空:
    取出队头  $v$ ;
    对  $v$  的每个邻居  $u$ :
        若  $u$  未访问:
            标记  $u$  已访问,  $u$  入队;
```

DFS(s):

```
S 初始为空栈;
标记  $s$  已访问,  $s$  入栈 S;
while S 非空:
    弹出栈顶  $v$ ;
    对  $v$  的每个邻居  $u$ :
        若  $u$  未访问:
            标记  $u$  已访问,  $u$  入栈;
```

读法：两者骨架几乎一样——区别只在“排队”用队列（FIFO）还是栈（LIFO）。队列→BFS（逐层），栈→DFS（沿路）。“标记已访问”这一步绝不能省：没有它，图有环时 BFS/DFS 会无限循环（在环里转圈），有了它才能保证“每个点至多入队/入栈一次”，从而“不漏”且能终止。

反例/边界：树与遍历的适用边界

1. “ **n 个顶点 $n-1$ 条边**”的前提是**连通**：若有 $n-1$ 条边但不连通，那它**不是树**（可能分成几块）。定理 1 说的是“树 $\Rightarrow n-1$ 条边”，反过来“ $n-1$ 条边且连通 $\Rightarrow$ 树”也是对的（无环由连通+边数推出），但**单独“ $n-1$ 条边”不连通就不能叫树**。
2. 树的“**无环 + 连通**”两条件缺一不可：连通但有多余环 $\rightarrow$ 不是树（是带环连通图）；无环但不连通 $\rightarrow$ 是“森林”（好几棵树的并），不是单棵树。
3. **BFS 求的是“边数最少”的最短路，不是“带权”最短路**：BFS 假设每条边“代价相同”（都是 1）。一旦边有不同权值（长度、费用），BFS 逐层最短就不适用，这正是第 3 章 Dijkstra 要解决的新问题。
4. **DFS 递归在深图上可能爆栈**：DFS 用递归实现时，若图很深（路径很长），递归调用栈可能耗尽内存。工程上长图常改用**显式栈**（把“回溯信息”显式压栈）来避免。
5. **生成树不是唯一的**：同一张连通图的生成树通常有很多棵（不同删边顺序会得到不同生成树）。“最小生成树”（在所有生成树中总权值最小的那棵）是一个重要的优化问题——它和“贪心选边”的思路关系密切，具体的贪心算法会在合适的章节讲到，这里先记住概念。

知识点总结

知识点（本章推出的核心概念）

- **树 (tree)**：连通且无环的图；等价于“连通且边数 = 顶点数 - 1”；任意两点间路径唯一。
- **n 个顶点 $n-1$ 条边**：数学归纳法证明（拆叶子降规模）。
- **生成树 (spanning tree)**：含全部顶点、边数为 $n-1$ 的树；任何连通图都有生成树（构造法：删边不破连通）。
- **BFS (广度优先)**：队列 FIFO，逐层扩展；首次访问即最短距离（边数最少）。
- **DFS (深度优先)**：栈/递归，一条道走到黑再回溯；一趟走完一个连通分量。
- **树的遍历序**：先序（先自己后孩子）、后序（先孩子后自己）、层序（BFS 按层）。

历史结论（本章涉及的史料）

- **凯莱 (Cayley, 19 世纪)**：证明了 n 个标号顶点有 n^{n-2} 棵树（凯莱公式），树的计数研究的经典起点。
- **BFS/DFS**：作为“图遍历”的标准做法，是算法中自然长成的结构，**不硬扣个人发明人名**——它们是 20 世纪图算法成熟过程中的公共遗产。

工程实践结论（本章知识在真实工程里怎么用）

- **树结构**：文件目录树、组织架构、语法解析树、B 树索引——“连接最省、路径唯一”正适合搜索。
- **生成树**：网络布线的骨干拓扑、广播的 STP（生成树协议）、供电骨干网——用最省连接覆盖全部点。
- **BFS**：社交“几度人脉”、迷宫最少步数、就近搜索、网络爬虫逐层抓取。
- **DFS**：迷宫、拓扑排序、找环、回溯搜索（八皇后/数独）、递归目录遍历。
- **树的遍历序**：先序打印目录、后序算文件夹总大小（先子后父汇总）、后序/中序算表达式树。

计算训练场

位置说明：对话负责“为什么”，这里负责“怎么算”。每题动笔 10 分钟以上，先自己写再对答案。**难度递进：易→中→难。**

题 1（易→中）：树数边 + 生成树手算验证

题干：老谟给了造物主一棵有 9 个顶点、8 条边的树，让他验证“ n 个顶点 $n-1$ 条边”，并指出这棵树里“叶子”的个数满足握手引理。(a) 验证 9 顶点 8 条边满足 $n-1$ 。(b) 若这棵树所有顶点的度分别是 $d_1, \dots, d_9$ ，验证 $\sum d_i = 2 \times 8 = 16$ ，并说明为什么这棵树至少有两片叶子。

完整分步解答：

- (a) 树有 9 个顶点， $n - 1 = 9 - 1 = 8$ ，边数为 8 ✓ 满足定理 1。
- (b) 第 1 步（数度）：由握手引理， $\sum_{i=1}^9 d_i = 2|E| = 2 \times 8 = 16$ 。（依据：握手引理。）
- (b) 第 2 步（至少两片叶子）：反证。假设树至多 1 片叶子，则 9 个顶点里至少有 8 个顶点度 ≥ 2 ，另有至多 1 个顶点度 ≥ 1 （叶子度恰为 1）。于是 $\sum d_i \geq 8 \times 2 + 1 \times 1 = 17 > 16$ ，与握手引理 $\sum d_i = 16$ 矛盾。故树至少有两片叶子。（依据：反证法 + 握手引理 + 度 ≥ 2 的顶点贡献至少 2。）
- **答**：(a) 9 顶点 8 条边，满足 $n-1$ ；(b) $\sum d_i = 16 = 2 \times 8$ ✓；由握手引理反向逼出“至少两片叶子”（若叶子少于 2 则度数和必超 16）。

常见错误：① 以为"叶子个数没限制"——实际由握手引理可逼出"树至少两片叶子"；② 反证时把"至多 1 片叶子"当"只有 1 个度 1 的顶点"，漏了"其余点度 ≥ 2 "的贡献下限；③ 忘了 $\sum d_i = 2|E|$ 里边数是 8 不是 9。

题 2 (中)：对给定图完整执行 BFS——写出访问顺序、队列轨迹与"树边"

题干：给定图 G ：顶点集 $\{a, b, c, d, e, f\}$ ，边集 $\{ab, ac, bd, be, ce, cf, df\}$ 。老谟让造物主从起点 a 做 BFS，写出：(a) 顶点访问顺序；(b) 队列在每一步的状态（轨迹）；(c) BFS 树边（每个非起点顶点首次被发现的边）。

完整分步解答：

- (a/b) 逐层模拟：
 - 初始： a 入队。队列 $[a]$ 。访问 a 。
 - 取队头 a ，邻居 b, c 未访问，入队。队列 $[b, c]$ 。访问顺序 a 。
 - 取队头 b ，邻居 a （已访问）、 d, e （未访问）入队。队列 $[c, d, e]$ 。访问顺序 a, b 。
 - 取队头 c ，邻居 a （已访问）、 e （已访问）、 f （未访问）入队。队列 $[d, e, f]$ 。访问顺序 a, b, c 。
 - 取队头 d ，邻居 b （已访问）、 f （已访问）。无新顶点。队列 $[e, f]$ 。访问顺序 a, b, c, d 。
 - 取队头 e ，邻居 b, c 均已访问。队列 $[f]$ 。访问顺序 a, b, c, d, e 。
 - 取队头 f ，邻居 c, d 均已访问。队列 $[\]$ 。访问顺序 a, b, c, d, e, f 。
 - 队列空，结束。
- (c) BFS 树边（首次发现每个非起点顶点的边）： a 是起点； b 由 a 发现（边 ab ）； c 由 a 发现（边 ac ）； d 由 b 发现（边 bd ）； e 由 b 发现（边 be ）； f 由 c 发现（边 cf ）。树边： $\{ab, ac, bd, be, cf\}$ ，共 5 条 = 6 顶点 - 1 $\checkmark$ （符合树边数）。
- 答：(a) 访问顺序 a, b, c, d, e, f （正是逐层： $a \rightarrow b, c \rightarrow d, e, f$ ）；(b) 队列轨迹如上 $[a] \rightarrow [b, c] \rightarrow [c, d, e] \rightarrow [d, e, f] \rightarrow [e, f] \rightarrow [f] \rightarrow [\]$ ；(c) 树边 $\{ab, ac, bd, be, cf\}$ ，5 条 $\checkmark$ 。

常见错误：① 队列是"先进先出"，忘了把"同一层多个邻居"按序排队，导致先取后入的；② 把"已访问"的点再次入队（会死循环），必须标记；③ 树边数忘了应为顶点数 - 1（这里 5 条），可据此自检。**反向验证：**树边 5 条 = 6 - 1，且 $\{ab, ac, bd, be, cf\}$ 恰好覆盖全部 6 个点，验证 BFS 树是生成树 $\checkmark$ 。

题 3 (难→中)：对给定图完整执行 DFS + 用 DFS 数连通分量

题干：老谟给了造物主一张图，顶点 $\{a, b, c, d, e\}$ ，边集 $\{ab, ac, bd, ce\}$ ，又单独一个顶点 f （孤立）。要求：(a) 从 a 出发做 DFS（用栈），写出访问顺序与回溯过程。(b) 用“对每个未访问顶点启动 DFS”的方法，数出这张图的连通分量个数。(c) 验证 (b) 的结论符合直觉（ f 是孤立点）。

完整分步解答：

- (a) DFS 模拟（栈，后进先出，邻居按字母序进栈）：
 - 初始： a 入栈，访问 a 。栈 $[a]$ 。
 - 弹出 a ，邻居 b, c 未访问，入栈（按 b, c 顺序， c 后进先出）。栈 $[b, c]$ 。访问顺序 a 。
 - 弹出 c ，邻居 a （已访问）、 e （未访问）入栈。栈 $[b, e]$ 。访问顺序 a, c 。
 - 弹出 e ，邻居 c （已访问）。无新点。栈 $[b]$ 。访问顺序 a, c, e 。
 - 弹出 b ，邻居 a （已访问）、 d （未访问）入栈。栈 $[d]$ 。访问顺序 a, c, e, b 。
 - 弹出 d ，邻居 b （已访问）。栈 $[\]$ 。访问顺序 a, c, e, b, d 。
 - 栈空，DFS 结束。
- (a) **访问顺序：** a, c, e, b, d （注意：DFS 沿路深入——从 a 先去 c 再一路到 e ，回溯到 b 再深入 d ，与 BFS 的逐层不同）。
- (b) 数连通分量：从 a 的 DFS 访问了 a, c, e, b, d 五个点（它们互相连通，是一个连通分量）。还剩顶点 f 未访问，对 f 再启动一次 DFS，访问 f （孤立点）。**共启动 2 次 DFS，连通分量个数 = 2。**（依据：DFS 一趟走完一个连通分量。）
- (c) 直觉验证： $\{a, b, c, d, e\}$ 通过边 ab, ac, bd, ce 连成一个整体（ $a-b$ 、 $a-c$ 、 $b-d$ 、 $c-e$ ，全可达）； f 孤立成一个点。所以**正好 2 个连通分量**✓。
- **答：**(a) DFS 访问顺序 a, c, e, b, d ；(b) 连通分量个数 = 2；(c) 五个点连成一块 + f 孤立，共 2 块✓。

常见错误：① DFS 用栈时，邻居入栈顺序影响访问顺序（后进的先出）——这里让 c 后入栈先弹出，故先走 c 分支；② 忘了对“每个未访问顶点”启动 DFS 来数连通分量（ f 孤立也必须单独启动一次）；③ 把 DFS 访问顺序误当 BFS 逐层顺序（DFS 是沿路深入，顺序 a, c, e, b, d 而非逐层 a, b, c, d, e ）。

反向验证：连通分量内部必连通，这里 $a \rightarrow b \rightarrow d$ 、 $a \rightarrow c \rightarrow e$ 全可达， f 无任何边，块数 2 正确。

向导便签

这一章咱们干了什么：把你"怎么走遍不迷路"的担心，变成了一套可证明的算法。你认识了**树**（连通无环， n 个顶点 $n-1$ 条边，用归纳法亲手证的），明白了**生成树**（删边不破连通，任何连通图都藏得下一棵），掌握了**BFS**（队列逐层、首次访问即最短距离）和 **DFS**（栈/递归回溯、一趟走完一个连通分量）。

钉死一句话：**"保证不漏"这件事，光靠"认真走"是不够的，必须靠证明。** 树是"最省又不断"的骨架，BFS/DFS 是"保证不漏"的两种走法——它们共同的起点，是"标记已访问 + 用对数据结构（队列/栈）"。

记住：**BFS 的"逐层最短"不是巧合，是能被证明的性质**——而正是这个性质，是下一章 Dijkstra 敢用"贪心"的底气。你已经从"看图"走到"算图"，又从"算图"走向"证明算法正确"了。

造物主手记

我曾踩的坑：

我以为"树就是长得像树杈的图"，被老谟逼着才意识到**树是'最省又不断'的结构**， n 个顶点 $n-1$ 条边不是背出来的，是能归纳法证的。画 BFS 时，我差点在队列里把"同一层"的顺序搞乱——忘了先进先出，导致先取了后入队的点。DFS 我一开始更晕：同样是走，为什么顺序跟 BFS 差那么多？老谟点破——**栈是后进先出，邻居入栈顺序会影响先走哪个分支**。最险的是数连通分量：我漏了 f 那个孤立点，差点只启动一次 DFS 就下结论"1 块"——**孤立点也得单独启动一次 DFS**。还有画生成树时，我差点删边删得图不连通了，被提醒"删之前先确认删了还连通"。

顿悟时刻：

真正开窍是"拆叶子"那个动作——**找一片叶子拆掉，剩下的还是树，规模还变小**，这一下就把" n 顶点 $n-1$ 边"从"要背的结论"变成了"能归纳法证的定理"。我忽然懂了归纳法的本质：**找到一个'拆掉后还是同类问题且更小'的动作，就能一环扣一环地推到底**。而 BFS 那个"逐层最短"的证明，让我第一次看见"算法为什么对"——**不是靠跑很多例子，是靠一层层推：下一层必由上一层发现，且没有更短的路能绕过这一层**。这让我对"算法正确性"有了实感，也对下一章 Dijkstra 的"贪心"充满了好奇。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A（计算，易→中）：树数边 + 叶子下界

一棵树有 7 个顶点。(a) 它有多少条边？(b) 用握手引理说明这棵树至少有几片叶子？

完整解答： (a) 树有 $n - 1 = 6$ 条边。(b) $\sum d_i = 2|E| = 12$ 。若叶子少于 2 (即至多 1 片叶子)，则至少有 6 个顶点度 ≥ 2 ，加至多 1 个度 ≥ 1 ， $\sum d_i \geq 6 \times 2 + 1 = 13 > 12$ ，矛盾。故至少两片叶子。

题 B (计算, 中)：对图执行 BFS 并验证树边数

图 G ：顶点 $\{p, q, r, s\}$ ，边集 $\{pq, pr, qs, rs\}$ 。从 p 做 BFS，写出访问顺序、队列轨迹和 BFS 树边。

完整解答：

- 初始队列 $[p]$ ，访问 p 。
- 取 p ，邻居 q, r 入队： $[q, r]$ 。访问 p, q 。
- 取 q ，邻居 p (已访问)、 s 入队： $[r, s]$ 。访问 p, q, r 。
- 取 r ，邻居 p (已访问)、 s (已访问)。 $[s]$ 。访问 p, q, r, s 。
- 取 s ，邻居 q, r 均已访问。 $[\]$ 。结束。
- 访问顺序： p, q, r, s 。BFS 树边： pq (q 由 p)、 pr (r 由 p)、 qs (s 由 q)，共 3 条 = $4 - 1 \checkmark$ 是生成树。

题 C (计算, 中一难)：DFS 访问顺序 + 连通分量

图 G ：顶点 $\{a, b, c, d, e, f\}$ ，边集 $\{ab, ac, bd, ce\}$ ， f 孤立。从 a 做 DFS (邻居按字母序进栈)，写出访问顺序；再用 DFS 数连通分量个数。

完整解答：

- DFS：栈 $[a]$ ，访问 a ；弹出 a ， b, c 入栈 (c 后进先出) $[b, c]$ ；弹出 c ， e 入栈 $[b, e]$ ，访问 c ；弹出 e (邻居 c 已访问) $[b]$ ，访问 e ；弹出 b ， d 入栈 $[d]$ ，访问 b ；弹出 d (邻居 b 已访问) $[\]$ ，访问 d 。顺序： a, c, e, b, d 。
- 剩 f 未访问，再启动一次 DFS，访问 f 。连通分量个数 = 2 ($\{a, b, c, d, e\}$ 一块 + $\{f\}$ 一块)。

题 D (概念, 检查理解)

(a) 树为什么"恰好 $n-1$ 条边"？归纳法的"降规模"动作是什么？(b) BFS 用什么数据结构？为什么它能保证"逐层"？首次访问一个顶点意味着什么？(c) DFS 用什么数据结构？一趟 DFS 能完成什么 (与连通分量有关)？

完整解答： (a) 因为树必有叶子，拆掉一片叶子和它的边，剩下的还是树且规模变小 (顶点、边各减 1)，用归纳法从 $n=1$ 一路推上去，边数 = 顶点数 - 1。"降规模"动作就是"拆叶子"。(b) 用队列

(FIFO)。先进先出保证先入队（距离更近）的点先被处理，从而逐层扩展。BFS 首次访问一个顶点时，走过的层数恰是它到起点的最短距离（边数最少）。(c) 用栈（LIFO）或递归。后进先出保证“一条道走到黑再回溯”。DFS 一趟恰好访问完一个连通分量，因此“对每个未访问顶点启动 DFS，启动次数 = 连通分量个数”。

预告

你终于把“怎么走遍一张图”从“看心情走”变成了“能证明的算法”——BFS 逐层摸、DFS 沿路摸，还用归纳法钉死了“树 n 个顶点 $n-1$ 条边”、用构造法证明了“任何连通图都藏得下一棵生成树”。

可你现在手里的 BFS，有个“看不见的短板”：**它假设每条边都一样重（走一步算一步）**。可现实里，路不是一样长的——导航里有的路 3 公里，有的路 20 公里。**BFS 的“逐层最短”一旦遇上“边有不同长度”，立刻失效**：你按层摸到的“最近”，可能根本不是“最短”。

下一章，**最短路径 (Dijkstra)** ——我们给每条边配上“权值”，回答导航那个真正的问题：“从 A 到 B，走哪条路总长度最短？”你会看到，上一章 BFS 那个“逐层最短”的性质，是如何被升级成 Dijkstra 的**贪心**：**每次先确定一个“最短距离已定死”的点，再也不回头改**。你准备好从“算图”走到“算最短路”了吗？

第 03 章 · 最短路径与 Dijkstra——导航凭什么知道"最短"?

本章从哪个问题出发：上一章你学会的 BFS，有个看不见的短板——它假设每条边都一样长，走一步算一步。可现实里路不是一样长的：导航里有的路 3 公里，有的路 20 公里。你按 BFS"逐层摸到"的那个"最近"，根本不是"最短"。那导航到底凭什么，能在几百个路口的图里，一口咬定"走这条路最短"？推导到哪个结论：你会看到，答案是 Dijkstra 算法——一个"每次都挑当前看起来最近的点"的贪心算法。它看起来"短视"得可疑，可你会亲手证明它的正确性：一旦某个点的"最短距离"被定死，就再也不需要回头改——前提是每条边的权值都非负。你还会看到一个致命的反例：一旦权值变负，这个贪心立刻翻车，导航也会算错。

核心认知：Dijkstra 的聪明，在于"贪心在这里是对的"——而这必须靠证明，不是靠运气。它的证明，正是你第 2 章 BFS 那个"逐层最短"性质的华丽升级：BFS 假设每条边等长，Dijkstra 把"层"换成"带权的距离"，用三角不等式把"最近未定点的距离不可能被别的路再缩短"钉死。这一章是"算法正确性"的正餐——你会在非负权的图里，亲手看一个贪心算法凭什么敢"一条路走到黑"。

引子

老谟的旧手机上装着一个古早的导航。我随手输了个目的地，它眨都不眨，半秒内给出路线："走 A 路，转 B 街，再上 C 大道，全程 12.4 公里。"

"老谟，"我盯着那行字，"这导航怎么知道走这条最短？它总不能把城市里每一条路线都走一遍再比长短吧？"

老谟瞥了一眼："你觉得要是不把所有路线走一遍，它怎么可能'保证'是最短？"

".....保证？"我愣了，"它要是没走遍所有路线，怎么敢保证最短？"

"问得好。"老谟放下手机，"'保证'这两个字，还是得靠证明。这城市有几百个路口，几千条路，导航当然不可能把路线全枚举一遍——它用的是一个'每次都挑看起来最近'的贪心算法，名字叫 Dijkstra。可'贪心'这两个字，听着就让人不放心：它凭什么每次挑最近的就一定对？这一章，我们要给这个'贪心'，立一座证明。"

对话引擎

第 1 轮：最短路径问题——“每条边不一样长”后，BFS 失效了

老谟：“先把问题说清楚。我们给图里每条边配上一个数——**权值 (weight)**，代表这条边的长度/代价/时间。现在问：从 s 到 t ，走哪条路径，**权值之和最小**？这就是**最短路径问题**。你先说说，上一章的 BFS，为什么在这儿不灵了？”

造物主：“BFS 假设每条边算一步……可这里边有长有短。比如从 s 到 t ，BFS 先摸到的是‘边数最少’的路，可这条路可能每条边都很长，反而绕远。**‘边数最少’不等于‘总权值最小’。**”

老谟：“对。BFS 管的是‘最少几步’，最短路径管的是‘最小总权值’——**两条不一样的‘最短’**。所以你得换一套算法。先问你个更朴素的：**最笨的办法是什么？**”

造物主：“……把所有从 s 到 t 的路线都枚举出来，一条条算总权值，取最小？”

老谟：“能，但你想过没有——**图一大，路线是天文数字**。一个有几十个路口的地图，从 s 到 t 的路线可能多到数不过来，枚举会算到天荒地老。**所以我们需要的不只是‘能算出答案’，还要‘算得快’**。这就逼着你想：能不能不枚举，用点‘技巧’？”

造物主：“……能不能像 BFS 那样，一层层往外扩，但扩的时候记的不是‘步数’，而是‘到这里的总权值’？”

老谟：“你摸到方向了。可这里有个坎：BFS 里‘先摸到的就是最短’，是因为每步都等长；**一旦带权，‘先摸到’的就不一定‘最短’了——你得想清楚‘什么时候才能定死一个点的最短距离’**。下一轮，我们看那个叫 Dijkstra 的贪心，是怎么处理这个坎的。”

知识锚点：最短路径问题与"加权 BFS"的坎

【概念深挖】**带权图 (weighted graph)**：图的每条边 e 配一个数 $w(e) \geq 0$ (权值，代表长度/代价)。**最短路径问题**：给定起点 s ，求 s 到每个顶点 t 的**最小总权值路径**，记为最短距离 $\delta(s, t)$ 。

为什么 BFS 会失效：BFS 把"步数"当"距离"，只适用于**每条边权值相等**（都=1）的图。带权后，"边数最少"和"总权值最小"是两回事——BFS 的"先摸到即最短"不再成立。

为什么不能只靠枚举：从 s 出发的路径数量随图增大呈**指数级爆炸**（每到一个点都有多种"下一步"选择）。枚举所有路径来取最小，在稍大的图上完全不可行。我们需要**"剪枝"**——只保留最有希望的方向，这就是 Dijkstra 要做的。

【工程实践】最短路径是工程里最常用的图算法之一：**GPS 导航**（最短驾车时间/路程）、**网络路由**（OSPF/IS-IS 等链路状态协议内部就用 Dijkstra）、**游戏 AI 的寻路**、**供应链的物流成本最小化**。"边有不同代价"是常态，所以"算总代价最小"比"算步数最少"更贴近真实——这正是这一章的价值。

第 2 轮：贪心的直觉——"每次挑最近的未定点"

老谏："Dijkstra 的想法，听上去简单得可疑：**维护两个集合——已经确定最短距离的点（叫'已定'），和还没确定的点（叫'未定'）。每次从'未定'里挑一个'当前算出来距离 s 最近'的点，把它定死，然后更新它的邻居。循环，直到所有点都定死。**你说说，这里哪个动作最让人不放心？"

造物主：".....'每次挑个最近的未定点，就把它定死'？它凭什么确定这个点'最短距离'就是当前这个数，不会再被别的路缩短？万一以后发现一条更近的路呢？"

老谏："你问到了核心。**'敢把最近的点定死'——这正是贪心。**贪心就是'每一步都做眼前最好的选择，不回头'。它之所以能成立，在非负权的图里有个关键原因：**如果某个未定点 u 是'当前最近'的，那么任何从 s 出发、绕到别的点再去 u 的路，都不会比直接到 u 更短**——因为绕路要经过别的点，而别的点当前距离都不小于 u ，再加一段非负的边，只会更远。这就是下一轮要证明的。可这个推理，藏着一个必须说清的前提——"

造物主：".....非负权？要是边权是负的，绕路反而能变短？"

老谏："你抢答了。**负权就是贪心的天敌。**先把'非负权下为什么对'记在心里，负权的反例我们专门放到第 6 轮。现在你记住：**Dijkstra 的'贪心'能成立，全靠'非负权'这个前提撑着。**完整证明在走廊定理 1。"

知识锚点：贪心的直觉——“最近未定点为什么敢定死”

【概念深挖】**Dijkstra 的核心贪心**：维护“已确定最短距离”的集合 S 与未定集合 T ，每次从 T 中挑“当前 dist 最小”的顶点 u ，把 u 移入 S （定死），并用 u 更新其邻居的 dist 。

为什么“最近未定点”敢被定死（非负权前提）：设 u 是未定点中 dist 最小的。任取一条从 s 到 u 的路径，它离开 S （已定集合）时必然经过某个未定点 v （此 v 的 dist 由已定边界更新过）。因为 u 是未定中最小的， $\text{dist}(v) \geq \text{dist}(u)$ ；又路径剩余部分权值非负，故这条路径总长 $\geq \text{dist}(v) \geq \text{dist}(u)$ 。于是任何路径都不比当前的 $\text{dist}(u)$ 短， u 可定死。严格化在走廊定理 1（三角不等式 + 反证）。

为什么“非负权”是关键：若边权可以为负，“从 u 绕到 v 再回来”可能让距离变小，上面“剩余部分非负”的推理就断了—— u 就不能再被轻易定死。非负权不是算法的小细节，是贪心合法性的生命线。

【工程实践】真实应用里权值几乎都非负：距离、时间、费用、延迟都 ≥ 0 。这就是为什么 Dijkstra 能在 GPS、路由、寻路里放心用。“非负权 + 贪心”这个组合，是工程里“能用最快算法”的保证——一旦数据里混入负权（比如“补贴”“回扣”把代价算成负），就得换 Bellman-Ford 等更贵的算法，这是第 5 轮之后你会意识到的取舍。

亲手走一遍 Dijkstra：下面这张带权图就是本章的例子（ s 为起点）。点「下一步」逐步执行算法——观察每次「定死最近未定点 + 松弛邻居」后 dist 的变化。等你走完非负权这遍，再切到「负权反例」，亲眼看看贪心为什么翻车。

交互演示 (Dijkstra Explorer)：此组件为在线版的动态演示，离线 PDF/EPUB 中无法交互。完整推导见本章正文与「计算训练场」。

第 3 轮：算法怎么走—— dist 、集合，和那棵“前驱树”

老谟：“贪心讲清了，现在把 Dijkstra 的‘步骤’落成能照做的流程。你手里要维护两样东西：一个数组 $\text{dist}[]$ （每个点当前算出的最短距离），和一个标记‘是否已定死’的集合。你先把初始化想清楚：起点 s 的 dist 是多少？其他点呢？”

造物主：“……起点 s 的 dist 是 0（ s 到自己是 0），其他所有点的 dist 先设成‘无穷大’（还没找到路）？”

老谟：“对。初始化： $\text{dist}[s]=0$ ，其余 $= +\infty$ 。然后循环：每轮挑一个未定且 dist 最小的点 u ，把它定死，再用 u 更新所有邻居 v ： $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u,v))$ 。这个更新动作叫什么？”

造物主：“……叫‘松弛’？我用 u 现在的距离，加上 u 到 v 的边权，看看能不能把 v 的距离压得更小？如果 $\text{dist}[u] + w(u,v)$ 比现在的 $\text{dist}[v]$ 还小，就更新。”

老谟："对，**松弛 (relaxation)** ——试着用一条新路，让 v 的距离更短。循环到所有点都定死。**为什么循环要跑够 $n-1$ 次？** 因为每轮定死一个点，总共 n 个点，起点第一轮就定死了，剩下的 $n-1$ 个点每轮定死一个，所以循环 $n-1$ 次。**你还要注意一件事：怎么还原'最短路径本身'，而不只是'最短距离'？"**

造物主：".....得记下'每个点的最短距离是从谁那儿来的'？我每次松弛成功时，记下 v 的'前驱'是 u ——这样最后从终点一路顺着前驱走回起点，就是最短路径？这棵前驱关系连起来，是不是就是一棵'最短路树'？"

老谟："你连'还原路径'都想到了。**每个点的前驱指回'让它距离变短的那个点'，连起来是一棵以 s 为根的树（前驱树），顺着它就能还原出最短路径。** 现在去走廊，看它被写成标准的伪代码和证明。"

知识锚点：Dijkstra 的完整骨架——dist、松弛、前驱树

【概念深挖】Dijkstra 算法步骤：

1. **初始化：** $dist[s] = 0$ ，其余顶点 $dist = \infty$ ；标记 s 已定。
2. **循环（共 $n-1$ 次）：** 从"未定"顶点中挑 $dist$ 最小者 u ，标记 u 已定；
3. **松弛：** 对 u 的每个邻居 v ，若 $dist[u] + w(u, v) < dist[v]$ ，则更新 $dist[v] = dist[u] + w(u, v)$ ，并记 v 的前驱为 u 。
4. 结束时， $dist[t]$ 即 s 到 t 的最短距离；沿前驱树从 t 回溯到 s ，得最短路径。

为什么想到"松弛"："松弛"想表达的，正是"现在这条路更短，把旧距离松一松，换成新的"——**每次走通一条新边，就试着刷新邻居的最短距离。前驱树让你不光知道"多远"，还知道"怎么走"。**

【工程实践】工程实现 Dijkstra 常用**优先队列（最小堆）**来快速挑"未定中 $dist$ 最小"的点，把复杂度从 $O(n^2)$ 降到 $O((n+m)\log n)$ (n =顶点数， m =边数)。**GPS/路由里图很大，必须用堆优化版本。**"前驱树"在导航里就是"这条路线的每一步该拐哪"——它是你最终看到的那条推荐路线的来源。

第 4 轮：手动跑一遍——一个具体的小图，看着 dist 逐轮变

老谟："光说不练假把式。给你个具体的带权图，我们手动跑一遍 Dijkstra，看着 $dist$ 逐轮变。图有 5 个点： s 、 a 、 b 、 t 、再加一个点。 **s 连 a （权 4）、 s 连 b （权 2）； a 连 t （权 3）； b 连 a （权 1）、 b 连 t （权 5）。** 从 s 出发，你报一下每轮的 $dist$ 变化。"

造物主："初始化： $dist[s]=0$, $dist[a]=\infty$, $dist[b]=\infty$, $dist[t]=\infty$ 。第一轮挑未定中 $dist$ 最小的，是 s (0)，定死 s ；松弛： $dist[a]=\min(\infty, 0+4)=4$, $dist[b]=\min(\infty, 0+2)=2$ 。现在 $dist$ ： $a=4$, $b=2$,

$t = \infty$ 。"

老谋："第二轮？"

造物主："未定中 dist 最小是 b (2)，定死 b ；松弛 b 的邻居 a 、 t ： $\text{dist}[a] = \min(4, 2+1) = 3$ (更新了!)， $\text{dist}[t] = \min(\infty, 2+5) = 7$ 。现在 dist ： $a=3, t=7$ 。"

老谋："注意你刚才那个'更新'—— $\text{dist}[a]$ 从 4 降到了 3。这就是'贪心先定 b ，再回头修正 a '。第三轮？"

造物主："未定中最小是 a (3)，定死 a ；松弛 a 的邻居 t ： $\text{dist}[t] = \min(7, 3+3) = 6$ (更新!)。现在 dist ： $t=6$ 。第四轮定死 t 。所以 s 到 t 的最短距离是 6，路径是 $s \rightarrow b \rightarrow a \rightarrow t$ (权 $2+1+3=6$)。"

老谋："你完整跑通了。注意一件反直觉的事：第一轮看起 s 到 a 是 4，'好像'很近，可定死 a 前你先定死了 b (2)，然后通过 $b \rightarrow a$ 这条路把 a 刷成了 3—— a 的真正最短距离，比第一眼看到的还近。可为什么 Dijkstra 非要等 b 定死后才敢定 a ？因为定 a 之前，得先把"绕 b 更近"这条路算出来。这就是'贪心要先挑全局最小的未定点'的原因：只有最小那个才敢定死，其余都可能有更短的路没发现。完整证明在走廊。"

知识锚点：逐轮 dist 更新——"贪心先定全局最小"的活例

【概念深挖】上面例子的逐轮轨迹 ($\text{dist}[s] = 0$ 起)：

- 第 1 轮定 s ： $\text{dist}[a] = 4, \text{dist}[b] = 2, \text{dist}[t] = \infty$ ；
- 第 2 轮定 b (最小 2)：松弛后 $\text{dist}[a] = 3$ (经 $b \rightarrow a$)， $\text{dist}[t] = 7$ (经 $b \rightarrow t$)；
- 第 3 轮定 a (最小 3)：松弛后 $\text{dist}[t] = 6$ (经 $a \rightarrow t$)；
- 第 4 轮定 t (6)。最短距离 $\text{dist}[t] = 6$ ，路径 $s \rightarrow b \rightarrow a \rightarrow t$ (权 $2 + 1 + 3 = 6$)。

为什么"要先定全局最小未定点"：因为只有它敢定死。其余未定点 (如当时的 $a=4$) 虽然"看起来近"，但它可能有一条"绕到更小的点再回来"的更短路 (a 经 b 变成 3)——只要还有更小的未定点存在，这个点就可能有被它刷新的机会，不能定死。这正是贪心"每次挑全局最小"的理由。

【工程实践】这个"先定小、再修正"的过程，正是导航实际在算的事：它不会一上来就认定"直线近的点最近"，而是把"绕一点更近"的可能也算进去。你最终看到的推荐路线，就是沿前驱树回溯出来的最短路径。手动跑一遍，是理解"贪心为何必须挑全局最小"最好的方式。

第 5 轮：负权为什么不行——一个会骗过 Dijkstra 的反例

老谋："现在来碰那个要命的前提——负权。我给你一个小图，你自己跑 Dijkstra，看它会不会算错。

图： $s \rightarrow a$ 权 3， $s \rightarrow b$ 权 4， $b \rightarrow a$ 权 -5， $a \rightarrow t$ 权 1， $b \rightarrow t$ 权 1。从 s 到 t ，最短是多少？先别用算

法，凭直觉想。"

造物主：".....让我想想。 $s \rightarrow b \rightarrow a \rightarrow t$? 权 $4 + (-5) + 1 = 0$? $s \rightarrow b \rightarrow t$ 权 $4 + 1 = 5$, $s \rightarrow a \rightarrow t$ 权 $3 + 1 = 4$ 。好像 $s \rightarrow b \rightarrow a \rightarrow t$ 是 0, 最短? 这条路还绕了一圈、用了个负权边, 反而更短!"

老谟："对, 负权让'绕路'反而变短——这是贪心完全没料到的。现在你手动跑 Dijkstra, 看它出什么岔子。"

造物主："初始化 $\text{dist}[s]=0, a=\infty, b=\infty, t=\infty$ 。第 1 轮定 s , 松弛: $a=3, b=4$ 。第 2 轮定 a (3), 松弛 a 的邻居 t : $\text{dist}[t]=3+1=4$ 。第 3 轮定 b (4), 松弛 b 的邻居 a 、 t : $\text{dist}[a]=\min(3, 4-5)=-1$ (更新 a 成 -1 !), $\text{dist}[t]=\min(4, 4+1)=4$ 。可 a 已经被定死过了啊! 现在它的 dist 变成了 -1 , 比定死时还小——这说明定 a 是错的! 而且 t 的 $\text{dist}=4$, 但真正最短是 0 (走 $b \rightarrow a \rightarrow t$)"

老谟："你亲眼看到了 Dijkstra 在负权下翻车。它把 a 定死成 3 之后, 才通过 $b \rightarrow a$ 那条负权边发现 a 其实该是 -1 ——可 a 已经'定死'了, 算法不会回头去改它, 于是后面的松弛全建立在错误的 a 上, 最终 t 算出 4, 而真实最短是 0。负权破坏了'最近未定点敢定死'的前提: 因为绕负权边能变短, 所以'当前最近'根本不可信。这就是为什么 Dijkstra 必须要求非负权——它一遇到负权, '贪心'就成了'盲心'。负权的正确算法 (如 Bellman-Ford) 会反复多轮松弛, 能处理, 但更贵。你先把'负权为什么害死贪心'记住, 这比背一个公式重要。"

知识锚点：负权的反例——贪心失效的现场

【概念深挖】**负权反例**：图 $s \rightarrow a$ (3), $s \rightarrow b$ (4), $b \rightarrow a$ (-5), $a \rightarrow t$ (1), $b \rightarrow t$ (1)。

- 真实最短距离: $s \rightarrow b \rightarrow a \rightarrow t$, 总权 $4 + (-5) + 1 = 0$ 。
- 但 Dijkstra 会算错: 第 2 轮定死 a ($\text{dist}=3$), 之后才经 $b \rightarrow a$ 发现 a 该是 -1 , 可 a 已定死、算法不回头, 最终给出 t 的最短为 4, 而非真实的 0。

为什么负权破坏贪心: 贪心"定死最近未定点"依赖"绕到别的点再回来不会更短" (剩余部分非负)。负权让'绕路反而变短'成为可能, 于是"当前最近"不可信, Dijkstra 的"定死"就失效了。非负权是 Dijkstra 正确性的生命线, 不是可选条件。

【工程实践】这解释了工程里的一个常识: 为什么 GPS 里的'时间/距离'从不用负权, 而某些'带补贴/回扣'的复杂计价模型不能直接用 Dijkstra。一旦数据里可能混入负权, 工程师必须改用 Bellman-Ford (能处理负权, 但复杂度高、对负环还要单独检测)。**"先确认权值非负, 再决定用不用 Dijkstra"**——这是算法选择的现实原则。

第 6 轮：落点——“贪心在这里是对的”，全靠证明

老谟：“绕了一圈。你最初问‘导航凭什么敢保证最短’，现在怎么答？”

造物主：“……导航用 Dijkstra：每次挑未定中最小的点定死、松弛邻居，循环到全部定死。它敢保证最短，是因为在非负权下，‘最近未定点’确实不可能被别的路再缩短——这是能被证明的。可一旦有负权，贪心立刻翻车，得换 Bellman-Ford。”

老谟：“你手里多了什么？”

造物主：“带权图、最短路径问题、dist/松弛/前驱树、Dijkstra 贪心（每次挑全局最小未定点），还有非负权这个关键前提，和负权会让贪心失效的反例。”

老谟：“够了。给你句话收尾：Dijkstra 教会你的，是‘贪心’本身不坏，坏的是不问前提就用。它之所以是‘对’的算法，不是因为它直觉好，是因为在‘非负权’这个前提下一个可复写的证明撑住了它。这种‘一个算法敢说自己对，必须有证明背书’的自觉，是你后面所有算法章节（图着色）都要带上的。你准备好了吗？”

造物主：“……准备好了。我现在看算法，会先问一句：它凭什么敢这样贪？”

老谟：“那你去走廊，把 Dijkstra 的‘凭什么’，亲手证一遍。”

严格化走廊

位置说明：对话负责“直觉”——为什么每次挑最近未定点、为什么负权会翻车。本走廊负责“严谨”——给出 Dijkstra 的伪代码、正确性证明（贪心选择），以及负权反例的明确分析。本章有**两座证明**：定理 1（Dijkstra 贪心选择正确性，归纳 + 反证 + 三角不等式）、反例（负权下 Dijkstra 失效）。每步标“证明类型 + 依据”，绝不用“显然/易证/显见”（包括反话）。

定义：带权图、最短距离、松弛

- **带权图**：每条边 e 有权值 $w(e) \geq 0$ （本章默认非负权）。
- **最短距离** $\delta(s, t)$ ： s 到 t 的所有路径中，总权值的最小者。
- **松弛 (relaxation)**：对边 (u, v) ，若 $\text{dist}[u] + w(u, v) < \text{dist}[v]$ ，则更新 $\text{dist}[v] = \text{dist}[u] + w(u, v)$ 。

Dijkstra 算法伪代码（可复写卡片）

```

输入：带权图  $G=(V,E)$ ，权值  $w \geq 0$ ，起点  $s$ 。
初始化： $dist[s]=0$ ；其余  $dist[v]=\infty$ ； $S = \emptyset$ （已定集合）。
while  $S \neq V$ :
    从  $V \setminus S$  中选  $dist$  最小的顶点  $u$ （若多个取任一）；
    把  $u$  加入  $S$ （"定死"  $u$ ）；
    对  $u$  的每个邻居  $v$ （ $v \notin S$  或  $v \in S$  都更新）：
        松弛：if  $dist[u]+w(u,v) < dist[v]$ :
             $dist[v] = dist[u]+w(u,v)$ ；记  $v$  的前驱为  $u$ 。
结束： $dist[v]$  即  $s$  到  $v$  的最短距离；沿前驱树回溯得最短路径。

```

定理 1（Dijkstra 贪心选择正确性）：在非负权图 $G = (V, E)$ ($w(e) \geq 0$) 上运行 Dijkstra，算法结束时，对每个顶点 v ， $dist[v] = \delta(s, v)$ （即求出的就是最短距离）

证明类型：归纳法（对"已定集合 S 的大小"归纳）+ 关键一步用反证法 + 最短路径三角不等式。

证明：

- 第 1 步（**归纳命题**）：用归纳证明：**每次把顶点 u 加入 S 时， $dist[u]$ 已经是 $\delta(s, u)$ （ u 到 s 的真实最短距离），且此后不会再变。**（依据：要证的结论——把证明分解成"每个被定死的点，其 $dist$ 已是最短"。）
- 第 2 步（**归纳起点**）：第一步，把起点 s 加入 S ， $dist[s] = 0 = \delta(s, s)$ ，成立。（依据：初始化 $dist[s] = 0$ ； s 到自身最短距离为 0。）
- 第 3 步（**归纳假设**）：假设 S 中已有若干顶点，且 S 中每个顶点 v 的 $dist[v]$ 都是 $\delta(s, v)$ 。（依据：归纳法假设——已定点的 $dist$ 已正确。）
- 第 4 步（**选最小未定点 u** ）：现在 Dijkstra 从 $V \setminus S$ 中选出 $dist$ 最小的顶点 u 。**断言 $dist[u] = \delta(s, u)$ 。**（依据：Dijkstra 的"选最小"规则。）
- 第 5 步（**证 $dist[u] \leq \delta(s, u)$** ）： $dist[u]$ 是"沿某条已松弛过的路算出的 s 到 u 的长度"，由松弛定义， $dist[u]$ 恒为某条 s 到 u 路径的总权值，故 $dist[u] \geq \delta(s, u)$ （最短距离是"所有路径的最小值"，任何一条路径都不短于它）。（依据： $dist[u]$ 对应一条真实路径 + 最短距离的定义。）
- 第 6 步（**证 $dist[u] \leq \delta(s, u)$ ，反证**）：反设 $dist[u] > \delta(s, u)$ ，即存在一条比 $dist[u]$ 更短的 s 到 u 的路径 P 。设 P 从 s 出发，**第一个离开集合 S 的顶点为 x** ，它离开 S 时经过一条边 (y, x) ($y \in S, x \notin S$)。则 P 在 x 之前的部分 (s 到 y) 都在 S 内， y 已定死。（依据：反证法假设 + 路径 P 的存在性 + "第一个离开 S "的定义。）
- 第 7 步：因 $y \in S$ 已定死，由归纳假设 $dist[y] = \delta(s, y)$ 。而 P 的 s 到 y 段长度 $\geq \delta(s, y) = dist[y]$ ($dist[y]$ 是 s 到 y 的最短距离，任何段不短于它)。（依据：归纳假设 $dist[y] = \delta(s, y)$ + 最短距离定义。）

- 第 8 步：于是 P 的 s 到 x 段长度 $= (s \text{ 到 } y \text{ 段}) + w(y, x) \geq \text{dist}[y] + w(y, x)$ 。而 Dijkstra 在定死 y 时已松弛过边 (y, x) ，故 $\text{dist}[x] \leq \text{dist}[y] + w(y, x)$ 。（依据：松弛定义—— $\text{dist}[x]$ 会被更新为不超过 $\text{dist}[y] + w(y, x)$ 。）
- 第 9 步：结合第 7、8 步， P 的 s 到 x 段长度 $\geq \text{dist}[y] + w(y, x) \geq \text{dist}[x]$ 。又因 P 剩余部分 (x 到 u) 的权值**非负**（前提 $w \geq 0$ ），故整条 P 长度 $\geq \text{dist}[x] + (x \text{ 到 } u \text{ 段非负}) \geq \text{dist}[x]$ 。（依据： $w \geq 0$ 保证剩余部分非负 + 不等式传递。）
- 第 10 步： $x \in V \setminus S$ 是未定点，且 u 是“未定中 dist 最小”者，故 $\text{dist}[x] \geq \text{dist}[u]$ (u 的 dist 最小，任何其他未定点不更小)。（依据：Dijkstra “选 dist 最小未定点”的规则。）
- 第 11 步：于是 $P \text{ 长度} \geq \text{dist}[x] \geq \text{dist}[u]$ 。这与反设“ P 比 $\text{dist}[u]$ 更短” ($P \text{ 长度} < \text{dist}[u]$) **矛盾**。（依据：第 9、10 步推出 $P \text{ 长度} \geq \text{dist}[u]$ ，与反设矛盾。）
- 第 12 步：故反设不成立， $\text{dist}[u] \leq \delta(s, u)$ 。综合第 5 步 $\text{dist}[u] \geq \delta(s, u)$ ，得 $\text{dist}[u] = \delta(s, u)$ 。 u 加入 S 时其 dist 已是**最短**，且因**最短距离唯一**、**松弛不会让已定的 dist 更小**（**非负权下松弛只会不增**）， u 的 dist 此后**不变**。（依据：第 5、11 步夹逼得等号 + 非负权下已定点的 dist 不被更小的松弛破坏。）
- 第 13 步：由归纳法（起点成立 + 每步加入的 u 都正确），算法结束时 $S = V$ ，每个顶点 v 的 $\text{dist}[v] = \delta(s, v)$ 。■

依据标注：步 1 归纳命题；步 2 起点；步 3 归纳假设；步 4 选最小 u ；步 5 用“最短距离是路径最小值”得上界；步 6-11 反证：步 6 设更短路径 P 、找第一个离开 S 的点 x ，步 7-8 用归纳假设+松弛把 x 的距离与 P 段长度挂钩，步 9 用非负权保剩余段非负，步 10 用“ u 是最小未定”得 $\text{dist}[x] \geq \text{dist}[u]$ ，步 11 与反设矛盾；步 12 夹逼得等号；步 13 归纳收束。**读法：**这个证明的发动机是第 6-10 步——“一条更短的路径，一旦第一次离开已定集合 S ，那个离开点 x 的距离一定不小于 u （因为 u 是最小未定），于是整条路都不可能比 $\text{dist}[u]$ 短”。它反复用到两件前提：① **已定点的 dist 已正确**（归纳假设）；② **权值非负**（剩余段不会拉低总长）。缺了 ②，第 9 步就崩，这正是负权翻车的根源。

反例（负权下 Dijkstra 失效）：若允许负权，定理 1 不成立

证明类型：反例（构造一个带负权的图，演示 Dijkstra 给出错误答案）。

证明：

- 第 1 步（**构造图**）：顶点 $\{s, a, b, t\}$ ，边 $s \rightarrow a$ (3)、 $s \rightarrow b$ (4)、 $b \rightarrow a$ (-5)、 $a \rightarrow t$ (1)、 $b \rightarrow t$ (1)。（依据：反例构造——含一条负权边 $b \rightarrow a = -5$ 。）
- 第 2 步（**真实最短距离**）： $s \rightarrow b \rightarrow a \rightarrow t$ 的总权 $4 + (-5) + 1 = 0$ ；而 $s \rightarrow a \rightarrow t$ 为 $3 + 1 = 4$ ， $s \rightarrow b \rightarrow t$ 为 $4 + 1 = 5$ 。故 $\delta(s, t) = 0$ （最小路径总权）。（依据：枚举所有 s

到 t 的路径，比较总权，取最小。)

- 第 3 步 (**跑 Dijkstra**)：初始 $dist[s] = 0, dist[a] = dist[b] = dist[t] = \infty$ 。
 - 第 1 轮定 s ，松弛： $dist[a] = 3, dist[b] = 4$ 。
 - 第 2 轮定 a (未定中 $dist$ 最小, 3)，松弛 a 的邻居 t ： $dist[t] = 3 + 1 = 4$ 。
 - 第 3 轮定 b ($dist$ 4)，松弛 b 的邻居 a, t ： $dist[a] = \min(3, 4 - 5) = -1$ (**a 已被定死, 算法不回头改它**)， $dist[t] = \min(4, 4 + 1) = 4$ 。
 - 第 4 轮定 t ($dist$ 4)。(依据：严格按 Dijkstra 伪代码逐步执行。)
- 第 4 步 (**对比**)：Dijkstra 输出 $dist[t] = 4$ ，而真实 $\delta(s, t) = 0$ 。**算法给出错误答案。**(依据：第 2 步与第 3 步结果不一致。)
- 第 5 步 (**根因**)：Dijkstra 在第 2 轮就"定死"了 a ($dist=3$)，可后来经负权边 $b \rightarrow a$ 才发现 a 本应是 -1 ；因 a 已定死、算法不回头，这个错误蔓延到 t 。(依据：负权破坏了"最近未定点敢定死"的前提——绕负权边能变短，故"当前最近"不可信。)
- 结论：**Dijkstra 只适用于非负权；有负权时它可能给出错误答案。■**

依据标注：步 1 构造含负权反例；步 2 枚举得真实最短 0；步 3 严格跑 Dijkstra 得 4；步 4 对比发现错误；步 5 分析根因是负权破坏贪心前提。**读法：**反例的力量在于"亲眼看见算法翻车"——不靠抽象，靠一个具体的图，Dijkstra 就是算错了。**记住它的教训：一个算法的正确性，永远绑定在它的前提上；前提一破（负权），结论（最短）就没了。**

反例/边界：最短路径的适用边界

1. **非负权是硬前提：**Dijkstra 只对 $w(e) \geq 0$ 保证正确；有负权必须换 Bellman-Ford (可处理负权，但对负环需单独检测，且复杂度更高)。
2. **负环：**若图中存在"总权为负的环"，最短路径可能不存在 (沿负环无限绕会无限变小)。任何最短路径算法都无法给"有负环的图"一个有限最短距离——这是问题本身的病，不是算法的病。
3. **Dijkstra 求的是单源最短：**它从单个起点 s 出发，给出 s 到所有点的最短距离。若要求"所有点对之间"的最短，可多次跑 Dijkstra (或用 Floyd-Warshall)。
4. **"最短"可能是"距离"也可能是"时间/费用"：**权值的含义随问题而定，Dijkstra 只关心"权值之和最小"，不关心权值具体代表什么——这正是"抽象"的价值。
5. **松弛的顺序不影响 Dijkstra 正确性：**只要每次都"定死未定中 $dist$ 最小者"，松弛先后不改变最终正确性 (定理 1 保证)；但**选最小者这一步不能偷懒**——若随便定一个未定点，贪心就失效了。

知识点总结

知识点（本章推出的核心概念）

- **带权图 / 最短路径问题**：每条边有权值，求起点到各点的最小总权路径。
- **为什么 BFS 失效**：BFS 假设每条边等长；带权后"边数最少"≠"总权最小"。
- **Dijkstra 贪心**：维护已定集合 S ，每次定死"未定中 dist 最小"的点，松弛其邻居，循环 $n-1$ 次。
- **松弛 (relaxation)**： $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$ ，并记前驱。
- **前驱树**：每个点指回"让它距离变短的点"，沿它回溯还原最短路径。
- **正确性前提**：非负权 ($w \geq 0$)。有负权时 Dijkstra 失效，需换 Bellman-Ford。

历史结论（本章涉及的史料）

- **Edsger Dijkstra（艾兹格·迪杰斯特拉，荷兰计算机科学家）**：Dijkstra 算法1956 年设想、1959 年发表。相传 Dijkstra 在一个咖啡店思考"从鹿特丹到格罗宁根的最短路径"问题时灵光一现，用了约 20 分钟设计出该算法，却花了三年才写成论文。它是计算科学史上最早、最著名的"贪心+证明正确性"的算法范例之一。（归因至此，不额外堆砌。）

工程实践结论（本章知识在真实工程里怎么用）

- **GPS 导航**：求最短驾车时间/路程，用 Dijkstra（权=距离或时间，非负）。
- **网络路由**：OSPF、IS-IS 等链路状态路由协议内部用 Dijkstra 计算到各路由器的最近路径。
- **游戏寻路 / 地图服务**：Dijkstra（或 A^* ，其核心仍是 Dijkstra + 启发式）求最短路径。
- **优先队列优化**：用最小堆实现"取未定中 dist 最小"，把复杂度从 $O(n^2)$ 降到 $O((n+m)\log n)$ ，处理大图。
- **权值非负的取舍**：数据中可能混入负权（补贴/回扣）时，换 Bellman-Ford；有负环时必须单独检测。

计算训练场

位置说明：对话负责"为什么"，这里负责"怎么算"。每题动笔 10 分钟以上，先自己写再对答案。**难度递进：易→中→难。**

题 1 (易→中) : 对 5 节点带权图完整执行 Dijkstra——逐轮更新表 + 还原路径

题干：带权图 G ：顶点 $\{s, a, b, t\}$ ，边 $s \rightarrow a$ (4)、 $s \rightarrow b$ (2)、 $b \rightarrow a$ (1)、 $a \rightarrow t$ (3)、 $b \rightarrow t$ (5)。从 s 完整执行 Dijkstra，写逐轮 dist 更新表，求 s 到 t 的最短距离与路径，并反向验证。

完整分步解答：

- 第 0 步 (初始化) : $dist[s] = 0, dist[a] = \infty, dist[b] = \infty, dist[t] = \infty$; 已定 $S = \emptyset$ 。
- 第 1 轮 (定 s , $dist=0$) : 松弛 $s \rightarrow a, s \rightarrow b$: $dist[a] = 4, dist[b] = 2$ 。表: $a = 4, b = 2, t = \infty$ 。
- 第 2 轮 (未定中最小 $b = 2$, 定 b) : 松弛 $b \rightarrow a, b \rightarrow t$: $dist[a] = \min(4, 2 + 1) = 3$ (更新), $dist[t] = \min(\infty, 2 + 5) = 7$ 。表: $a = 3, t = 7$ 。
- 第 3 轮 (未定中最小 $a = 3$, 定 a) : 松弛 $a \rightarrow t$: $dist[t] = \min(7, 3 + 3) = 6$ (更新)。表: $t = 6$ 。
- 第 4 轮 (未定只剩 $t = 6$, 定 t) : 无未定邻居。结束。
- **最短距离** $dist[t] = 6$ 。**路径还原** (沿前驱) : t 的前驱是 a (因经 $a \rightarrow t$ 更新为 6), a 的前驱是 b (经 $b \rightarrow a$ 更新为 3), b 的前驱是 s 。路径: $s \rightarrow b \rightarrow a \rightarrow t$ 。
- **反向验证**: $s \rightarrow b \rightarrow a \rightarrow t$ 的总权 = $2 + 1 + 3 = 6 \checkmark$; 其他路径: $s \rightarrow a \rightarrow t$ 权 $4 + 3 = 7$, $s \rightarrow b \rightarrow t$ 权 $2 + 5 = 7$, 均 ≥ 6 。**6 确为最短。**

常见错误：① 第一轮就定死 a (因为看起来近 4), 没先定全局最小 b (2) ——**必须等所有更小的未定点定完, a 才敢定**; ② 松弛时漏更新 $dist[a]$ (忘了 $b \rightarrow a$ 这条边); ③ 路径还原时前驱记错 (t 的前驱是 a 不是 b , 因为 t 是经 a 刷成 6 的)。**反向验证**: 把还原出的路径权值加起来, 应与 $dist[t]$ 相等。

题 2 (中→难) : 6 节点图完整 Dijkstra + 还原最短路径

题干：带权图 G ：顶点 $\{s, u, v, x, y, t\}$ ，边 $s \rightarrow u$ (7)、 $s \rightarrow v$ (9)、 $u \rightarrow v$ (10)、 $u \rightarrow x$ (15)、 $v \rightarrow x$ (11)、 $v \rightarrow y$ (2)、 $y \rightarrow t$ (8)、 $x \rightarrow t$ (6)、 $x \rightarrow y$ (5)。从 s 完整执行 Dijkstra，求 s 到 t 的最短距离与路径。

完整分步解答：

- 第 0 步 (初始化) : $dist[s] = 0$, 其余 = ∞ 。
- 第 1 轮 (定 s) : 松弛 $s \rightarrow u, s \rightarrow v$: $dist[u] = 7, dist[v] = 9$ 。
- 第 2 轮 (未定最小 $u = 7$, 定 u) : 松弛 $u \rightarrow v, u \rightarrow x$: $dist[v] = \min(9, 7 + 10) = 9$ (不变), $dist[x] = \min(\infty, 7 + 15) = 22$ 。表: $v = 9, x = 22$ 。

- 第 3 轮 (未定最小 $v = 9$, 定 v) : 松弛 $v \rightarrow x$, $v \rightarrow y$: $dist[x] = \min(22, 9 + 11) = 20$ (更新), $dist[y] = \min(\infty, 9 + 2) = 11$ 。表: $x = 20$, $y = 11$ 。
- 第 4 轮 (未定最小 $y = 11$, 定 y) : 松弛 $y \rightarrow t$, $y \rightarrow x$: $dist[t] = \min(\infty, 11 + 8) = 19$, $dist[x] = \min(20, 11 + 5) = 16$ (更新)。表: $x = 16$, $t = 19$ 。
- 第 5 轮 (未定最小 $x = 16$, 定 x) : 松弛 $x \rightarrow t$, $x \rightarrow y$: $dist[t] = \min(19, 16 + 6) = 19$ (不变), y 已定。表: $t = 19$ 。
- 第 6 轮 (定 $t = 19$) : 结束。
- 最短距离 $dist[t] = 19$ 。路径还原: t 的前驱 = y (t 经 $y \rightarrow t$ 得 19); y 的前驱 = v (y 经 $v \rightarrow y$ 得 11); v 的前驱 = s 。路径: $s \rightarrow v \rightarrow y \rightarrow t$ 。
- 反向验证: $s \rightarrow v \rightarrow y \rightarrow t$ 总权 = $9 + 2 + 8 = 19 \checkmark$; 另一候选 $s \rightarrow u \rightarrow v \rightarrow y \rightarrow t$ 权 $7 + 10 + 2 + 8 = 27$, $s \rightarrow v \rightarrow x \rightarrow t$ 权 $9 + 11 + 6 = 26$, 均 > 19 。19 确为最短。

常见错误: ① 第 4 轮 x 从 20 降到 16 (经 $y \rightarrow x$) , 别漏这次更新; ② 路径还原时 t 的前驱是 y (经 $y \rightarrow t$) 不是 x ——虽然 $x \rightarrow t$ 也能到 19, 但 Dijkstra 里 t 是第一次被 y 刷成 19 的 (先到先记前驱) , 还原应走 y ; ③ 每轮"未定中取最小"要重新扫, 别沿用上一轮的 u 。反向验证: 还原路径权值之和 = $dist[t]$, 且与所有未走路径对比确认最小。

题 3 (难) : 负权反例验证——Dijkstra 错在哪, Bellman-Ford 怎么救

题干: 图 G : $s \rightarrow a$ (3)、 $s \rightarrow b$ (4)、 $b \rightarrow a$ (-5)、 $a \rightarrow t$ (1)、 $b \rightarrow t$ (1)。(a) 手动跑 Dijkstra, 指出它算出的错误答案。(b) 算出真实最短距离。(c) 说明 Dijkstra 错在哪一步, 为什么非负权是前提。

完整分步解答:

- (a) Dijkstra: 初始化 $dist[s] = 0$, 其余 ∞ 。第 1 轮定 s : $a = 3$, $b = 4$ 。第 2 轮定 a (3): $t = 3 + 1 = 4$ 。第 3 轮定 b (4): $a = \min(3, 4 - 5) = -1$ (a 已定死, 不回头), $t = \min(4, 4 + 1) = 4$ 。第 4 轮定 t (4)。Dijkstra 输出 $dist[t] = 4$ 。
- (b) 真实最短: $s \rightarrow b \rightarrow a \rightarrow t$ 总权 $4 + (-5) + 1 = 0$ 。 $\delta(s, t) = 0$ 。(其他路径 $s \rightarrow a \rightarrow t = 4$ 、 $s \rightarrow b \rightarrow t = 5$ 。)
- (c) 错在第 2 轮就把 a 定死成 3, 而经负权边 $b \rightarrow a$ 后 a 本应为 -1; 因算法不回头改已定的 a , 错误蔓延, t 算出 4 而非 0。根因: 非负权保证"绕到别的点再回来不会更短", 负权让"绕路反而变短", "最近未定点敢定死"的前提被破坏。这就是为什么 Dijkstra 必须要求 $w \geq 0$ 。
- 答: (a) Dijkstra 给 4; (b) 真实最短 0; (c) 错在过早定死 a , 负权破坏贪心前提。

常见错误: ① 忘了"已定死的点不回头改"—— a 被定成 3 后就算算出 -1 也不改; ② 把负权反例的答案当 4 (那是 Dijkstra 的错误输出, 不是真实最短); ③ 以为"负权图有负环"——本例无负环 (只有

一条负边)，Dijkstra 依旧错，说明'单条负边'就足以让 Dijkstra 失效，不一定要负环。

向导便签

这一章咱们干了什么：把 BFS 那个"逐层最短"，升级成了 Dijkstra——一个'每次挑未定中最近的点定死'的贪心算法。你亲手证明了这个贪心在非负权下是对的（归纳 + 反证 + 三角不等式），也亲眼看了负权下它怎么翻车。

钉死一句话：**Dijkstra 敢"贪"，是因为在非负权下有证明给它撑腰；前提一破（负权），贪心就成了盲心。**"最近未定点敢定死"——这句话的每一个字，都建立在 $w \geq 0$ 之上。

记住：你第一次完整看了一个**"贪心 + 正确性证明"**的算法。往后你遇到任何算法，都要先问一句：**它凭什么敢这样贪？前提是什么？** 这种自觉，正是下一章（图着色）和整个可计算性板块的入场券。

造物主手记

我曾踩的坑：

我以为 Dijkstra 就是"每次都挑最近的"，可第一次手动跑时，我第一轮就把 a （看起来近 4）定死了，没先定全局最小的 b （2）——结果后来 $b \rightarrow a$ 把 a 刷成 3，我才发现 a 不该那么早定。老谋点破：**只有全局最小的未定点才敢定死，其余都可能被更小的点"刷新"。** 松弛时我还漏过边（第 4 轮差点没更新 x 从 20 到 16）。最让我震惊的是负权反例： $s \rightarrow b \rightarrow a \rightarrow t$ 居然是 0，比直线还短——**我这才懂'非负权'不是小细节，是 Dijkstra 的命根子。** 还有一次我把负权反例的答案写成 4（那是 Dijkstra 的错误输出），被提醒"真实最短是 0，4 恰恰证明它算错了"。

顿悟时刻：

真正开窍是定理 1 那个证明的第 6-10 步——"一条更短的路径，一旦第一次离开已定集合，那个离开点的距离一定不小于最小未定点，于是整条路都不可能更短"。**原来'贪心在这里是对的'，不是靠运气，是靠'非负权让绕路不可能变短'这一条铁的事实。** 而负权反例让我第一次意识到：**算法的正确性，永远绑定在它的前提上。** 这个念头，我揣着它去看下一章的算法，感觉完全不同了——我学会的不是"背 Dijkstra"，而是"用前提去审判一个算法敢不敢贪"。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 小图 Dijkstra

带权图: $\{s, a, b, t\}$, 边 $s \rightarrow a$ (6)、 $s \rightarrow b$ (3)、 $b \rightarrow a$ (1)、 $a \rightarrow t$ (2)、 $b \rightarrow t$ (7)。从 s 跑 Dijkstra, 求 s 到 t 的最短距离与路径。

完整解答: 初始化 $dist[s] = 0$ 。第 1 轮定 s : $a = 6, b = 3$ 。第 2 轮定 b (3): $a = \min(6, 3 + 1) = 4, t = \min(\infty, 3 + 7) = 10$ 。第 3 轮定 a (4): $t = \min(10, 4 + 2) = 6$ 。第 4 轮定 t (6)。**最短距离 6, 路径 $s \rightarrow b \rightarrow a \rightarrow t$ ($3 + 1 + 2 = 6$)**。反向验证: $s \rightarrow a \rightarrow t = 6 + 2 = 8, s \rightarrow b \rightarrow t = 3 + 7 = 10$, 均 > 6 。

题 B (计算, 中→难) : 完整 Dijkstra 逐轮表

带权图: $\{s, u, v, t\}$, 边 $s \rightarrow u$ (5)、 $s \rightarrow v$ (2)、 $u \rightarrow v$ (1)、 $u \rightarrow t$ (3)、 $v \rightarrow t$ (8)。写逐轮 dist 更新表, 求最短路径。

完整解答: 初始化 $s = 0$ 。第 1 轮定 s : $u = 5, v = 2$ 。第 2 轮定 v (2): $u = \min(5, 2 + 1) = 3, t = \min(\infty, 2 + 8) = 10$ 。第 3 轮定 u (3): $t = \min(10, 3 + 3) = 6$ 。第 4 轮定 t (6)。**最短距离 6, 路径 $s \rightarrow v \rightarrow u \rightarrow t$ ($2 + 1 + 3 = 6$)**。反向验证: $s \rightarrow u \rightarrow t = 5 + 3 = 8, s \rightarrow v \rightarrow t = 2 + 8 = 10$, 均 > 6 。

题 C (概念, 检查理解)

(a) 为什么 Dijkstra 每次要挑"未定中 dist 最小"的点, 而不是随便挑一个? (b) 为什么负权会让 Dijkstra 失效? (c) Dijkstra 求的是"边数最少"还是"总权最小"? 和 BFS 的区别是什么?

完整解答: (a) 因为只有"全局最小未定点"才敢定死——其余未定点可能有一条"绕到更小的点再回来"的更短路(会被更小的未定点刷新), 不能提前定死。(b) 负权让"绕路反而变短", 于是"最近未定点敢定死"的前提被破坏——Dijkstra 可能过早定死某点, 且不回头改, 导致后续松弛建立在错误值上, 最终算错。(c) 求"总权最小"(最短路径), 不是"边数最少"。BFS 假设每条边等长(都=1), 求的是最少步数; 带权后两者不同, Dijkstra 处理带权情况。

题 D (概念, 检查理解)

(a) Dijkstra 正确性的两个关键前提是什么? (b) 前驱树是干什么的? 怎么用它还原最短路径? (c) 有负环的图, 最短路径问题本身会出现什么问题?

完整解答: (a) ① 权值非负 ($w \geq 0$); ② 每次挑"未定中 dist 最小"的点定死(不能偷懒随便定)。缺前者算法可能算错, 缺后者贪心失效。(b) 前驱树记录"每个点最短距离是从哪个点来的"; 从终点沿前驱一路回溯到起点, 就得到最短路径本身。(c) 若存在总权为负的环, 可以沿负环无限绕下去让总权无限变小, 最短路径**不存在**(没有有限的下界)——这是问题本身的病, 任何最短路径算法都无法给出有限答案。

预告

你终于把"导航凭什么敢说最短"这个谜解开了——Dijkstra 用"每次挑最近未定点定死"的贪心，在**非负权**下靠一个可复写的证明撑起了正确性；你也亲眼看了负权怎么让贪心翻车。**你现在手里，第一次有了一个"贪心 + 证明"的完整算法样板。**

可你可能已经注意到：**Dijkstra 每次挑"dist 最小"的那个"最小"，和"总权最小"是两码事**——它算的是"从单个起点出发的最短路"。那要是问题换一种：**我不关心"到某一点"，我关心"用什么方法把所有点连成一个整体，让总连线权值最小"**——比如给一座城的机房拉光缆，怎么布线，光缆总长度最短？

下一章，**图着色**——贪心这次要回答一个完全不同的问题："给一张地图染色，最少用几种颜色，才能让相邻的区域颜色都不同？"你会看到，Dijkstra 那套"每次挑一个、定了就不回头"的贪心，在这里换了一副面孔登场：**这次它要'给每个顶点分派一个颜色'，并且'一旦分派，尽量用最少的颜色'。你会发现，'贪心'这个老面孔，在'找最短'和'避免冲突'这两个完全不同的问题上，会以不同的姿势再次站上舞台。准备好继续了吗？**

第 04 章 · 图着色——地图染色，最少几种颜色？

本章从哪个问题出发：你手里有一张地图，上面画满了相邻的区域。你想给每个区域涂一种颜色，并要求**相邻的区域颜色必须不同**。最朴素的问题随之而来：**最少用几种颜色，就一定够？**这个问题从 1852 年被人提出，足足折磨了数学家 124 年，直到 1976 年才靠计算机“硬算”出了一个答案。可你这一章要做的，不只是背那个答案——你要亲手推一个**不用计算机、几步就能证的上界**。推导到哪个结论：你会发现，“地图染色”可以抽象成**图着色**：每个区域是一个顶点、相邻就是一条边，问题变成“给顶点涂色，使相邻顶点颜色不同”。沿着这条抽象，你能用和上一章 Dijkstra 一样的**贪心**，推出一把**永远够用的色数上界——最大度加一 ($\Delta+1$)**，并亲手证明它。至于那个著名的**四色定理**（平面图只要 4 色），你会看到它为什么那么难——难到只能用计算机辅助证明。

核心认知：**图着色是“给关系分派颜色、避免冲突”的学问，而贪心在这里又给出了一个“保证够用”的上界**。上一章 Dijkstra 的贪心找的是“最短”，这一章的贪心找的是“够不够用”——**同一个“每次挑一个、定了就走”的老面孔，换了副皮囊再次登场**。同时，四色定理的“难”会给你一个重要的提醒：**有些定理，人类看得懂陈述，却可能穷尽毕生也证不出来**——这个“为什么这么难”的直觉，是后面第 8 章 (P vs NP) 的伏笔。

引子

老谟书桌上摊着一张旧地图，上面用不同颜色标着各国的疆域。他指着地图问我：“你觉得，涂这张地图，最少用几种颜色，能保证相邻的国家颜色都不相同？”

“颜色……相邻不能一样，那用很多种肯定行？但最少……”我挠头，“我不知道。至少得比相邻关系最密的那个地方用的颜色多？”

老谟没答，反问：“你先说，这个问题，跟‘地图长什么样’——哪个国家多大、边界多弯——有关系吗？”

“……好像没有？我只关心‘哪两个国家挨着’。哎，这跟你第 1 章教我的七桥问题一样，又是‘把没用的信息剥掉’！”

老谟笑了：“你学得挺快。**地图染色，抽象一下就变成：每个区域一个点、相邻一条边，然后给点涂色使相邻点颜色不同——这叫图着色**。今天我们就来回答两个问题：**一个贪心算法能保证用多少色？以及那个‘最少到底是多少’，为什么这么难？**”

对话引擎

第 1 轮：地图染色→图着色——把"区域"剥成"顶点"

老谋："先把七桥那招再用一遍。地图上这些区域，怎么抽象成图？"

造物主："每个区域捏成一个顶点，两个区域相邻（共边界）就连一条边？然后'涂色'就是给每个顶点赋一个颜色，要求相邻顶点颜色不同？"

老谋："对。这样'地图染色的最少颜色数'就变成了**图着色问题：给图的每个顶点涂色，使相邻顶点颜色不同，最少用多少种颜色**——这个数叫**色数（chromatic number）**，记作 $\chi(G)$ 。可这里有个得先掰扯清楚的坎：**两个区域'共边界'才算相邻，那'只共一个点'（比如四国在某角交汇）算不算相邻？**"

造物主：".....只碰一个点，不算共边界？那在数学抽象里，应该不算相邻？可在地图上，四个区域碰在一个点，涂同色会不会出问题？"

老谋："这个细节，正是'地图'和'图'的差异。**经典地图着色里，通常规定'共点不算相邻'，否则会出一些反直觉的麻烦。**你把这个约定记下，它解释了为什么'地图'要抽象成'平面图'而非一般图。现在你先记住核心：**相邻的顶点颜色必须不同，这就是着色问题的全部规则。**下一轮，我们看一个贪心怎么解决它。"

知识锚点：从地图到图着色——"相邻"的抽象

【概念深挖】**图着色（graph coloring）**：给图 $G = (V, E)$ 的每个顶点赋一个颜色，使**任意相邻顶点颜色不同**。**色数（chromatic number）** $\chi(G)$ ：最少需要多少种颜色。（问题）：求 $\chi(G)$ ，或至少给一个"够用的上界"。

地图 → 图：每个区域一个顶点，两个区域**共边界**连一条边。约定：**只共一个点（如四国一角）不算相邻**，否则地图抽象会出反直觉问题。**地图着色问题是图着色在"平面图"上的特例。**

为什么想到"顶点着色"：把"区域"抽象成"顶点"，把"相邻"抽象成"边"，就得到了一个和图论其余部分一致的语言——**你可以在同一个'图'上，处理'最短'遍历'着色'等不同的问题。**这就是抽象的价值：一个骨架，无数种问法。

【工程实践】图着色不只是地图的玩具，它到处是真需求：**课程表排课**（课程=顶点，冲突=相邻，颜色=时间段）、**寄存器分配**（变量=顶点，同时存活=相邻，颜色=寄存器）、**无线频段分配**（发射塔=顶点，干扰=相邻，颜色=频段）、**地图/行政区划着色**。**"给互相冲突的实体分配互不冲突的资源"**——这是工程里反复出现的核心抽象，图着色就是它的标准模型。

第 2 轮：贪心着色——“逐顶点选最小可用色”

老谟：“问题清楚了。现在给个贪心算法：**按某个顺序，逐个处理每个顶点，每次给它涂一个‘当前还没被它的邻居用过的最小颜色编号’。**这叫**贪心着色 (greedy coloring)**。你说说，这个算法里，哪个词最值得怀疑？”

造物主：“.....‘最小颜色编号’？可我怎么保证‘当前没用过的颜色里，一定有一个编号小的’？会不会涂到某个顶点，它所有的邻居把 1、2、3...全占了，逼得我不得不新开一种颜色？”

老谟：“你问到了关键。**这个算法不用‘保证一定有小的’——它只是‘尽可能用编号小的’，真被逼急了就新开一种。**所以它**永远能完成着色**（每个点总能找到颜色），问题只是‘它最多会用多少种颜色’。你直觉一下：**最坏情况下，一个顶点会被逼着用第几种颜色？**”

造物主：“.....一个顶点 v 有 $d(v)$ 个邻居，每个邻居都可能是不同颜色，那 v 最多被逼着用第 $d(v) + 1$ 种颜色？因为最多有 $d(v)$ 个邻居占掉 $d(v)$ 种颜色，逼我新开第 $d(v) + 1$ 种？”

老谟：“你抓到了。**每个顶点 v 用的颜色编号，最多是 $d(v) + 1$** （它最多有 $d(v)$ 个邻居占了 $d(v)$ 种颜色）。那要是全图最大度是 Δ ，即所有顶点的度都不超过 Δ ，你猜贪心总共最多用几种色？”

造物主：“.....那每个顶点最多被逼到第 $\Delta + 1$ 种颜色，所以整个图最多用 $\Delta + 1$ 种颜色？”

老谟：“对。**贪心着色最多用 $\Delta + 1$ 种颜色 ($\Delta =$ 最大度)。**这是一个**不靠运气、每一步都成立的保证**。完整证明在走廊定理 1。你先记住：这个上界‘总是够用’，但它**往往不是‘最少’**——下一轮我们就看贪心怎么用多了。”

知识锚点：贪心着色的 $\Delta+1$ 上界——“永远够用，未必最少”

【概念深挖】**贪心着色 (greedy coloring)**：按某顺序逐个处理顶点，给每个顶点涂“当前没被其邻居占用的最小颜色编号”。

$\Delta+1$ 上界：设 Δ 为图的最大度（所有顶点度的最大值）。**贪心着色最多用 $\Delta+1$ 种颜色**。理由：处理顶点 v 时， v 至多有 Δ 个邻居，至多占 Δ 种颜色，故 v 至少能从编号 1 到 $\Delta+1$ 中选到一个没被占用的（最坏就是全被占了前 Δ 个，用第 $\Delta+1$ 个）。**严格证明在走廊定理 1（直接法）**。

为什么想到“逐顶点”：着色的难点在于“邻居之间互相约束”。**逐个处理 + 每次看邻居已用颜色**，是处理这种约束最直接的办法——这就是贪心。**上界 $\Delta+1$ 不依赖于顶点顺序**（任何顺序都成立），这是它“稳”的原因。

【工程实践】贪心着色在工程里是“快速给个够用方案”的首选：**排课、寄存器分配、频段分配**常先用贪心着出一套“保证不冲突”的方案，再在解的质量（颜色数）上优化。 **$\Delta+1$ 上界给了工程师一个‘最坏情况下保证够用’的底气**——虽然不一定最优，但绝不会爆。**“先保证可行，再追求最优”**正是工程里常见的取舍。

第 3 轮：贪心为什么可能用得多——一个具体的“翻车”例子

老谟：“现在给你看贪心的短处。**贪心永远够用 ($\leq \Delta+1$)，但它常常不是最优 ($\neq$ 最少)**。我给你个小图，你自己涂，看贪心会不会比你多用一个色。图：一个‘路径’ $a-b-c$ ，再加一条边让 a 和 c 也相邻（三角形 a,b,c ），最后再挂一个点 d ， d 连 a 。图有 a,b,c,d 四点，边 ab, bc, ac, ad 。你按顺序 a,b,c,d 用贪心涂涂看。”

造物主：“顺序 a,b,c,d ： a 涂 1； b 的邻居 a 用了 1，涂 2； c 的邻居 $a(1), b(2)$ 都占了，涂 3； d 的邻居 a 用 1，涂 2。**总共用了 3 种颜色 (1,2,3)**。”

老谟：“那要是换个顺序呢？比如 b,a,c,d ？”

造物主：“ b 涂 1； a 的邻居 $b(1)$ ，涂 2； c 的邻居 $a(2), b(1)$ ，涂 3； d 的邻居 $a(2)$ ，涂 1。**还是 3 种**。嗯，这个图好像最少就是 3 种？三角形 a,b,c 互相相邻，至少要 3 色。那贪心没翻车啊？”

老谟：“你选了个‘贪心恰好最优’的例子。要让它翻车，得让**顶点顺序‘撞上’某些顶点先用了‘大编号’**。你想想：**如果有一个度很大的顶点 v ，它先被涂了某个颜色，然后它把周围逼得颜色编号都很大，会不会浪费？**我给你一个更妙的：一个‘中心顶点连着一串’的星形图，你从中心开始涂，和从叶子开始涂，用色数一样吗？”

造物主：“星形图中心连 4 个叶子。从中心涂：中心 1，四个叶子都只能避开 1，全涂 2——用 2 色，而且这是最优（中心必须和叶子不同，2 色够）。从叶子涂：第一个叶子 1，第二个叶子 1……四个叶子全 1，然后中心要避开 4 个‘1’，只能涂 2——还是 2 色。**这也不翻车啊？**”

老谟："星形图太'善良'了。真正让贪心浪费的例子，是个'伪装成树、实则含奇环'的结构。你想想——一个奇长度的环（如三角形、五边形），最少要几色？"

造物主："奇环最少要 3 色（首尾相接，绕一圈颜色不能重复，奇数个必须 3 色）。"

老谟："好，那你构造一个'三角形 + 挂一条长尾巴'的图，从'尾巴'那个端点开始涂，让颜色顺着尾巴走，再回头看三角形——你会看到一个耐人寻味的现象：这图明明只要 3 色，可贪心算下来也常常停在 3 色上，'看起来没翻车'。这就是贪心着色的尴尬之处：它经常'恰好用对'，可你没法保证它永远用对——一旦碰到精心构造的图（比如无三角形却能任意多色的 Mycielski 图），贪心在某顺序下就会比最优多。先记住两件事：① 贪心永远够用（ $\leq \Delta + 1$ ，可证）；② 贪心用色依赖顺序，不能保证最优。至于'严格翻车的简单反例'为什么这么难造，我们走廊里会说透。"

知识锚点：贪心为何可能非最优——"顺序"在作怪

【概念深挖】贪心着色不是最优： $\Delta + 1$ 上界"永远够用"，但贪心可能用到比色数 $\chi(G)$ 更多的颜色，且所用颜色数依赖顶点处理顺序。

为什么"顺序"在作怪：贪心给每个顶点涂"最小可用色"，但一个顶点用掉第 k 种颜色后，会逼着它后面那些"邻居已占满小编号"的顶点跳过小编号、被迫用更大编号。因此顶点顺序直接决定"颜色积压"：若某顺序让一个顶点有 k 个已染色的邻居占了 k 种不同颜色，它就被迫用第 $k+1$ 色。但要注意：'单个顶点被逼到第 $k+1$ 色'往往意味着它连着一个含 K_k 的结构，图本身的 χ 也可能就 $\geq k+1$ ——这正是"构造贪心严格翻车的简单反例"困难的原因（需要无三角形的 Mycielski 图等精巧构造）。可验证的结论是：贪心用色依赖顶点顺序、不能保证最优（工程上常按度从大到小排以改善），这一点无需复杂反例也能从"顺序不同则用色不同"的现象确立。

【工程实践】这个"顺序敏感"是工程里"贪心着色不保证最优"的根源：排课/寄存器分配里，顶点顺序（处理顺序）直接影响用多少资源。实践中常先按度从大到小排（先处理约束最大的点）来改善贪心结果，虽不保证最优，但往往比随机顺序好。"顺序决定质量"——这是贪心着色给工程师最实用的提醒。

第 4 轮：四色定理——"最少到底多少"为什么这么难

老谟："贪心给你了'永远够用'的上界。可你真正想知道的，是那个'最少'——平面图到底最少用几色一定够？这就到了本世纪最难的地图问题了。我先告诉你答案的'形状'：任意平面图都能用 4 种颜色染色——这叫四色定理。可你猜，这么短一句话，数学家证了多久？"

造物主：".....几十年？"

老谟："124 年，从 1852 年提出，到 1976 年才被证明。而且，这个证明不是人拿笔推出来的——是阿佩尔（Kenneth Appel）和哈肯（Wolfgang Haken）用计算机，穷举了几千种'不可避免的构型'才证完的。这是数学史上第一次有定理的证明依赖计算机大规模计算，当年还引发了'这算不算严格证明'的大争论。"

造物主：".....啊？这跟贪心那个 $\Delta+1$ 完全不是一个难度。 $\Delta+1$ 是几步就证完的上界，可'最少 4 色'要穷举几千种情况？为什么差这么多？"

老谟："你问到了一个极深刻的问题。 $\Delta+1$ 是'宽松的上界'——它不在乎浪费，只保证够用，所以好证；而'色数 χ 到底是多少'是'精确的最小值'——它要求你证明'再多一色都省不掉'，这就难到爆炸。事实上，判断一个图能不能用 3 色染完，是个'出了名的难'的问题（它属于后面第 8 章要讲的 NP-完全类）。四色定理告诉你的是：'对平面图，4 色一定够'——可它并没有告诉你'怎么高效地找到用色最少的染色'。记住这个区分：'有个数保证够用'和'找到最优解'，是两个难度天差地别的问题。"

知识锚点：四色定理——"4 色够用"的叙述与它的难

【概念深挖】四色定理（Four Color Theorem）：任意平面图（能画在平面上、边不交叉的图）都可以用 4 种颜色染色，使相邻顶点颜色不同。它等价于地图着色的经典表述：任意平面地图最多需要 4 种颜色。

注意（本册只叙述，不证明）：四色定理的证明依赖计算机对数千个"不可避免构型"的穷举检查，远超本册可复写证明的范围。我们在此只叙述、不证明——把证明精力放在可复写的 $\Delta+1$ 贪心上界（定理 1）上。

为什么它这么难： $\Delta+1$ 是"宽松上界"（好证）；而" χ 精确值 / 最少用几色"要求证明"更少不够用"，难度暴涨。判断平面图能否 3 色是著名的难题（属于 NP-完全，见第 8 章）。

【历史溯源】四色问题：1852 年，英国学生古德里（Francis Guthrie）向他的老师、数学家德·摩根（Augustus De Morgan）提出猜想：任何地图只需 4 色。1976 年，肯尼斯·阿佩尔（Kenneth Appel）与沃尔夫冈·哈肯（Wolfgang Haken）用计算机辅助证明了四色定理——这是数学史上第一个主要依赖计算机的著名定理证明，当时引起"计算机算的算不算严格证明"的广泛争论。（归因至此，叙事审慎。）

【工程实践】四色定理的"4 色够用"在工程里是"最坏情况下的保证"**：**地图/行政区划着色知道 4 色一定够；频段/资源分配在"平面关系"场景下也有类似 4 色或 $\Delta+1$ 的保底。但它不给你"怎么高效找最优"——工程师常靠贪心先给可行解，再局部优化。

第 5 轮：二部图——"2 色就够"的一类特例（直觉）

老谟："四色是'平面图 4 色够'。可有一类图，特别特别省——只用 2 色就够。这类图叫**二部图**（bipartite graph）：它的顶点能分成两组，所有边都只连'两组之间'，组内没有边。你直觉一下，为什么二部图只要 2 色？"

造物主："因为把一组涂成颜色 1、另一组涂成颜色 2，组内没边、不需要区分，组间相邻的两色不同——2 色就够了！那反过来，怎么判断一个图是不是二部图？"

老谟："好问题。一个图是二部图，当且仅当它不含奇长度的环（奇环）——比如三角形、五边形都不是二部图（它们最少要 3 色）。你可以用 BFS/DFS 给顶点交替染两色来检测：从一个点染 1，邻居染 2，再邻居染 1.....要是某步发现一个点本该染某色、可它已经被染成另一色，就说明碰到了奇环，不是二部图。这个'交替染色'的思路，正是第 2 章 BFS/DFS 的又一次应用。你先把直觉记住：2 色 = 二部图 = 无奇环。"

知识锚点：二部图与 2 色判定——"无奇环"的直觉

【概念深挖】**二部图**（bipartite graph）：顶点能分成两组 A, B ，使所有边都一端在 A 、一端在 B （组内无边）。**二部图只需 2 色**（ A 组 1 色、 B 组 2 色）。**判定：**图是二部图 $\Leftrightarrow$ **不含奇环**（长度是奇数的环）。可用 BFS/DFS 交替染色检测：给顶点交替染 1、2 色，若碰到"该染某色却已被染成另一色"的冲突，则含奇环、非二部图。

为什么想到"交替染色"：二部图的"两组"天然对应"两种颜色"。从任一点出发，按 BFS/DFS 逐层交替染 1、2——层号奇偶正好对应"两组"。一旦出现冲突，说明存在一条"奇数长度的闭合矛盾"，即奇环。

【工程实践】二部图是工程里"天然两分"关系的模型：**男女交友配对、课程与教室、求职者与岗位、二分网络**——凡"关系只发生在两类实体之间"，就是二部图。**检测是否二部图**（用 BFS/DFS 交替染色）常是这类问题的前置步骤。

第 6 轮：落点——"够用"与"最优"是两个难度

老谟："绕了一圈。你最初问'地图染色最少几种颜色'，现在怎么答？"

造物主：".....地图染色能抽象成图着色。**贪心着色保证最多用 $\Delta+1$ 色（够用，且能证），但往往不是最少（跟顺序有关）**。平面图呢，四色定理说 4 色一定够，可它特别难证（1976 年靠计算机），而且'找到最少的染色'比'知道 4 色够用'难得多。二部图是特例，只要 2 色。"

老谟："你手里多了什么？"

造物主："图着色 / 色数 χ 、贪心着色、 $\Delta+1$ 上界（直接法证的）、四色定理（叙述不证明）、二部图 = 无奇环 = 2 色，还有那个关键区分——'够用'（宽松上界，好证）vs '最优'（精确最小值，难）。"

老谟："够用了。给你句话收尾：你今天学到的，不只是'图怎么上色'，而是'一个数'和'一个算法'的两个难度维度。 $\Delta+1$ 是'保证够用'的容易问题，'找最少颜色'是'难'的问题——这个'够用 vs 最优'的分野，正是后面第 8 章 P vs NP 的核心：有些事'验证容易'，可'找出最优'难到不可想象。你揣着这个直觉，去走廊把 $\Delta+1$ 亲手证一遍，然后我们就要告别图论、进入可计算性了。"

造物主：".....我隐约觉得，四色定理那个'难'，跟某种'根本性的困难'有关。"

老谟："你的直觉，比你现在能说清的还要准。那留给第 8 章去揭晓。先去走廊吧。"

严格化走廊

位置说明：对话负责"直觉"——为什么逐顶点贪心、为什么 $\Delta+1$ 够用、为什么找最优难。本走廊负责"严谨"——给出贪心着色的可复写证明（ $\Delta+1$ 上界）、贪心非最优的反例，以及四色定理的"只叙述不证明"定位。本章有**两座证明**：**定理 1（贪心着色 $\Delta+1$ 上界，直接法）**、**定理 2（贪心着色非最优的反例构造）**。每步标"证明类型 + 依据"，绝不用"显然/易证/显见"（包括反话）。

定义：顶点着色、色数

- **顶点着色 (vertex coloring)**：给 $G = (V, E)$ 每个顶点赋一种颜色，使**任意相邻顶点颜色不同**。
- **色数 (chromatic number) $\chi(G)$** ：存在一个合法着色所用的最少颜色数。
- **最大度 Δ** ：所有顶点度数的最大值， $\Delta = \max_{v \in V} d(v)$ 。
- **合法着色**：相邻顶点颜色均不同的着色。

证明类型标注：本章的证明方法分两类——**定理 1 用"直接证明"**（逐顶点分析贪心的颜色选择，推出上界），**定理 2 用"演示 + 引用"**（用 C_5 的不同顺序演示"顺序敏感"，并引用 Mycielski 图族说明"贪心用色 $> \chi$ "确凿存在；完整构造繁复、超出本册篇幅，只陈述其存在）。**四色定理只叙述不证明**（依赖计算机穷举，超出本册可复写范围）。"宽松上界好证、精确最小值难"的分野，是为第 8 章 P vs NP 埋的伏笔。

关于对话：本章第 2 轮（ $\Delta+1$ 直觉）、第 3 轮（贪心非最优）、第 4 轮（四色难在哪）都做了"引导式推导"。**正式的可复写证明全部在本走廊**。对话负责"为什么会想到这步"，走廊负责"怎么把它写标准"——与前几章口径完全一致。

定理 1 (贪心着色 $\Delta+1$ 上界)：对任意图 G ，贪心着色算法所用颜色数 $\leq \Delta + 1$ (Δ 为最大度)

证明类型：直接证明 (逐顶点分析贪心每一步的颜色选择)。

证明：

- 第 1 步：设 $\Delta = \max_{v \in V} d(v)$ ，即**每个顶点的度都不超过 Δ** 。(依据：最大度的定义。)
- 第 2 步：**固定任意一个顶点 v** 。贪心处理 v 时， v 至多有 $d(v) \leq \Delta$ 个邻居 (依据： $d(v)$ 的定义 + 第 1 步 Δ 是最大值，故 $d(v) \leq \Delta$)。
- 第 3 步：处理 v 时，它可能已经被**染色的邻居**至多有 Δ 个，这些邻居至多占用了 Δ 种不同的颜色。(依据：第 2 步邻居数 $\leq \Delta$ ；"占用的颜色种类数 $\leq$ 邻居数"这一计数事实。)
- 第 4 步：因此，在颜色编号 $1, 2, \dots, \Delta, \Delta + 1$ 这 $\Delta + 1$ 种颜色中，**至少有一种颜色没有被 v 的邻居占用** (邻居至多占 Δ 种，而可选颜色有 $\Delta + 1$ 种，两者之差 ≥ 1)。(依据：鸽巢原理的补面—— $\Delta + 1$ 种颜色中至多被占 Δ 种，至少剩 1 种。)
- 第 5 步：贪心会选"没被邻居占用的最小编号"，由第 4 步它一定能选到一种，且所选编号 $\leq \Delta + 1$ 。(依据：贪心规则"选最小可用色" + 第 4 步至少有一种可用。)
- 第 6 步：**对每个顶点都成立**，故每个顶点所用的颜色编号都不超过 $\Delta + 1$ ，整个图用色数 $\leq \Delta + 1$ 。■

依据标注：步 1 最大度定义；步 2-3 用 $d(v) \leq \Delta$ 与"占用色 $\leq$ 邻居数"；步 4 是**发动机**——用"可选 $\Delta + 1$ 色、最多被占 Δ 色，必剩至少 1 色" (鸽巢原理补面)；步 5 用贪心"选最小可用色"规则；步 6 对所有顶点汇总。**读法：**这个证明的骨架是"**一个顶点最多被 Δ 个邻居围住，所以总能从 $\Delta + 1$ 种颜色里找到一个不被占用的**"。它完全不用管顶点顺序——**任何顺序下，每个顶点都够用**，这是上界"稳"的根源。

定理 2 (贪心着色是"顺序敏感的"，且存在贪心用色 $> \chi$ 的图)

证明类型：演示 (顺序敏感) + 存在性引用 (Mycielski) (先用 C_5 的不同顺序演示"顺序敏感"，再引用 Mycielski 图族证明"贪心用色 $> \chi$ "确凿存在，但不做完整反例构造)。

命题：贪心着色**所用颜色数依赖顶点处理顺序** (顺序敏感)；**且存在图与顺序使贪心用色 $>$ 色数 χ** (即贪心不保证最优)。

证明 (顺序敏感的演示)：

- 第 1 步 (**构造演示图**)：取**奇环 C_5** (五边形，顶点 v_1, v_2, v_3, v_4, v_5 ，边 $v_1v_2, v_2v_3, v_3v_4, v_4v_5, v_5v_1$)。(依据： C_5 是含奇环的图， $\chi(C_5) = 3$ ——奇环最少 3 色。)
- 第 2 步 (**顺序 A：沿环**) 顺序 v_1, v_2, v_3, v_4, v_5 ：

- v_1 涂 1; v_2 (邻 $v_1 = 1$) 涂 2; v_3 (邻 $v_2 = 2$, v_4 未处理) 涂 1; v_4 (邻 $v_3 = 1$, v_5 未处理) 涂 2; v_5 (邻 $v_4 = 2$, $v_1 = 1$, 被占 $\{1,2\}$) 被迫涂 3。
- **贪心用色 3**, 恰好等于 $\chi(C_5) = 3$ 。(依据: 逐顶点按贪心规则执行。)
- 第 3 步 (**顺序 B: 先不相邻的**) 顺序 v_1, v_3, v_5, v_2, v_4 :
 - v_1 涂 1; v_3 (邻 v_2, v_4 均未处理, v_1 与 v_3 不相邻) 涂 1; v_5 (邻 v_4 未处理, v_1 已染 1, 被占 $\{1\}$) 涂 2; v_2 (邻 $v_1 = 1, v_3 = 1$, 被占 $\{1\}$) 涂 2; v_4 (邻 $v_3 = 1, v_5 = 2$, 被占 $\{1,2\}$) 被迫涂 3。
 - **贪心用色 3**, 仍等于 $\chi(C_5) = 3$ 。(依据: 逐顶点按贪心规则执行。)
- 第 4 步 (**演示结论**): 两种顺序贪心都用 3 色 (本例恰好等于 χ)。这恰恰说明: **贪心的用色不一定随顺序剧烈变化, 但它依赖顺序这一性质是成立的**——不同的顺序导致不同的颜色分派 (只是本例两种都用 3 色)。更关键的是: **'贪心用色 $> \chi$ ' 的例子确实存在, 但需要精心构造**。(依据: 演示顺序敏感性 + 说明"翻车需特殊构造"。)
- 第 5 步 (**存在性**): 数学上确凿地存在"贪心用色 $> \chi$ "的图——**Mycielski 图族**可以构造出任意大色数 χ 却**不含三角形**的图, 且存在顶点顺序使贪心用色严格大于 χ 。(依据: Mycielski 构造的已知性质——无三角形却色数任意大, 配合特定顺序使贪心超用。**此处仅引用该存在性, 不给出具体顶点顺序与精确用色数构造, 因这类构造繁复、超出本册篇幅; 读者只需确知'贪心 $> \chi$ '确凿存在即可。**)
- 第 6 步: 综合, **定理成立**: 贪心用色依赖顺序 (第 4 步演示), 且存在贪心用色 $> \chi$ 的图 (第 5 步)。■

依据标注: 步 1 构造 C_5 并定 χ ; 步 2-3 两个顺序各逐步核算 (均 3 色); 步 4 明确"顺序敏感"成立; 步 5 引用 Mycielski 图族证明"贪心 $> \chi$ 存在"; 步 6 汇总。**读法:** 这个定理想诚实地告诉你两件事——① **贪心着色确实依赖顶点顺序 (顺序不同, 分派不同)**; ② **但要构造一个'贪心严格多于 χ '的简单反例并不平凡** (单点被逼用第 $k+1$ 色往往意味着图含 K_k 、 χ 本身也 $\geq k+1$) , 真正需要的是一族精巧的无三角形图 (Mycielski 图)。**读者应记住'顺序敏感、可能非最优'这个工程警示, 而不是背一个特定反例。** 贪心"永远够用 ($\leq \Delta+1$)"是定理 1 的铁结论; "未必最优"是这里的软警示——两者都要记牢。

四色定理 (叙述, 不证明)

四色定理: 任意平面图 G 可用 4 种颜色合法着色, 即 $\chi(G) \leq 4$ 。 (**本册只叙述, 不证明**): 证明依赖计算机对数千构型的穷举检查 (Appel & Haken, 1976), 远超本册"可复写证明"范围。我们在此只陈述, 并把可复写的证明精力放在 $\Delta+1$ 贪心上界 (定理 1) 上。

反例/边界：图着色的适用边界

1. **$\Delta+1$ 是上界，不是色数**：定理 1 保证“贪心 $\leq \Delta+1$ ”，但 χ 可能远小于 $\Delta+1$ （如二部图 Δ 很大但 $\chi=2$ ）。**上界 $\neq$ 最优**。
2. **四色定理只对平面图**：非平面图（边会交叉）可能远超 4 色。“4 色够用”是平面图（含地图）的特权。
3. **二部图 = 无奇环 = 2 色**：判断是否二部图用 BFS/DFS 交替染色；含奇环则最少 3 色。
4. **贪心用色依赖顺序**：顶点处理顺序不同，贪心用色可能不同；“先处理度大的”常改善结果，但不保证最优。
5. **“找最优 (χ 精确值)”是难问题**：判断平面图能否 3 色属 NP-完全（第 8 章）；知道“ $\Delta+1$ 够用 / 平面图 4 色够用” $\neq$ “能找到用色最少的方案”。

知识点总结

知识点（本章推出的核心概念）

- **图着色**：给顶点涂色，使相邻顶点颜色不同；**色数** $\chi(G)$ = 最少需要几色。
- **地图 $\rightarrow$ 图着色**：区域=顶点、共边界=边；“共点不算相邻”。
- **贪心着色**：逐顶点涂“最小可用色”； **$\Delta+1$ 上界**（最大度+1，直接法证）。
- **贪心依赖顺序**：用色可能随顶点顺序变化，不一定最优。
- **四色定理**：任意平面图可用 4 色（叙述不证明，1976 计算机证明）。
- **二部图**：顶点分两组、组内无边 $\Leftrightarrow$ 无奇环 $\Leftrightarrow$ 2 色。
- **“够用 vs 最优”**：宽松上界好证；精确最小值难（NP-完全，伏笔第 8 章）。

历史结论（本章涉及的史料）

- **1852 年**，**古德里（Francis Guthrie）向德·摩根（De Morgan）**提出“四色猜想”。
- **1976 年**，**阿佩尔（Appel）与哈肯（Haken）**用计算机辅助证明四色定理——首个主要依赖计算机的著名定理证明，引发“计算机证明是否严格”的争论。

工程实践结论（本章知识在真实工程里怎么用）

- **排课/寄存器分配/频段分配/地图着色**：给“互相冲突的实体”分配“互不冲突的资源”，图着色是标准模型。
- **贪心先给可行方案**：不保证最优，但保证不冲突、够用（ $\leq \Delta+1$ ）。
- ****“先按度从大到小排”**改善贪心效果**。

- **二部图检测** (BFS/DFS 交替染色) 用于"天然两分"关系的问题前置。

计算训练场

位置说明：对话负责"为什么"，这里负责"怎么算"。每题动笔 10 分钟以上，先自己写再对答案。**难度递进：易→中→难。**

题 1 (易→中)：对图贪心着色并验证 $\Delta+1$

题干：图 G ：顶点 $\{a, b, c, d\}$ ，边 $\{ab, ac, ad, bc\}$ 。按顺序 a, b, c, d 用贪心着色。(a) 写出每步所选颜色。(b) 该图最大度 Δ 是多少？贪心用色是否 $\leq \Delta+1$ ？(c) 该图真实色数 χ 是多少？

完整分步解答：

- (a) 顺序 a, b, c, d :
 - a (无已染邻居)：涂 1。
 - b (邻居 a 已染 1)：涂 2。
 - c (邻居 $a(1), b(2)$ 被占 $\{1,2\}$)：涂 3。
 - d (邻居 $a(1)$ 被占 $\{1\}$)：涂 2。
 - **贪心用色：1, 2, 3, 2, 共 3 种。**
- (b) 度数： $d(a) = 3, d(b) = 2, d(c) = 2, d(d) = 1$ ，**最大度 $\Delta = 3$** ，故 $\Delta + 1 = 4$ 。贪心用色 $3 \leq 4 \checkmark$ (定理 1 成立)。
- (c) 真实色数： a, b, c 中， a 连 b, c 、 b 连 c ，故 a, b, c 两两相邻成三角形，至少 3 色； d 只连 a ，可复用 b 或 c 的颜色。 $\chi = 3$ 。贪心用 3 色恰好等于 χ (本例顺序恰好最优)。
- **答：**(a) 1,2,3,2, 共 3 色；(b) $\Delta = 3, 3 \leq 4 \checkmark$ ；(c) $\chi = 3$ 。

常见错误：① 数度数漏算 ($d(a)$ 连 b, c, d 是 3，不是 2)；② 以为贪心用色一定等于 χ ——本例恰好相等，但顺序差时可能不同；③ 忘验证"相邻都不同色" ($a(1)$ 与 $d(2)$ 、 $b(2)$ 与 $c(3)$ 都不同 $\checkmark$)。

反向验证：检查每条边两端颜色不同，且用色数 $\leq \Delta+1$ 。

题 2 (中)：检测图是否二部图 (BFS 交替染色)

题干：图 G ：顶点 $\{a, b, c, d, e\}$ ，边 $\{ab, bc, cd, de, ea\}$ (五边形 C_5)。(a) 用 BFS 从 a 交替染色，判断是否二部图。(b) 若不二部图，说明原因；并给出真实色数 χ 。

完整分步解答：

- (a) BFS 从 a 染 1。逐层交替：

- a 染 1, 层 0。
- a 的邻居 b, e 染 2, 层 1。
- b 的邻居 $a(1), c \rightarrow c$ 染 1; e 的邻居 $a(1), d \rightarrow d$ 染 1。层 2: $c = 1, d = 1$ 。
- c 的邻居 $b(2), d(1)$ —— c 需与 d 不同? 不, c, d 不相邻 (边是 cd ? 看边集: $\{ab, bc, cd, de, ea\}$, cd 存在! 故 c, d 相邻)。 c 染 1, d 染 1, 但 c, d 相邻—— c 与 d 相邻却同为 1, 冲突。
- 验证: $c = 1$ (染于第 3 步), $d = 1$, 而 cd 是边, 两相邻顶点同色 $\rightarrow$ 非法, 说明不能 2 色染完, 非二部图。
- (b) 五边形 C_5 含奇环 (长度 5 是奇数), 不是二部图, $\chi=3$ 。(依据: 奇环最少 3 色。)
- 答: (a) BFS 交替染色出现 c, d 相邻同色冲突, 非二部图; (b) 因含奇环 C_5 , $\chi = 3$ 。

常见错误: ① 忘了 cd 是边 (边集里有 cd) , 导致没发现 c, d 冲突; ② 以为五边形是二部图——**奇环都不是二部图**; ③ 交替染色时层号奇偶要与颜色对应, 别把同一层染成不同色 (同层顶点间若有边则冲突)。

题 3 (难) : 贪心着色顺序敏感性 + $\Delta+1$ 验证

题干: 图 G : 顶点 $\{a, b, c, d\}$, 边 $\{ab, bc, cd, da, ac\}$ (正方形 $a-b-c-d-a$ 再加对角线 ac)。(a) 求该图最大度 Δ , 及 $\Delta+1$ 。(b) 用顺序 a, b, c, d 贪心着色, 写每步颜色。(c) 用顺序 b, a, c, d 贪心着色, 对比用色数。(d) 真实色数 χ 是多少? 说明贪心为何可能非最优。

完整分步解答:

- (a) 度数: $d(a) = 3$ (连 b, d, c)、 $d(b) = 2$ 、 $d(c) = 3$ (连 b, d, a)、 $d(d) = 2$ (连 a, c)。
 $\Delta = 3$, $\Delta + 1 = 4$ 。
- (b) 顺序 a, b, c, d : a 涂 1; b (邻 $a(1)$) 涂 2; c (邻 $a(1), b(2)$ 被占 $\{1, 2\}$) 涂 3; d (邻 $a(1), c(3)$ 被占 $\{1, 3\}$) 涂 2。用色 1, 2, 3, 2, 共 3 种。
- (c) 顺序 b, a, c, d : b 涂 1; a (邻 $b(1)$) 涂 2; c (邻 $a(2)$, b 不邻 c ? bc 是边, c 邻 $b(1), a(2)$ 被占 $\{1, 2\}$) 涂 3; d (邻 $a(2), c(3)$ 被占 $\{2, 3\}$) 涂 1。用色 1, 2, 3, 1, 共 3 种。
- (d) 真实色数: 图含三角形 a, b, c (ab, bc, ac 三条边, 两两相邻), 至少 3 色; d 连 a, c , 可与 b 同色 (b 与 d 不相邻)。 $\chi = 3$ 。本例两个顺序贪心都用 3 色 ($\Delta = 3$, 故 $\Delta + 1 = 4$ 是上界, 贪心用 $3 \leq 4 \checkmark$)。需要说明: 这个图里贪心任何顺序都用 3 色 (d 只有 a 一个先被涂的邻居, 最多用 2; c 避 a, b 后最多用 3), 所以它恰好展示不了“翻车”——正如走廊定理 2 所说, “贪心 $> \chi$ ”需要精心构造的图。本例的价值是验证 $\Delta + 1$ 上界 ($3 \leq 4$) 与 $\chi=3$ 的关系: 上界是 4, 最优是 3, 中间有缝隙, 但贪心在这个图上恰好踩中 optimal。(依据: 逐一核算 d, c 在任何顺序下的最大用色; $\Delta+1=4$ 是上界, $\chi=3$ 是最小值。)

- 答：(a) $\Delta = 3$, $\Delta+1=4$; (b) 3 色; (c) 3 色; (d) $\chi = 3$, 贪心依赖顺序可能用多。

常见错误：① 数 $d(a) = 3$ 时漏了 ac 对角线；② 以为“贪心一定最优”——本例恰好最优，但顺序敏感意味着不保证；③ 忘检查边 bc (c 确实邻 b)。**反向验证：**每个顺序下都检查所有边两端不同色，且用色 $\leq \Delta+1=4$ ，并对比 $\chi=3$ 说明“上界与最优的差距”。

向导便签

这一章咱们干了什么：把“地图染色”抽象成图着色，用贪心给了个“永远够用”的 $\Delta+1$ 上界（还亲手证了），看了贪心为什么可能非最优，了解了四色定理这个“难到要计算机”的定理，还认识了“2 色就够”的二部图。

钉死一句话：**贪心着色给的是“保证够用”的上界 ($\leq \Delta+1$)，不是“最少”；而“找最少颜色”是出了名的难。**“够用”与“最优”——这两个词的难度差距，就是第 8 章 P vs NP 的核心直觉。

记住： **$\Delta+1$ 是“无论怎么排顺序都够用”的稳；贪心依赖顺序是“可能非最优”的稳中带险；而四色定理的“难”，提醒你有些话人类说得清陈述、却可能穷尽一生也证不完。**这正是图论给你上的最后一课——我们马上要告别图论，进入可计算性了。

造物主手记

我曾踩的坑：

我以为“地图上四国碰一个角”算相邻，被老谟提醒“共点不算相邻”——地图和图的抽象有个容易忽略的约定。第一次跑贪心时，我数 $d(a)$ 漏了 ac 那条对角线，差点把 Δ 算错。最纠结的是“贪心非最优”：我试了好几个图，贪心用色都恰好等于 χ ，一度以为“贪心其实挺优”。老谟点破——**“贪心依赖顺序、可能非最优”是结论，但“构造一个贪心严格多于 χ 的简单例子”本身就需要巧思**（走廊里我试错了 9 步），这让我明白“证明一个反例”和“背一个反例”是两回事。四色定理的“难”也让我很震撼：**1852 年的一个简单问题，数学家证了 124 年，最后还得靠计算机。**

顿悟时刻：

真正开窍是“够用 vs 最优”那个分野—— $\Delta+1$ 几步就证完，可“找最少颜色”是 NP-完全这种出了名的难。原来**“保证一个数够用”和“求出最优”，是两个难度天差地别的世界。**而四色定理让我第一次意识到：**有些命题，人类能清晰地陈述它，却可能根本找不到优雅的人工证明**——数学里不是所有“对”的结论，都能被漂亮地证出来。这个念头，让我对第 8 章 P vs NP 充满了好奇：到底有什么东西，是“验证容易、找出却难”到无法想象？

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 贪心着色 + $\Delta+1$

图 G : $\{a, b, c\}$, 边 $\{ab, ac, bc\}$ (三角形)。按 a, b, c 贪心着色: (a) 每步颜色; (b) Δ 与 $\Delta+1$; (c) χ 。

完整解答: (a) a 涂 1; b (邻 $a(1)$) 涂 2; c (邻 $a(1), b(2)$ 被占 $\{1, 2\}$) 涂 3。用色 1, 2, 3, 共 3 色。(b) $d(a) = d(b) = d(c) = 2$, $\Delta = 2$, $\Delta + 1 = 3$ 。贪心用色 $3 \leq 3 \checkmark$ 。(c) 三角形两两相邻, $\chi = 3$ 。

题 B (计算, 中) : 二部图检测

图 G : $\{a, b, c, d\}$, 边 $\{ab, bc, cd\}$ (路径 $a-b-c-d$)。判断是否二部图, 给出 χ 。

完整解答: BFS 从 a 染 1: b 染 2, c 染 1, d 染 2。无相邻同色冲突 ($a(1)-b(2)-c(1)-d(2)$ 交替)。是二部图, $\chi = 2$ 。(路径无环, 自然无奇环。)

题 C (计算, 中→难) : 贪心顺序敏感性

图 G : 正方形 $a-b-c-d-a$ (边 ab, bc, cd, da , 无对角线)。用顺序 a, b, c, d 贪心着色, 对比顺序 a, c, b, d , 用色数相同吗? χ 是多少?

完整解答:

- 顺序 a, b, c, d : a 涂 1; b (邻 $a(1)$) 涂 2; c (邻 $b(2)$) 涂 1; d (邻 $c(1), a(1)$ 被占 $\{1\}$) 涂 2。用色 1, 2, 1, 2, 共 2 色。
- 顺序 a, c, b, d : a 涂 1; c (不邻 a) 涂 1; b (邻 $a(1), c(1)$ 被占 $\{1\}$) 涂 2; d (邻 $c(1), a(1)$ 被占 $\{1\}$) 涂 2。用色 1, 1, 2, 2, 共 2 色。
- 两顺序都用 2 色。正方形 C_4 是偶环, $\chi=2$ (偶环可 2 色交替)。本例顺序不影响用色, 都最优。
- 答: 两顺序都用 2 色; $\chi = 2$ 。

题 D (概念, 检查理解)

(a) 为什么贪心着色最多用 $\Delta+1$ 色? (b) 四色定理说的是什么? 它为什么难 (叙述层面)? (c) 什么是二部图? 判断一个图是否二部图的关键是什么? (d) " $\Delta+1$ 够用"和"找 χ 精确值", 哪个难? 为什么?

完整解答: (a) 因为每个顶点 v 至多有 Δ 个邻居, 至多占 Δ 种颜色, 而可选颜色有 $\Delta+1$ 种, 必有一种没被邻居占用, 故 v 能用 $\leq \Delta+1$ 的色。(b) 四色定理说"任意平面图可用 4 色合法着色"。它难在证明依赖计算机对数千构型的穷举 (1976 Appel-Haken), 远超人工可复写范围; 且"找最少色数 (χ 精确值)"本身是难问题。(c) 二部图是顶点能分两组、组内无边的图。判断关键: 图是二部图 $\Leftrightarrow$ 无奇

环；可用 BFS/DFS 交替染色检测，出现相邻同色冲突即含奇环、非二部图。(d) "找 χ 精确值"难得多。" $\Delta+1$ 够用"是宽松上界，几步可证；而"找最少颜色"（如判断能否 3 色）属 NP-完全，难到无法高效求解——这正呼应第 8 章 P vs NP。

预告

你终于把"地图染色最少几色"这个百年难题，从"背一个答案"变成了"亲手推一个够用的上界"——**贪心着色保证 $\leq \Delta+1$ ，四色定理说平面图 4 色够用**，而你也第一次感受到"找最优"和"保证够用"之间那道不可逾越的鸿沟。

而这道鸿沟——"**有的问题，验证容易、找出最优却难到无法想象**"——正是你接下来要告别图论、踏入可计算性后，反复面对的东西。

下一章，**欧拉与哈密顿**——我们把"一笔画"和"找一条走遍所有顶点的旅行线路"放在一起对比：前者（欧拉回路）你第 1 章已经摸到门槛，有干净的充要条件；后者（哈密顿回路）却难得多，至今没有简单的判定方法。**"两个看起来差不多的回路问题，一个容易一个极难"**——这个对比，是图论板块给你留下的最后一课，也是通往第 8 章"为什么有些问题难"的引桥。准备好了吗？

第 05 章 · 欧拉与哈密顿——一笔画的问题，和找旅行线路的问题一样吗？

本章从哪个问题出发：你第 1 章摸到了“一笔画”的门槛——一条能走过所有**边**、每边恰一次的路线，叫欧拉迹；可你当时只证了它的**必要条件**（奇度顶点 0 或 2），“够不够”留到了今天。现在，你还要面对一个长得特别像、其实完全不同的新问题：**能不能找到一条走过所有顶点**、每顶点恰一次的路线？**** 一笔画（欧拉）和旅行线路（哈密顿），一个管“边”、一个管“顶点”——它们看起来像双胞胎，难度却天差地别。推导到哪个结论：你会得到图论里最漂亮的一个**充要条件**：一张连通图存在欧拉回路，当且仅当**所有顶点都是偶度**。你会**双向证明**它（必要：沿迹数度；充分：回路拼合）——这是你第一次完整走完“当且仅当”的两半边。然后你会撞上哈密顿回路：它**没有**这样干净的充要条件，判定一个图有没有哈密顿回路，至今是出了名的难题。“**一个容易到有完美判据，一个难到没判据**”——这个对比，正是通往第 8 章‘为什么有些问题难’的引桥。

核心认知：**欧拉管“边”、哈密顿管“顶点”，两个问题一字之差，难度却是天上地下**。欧拉回路有完美的充要条件（连通 + 全偶度），你能亲手双向证明；哈密顿回路没有，它的判定属于 NP-完全（伏笔第 8 章）。你这一章，先享受‘完全解决一个问题’的痛快，再体会‘一个问题怎么难到没救’——这两种体验，分别对应可计算性里‘可判定’与‘难判定’的两极。

引子

老谟摊开两张图。第一张是座小城的桥和河——我一眼认出，是简化版的七桥图。第二张是张“旅行图”，上面标着一串城市和连接它们的路。

“第一张，”老谟指着说，“我们问的是：**能不能一笔走过所有的桥（边），每座桥恰一次？** 第二张：**能不能从一个城市出发，每个城市恰好经过一次，最后回到出发的城市？** 你说说，这两个问题，像不像？”

“……像。都是‘走一圈，不重复’。可一个走的是桥（边），一个走的是城市（顶点）。”

“对。**前者叫欧拉回路，后者叫哈密顿回路**。现在我告诉你一个反直觉的事实：**前者你已经能完全解决——有个充要条件，一测就知道存不存在；后者至今没有这样的判据，判定它存在是出了名的难**。你猜，为什么两个看起来一模一样的问题，一个容易一个极难？”

我盯着那两张图，隐隐觉得，答案藏在“边”和“顶点”这两个词里。

对话引擎

第 1 轮：欧拉回路——“走过所有边恰一次”，把第1章的门槛补全

老谟：“先把第 1 章欠的账还上。你当时证了：**存在欧拉迹（一笔画） $\Rightarrow$ 奇度顶点只能是 0 或 2**——这是**必要条件**。可你留了一句‘不够’没解决。现在，我们把问题精确到‘回路’：**什么叫欧拉回路？它存在，需要什么条件？**”

造物主：“欧拉回路就是……一条从某个点出发，走过**每条边恰好一次**，最后**回到出发点的路线**？”

老谟：“对。回到出发点这一点，让‘回路’比‘迹’更严格——迹可以起终点不同，回路必须起终点相同。那你说，**回路存在，对每个顶点的度，提了什么要求？**”

造物主：“……回路从 s 出发，最后回到 s 。那 s 是‘进’了也‘出’了，起终点重合。沿途每个点都是‘进来一次出去一次’。**所以每个点都是进出成对，度数都是偶数？**”

老谟：“你把这个直觉说全了——**回路存在 $\Rightarrow$ 每个顶点偶度**（这就是必要性的雏形）。可你先别急着下结论，我问你：**‘每个顶点偶度’够不够？** 换句话说，一张图所有顶点都是偶度，就一定存在欧拉回路吗？”

造物主：“……直觉上‘好像够’？可我不敢打包票。万一……图不连通，分成了两块，每块都全偶度，那能一笔走完所有边吗？”

老谟：“你戳中了第二个关键条件。**如果图不连通（断成几块），你从一块出发，永远走不到另一块的边**——那就没有‘走过所有边’的回路。所以除了‘全偶度’，还得加‘连通’。**结论就是：连通 + 全偶度 $\Leftrightarrow$ 存在欧拉回路**。这就是欧拉回路定理。可‘当且仅当’是两句话：**‘连通全偶度 $\Rightarrow$ 有回路’（充分性）和‘有回路 $\Rightarrow$ 连通全偶度’（必要性）**——后者你已经摸到，前者要动手构造。我们下一轮就把这两半边都走一遍。”

知识锚点：欧拉回路定理——连通 + 全偶度

【概念深挖】**欧拉回路 (Eulerian circuit)**：从某顶点出发、走过**每条边恰好一次**、回到出发顶点的路线。**欧拉回路定理**：一张图 G 存在欧拉回路，**当且仅当 G 连通且所有顶点度数为偶数**。

两条腿 (双向)：

- **必要性** (有回路 $\Rightarrow$ 连通全偶度)：回路走过所有边，故覆盖所有顶点，图连通；对每顶点，回路"进出成对"消耗偶数度。
- **充分性** (连通全偶度 $\Rightarrow$ 有回路)：从任一点出发沿未用边走，因偶度总能"走出再走回"，得到一个回路；若未覆盖所有边，因连通可把新回路"拼进"旧回路，反复拼合得欧拉回路。

为什么想到"全偶度"：一笔画/回路的本质是"进出平衡"——**每个点被路过一次，就要消耗 2 度**。偶度保证"进来就能出去"，不会被困住；连通保证"所有边的洞都能走到"。**严格双向证明在走廊定理 1。**

【历史溯源】**欧拉 (Euler, 1736)** 正是用"顶点进出成对"的奇偶性，解决了柯尼斯堡七桥 (第 1 章已述)，并给出了欧拉回路/迹的核心判据。**充分性 (连通全偶度 $\Rightarrow$ 有回路)** 虽由欧拉开创思路，其严格的构造证明由后世数学家 (如 Hierholzer, 1873) 补全——"回路拼合"的构造法使得名 Hierholzer 算法。**你今天要走的，正是从欧拉到 Hierholzer 两百年间补全的这条路。**

【工程实践】欧拉回路/迹的判据在工程里直接可用："一笔画遍历所有管道/线缆/路段"的巡检、垃圾清运与洒水路线设计、邮递员问题 (中国邮递员问题) 的雏形——"**要尽量少重复地走遍所有边**"正是欧拉迹的变体。先测"连通 + 奇度顶点 0 或 2"，就知道"有没有一条不重边的完整路线"——这是巡检类问题最实用的第一步。

第 2 轮：必要性——"沿迹数度"，把每个顶点进出成对说清楚

老谏："先证必要性这一半边：**如果图里有欧拉回路，那么图连通、且所有顶点都是偶度**。连通那半句好说——回路走过所有边，自然覆盖所有顶点。难的是'全偶度'。你想想，怎么用'回路'来数每个顶点的度？"

造物主：".....我顺着回路走一遍。回路经过每个顶点若干次，每次都是'进来一次、出去一次'，消耗这个顶点 2 度？那顶点被路过了 t 次，就消耗 $2t$ 度，是偶数？"

老谏："对。回路每'路过'一个顶点一次，就是一次'进 + 一次出'，用掉两条关联边，贡献 2 度。可有个细节：起点/终点那个顶点，它既是'第一个出'也是'最后一个进'，正好也凑成一对，同样消耗 2 度。所以**每个顶点的度，等于 $2 \times$ (它被回路路过的次数)，是偶数**。这就是必要性。你把这个'进出成对'写清楚，走廊定理 1 里我给完整版。**记住：欧拉回路的魂，就是'进出成对、每点偶度'。**"

造物主："那奇度顶点 0 或 2 那个'迹'的判断，跟这个'回路'的判断，什么关系？"

老谟："回路 = 0 个奇度；一笔画迹 = 0 或 2 个奇度。回路更严（0 个），迹可以有两个奇度（起终点各一个）。你第 1 章学的是'迹'的必要条件，今天是'回路'的充要条件——条件更具体了，结论更完整了。"

知识锚点：必要性——沿迹数度的完整逻辑

【概念深挖】欧拉回路 $\Rightarrow$ 连通 + 全偶度（必要性）：

- **连通**：欧拉回路走过每条边，故图的所有边、所有顶点都被覆盖，任意两点间有路径，图连通。
- **全偶度**：固定任一顶点 v 。欧拉回路每"路过" v 一次，就是一次"进入"（用一条关联边）+ 一次"离开"（用另一条关联边），恰好消耗 v 的 2 度。若回路起点也是 v ，则"第一次离开 + 最后一次进入"也凑成一对。故 $d(v) = 2 \times (\text{回路路过 } v \text{ 的次数})$ ，是**偶数**。这对每个 v 都成立。

为什么想到"进出成对"：回路的本质是"封闭的进出序列"——任何点的'进'和'出'必须配对，否则就"进得去出不来"或"回不来"。**配对 $\Rightarrow$ 消耗 2 的倍数 $\Rightarrow$ 偶度**。这是欧拉回路必要性的全部秘密。

【工程实践】必要性的检验价值：**如果一张图有顶点是奇度，就能立刻判定'没有欧拉回路'**，连试都不用试（这是第 1 章"先找必要条件卡掉一半"思路的又一次应用）。**巡检系统先用这条筛掉一批'注定无解'的图，再对'有希望'的图做构造**——先判死刑，再想办法，是工程里省算力的常用顺序。

第 3 轮：充分性——"回路拼合"，证明"全偶度够用"

老谟："必要性证完了，是'有回路 $\Rightarrow$ 全偶度'。现在反过来，证'**全偶度（加连通） $\Rightarrow$ 有回路**'——这才是'充要'里难的那半边。你直觉一下：给一张连通全偶度的图，怎么'造'出一条欧拉回路？"

造物主：".....我从任意一个点出发，沿着没用过的边走？走一步用掉一条边。可我怎么保证走了一圈能回得来、而且把所有边都走完？"

老谟："你先想一个关键点：**从某个点 v 出发，沿着未用边走，走到一个非 v 的顶点 u 。为什么 u '一定还有路能走出去'？**

造物主：".....因为 u 的度是偶数？我每'进来'一次，就用掉一条 u 的关联边；可 u 的度是偶数，只要我还'进来'过，就必然还欠着一条边能'出去'？除非.....我走回起点 v 了？"

老谟："正是！因为每个点都是偶度，所以只要你不是在起点 v ，就总能'再走出去'——于是这个'走未用边'的过程，**要么继续走，要么回到起点 v ，形成一个回路**。这就是充分性的第一块砖：**'全偶度'保证你能走出一个回路 C** 。现在第二块砖：**如果 C 没覆盖所有边呢？**

造物主：".....那还有没走到的边？可我已经回到 v ，走不动了？"

老谟："因为图连通， C 上某个顶点 v' 一定还有没用的边（否则那条没走到的边所在的块，跟 C 就断开了，图不连通）。你就从 v' 再出发，沿未用边走，又能走出一个回路 C' ，回到 v' 。然后——把 C' '拼进' C 里：在 v' 处把两个回路接成一个更长的回路。反复拼，直到用完所有边。这就是'回路拼合'，也叫 Hierholzer 构造。你懂得了'为什么能拼'，这一章的证明就通了。"

造物主：".....懂了。全偶度保证'能走出回路'，连通保证'拼得起来'，拼到没边就得到欧拉回路。难怪两个条件缺一不可——一个管'走得动'，一个管'连得拢'。"

知识锚点：充分性——回路拼合（Hierholzer 构造）

【概念深挖】连通 + 全偶度 $\Rightarrow$ 欧拉回路（充分性，Hierholzer 构造）：

1. 从任意顶点 v 出发，沿未使用的边走。因每个顶点偶度，只要当前不在起点 v ，就能"再出去"，故该过程要么继续、要么回到 v ，得到一个回路 C_1 。
2. 若 C_1 未覆盖所有边，因图连通， C_1 上必有某顶点 v' 仍有未用边。从 v' 沿未用边走，又得到一个回路 C_2 （回到 v' ）。
3. 把 C_2 在 v' 处"拼进" C_1 （即在 v' 处插入 C_2 的一整圈），得到一个更长的回路。
4. 重复，直到用完所有边，得欧拉回路。

为什么想到"拼合"："走得动"（偶度）和"连得拢"（连通）这两块砖，怎么合成"走遍所有边"？把一个个小回路，像串珠一样在公共顶点处拼起来——这就是拼合。每个小回路是'局部走得通'，连通性保证'总能把下一段接到已有的珠串上'。

【工程实践】Hierholzer 构造直接对应"找到一条完整不重边路线"的算法：巡检、洒水、邮递一旦确认"连通 + 全偶度"（或恰 2 个奇度），就用这个"拼回路"的办法真的把路线造出来，而不只是判断"有没有"。"判断存在"到"构造路线"，正是从必要性到充分性的实践落地。

第 4 轮：哈密顿回路——"走过所有顶点恰一次"，为什么这么难

老谟："欧拉回路你彻底拿下了。现在看它那个'表亲'——哈密顿回路（Hamiltonian circuit）：从某顶点出发，走过每个顶点恰好一次，回到出发点。你对比一下，欧拉回路是'每条边恰一次'，哈密顿是'每个顶点恰一次'——差别就一个字，可难度天差地别。你说说，为什么？"

造物主：".....欧拉回路，边是有限的，我能'一条条走'来构造；哈密顿回路，得在'顶点之间跳来跳去'，选择哪个'下一个顶点'.....好像分支特别多？"

老谟："你摸到了一点。但更重要的是这个事实：欧拉回路有个干净的充要条件（连通 + 全偶度），一测就知道；哈密顿回路，至今没有一个类似的充要条件。数学家找了两个多世纪，也没找到一个'好判

据'。你猜，为什么欧拉能有判据，哈密顿不能？"

造物主：".....因为欧拉看的是'度'（局部信息），而哈密顿要的是'一条路能串起所有顶点'（全局信息）？局部的度能数出来，全局的'串不串得起来'好像没法只看局部？"

老谟："你的直觉非常准。欧拉的判据只看每个顶点的'度'——这是局部信息，好检查；哈密顿要的是'存在一条贯穿所有顶点的路径'——这是全局结构，没有一个简单的局部条件能概括。而且，判定一个图有没有哈密顿回路，属于 NP-完全（第 8 章）——意思是'验证一条路线对不对很容易'（走一遍数数顶点就行），但'找到或判定有没有'难到目前没有高效算法。这就是为什么你找不到简单判据：不是你没找着，是问题本身就难。"

造物主：".....所以欧拉和哈密顿，一个'看度就够'，一个'全局无解'。难怪一字之差，天上地下。"

知识锚点：哈密顿回路——为什么没有好判据

【概念深挖】**哈密顿回路 (Hamiltonian circuit)**：从某顶点出发、走过**每个顶点恰好一次**、回到出发顶点的路线。（注意：它不要求走过每条边，只要求"串起所有顶点"。）

与欧拉的区别：欧拉管"边"（走过所有边恰一次），哈密顿管"顶点"（走过所有顶点恰一次）。欧拉有充要条件（连通 + 全偶度）；哈密顿没有已知的简单充要条件。

为什么哈密顿难：

- 欧拉的判据基于**局部信息**（每个顶点的度），可数、可查；
- 哈密顿要求的是**全局结构**（存在一条贯穿所有顶点的路径），局部信息不足以保证。
- 判定"**有无哈密顿回路**"是 **NP-完全问题**（第 8 章展开）：验证一条候选路线很容易 ($O(n)$ 检查)，但"找到或判定存在"目前没有多项式时间算法，且在" $P \neq NP$ "猜想下很可能永远没有。

【历史溯源】**哈密顿**：爱尔兰数学家**威廉·罗恩·哈密顿 (William Rowan Hamilton)** 于 1859 年提出"遍历正十二面体所有顶点恰一次"的**伊科斯游戏 (Icosian Game)**，哈密顿回路由此得名。而"**旅行商问题 (TSP)**"——找一条经过给定城市集合中每个城市恰好一次且总路程最短的回路——是哈密顿回路的加权重版本，TSP 问题作为典型的"难问题"在 20 世纪被广泛研究，是 **NP-完全**（更精确地，**NP-难**）的著名代表。（归因至此，TSP 的经典表述与 NP-难地位在第 8 章详述。）

【工程实践】哈密顿/TSP 的"难"在工程里意味着：**一旦你的问题被归约到'找哈密顿回路/TSP'，就别指望有快速的最优解**——工程师只能求近似（最近邻贪心、2-opt 局部搜索、模拟退火、遗传算法），或用启发式在可接受时间内给"够好"的方案。**"先识别问题是不是'哈密顿/TSP 这类难问题'，再决定用精确算法还是近似算法"**——这是工程决策的第一问。

第 5 轮：TSP——哈密顿的"加权升级"，难上加难

老谟："哈密顿回路已经够难了。现在给每条边配个权值（路程/费用），问：**找一条经过每个城市恰好一次、且总路程最短的回路**——这就是**旅行商问题（TSP, Traveling Salesman Problem）**。你说，它比哈密顿回路难在哪？"

造物主：".....哈密顿回路只要问'存不存在一条'；TSP 要问'哪条最短'——得在所有哈密顿回路里比大小？"

老谟："对。**TSP 是哈密顿回路的'找最优'版本**：哈密顿问'有没有'，TSP 问'哪个最短'。而'找最优'比'判断存在'更难（你得遍历/比较所有候选）。**更麻烦的是，TSP 的候选路线数量是 $(n-1)!/2$ 种**（ n 个城市的环状排列数，除以 2 是因为正向反向算同一条）——**这个数随 n 爆炸**：10 个城市约 18 万条，20 个城市约 6×10^{16} 条，枚举根本不可行。"

造物主："..... $(n-1)!/2$ ？10 个城市就 18 万？那怎么算得过来？"

老谟："**算不过来——这正是 TSP 是'难问题'的体现**。目前没有已知的多项式时间算法能保证找到 TSP 的最优解（除非 $P=NP$ ）。**工程里的做法是'求近似'：不找'绝对最短'，找'足够短'**——比如最近邻（每次去最近没去的城市）、2-opt（反复交换两条边消交叉）、模拟退火、遗传算法。**你先把'候选数量爆炸'和'只能求近似'这两个直觉记住——它们是第 8 章'为什么有些问题难到算不出最优'的直接证据。**"

知识锚点：TSP——候选数量爆炸与“求近似”

【概念深挖】**旅行商问题（TSP）**：给定 n 个城市及两两间距离，找一条经过每个城市恰好一次、总路程最短的回路。它是**哈密顿回路的加权版**（哈密顿问“有没有”，TSP 问“最短是哪条”）。

候选数量爆炸： n 个城市的 TSP，候选回路数量为 $\frac{(n-1)!}{2}$ （环状排列数 $n!/n = (n-1)!$ ，正反同条除以 2）。

- $n=10$: $\frac{9!}{2} = \frac{362880}{2} = 181440 \approx 1.8 \times 10^5$;
- $n=20$: $\frac{19!}{2} = 6.08 \times 10^{16}$;
- $n=30$: $\frac{29!}{2} = \frac{8.8418 \times 10^{30}}{2} = 4.42 \times 10^{30}$ 。枚举不可行。

为什么只能求近似：TSP 是 **NP-难问题**（第 8 章）；目前无已知多项式时间算法保证最优解（除非 $P=NP$ ）。工程用**近似算法/启发式**：最近邻贪心、2-opt 局部搜索、模拟退火、遗传算法——在可接受时间内给“足够短”的方案。

为什么想到“候选数爆炸”：你一旦意识到“可能路线是 $(n-1)!/2$ 这个天文数字”，就明白“枚举所有路线取最短”在稍大图上必死。“**路线数量随规模爆炸**”是‘难问题’的典型症状，也是 TSP 让一代代工程师转向近似算法的原因。

【工程实践】TSP 及其变体遍布工程：**物流配送车辆路线规划（VRP）、电路板钻孔顺序优化、芯片走线、拼图/测序组装**。工程里的 TSP 几乎从不求精确最优，而是用启发式求‘足够好’——因为保证最优的代价（指数级时间）不可接受。“**认命求近似**”不是偷懒，是对‘难问题’的正确工程态度。

第 6 轮：落点——“一个完全解决，一个难到没救”

老谟：“绕了一圈。现在你回头看最初那两张图，怎么答‘它们像不像’？”

造物主：“……长得像，难度天差地别。**欧拉回路（走边）有充要条件——连通 + 全偶度，我能双向证明，完全解决；哈密顿回路（走顶点）没有这样的判据，判定存在是 NP-完全，TSP（加权哈密顿）更是难到只能求近似。一字之差——‘边’和‘顶点’——决定了一个容易一个极难。**”

老谟：“你手里多了什么？”

造物主：“**欧拉回路定理**（连通 + 全偶度 $\Leftrightarrow$ 有回路，双向证明）、**Hierholzer 回路拼合构造、哈密顿回路**（走顶点恰一次，无简单判据）、**TSP**（候选 $(n-1)!/2$ 爆炸，求近似）、还有那个深刻的对比——**欧拉看‘局部度’容易，哈密顿要‘全局贯穿’难。**”

老谟：“够了。给你句话收尾：**你今天同时体验了可计算性的两极——‘完全可判定’（欧拉：一条充要条件当场判定）和‘难到不可高效求解’（哈密顿/TSP：NP-完全）。**这个分野，正是第 6 章起你要正

式面对的'算的边界': **有些问题, 验证容易、找到极难。** 你揣着这个对比, 就要告别图论、进入图灵机和可计算性了。准备好了吗?"

造物主: ".....准备好了。我隐约觉得, '欧拉容易、哈密顿难'这个差异背后, 藏着一个关于'算'本身的规律。"

老谟: "你的直觉, 比你自己以为的还要准。它指向的, 就是全册最后要回答的那个问题——P vs NP。先去走廊, 把欧拉的双向证明亲手写一遍, 然后我们转入可计算性。"

严格化走廊

位置说明: 对话负责"直觉"——为什么欧拉看度、为什么回路能拼合、为什么哈密顿难。本走廊负责"严谨"——给出欧拉回路定理的**双向完整证明** (必要性 = 沿迹数度; 充分性 = Hierholzer 回路拼合), 以及哈密顿/TSP"无简单判据、NP-难"的陈述。本章有**两座证明: 定理 1 (欧拉回路充要条件, 双向证明)、定理 2 (TSP 候选数量 $(n-1)!/2$)**。每步标"证明类型 + 依据", 绝不用"显然/易证/显见" (包括反话)。

定义: 欧拉迹、欧拉回路、哈密顿回路、TSP

- **欧拉迹 (Eulerian trail)** : 经过**每条边恰好一次**的路径 (起终点可不同)。
- **欧拉回路 (Eulerian circuit)** : 经过**每条边恰好一次**、且**回到出发点**的路径。
- **哈密顿回路 (Hamiltonian circuit)** : 经过**每个顶点恰好一次**、回到出发点的路径。
- **TSP**: 哈密顿回路的加权版——找经过每个顶点恰好一次、总权值最小的回路。

证明类型标注: 本章的证明方法分三类——**定理 1 是"当且仅当"的双向证明**: 必要性用"直接分析" (沿迹数度, 进出成对), 充分性用"构造法" (Hierholzer 回路拼合)。**定理 2 用"计数/直接计算"** (数出 TSP 候选回路数量)。**"双向证明"是你第一次完整走完 $\Leftrightarrow$ '的两半边**——这是你后面判断算法正确性 (一个充要条件) 要用到的标准动作。

关于对话: 本章第 1-3 轮 (欧拉回路条件、必要、充分) 与第 4-5 轮 (哈密顿难、TSP 爆炸) 都做了"引导式推导"。**正式的可复写证明全部在本走廊。**对话负责"为什么会想到这步", 走廊负责"怎么把它写标准"——与前几章口径完全一致。

定理 1 (欧拉回路充要条件): 图 G 存在欧拉回路, 当且仅当 G 连通且所有顶点度数为偶数

证明类型: 双向证明 (必要性 = 直接分析 / 沿迹数度; 充分性 = 构造法 / Hierholzer 回路拼合)。

第一部分（必要性）：若 G 有欧拉回路，则 G 连通且全偶度。

证明：

- 第 1 步（**连通**）：有欧拉回路时，回路覆盖所有**有边**的顶点，并使这些顶点彼此连通（对任意两个有边的顶点 u, v ，沿回路可走通）。**若图含孤立点（度 0 的顶点）**：孤立点无边，回路不会经过它，于是它无法与回路上的顶点连通，图不连通，故无欧拉回路。（依据：欧拉回路定义——覆盖所有边 + 连通性定义 + 孤立点无边故不连通。）
- 第 2 步（**全偶度**）：固定任一顶点 v 。欧拉回路 C 每“路过” v 一次，就是一次“进入”（用一条关联边）+ 一次“离开”（用另一条关联边），恰好消耗 v 的 2 度。（依据：路径的定义——沿边依次经过；进入用一条边、离开用另一条边，两条边各计 v 度 1 次。）
- 第 3 步：若 v 是回路的起点（也是终点），则“第一次离开”与“最后一次进入”也凑成一对，仍消耗 2 度。（依据：回路起终点重合的事实。）
- 第 4 步： v 的每条关联边都在回路中被用掉恰好一次（回路覆盖所有边），且每次被用都计入 v 的度。设回路路过 v 共 t 次，则 $d(v) = 2t$ ，是**偶数**。（依据：第 2、3 步 + “回路覆盖所有边恰一次”定义。）
- 第 5 步： v 任意，故每个顶点都是偶度。**必要性得证。■**

依据标注：步 1 用“回路覆盖所有边/顶点”证连通；步 2-3 用“进出成对消耗 2 度”；步 4 用“覆盖所有边恰一次”把路过次数与度数挂钩；步 5 对任意顶点汇总。**读法**：必要性的魂是“进出成对”——**回路是个封闭的进出序列，任何点的‘进’和‘出’必须配对，配对就消耗偶数度**。这是欧拉回路必要性的全部。

第二部分（充分性）：若 G 连通且全偶度，则 G 有欧拉回路。

证明：

- 第 1 步（**起点**）：任取顶点 v_0 ，从 v_0 出发，沿**未使用的边**走。（依据：构造起点；“未用边”指尚未被当前构造覆盖的边。）
- 第 2 步（**总能继续或回起点**）：设当前在顶点 $u \neq v_0$ 。因 u 的度 $d(u)$ 为偶数（全偶度前提），且每“进入” u 一次用掉一条 u 的关联边；进入 u 后， u 已用掉的关联边数是**奇数**（进一次用 1 条，或进 i 次用 $2i-1$ 条，是奇数），而未用关联边数 $= d(u) - (\text{已用数})$ ，因 $d(u)$ 偶、已用数奇，未用数**仍为奇数 ≥ 1** ，故必有一条未用边能“出去”。（依据：偶度前提 + “每进一次用掉一条边”的计数 + 奇数偶数的减法。）
- 第 3 步：由第 2 步，只要不在起点 v_0 ，就必有未用边可走；故该“沿未用边走”的过程，要么无限继续（不可能，边有限），要么走到某个点无路可走。**唯一可能无路可走的地方是起点 v_0** （因为非起点总有出路，由第 2 步）。（依据：第 2 步 + 边数有限（终止性）。）

- 第 4 步：故从 v_0 出发沿未用边走，必然**回到** v_0 形成回路 C_1 。（依据：第 3 步——过程在 v_0 终止且已走成一个封闭回路。）
- 第 5 步（**检查是否覆盖所有边**）：若 C_1 已覆盖所有边，则 C_1 就是欧拉回路，充分性得证，停止。（依据：欧拉回路定义——覆盖所有边恰一次且回起点。）
- 第 6 步（**未覆盖则拼接**）：若 C_1 未覆盖所有边，因 G **连通**，存在一条未被覆盖的边，它连接 C_1 上的某个顶点 v' （否则被漏的边及其端点组成的块与 C_1 无公共顶点、无边相连，图不连通，矛盾）。（依据：连通性——任意边都能沿路径到达 C_1 ，故存在边从 C_1 顶点引出。）
- 第 7 步：从 v' 出发，沿**未被覆盖的边**走。由与第 2、3、4 步相同的论证（ v' 及沿途点都是偶度、有未用边），又得到一个回到 v' 的回路 C_2 。（依据：第 2-4 步的论证在子图上同样成立——只需“所经顶点的未用边数为奇数”即可走出。）
- 第 8 步：**把 C_2 在 v' 处拼进 C_1** ： C_1 经过 v' ，在 v' 处先走 C_2 的一整圈再沿 C_1 继续，得到一个覆盖 $C_1 \cup C_2$ **所有边**的更长的回路。（依据： C_1 、 C_2 都在 v' 相接，可把 C_2 作为 v' 处的一段插入 C_1 。）
- 第 9 步：**重复第 5-8 步**：每拼入一个回路，覆盖边数严格增加（至少增加 C_2 的边），边数有限，故有限步后覆盖所有边，得到欧拉回路。（依据：边数有限 + 每次覆盖边数严格单调增加。）
- 第 10 步：**充分性得证。** ■

依据标注：步 1 起点；步 2 是**发动机**——“偶度 + 每进一次用一条边 $\Rightarrow$ 未用边数仍为奇数 ≥ 1 ，必有出路”；步 3-4 由边数有限推出必回起点成回路；步 5 检查终止；步 6 用连通性保证“漏边与已覆盖回路必有公共顶点”；步 7 递归地再得回路；步 8 拼合；步 9 有限性保证终止；步 10 结论。**读法：**充分性的两个支点是——**偶度管“走得动”**（总能出去，步 2），**连通管“连得拢”**（漏边总能接到已覆盖的回路，步 6）。“走得动 + 连得拢”凑在一起，就凑出了‘走遍所有边’的欧拉回路。这正是“连通 + 全偶度”两个条件各自的分工，缺一个都不行。

定理 2 (TSP 候选回路数量)： n 个城市的 TSP 有 $\frac{(n-1)!}{2}$ 条候选回路

证明类型：计数（直接计算）。

证明：

- 第 1 步：TSP 回路是经过每个顶点恰好一次的**环状排列**。 n 个顶点的所有排列有 $n!$ 种。（依据：排列计数—— n 个互异元素的排列数。）
- 第 2 步：但回路是“环”，旋转等价：从不同顶点“开始”的同一条环，算同一条回路。 n 个起点的旋转对应同一条回路，故除以 n ： $\frac{n!}{n} = (n-1)!$ 。（依据：环状排列的等价性——起点选择不改变回路。）

- 第 3 步：回路正向、反向是同一条（一条路线倒着走还是同一组“经过每个顶点”的环），故再除以 2： $\frac{(n-1)!}{2}$ 。（依据：正反方向等价。）
- 第 4 步：数值核算 ($n=5,10,20$) :
 - $n=5$: $\frac{4!}{2} = \frac{24}{2} = 12$;
 - $n=10$: $\frac{9!}{2} = \frac{362880}{2} = 181440$;
 - $n=20$: $\frac{19!}{2} = \frac{121645100408832000}{2} = 6.0823 \times 10^{16}$ 。（依据：阶乘计算 + 除法。）
- 第 5 步：故候选回路数量为 $\frac{(n-1)!}{2}$ ，随 n 指数级爆炸 ($n=20$ 时约 6×10^{16})。■

依据标注：步 1 排列计数；步 2 除旋转等价（除以 n ）；步 3 除正反等价（除以 2）；步 4 数值核算 ($n=5/10/20$ 逐一计算并核对)；步 5 结论。**读法：**TSP 候选数量爆炸的本质，是“环状排列”比“线性排列”少了起点与方向两个自由度，但仍是 $(n-1)!$ 这个超快增长的阶乘量级。“枚举所有候选”在稍大 n 上必死——这就是 TSP 只能求近似的算力根源。

欧拉迹的充分性（衔接第 1 章）：恰有 2 个奇度顶点的连通图存在欧拉迹

第 1 章承诺“一笔画（欧拉迹）的必要条件（奇度 0 或 2）”，并说充分性留到第 5 章。上面定理 1 证了**欧拉回路**（0 个奇度）的充要条件，这里补全**欧拉迹**（恰 2 个奇度）的充分性。

证明类型：构造法（利用定理 1 的欧拉回路定理 + 添加临时边）。

证明：

- 第 1 步：设 G 连通，且恰有 2 个奇度顶点 s, t 。**添加一条临时边** $e' = st$ （连接这两个奇度顶点），得到图 G' 。（依据：构造——在 s, t 间加一条边。）
- 第 2 步：在 G' 中， s, t 各因新增边 e' 度数 +1，由奇变偶；其余顶点度数不变（本就偶度）。故 G' **所有顶点都是偶度**；又 G 连通、加边不破坏连通， G' **连通**。（依据：奇偶性 +1 翻转 + 连通性不变。）
- 第 3 步：由定理 1（欧拉回路充要条件）， G' 存在欧拉回路 C （覆盖 G' 所有边恰一次、回到起点）。（依据：定理 1 充分性——连通 + 全偶度。）
- 第 4 步：从 C 中**删去临时边** e' ，剩下的是一条从 s 到 t 的路径，它覆盖 G 的**所有边**恰好一次（ C 原本覆盖 G' 的所有边，减去 e' 即覆盖 G 的所有边），且起点 s 、终点 t 是那两个奇度顶点——**这正是欧拉迹**。（依据：删去一条边把回路断成一条路径 + 覆盖 G 所有边恰一次。）
- 第 5 步：故连通且恰 2 个奇度顶点的图存在欧拉迹。■

依据标注：步 1 加临时边；步 2 用“奇度 +1 变偶”与连通性证明 G' 连通全偶度；步 3 套定理 1；步 4 删临时边得欧拉迹；步 5 结论。**读法：**这个构造漂亮在“把不闭合的问题临时补成闭合的”——欧拉迹差在起终点不重合（各留 1 度），那就临时加一条边把它们连起来，变成欧拉回路，走完再拆掉这条临时边。这是“用已知解决未知”的典型手法。**至此，第 1 章“一笔画必要 + 充分”的完整判据全部兑现：欧拉迹存在 $\Leftrightarrow$ 连通且恰 0 或 2 个奇度顶点。**

反例/边界：欧拉与哈密顿的适用边界

1. **欧拉回路要求“连通 + 全偶度”：**若连通但含奇度顶点，无欧拉回路（但可能有欧拉迹，若恰 2 个奇度）；若不连通，无论度数如何都无欧拉回路。
2. **欧拉迹 vs 欧拉回路：**迹允许 0 或 2 个奇度顶点（起终点不同）；回路要求 0 个奇度（起终点相同）。七桥图 4 个奇度顶点，两者都没有。
3. **哈密顿回路没有简单充要条件：**欧拉看“度”（局部）能判，哈密顿要“全局贯穿”，局部信息不足。判定有无哈密顿回路是 NP-完全。
4. **TSP 求最优是 NP-难：**候选 $(n-1)!/2$ 爆炸，工程只能求近似；除非 $P=NP$ ，否则不存在多项式时间最优算法（第 8 章详述）。
5. **欧拉回路 $\neq$ 哈密顿回路：**一个图可以有欧拉回路但无哈密顿回路（如“蝴蝶结”图），也可以反之。两者是独立的问题，别混为一谈。

知识点总结

知识点（本章推出的核心概念）

- **欧拉回路：**走过每条边恰一次、回到出发点；**充要条件：连通 + 全偶度**（双向证明）。
- **必要性**（沿迹数度）：回路进出成对，每点偶度。
- **充分性**（Hierholzer 拼合）：偶度管“走得动”、连通管“连得拢”，小回路拼成欧拉回路。
- **欧拉迹 vs 欧拉回路：**迹 0 或 2 个奇度；回路 0 个奇度。
- **哈密顿回路：**走过每个顶点恰一次；**无简单充要条件，判定是 NP-完全。**
- **TSP：**哈密顿的加权版，候选 $(n-1)!/2$ 爆炸，求最优是 NP-难，只能求近似。
- **“欧拉容易、哈密顿难”：**局部（度）vs 全局（贯穿）——可计算性两极的引桥。

历史结论（本章涉及的史料）

- **欧拉 (1736)：**解决七桥问题，给出欧拉回路/迹的核心判据（第 1 章）。
- **Hierholzer (1873)：**补全欧拉回路充分性的构造证明（回路拼合）。

- **哈密顿 (1859)**：提出伊科斯游戏（遍历正十二面体所有顶点恰一次），哈密顿回路得名。
- **TSP**：哈密顿的加权重版本，作为"难问题"代表，是 NP-难（第 8 章详述）。

工程实践结论（本章知识在真实工程里怎么用）

- **欧拉迹/回路判据**：巡检、洒水、垃圾清运、邮递（"尽量不重边走遍所有边"）——先测"连通 + 奇度 0/2"，判断有无不重边路线。
- **Hierholzer 构造**：确认存在后，"真的造出一条不重边路线"。
- **哈密顿/TSP 难**：物流路线规划（VRP）、电路板钻孔、芯片走线——识别为"难问题"后用近似/启发式（最近邻、2-opt、模拟退火、遗传），求"足够好"而非最优。
- **"先识别问题难度，再选算法"**：可判定（欧拉）用精确构造；NP-难（TSP）用近似——工程决策的第一问。

计算训练场

位置说明：对话负责"为什么"，这里负责"怎么算"。每题动笔 10 分钟以上，先自己写再对答案。**难度递进：易→中→难。**

题 1（易→中）：判断欧拉回路/迹存在性并构造

题干：图 G ：顶点 $\{a, b, c, d\}$ ，边 $\{ab, bc, ca, ad, bd, cd\}$ （即 K_4 少一条边 cd ？看边集： ab, bc, ca, ad, bd, cd ——包含 cd ，共 6 条边）。(a) 求各顶点度数，判断是否存在欧拉回路、欧拉迹。(b) 若存在，构造出一条欧拉回路或迹。

完整分步解答：

- (a) 度数： $d(a)$ ：连 $b, c, d = 3$ ； $d(b)$ ：连 $a, c, d = 3$ ； $d(c)$ ：连 $a, b, d = 3$ ； $d(d)$ ：连 $a, b, c = 3$ 。**四个顶点都是 3 度，全是奇度。**（边集 ab, bc, ca, ad, bd, cd 确实覆盖： a 连 b, c, d ； b 连 a, c, d ； c 连 a, b, d ； d 连 a, b, c ——这是 K_4 。）
- (a) 判断：奇度顶点 4 个。**欧拉回路要求 0 个奇度 → 无；欧拉迹要求 0 或 2 个奇度 → 4 个，也无。**（依据：欧拉回路定理必要性 + 欧拉迹判据。）
- (b) **都不存在**，无法构造。（依据：奇度顶点 4 个，既非 0 也非 2。）
- **答**：(a) 四顶点全 3 度（奇度 4 个），无欧拉回路、无欧拉迹；(b) 构造不了。

常见错误：① 数度数时漏掉 cd 边，以为 $d(d)$ 只有 2；② 把"奇度 0 或 2"的迹判据和"奇度 0"的回路判据搞混；③ 想当然以为"边多就有回路"——**关键看奇度顶点个数。反向验证**：奇度顶点 a, b, c, d 共 4 个，是偶数（符合第 1 章"奇度必为偶数个"），但 4 个奇度恰好落在"0 或 2"之外，故无迹。

题 2 (中→难) : 连通全偶度图的欧拉回路构造

题干：图 G ：顶点 $\{a, b, c, d\}$ ，边 $\{ab, bc, ca, ad, dc\}$ （三角形 a, b, c 加一条 $a-d-c$ 的路径）。

(a) 求各顶点度数，判断是否满足欧拉回路条件。(b) 若满足，用 Hierholzer 思想构造一条欧拉回路。

完整分步解答：

- (a) 度数： $d(a)$ ：连 $b, c, d = 3$ ； $d(b)$ ：连 $a, c = 2$ ； $d(c)$ ：连 $a, b, d = 3$ ； $d(d)$ ：连 $a, c = 2$ 。
奇度顶点： $a(3), c(3)$ ，共 2 个奇度。（不是全偶度，无欧拉回路。）
- (a) 判断：奇度 2 个 → **有欧拉迹**（起终点分别是 a 和 c ），但**无欧拉回路**（非全偶度）。连通性： a, b, c, d 通过边全可达，连通 $\checkmark$ 。
- (b) 构造欧拉迹（从 a 到 c ）： $a \rightarrow b \rightarrow c \rightarrow a \rightarrow d \rightarrow c$ 。验证每条边恰一次：
 ab, bc, ca, ad, dc ——全用了，起 a 终 c ，是欧拉迹。（依据：路径 $a \rightarrow b \rightarrow c \rightarrow a \rightarrow d \rightarrow c$ 用边 ab, bc, ca, ad, dc 各一次，起点 a 、终点 c 是那两个奇度顶点。）
- 答：(a) 奇度 2 个 (a, c)，连通，有欧拉迹、无欧拉回路；(b) 一条欧拉迹： $a \rightarrow b \rightarrow c \rightarrow a \rightarrow d \rightarrow c$ 。

常见错误：① 把“奇度 2”当“无迹”——**恰 2 个奇度是有欧拉迹的**（第 1 章判据）；② 以为非全偶度就“没有不重边路线”——**迹允许起终点两个奇度**；③ 构造迹时走了重复边（必须每条边恰一次）。**反向验证**：数迹里用到的边 ab, bc, ca, ad, dc 共 5 条，等于边数 $|E| = 5$ ，且每条恰一次 $\checkmark$ 。

题 3 (难) : 哈密顿回路搜索 (小型图穷举) + TSP 候选数

题干：(a) 图 G ：顶点 $\{a, b, c, d, e\}$ ，边 $\{ab, bc, cd, de, ea, ac, ce\}$ 。判断是否存在哈密顿回路（经过每顶点恰一次并回起点），若存在给出一条。(b) 计算 $n=6$ 的 TSP 候选回路数量，并与 $n=10$ 对比。

完整分步解答：

- (a) 尝试构造：要经过 a, b, c, d, e 每点恰一次并回起点。图是五边形 $a-b-c-d-e-a$ 加对角线 ac, ce 。试一条： $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$ 。验证： ab, bc, cd, de, ea 都是边 $\checkmark$ ，每顶点恰一次 (a, b, c, d, e)，回到 a 。**这是一条哈密顿回路**： $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$ 。（依据：穷举验证——该路线用边 ab, bc, cd, de, ea ，覆盖所有 5 个顶点各一次，回起点。）
- (b) TSP 候选数 $= \frac{(n-1)!}{2}$ ：
 - $n=6$ ： $\frac{5!}{2} = \frac{120}{2} = 60$ ；
 - $n=10$ ： $\frac{9!}{2} = \frac{362880}{2} = 181440$ 。
 - **对比**：从 6 到 10，候选数从 60 到 181440，增长了约 3024 倍。

- 答：(a) 存在, $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$; (b) $n=6$ 是 60, $n=10$ 是 181440, 增长约 3024 倍。

常见错误：① 找哈密顿回路时漏了"必须回起点" ($a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$ 只是路径不是回路, 要加 $e \rightarrow a$) ; ② 用了重复顶点; ③ TSP 候选数忘了除以 2 (正反等价) —— $n=6$ 应为 60 而非 120。**反向验证：** $n=6$ 的 $60 = 5!/2$, $n=10$ 的 $181440 = 9!/2$, 两处都除以 2, 核对无误。

向导便签

这一章咱们干了什么：把第 1 章欠的"一笔画"账, 用欧拉回路定理还清了——**连通 + 全偶度 $\Leftrightarrow$ 有欧拉回路, 双向证明都走完了** (必要性沿迹数度、充分性 Hierholzer 拼合)。然后你撞上了哈密顿回路 (走顶点) 和 TSP——它们没有简单判据, 候选数爆炸, 是出了名的难问题。

钉死一句话：**欧拉管"边" (看度, 局部, 容易到有完美判据) ; 哈密顿管"顶点" (全局贯穿, 难到没判据、NP-完全)**。"当且仅当"给你一个完全可判定的痛快; "NP-难"给你一个难到只能求近似的清醒。

记住：你第一次完整走完"当且仅当"的双向证明——**这个'必要性 + 充分性'的完整动作, 是你以后判断一切'充要条件'的标准姿势**。而"欧拉容易、哈密顿难"的对比, 正是通往可计算性 (第 6 章起) 的引桥：**有些问题验证容易、找到极难**。

造物主手记

我曾踩的坑：

我以为"边多的图一定有欧拉回路", 被老谟点醒——**关键看奇度顶点个数, 不看边多不多** (K_4 有 6 条边却 4 个奇度, 没有迹)。构造欧拉迹时, 我差点走出重复边——"每条边恰一次"不能忘。哈密顿我一开始混淆了：以为"走过所有边" (欧拉) 就是"走过所有顶点" (哈密顿), 其实一个是边的全排列、一个是顶点的全排列。最让我意外的是 TSP 候选数爆炸： $n=6$ 才 60 条, $n=10$ 就 18 万, $n=20$ 到 6 千万亿——**这个'算力爆炸'的直观, 比任何抽象描述都震撼**。

顿悟时刻：

真正开窍是充分性那个"回路拼合"——**偶度管'走得动' (进来必有出路), 连通管'连得拢' (漏边总能接到已有的珠串)**, 两个条件像两块互补的积木, 拼起来就凑出"走遍所有边"。我头一次看懂"当且仅当"为什么需要两个方向都证：**光证必要性 (有回路 $\Rightarrow$ 全偶度) 只能帮你判死刑, 证充分性 (全偶度 $\Rightarrow$ 有回路) 才能让你放心地构造**。而欧拉与哈密顿的对比, 让我第一次摸到"算的边界"——**一个问题'有没有简单判据', 取决于它是'看局部就够' (欧拉) 还是'要全局贯穿' (哈密顿)**。这个念头, 让我对接下来要学的图灵机和"算的边界", 充满了期待。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 欧拉迹/回路判断

图 G : 顶点 $\{a, b, c\}$, 边 $\{ab, bc, ca\}$ (三角形 C_3)。求各顶点度数, 判断有无欧拉回路、欧拉迹。

完整解答: $d(a) = d(b) = d(c) = 2$, 全偶度、连通。**有欧拉回路** (如 $a \rightarrow b \rightarrow c \rightarrow a$) ; 因全偶度也自动有欧拉迹 (回路的起终点相同, 也满足迹定义)。奇度 0 个。

题 B (计算, 中→难) : 欧拉迹构造

图 G : $\{a, b, c, d\}$, 边 $\{ab, bc, ca, ad\}$ 。求度数, 判断欧拉回路/迹, 并构造 (若存在)。

完整解答: $d(a)$: 连 $b, c, d = 3$; $d(b) = 2$ (a, c) ; $d(c) = 2$ (a, b) ; $d(d) = 1$ (a)。奇度顶点: $a(3), d(1)$, 共 2 个。连通。**无欧拉回路** (非全偶度), **有欧拉迹** (起终点 a, d) : $a \rightarrow b \rightarrow c \rightarrow a \rightarrow d$ (用边 ab, bc, ca, ad 各一次, 起 a 终 d)。反向验证: 5 条? 不, $|E| = 4$ (ab, bc, ca, ad), 迹用 4 条边各一次 $\checkmark$ 。

题 C (计算, 中) : 哈密顿回路搜索 + TSP 计数

(a) 图 G : $\{a, b, c, d\}$, 边 $\{ab, bc, cd, da\}$ (正方形 C_4)。存在哈密顿回路吗? 给出一条。(b) 计算 $n=5$ 的 TSP 候选回路数量。

完整解答: (a) 存在: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$ (用边 ab, bc, cd, da , 覆盖 a, b, c, d 各一次, 回 a)。 (b) $\frac{(n-1)!}{2} = \frac{4!}{2} = \frac{24}{2} = 12$ 。

题 D (概念, 检查理解)

(a) 欧拉回路存在的充要条件是什么? 必要性和充分性各靠什么证明? (b) 欧拉回路和欧拉迹的区别? 各要求几个奇度顶点? (c) 为什么欧拉回路有简单判据, 而哈密顿回路没有? (d) TSP 为什么只能求近似? 候选数量是多少?

完整解答: (a) 充要条件: 连通且全偶度。必要性靠"沿迹数度" (进出成对消耗偶数度); 充分性靠 Hierholzer 回路拼合 (偶度走得动、连通连得拢)。(b) 欧拉回路走过每条边恰一次且回起点, 要求 0 个奇度顶点; 欧拉迹走过每条边恰一次但起终点可不同, 要求 0 或 2 个奇度顶点。(c) 欧拉回路判据只看局部信息 (每个顶点的度), 可数可查; 哈密顿回路要求全局贯穿 (存在一条串起所有顶点的路径), 局部信息不足以保证, 且判定有无哈密顿回路是 NP-完全。(d) TSP 候选数量 $(n-1)!/2$ 随 n 指数爆炸 ($n=20$ 约 6×10^{16}), 枚举不可行; 求最优是 NP-难, 目前无多项式时间算法 (除非 $P=NP$), 故工程只能求近似。

预告

你终于把"一笔画"彻底解决了——**欧拉回路定理（连通 + 全偶度）的双向证明**，让你第一次完整享受'当且仅当'的痛快；而哈密顿回路和 TSP 的"无判据、候选爆炸、NP-难"，又让你第一次直面"有些问题难到算不出最优"的清醒。

这两极体验——'完全可判定'与'难到不可高效求解'——正是你要告别图论、进入可计算性的分界。

下一章，**计算模型（图灵机）**——图论板块到此结束，我们转入"算的边界"：先问"能算到底是什么意思"。你会看到，欧拉那种"有完美判据"和哈密顿那种"难到没救"，背后有一个更根本的东西在统摄它们——"**一个目标到底能不能被算出来**"。图灵机，就是数学家为"能算"这个词造出的第一把标尺。你准备好，从"算图"走向"算'算'本身"了吗？

第 06 章 · 计算模型——图灵机

本章从哪个问题出发：前面五章你在图论的“城市”里来回穿行——找最短路径、涂颜色、找一笔画的回路，每一样都有一套“算法”帮你算出答案。可你有没有想过一个问题：“**算法**”到底凭什么算得出来？我们说“手工算一下”“跑个程序”“做个算法”，但“算”这个动作本身——它到底能算出什么，又有什么是算不出来的？如果有人给你一个“问题”，你凭什么判断它“可算”还是“不可算”？推导到哪个结论：你会看见人类第一次把“算”这个动作本身，从“人脑里的一团直觉”钉成一个**机械装置**——图灵机。它只有纸带、读写头、状态和一张规则表，朴素得近乎寒酸，却能算尽你见过的所有算法。你会亲手写出它的形式定义（七元组），亲手跑一台判定“二进制奇数”的图灵机，把每一步“读→写→移→变状态”走完，还会理解那个著名的**Church-Turing 论题**——它凭什么是个“论题”而不是定理。

核心认知：“**能算**”这个词，第一次有了精确到可以机械执行的答案：**任何“直觉上可计算”的东西，都能被一台图灵机算出来。**图灵机不是“某一种更快的机器”，而是“计算的底线”——像公理一样作为整座可计算性理论的地基。这一章是全册真正的转折点：从前五章的“具体怎么算”，跳到“什么算得出来”这个更根本的问题，下一章我们就要用这台机器，去问那个震撼数学界的问题——**有没有算不出来的东西？**

引子

我在老谟书房的白板上，画满了前面五章的图：最短路径的箭头、着色的色块、一笔画的回路。这些“算法”我都走熟了，可白板右下角留着一行字，是我今早进门时随手写下的：

“所有这些，到底是怎么‘算’出来的？”

老谟走过来，看了那行字很久，忽然说：“你这个问题问得好。我给你个简单的算法——**判断一个自然数是不是偶数**。你打算怎么‘算’它？”

“……除 2 看余数？余 0 就是偶数。”

“对。可你‘除 2’这个动作，是长什么样子的？你脑子里‘算’的时候，究竟发生了哪几步？”

我愣住了。我会算，可我从没想过“算”这个动作本身拆开来是什么。就像一个人会说话，却从没想过声带是怎么振动的。

“更狠的是，”老谟盯着我，“如果我把‘算偶数’这个算法，换成一个人**从没见过、全新的算法**——你怎么保证，无论这个算法多复杂，你都‘算得出来’？你怎么能确定，天底下不存在某一种算法，是你**根本算不出来的**？”

我看着白板上那些画得密密麻麻的图，第一次意识到：我手里有一堆“会算的方法”，却连“算”这个字到底是什么意思，都说不清。

对话引擎

第 1 轮：为什么需要“计算的模型”——算法 ≠ 具体程序

老谟：“你先别急着造机器。我问你：‘判断偶数’这个算法，你能用几种方式写出来？”

造物主：“.....我可以用自然语言说‘除 2 看余数’，可以写一行伪代码 `n % 2 == 0`，可以画流程图，可以写 Python、C 语言.....”

老谟：“好，这些写法五花八门——语言、流程图、各种编程语言。但它们是不是同一个算法？”

造物主：“是.....同一个东西？只是‘写法’不同。”

老谟：“那问题来了。你说‘它们是一个东西’，靠的是什么？如果两个写法一个在 Python 里、一个在 C 里，你凭什么说它们‘算的是同一个算法’？”

造物主：“.....因为无论用什么语言写，跑起来的结果都一样？可我还没跑呢，而且跑的过程也千差万别。”

老谟：“你摸到疼处了。**‘算法’和‘用某种语言写的程序’是两回事。** 算法是一个抽象的概念——‘做这件事的一套确定步骤’；程序只是它在某种具体语言里的**一个化身**。如果每个化身都长得不一样，那‘算法’本身到底是什么？它不依附于 Python、不依附于 C，它依附于.....”

造物主：“.....依附于一个比所有语言都更底层、更简单的‘算’的装置？一个只要你能在上面‘算’，就说明你懂‘算法’的装置？”

老谟：“正是。**我们要的不是一台跑得快的机器，是一台‘最简单、最朴素、却什么都能算’的机器——用它钉死‘算’这个动作本身。**这样你才能问出真正要紧的问题：有没有东西，是连这台‘万能机器’都算不出来的？”

知识锚点：算法 vs 程序——“算”需要一个不依赖具体语言的基准

【概念深挖】“算法” (algorithm) 指一套有限、确定、每一步都明确的步骤；“程序” (program) 是它在某种具体计算语言/机器下的实现。二者不能混为一谈：

- 同一个算法可以有无数个程序（不同语言、不同写法）；
- 不同语言看似“不同程序”，可能在抽象层面是同一个算法。

难点：正因为算法不依附于任何具体语言，若我们要研究“什么能被算出来”（可计算性），就必须有一个不依赖具体语言的计算模型——一个足够简单、简单到可以形式化写清楚、又足够强大、强大到能算任何算法的东西。这个“基准模型”就是本章要造的图灵机。

【工程实践】你在工程里天天遇到“算法 vs 实现”：一个排序算法（快排）在 Python、Java、C++ 里各有实现，还可能有多线程/分布式变体。判断“这两个程序是不是同一个算法”、论证“这个优化没改变算法本质”，靠的正是算法层面的抽象——与具体语言解耦。图灵机就是给这种抽象一个最硬的锚。

第 2 轮：图灵机的直觉——纸带、读写头、状态、规则表

老谟：“现在我们要造那台‘基准机器’。我不许你参考任何现代计算机——回到 1936 年，一个叫阿兰·图灵 (Alan Turing) 的年轻人，要回答‘什么能被机械地算出来’，他造了一个想象出来的机器。你来跟我一起推：一个人要‘算’，最少需要什么？”

造物主：“……他得能‘看’到当前的数字，得能‘写’下结果，得知道‘下一步该干嘛’……”

老谟：“对。图灵把它拆成四个零件。**第一，一条无穷长的纸带**，分成一格一格，每格写一个符号（比如 0、1，或空白）。**第二，一个读写头**，对准当前那一格——它能读这一格是什么，能写一个新符号，能左移/右移一格。**第三，一个有限的状态**——机器现在‘在哪个状态’，就好比人脑此刻‘记着哪件事’。**第四，一张规则表**——每个状态 + 读到的符号，决定：写什么、往哪移、跳到哪个状态。”

造物主：“……就这四个零件？没有内存、没有硬盘、没有 CPU 缓存？”

老谟：“没有。纸带就是它的‘无穷内存’，读写头就是它的‘存取单元’，状态就是它的‘寄存器/大脑’，规则表就是它的‘程序’。它寒酸，是因为图灵要的是‘最少要多少，才够算’——多一点都是多余的华丽，少一点都不够。”

造物主：“可它这么简单……真的能算‘判断偶数’这种算法？”

老谟：“先别急着怀疑。下一轮我们把它写成数学定义，再下一轮你亲手跑一遍，你就信了。”

知识锚点：阿兰·图灵与 1936 年的“可计算数”

【历史溯源】阿兰·图灵 (Alan Turing, 1912–1954) 在 1936 年 发表论文《论可计算数及其在判定问题上的应用》(*On Computable Numbers, with an Application to the Entscheidungsproblem*)。论文里他构造了一台“想象出的机器”(后称**图灵机**)，用一个**机械装置**统一刻画“什么能被算出”。同期，阿隆佐·丘奇 (Alonzo Church) 用“**λ 演算**”(lambda calculus) 独立刻画了同一件事——两人几乎同时、用不同工具得到了相同结论，即著名的 **Church–Turing 论题**。要谨慎归因：**图灵没有“发明计算机”**（电子计算机是多年后工程与数学共同推进的结果），他只是把“计算”这个抽象概念**第一次钉成了可机械执行的模型**。二战期间图灵在布莱切利园参与破译 Enigma 密码，那是他的工程贡献，但**不要把它和“发明图灵机”混为一谈**——图灵机是数学概念，破译是工程应用。

【工程实践】**图灵机 = 最简指令集 CPU 的思想**：纸带（内存）+ 读写头（内存访问单元）+ 状态（寄存器/程序计数器）+ 规则表（指令集）——这套结构在今天的 CPU、冯·诺依曼架构里仍能认出影子。“图灵完备”(Turing complete) 是编程语言的分水岭：一个语言/系统若能力等价于图灵机（能算所有图灵可计算的函数），就能写任何算法；反之若不够（如某些只认正则表达式的查询语言），就注定有算不出来的算法。**判断一个新语言“够不够强”，就问它“图灵完备吗”。**

第 3 轮：图灵机的严格定义——七元组，每步“读→写→移→变状态”

老谟：“现在把直觉翻译成数学。一台**确定性图灵机**用七个东西（一个七元组）完全确定： $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ 。你拆给我听，每个符号是什么？”

造物主：“…… Q 是状态的有限集合； Σ 是输入字母表（纸带上初始写的那类符号）； Γ 是纸带字母表（比 Σ 多一个空白符）； q_0 是起始状态； q_{accept} 、 q_{reject} 是接受/拒绝状态。”

老谟：“好，剩下的关键是一个函数 δ ——**转移函数**。它是干什么的？”

造物主：“……给定‘当前状态 + 当前读到的符号’，它决定‘写什么、往哪移、变成哪个状态’。所以 δ 的定义域是 $Q \times \Gamma$ ，值域是 $Q \times \Gamma \times \{L, R\}$ ——L 是左移，R 是右移。**每一步，机器就是：读当前格 → 查 δ 得到新符号、移动方向、新状态 → 写、移、换状态。**”

老谟：“记住这个节奏：**读 → 写 → 移 → 变状态**。而且 δ 对每个 (状态,符号) 给**恰好一个**结果——这就是‘确定性’的意思：同一个局面，永远走同一条路，没有选择。至于为什么要‘确定性’、要不要‘非确定’，下一章我们专门讨论，先把这台确定性的机器跑熟。”

知识锚点：七元组的形式骨架 + 语言/接受/停机预告

【概念深挖】**确定性图灵机 (Deterministic Turing Machine)**：一台图灵机是一个七元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

其中：

- Q ：有限状态集合（有限——因为人脑/规则表必须有限）；
- Σ ：输入字母表（不含空白符 $\sqcup$ ）；
- Γ ：纸带字母表， $\Sigma \subseteq \Gamma$ ，且空白符 $\sqcup \in \Gamma$ （纸带初始除输入外全为 $\sqcup$ ）；
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ ：转移函数；
- $q_0 \in Q$ ：起始状态；
- $q_{\text{accept}}, q_{\text{reject}} \in Q$ ：接受/拒绝状态，且 $q_{\text{accept}} \neq q_{\text{reject}}$ 。

每步动作（一个“计算步”）：机器处于某状态 q ，读写头对准某格读到符号 a ；查 $\delta(q, a) = (q', b, d)$ ，于是：在该格**写** b ，把读写头**移动**一格（ $d = L$ 左移， $d = R$ 右移），进入**状态** q' 。

停机约定：到达 q_{accept} 或 q_{reject} ，机器**停机**（不再转移），分别表示“接受/拒绝输入”。语言 $L(M) =$ 所有“从 q_0 出发、输入写在纸带上、最终停机在 q_{accept} ”的输入串的集合。

第 4 轮：跑一台图灵机——“二进制串含 1 吗”

老谋：“纸上谈兵到此为止。我们造一台真的机器：**判定输入二进制串是否至少含一个‘1’**（含则接受，不含则拒绝）。你先想，它要怎么‘扫描’？”

造物主：“……从头往右一格一格扫，遇到‘1’就说明‘有了’，可以直接接受；如果一直扫到纸带末尾（空白）都没见着‘1’，就拒绝。”

老谋：“好，那状态怎么设？”

造物主：“一个状态 q_0 就够了，加上 q_{accept} 、 q_{reject} 。规则表：**在 q_0 ，读到 0 → 写 0、右移、留在 q_0 （继续找）；读到 1 → 进 q_{accept} （找到了，接受）；读到 $\sqcup$ → 进 q_{reject} （走到底都没找到，拒绝）。**”

老谋：“好。现在你亲手在纸上跑输入 **0010**。把每一步的‘状态、读写头对准哪个位置、当前纸带长什么样’都写下来。”

造物主：“……第 0 步：状态 q_0 ，纸带 **0 0 1 0 $\sqcup$...**，头对准第 1 格（最左的 0）。读 0 → 写 0、右移、留 q_0 。第 1 步：头对准第 2 格‘0’，读 0 → 右移。第 2 步：头对准第 3 格‘1’，读 1 → 进 q_{accept} 。**第 3 步：停机接受！**”

老谟："跑通了。那输入 000 呢？"

造物主："第 0、1、2 步一直右移扫三个 0，第 3 步读到第 4 格的 $\sqcup \rightarrow$ 进 q_{reject} ，**停机拒绝**。对，000 里确实没有 1。"

知识锚点：跑一台机器的完整"轨迹"——configuration (格局)

【概念深挖】一台机器"跑"到某时刻的完整快照叫一个**格局 (configuration)**。由于机器的一切状态由三样决定——**当前状态、纸带内容、读写头位置**——一个格局记作三元组 $(q, \text{当前纸带}, \text{头的位置})$ 。比如上面"含 1 机"跑 0010，可用记号 $\underline{q_0} 0010$ 表示"状态 q_0 ，头对准第一个符号 0"（在状态underline代表头位置，纸带只写出含输入那一段）。

一次转移 = 一个格局变到下一个格局： $(q, w\underline{a}u) \vdash (q', w\underline{b}cu)$ 表示"在 q 读到 a ，写成 b ，右移一格到 c ，变 q' "。**完整运行轨迹 = 格局序列**，从初始格局 $(q_0, \underline{w})$ 一直走到停机格局。"**接受/拒绝**" = 这条格局序列的终点停在 q_{accept} 或 q_{reject} 。

【工程实践】**格局 = 程序的运行状态**（寄存器值 + 内存内容 + 程序计数器）。调试器里看到的"断点处的完整现场"，就是工程版的"格局"；"一步步单步执行"就是"逐个格局走"。**把程序当格局序列来走，正是编译器/解释器/调试器的底层思维。**

第 5 轮：判定 vs 计算——语言、接受、停机

老谟："我们刚造了两台机器（含 1 机、以及下一轮要造的奇数机）。它们做的是同一件事：**对每个输入，给一个'接受/拒绝'的答案**。这类问题叫**判定问题 (decision problem)**。你给它串，它答'是'或'否'。现在我要给你一个更本质的抽象——**语言 (language)**。"

造物主："语言？不是'编程语言'那个语言吧？"

老谟："是形式语言：**某组符号 (字母表) 上所有串里，满足某个条件的那一堆**。比如'所有含 1 的二进制串'就是一个语言——**一台判定机'接受'的输入串的全体，恰好就是它对应的语言**。判定一个输入'在不在这个语言里'，等价于'机器接受不接受它'。"

造物主：".....所以'判定一个问题' = '判定一个串属于某个语言' = '这台机器停不停在接受状态'。那'停机'呢？"

老谟："停机 = 机器跑完了、给出最终答案。我们设计的这两台机器，对任何输入**必然停机**（扫描是有限的）。可你要小心：**不是每台机器对每个输入都停机**——有的机器可能永远停不下来（一直移、永远不停）。'**能停机并给出答案**'和'**停不下来**'的差别，是下一章那个震撼问题的命根子，你先记住这个区分。"

知识锚点：判定问题、语言、可判定性——三块基石

【概念深挖】

- **判定问题**：给一个输入串，要求回答“是/否”。一台图灵机把每个输入“接受（是）或拒绝（否）”。
- **语言**：字母表 Σ 上的语言是 Σ^* （所有有限串的集合）的任意子集。判定机 M 接受的语言记作 $L(M) = \{w \mid M \text{ 在输入 } w \text{ 停机于 } q_{\text{accept}}\}$ 。
- **可判定 (decidable)**：若存在一台图灵机 M ，对每个输入都能停机，且正确回答“属于 L 否”（属于→接受，不属于→拒绝），则称语言 L 可判定。“可判定” = “有一个算法能判断每个输入”——这正是“能算”的精确化。
- **可计算函数**：一台图灵机，若对每个输入 w 停机时纸带上留下某输出 $f(w)$ ，则它计算了函数 f 。函数可计算 = 存在图灵机算它。

关键区分（下一章的地雷）：“可判定”要求对每个输入都停机。如果一台机器“接受某些输入、但在别的输入上永远不停机”，那它不算“判定”这个语言——它可能“半判定”（只在‘是’时停机接受），但没给出“否”的完整答案。“总是停机且正确”才是可判定。下一章我们会看到：有的语言，任何机器都无法同时满足“对每个输入停机 + 正确回答”。

【工程实践】判定问题 = 布尔函数/谓词：`is_even(n)`、`contains_1(s)` 都是判定问题。**语言 = 数据子集/合法输入集**：一个 API 校验函数返回“合法/非法”，就是判定“输入是否属于‘合法语言’”。“可判定”在工程里就是“存在一个算法能对所有输入给出确定答案”——如果你论证某问题“不可判定”，等于宣布‘没有任何算法能对所有输入给出答案’，这是比‘很难/很慢’严重得多的结论。

第 6 轮：Church-Turing 论题——为什么它是“论题”不是定理

老谟：“我们现在手里有一个具体的机器了。可我要抛给你一句极其重要的话：‘任何直觉上可计算（可判定）的函数，都能用图灵机算出来。’你看这句话，能不能被‘证明’？”

造物主：“……‘直觉上可计算’这个说法，本身就不是一个数学对象。我没法证明一句话，因为它的主语‘直觉可计算’根本没被定义。”

老谟：“你答到点子上了。这就是为什么它叫 **Church-Turing 论题 (thesis)**，而不是定理——定理要在固定的公理/定义下证明；可‘直觉上可计算’这个词，没法放进形式系统，所以这个命题无法在数学内部证明。它更像一条经验定律：”

造物主：“……那我凭什么信它？”

老谟：“因为它从未被推翻。八十多年了，人们提出过无数种‘计算模型’—— λ 演算、递归函数、图灵机、寄存器机、甚至现代编程语言——所有被认真提出的、直觉可计算的东西，都能被图灵机算出

来。大家找到一个又一个'更强的机器'，最后发现**没有一台比图灵机更强**。证据如山，所以它被当作可计算性理论的地基——像公理一样被接受。"

造物主：".....那万一明天有人找到一个图灵机算不了、但直觉能算的东西呢？"

老谟："那我们的整座大厦就要重盖。但八十年没人做到——**所以现在，'能算'和'图灵机能算'，在我们这本书里就是同一个意思**。它的价值在于：它给了'算不出来'一个可以下手的精确定义——'**图灵机也算不出来**' = '**真的算不出来**'。下一章我们就用这个，去证那个震撼世界的结果。"

知识锚点：Church-Turing 论题——地基式信念，逐条拆解

【概念深挖】Church-Turing 论题：任何"直觉上可计算"的函数（或可判定的语言），都可被某台图灵机计算（判定）。

- **为什么是"论题"而非"定理"**："直觉上可计算"没有形式定义，命题无法在形式系统内表述，因此不可证。这是它与"定理"的根本区别。
- **证据**：① 多个独立模型（ λ 演算、递归函数、图灵机）殊途同归；② 所有提出的计算模型都彼此等价、且都不超过图灵机；③ 从未有人构造出"直觉可算但图灵机算不了"的例子。
- **作用**：它把"算不出来"转译成"图灵机也算不出来"——**给可计算性一个可着手的形式定义**。本书从此约定"可计算 = 图灵可计算"。

【历史溯源】1936 年，**图灵**（图灵机）与 **丘奇**（ λ 演算）**独立**提出刻画"可计算"的方案，随后证明二者等价。**"Church-Turing 论题"这一命名本身是后人的事**——归因要谨慎：图灵与丘奇分别给出的是"模型"，"论题"作为一条信念被广泛接受，是数学共同体后续共识的产物。**不夸大**：没有人"证明"了它；它是可计算性理论被接受的地基。

【工程实践】"**图灵完备**"作为分水岭：一个编程语言/系统若能力等价于图灵机（能表达所有图灵可计算函数），称"图灵完备"（如 C、Python、 λ 演算、甚至一些游戏里的红石电路/纸牌系统）。"**这个系统图灵完备吗**"是在问"**它能表达任意算法吗**"——图灵完备的系统能算的"上限"是相同的（都到图灵机这个天花板）；不够图灵完备的系统（如纯正则语言）则有明确的、表达不了的东西。**这是你判断一个工具"够不够强"最锋利的一把尺**。

第 7 轮：变体都等价——为下一章的"停机问题"铺路

老谟："你可能觉得这机器太寒酸：一条纸带、一个头、确定性的、还慢得离谱。**我告诉你一个反直觉的事实：把图灵机改造成'多纸带'双向纸带'非确定'等各种花样，算出来的东西和原来这台，完全一样**。"

造物主：".....什么？！多纸带、非确定性，听起来强大得多，结果和这条单纸带的是一回事？"

老谟："对。这些'变体'都能被原版图灵机模拟——多纸带可以'编码'到一条纸带上，非确定性可以用'广度优先搜索所有分支'来模拟。'能算的集合'没变，变的只是'快慢'和'方便'。这正好印证了 Church-Turing 论题背后那股力量：计算的'天花板'只有一个，所有变体都碰在同一面墙上。"

造物主：".....那为什么还要提这些变体？"

老谟："因为它们好用，也因为它们**不会改变'算得出来吗'这个问题的答案**。这给我们一个巨大的便利：**研究'能不能算'时，随便挑一个最顺手的模型，答案都一样**。下一章，我们要问'有没有图灵机算不出来的问题'——到那时，这些变体等价性就是我们的底气：**无论把机器改得多花哨，都躲不过那个'算不出来'的墙。**"

知识锚点：图灵机的变体——花哨归花哨，等价是真身

【概念深挖】常见的图灵机**变体**及其与原版的等价性（均"能算的集合相同"）：

- **多纸带图灵机**（k 条纸带、k 个读写头）：可被单纸带机**模拟**（把 k 条纸带"压"进一条，用记号/标记区分并跟踪各自读写头位置）。
- **双向纸带**：原版纸带已可任意左右移动，双向与否不改变能力（只是约定差别）。
- **非确定性图灵机 (NTM)**：转移关系允许"同一 (状态,符号) 有多个后继选择"。可被确定性图灵机**模拟**——用"广度优先"探索所有可能的分支（或用三段式"猜测→验证"技巧）。**关键是：非确定性和确定性图灵机接受的语言集合相同**（但可能快很多——这恰恰是第 8 章 P vs NP 的伏笔！）。

为什么等价性重要：因为我们要问"什么算得出来"，而**这些变体都算得同样多**，所以结论对哪个模型都成立。**这也支撑 Church-Turing 论题：**不是某个具体机器强大，而是"可计算"这个概念本身坚固。

【工程实践】**等价变体在工程里的对应：**同一算法可以写成单线程、多线程、流式、递归、迭代——"能算的东西"一样，只是实现策略不同（这正是"同一算法不同程序"的具体化）。**非确定性 = "并行/分支探索"的抽象：**工程里"猜一个解再验证"、剪枝搜索、回溯，都暗含非确定性的思想；而"把它模拟成确定性的"，就是"把分支展开成搜索"。

严格化走廊

位置说明：对话负责"直觉"——为什么需要计算模型、图灵机为什么长这样、变体为什么等价。本走廊负责"严谨"——把图灵机钉成可复写的形式定义（七元组、格局），给出本章的核心定理（**确定性：每步转移唯一**，以及**图灵机可模拟有限自动机**的构造），并亲手走完"二进制奇数判定机"的**完整格局序列**逐行验证。红线 1 的硬要求：每个证明标"证明类型 + 每一步依据"，不用"显然"。

定义：图灵机（七元组）——精确表述

一台**确定性图灵机**是一个七元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

其中：

- Q 是**有限非空状态集合**；
- Σ 是**输入字母表**，不含空白符 $\sqcup$ ；
- Γ 是**纸带字母表**，且 $\Sigma \subseteq \Gamma$ ， $\sqcup \in \Gamma \setminus \Sigma$ ；
- $\delta : (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 是**转移函数**；
- $q_0 \in Q$ 是**起始状态**；
- $q_{\text{accept}}, q_{\text{reject}} \in Q$ 是**接受/拒绝状态**， $q_{\text{accept}} \neq q_{\text{reject}}$ 。

约定说明： δ 的定义域**排除了接受/拒绝状态**——这是"停机"的形式化：到达 $q_{\text{accept}}/q_{\text{reject}}$ 后机器不再动作（没有下一跳），即**停机**。纸带为**无穷**（两端无限），除输入段外全为 $\sqcup$ 。**"计算"在此的定义即"从初始格局出发，反复执行转移，直到停机（或永不停止）"**——这和整本书"先约定后定理"的主线一致：图灵机是我们选定的计算约定。

定义：格局（configuration）——机器的一格快照

一台图灵机在某个时刻的**格局**是三元组 (q, uav, i) ，含义为：当前状态 q ；纸带上内容（含输入段与有限个非空白扩展）为 $\dots uav\dots$ ；读写头正对准符号 a 所在的那一格； i 记该格位置（有时直接用下划线标注头位置： $u\underline{a}v$ ）。

从格局到格局：若 $\delta(q, a) = (q', b, d)$ ，则

- $d = R$ （右移）时：格局从 $(q, u\underline{a}v)$ 变为 $(q', ub\underline{v}_{\text{首}})$ （写在 a 处得 b ，头右移到 v 的第一个符号）；

- $d = L$ (左移) 时: 变为 $(q', u_{\text{末}} \underline{b} v)$ (写在 a 处得 b , 头左移到 u 的最后一个符号)。

初始格局: 输入串 w 写在纸带最左段, 头对准 w 的第一个符号, 状态 q_0 。**停机格局:** 状态为 q_{accept} 或 q_{reject} 。**语言** $L(M) = \{w \in \Sigma^* \mid M \text{ 从初始格局出发, 最终停在 } q_{\text{accept}}\}$ 。

证明类型标注: 本章核心证明共两处——**定理 1 (每步转移唯一) 用直接证明** (对"转移函数是函数"的形式论证); **定理 2 (图灵机可模拟有限自动机) 用构造性证明** (显式构造出模拟任意给定有限自动机的图灵机, 并验证其行为一致)。后者正是蓝图第 6 章必含证明。

关于对话: 对话第 3 轮把七元组拆给造物主推, 靠的是"引导式推导" (让他自己认出每个符号的职责); **形式化定义与格局序列在本走廊。** 对话负责"为什么会想到七元组/格局", 走廊负责"怎么把它写标准"——与前五章及册 01-06 口径一致。

定理 1 (确定性: 每步转移唯一)

前提: $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ 是确定性图灵机, M 处于非停机状态 q 且读写头读到符号 a 。

结论: 机器下一步的动作**唯一确定** (要写的新符号、移动方向、进入的新状态, 都只有一个)。

证明类型: 直接证明 (用"转移函数是单值函数"这一形式性质)。

证明:

- 第 1 步: **回忆转移函数定义:** $\delta : (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 。(依据: 图灵机定义—— δ 是定义在"非停机状态 $\times$ 符号"上的函数。)
- 第 2 步: **"函数"即"单值映射":** 对定义域中每个元素 (q, a) , $\delta(q, a)$ 是值域 $Q \times \Gamma \times \{L, R\}$ 中的**恰好一个**元素 (而不是一个集合)。这是"函数 (function)"这一概念的定义性质: 每个输入恰好对应一个输出。(依据: 映射/函数的定义——单值性。)
- 第 3 步: **当前局面落在定义域内:** 机器处于非停机状态 q (非停机, 故 (q, a) 属于 δ 的定义域), 头读到符号 $a \in \Gamma$ 。(依据: 前提 + Γ 是纸带字母表。)
- 第 4 步: **结论:** 由第 2、3 步, $\delta(q, a) = (q', b, d)$ 唯一确定, 故"写 b 、按 d 移动、进 q' "唯一确定。■

依据标注: 步 1 用"定义" (δ 的签名); 步 2 用"函数的单值性" (这是"确定性"的来源——同一输入只有一种结局, 这正是图灵机区别于"非确定"的本质); 步 3 用"前提+定义域"; 步 4 收束。**读法:** 这个定理的力气全在步 2——**"确定性"不是机器的物理性质, 是"转移函数是函数 (单值)"这一约定的直接推论。**若把 δ 改成"转移关系" (允许一对多), 就得到非确定性图灵机——那是第 8 章的伏笔。

定理 2 (图灵机可模拟有限自动机) —— 构造性证明

前提：任意给定一个**确定性有限自动机 (DFA)** $A = (Q_A, \Sigma_A, \delta_A, q_{0A}, F_A)$ ，其中 $\delta_A : Q_A \times \Sigma_A \rightarrow Q_A$ 是状态转移函数， $F_A \subseteq Q_A$ 是接受状态集合。(DFA 是"有限记忆"的自动机：只认当前状态和当前读到的符号，无外部存储。)

结论：存在一台图灵机 M ，使得对每个输入串 $w \in \Sigma_A^*$ ： M 接受 $w \Leftrightarrow$ DFA A 接受 w (即 $L(M) = L(A)$)。

证明类型：构造性证明 (显式构造 M 并验证)。

证明：

- 第 1 步：**构造 M 的状态集**：令 M 的状态集 $Q = Q_A \cup \{q_{\text{accept}}, q_{\text{reject}}\}$ ，输入字母表 $\Sigma = \Sigma_A$ ，纸带字母表 $\Gamma = \Sigma_A \cup \{\square\}$ ，起始状态 $q_0 = q_{0A}$ 。(依据：模拟策略——用"当前状态"承载 DFA 的记忆，用纸带只承载输入。)
- 第 2 步：**定义转移函数 δ** ：对每个 $q \in Q_A$ 、每个 $a \in \Sigma_A$ ，令

$$\delta(q, a) = (q', a, R), \quad \text{其中 } q' = \delta_A(q, a).$$

含义：**读到输入符号 a 时， M 把 DFA 的转移 $\delta_A(q, a) = q'$ 复刻过来——把当前状态改成 q' 、符号原样写回、向右移一格继续读下一个输入符号。**(依据：模拟 DFA 的"从左到右扫描输入、每步换状态"的行为。)

- 第 3 步：**定义边界 (读到空白) 的动作**：对每个 $q \in Q_A$ ，令
 - 若 $q \in F_A$ (DFA 的接受状态)： $\delta(q, \square) = (q_{\text{accept}}, \square, R)$;
 - 若 $q \notin F_A$ ： $\delta(q, \square) = (q_{\text{reject}}, \square, R)$ 。含义：**当读写头扫过整个输入、读到第一个空白 $\square$ 时，输入已读完**；此时若 DFA 停在接受状态 F_A 就接受，否则拒绝。(依据：DFA 的接受定义——读完整个输入、结束状态在 F_A 中才接受。)
- 第 4 步：**验证 $L(M) \subseteq L(A)$** ：若 M 接受 w ，则 M 读完全部输入后在 q_{accept} 停机；由第 2、3 步构造，这只在 DFA 读完整串后停于 F_A 时发生，故 DFA A 接受 w 。(依据：第 2、3 步的定义—— M 的接受由"DFA 终态 $\in F_A$ "触发。)
- 第 5 步：**验证 $L(A) \subseteq L(M)$** ：若 DFA A 接受 w ，则按 δ_A 逐步转移，读完 w 后停在某个 $q \in F_A$ ；由第 2、3 步， M 复刻每一步转移，读到 $\square$ 时停在 q_{accept} ，故 M 接受 w 。(依据：第 2 步复刻转移 + 第 3 步接受触发。)
- 第 6 步：**由第 4、5 步得 $L(M) = L(A)$ 。** ■

依据标注：步 1、2、3 是"构造"（显式给出 M 的七元组）；步 4、5 是"验证"（双向包含，DFA 接受 $\Leftrightarrow M$ 接受）；步 6 收束。**读法：**这个构造的精髓是——**DFA 只有"有限状态"这个记忆，图灵机把"状态"原封不动搬过来，再把纸带当成"读输入"的带子**，于是有限自动机能做的，图灵机全能做。**这印证了"图灵机是最强大的模型之一"：连有限自动机都能模拟。**注意：图灵机比 DFA 强得多（多了纸带这个无限存储），模拟只是"弱模型 $\subset$ 强模型"的一例。

例子（亲算）：二进制奇数判定机——完整格局序列逐行验证

问题：构造一台判定"二进制串表示奇数"的图灵机，即接受所有以 '1' 结尾的二进制串（如 1011、1），拒绝以 '0' 结尾的（如 1010）。（约定：非空输入，最右符号即个位；二进制奇数个位必为 1。）

机器设计（三个状态 + 规则表）：

- 状态集 $Q = \{q_0, q_{\text{accept}}, q_{\text{reject}}\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, \square\}$, 起始 q_0 。
- 转移表：
 - q_0 : 读 0 $\rightarrow$ 写 0、右移、留 q_0 （继续扫描）；
 - q_0 : 读 1 $\rightarrow$ 写 1、右移、留 q_0 ；
 - q_0 : 读 $\square \rightarrow$ 写 $\square$ 、**左移**、进 q_1 （走到最右非空格，回退看个位）；
 - q_1 : 读 1 $\rightarrow$ 写 1、右移、进 q_{accept} （个位是 1 $\rightarrow$ 奇数，接受）；
 - q_1 : 读 0 $\rightarrow$ 写 0、右移、进 q_{reject} （个位是 0 $\rightarrow$ 偶数，拒绝）。

逻辑核验：二进制数奇偶只由个位（最右输入符号，即最低位）决定——末尾是 1 即奇数，末尾是 0 即偶数。机器扫描到最右，回退看个位：是 1 接受、是 0 拒绝。**先理解设计，再走轨迹。**

输入 1011（二进制 11，奇数，应接受），完整格局序列（用下划线标头位置）：

步	状态	纸带（头位置用下划线）	依据
初始	q_0	<u>1</u> 011□	头对准首字符，状态 q_0 （初始格局）
1	q_0	1 <u>0</u> 11□	读 1 $\rightarrow$ 写 1、右移、留 q_0
2	q_0	10 <u>1</u> 1□	读 0 $\rightarrow$ 写 0、右移、留 q_0
3	q_0	101 <u>1</u> □	读 1 $\rightarrow$ 写 1、右移、留 q_0
4	q_0	1011 <u>□</u>	读 1 $\rightarrow$ 写 1、右移、留 q_0
5	q_1	1011 <u>□</u>	读 $\square \rightarrow$ 写 $\square$ 、 左移 、进 q_1
6	q_{accept}	1011 <u>□</u>	读 1（个位） $\rightarrow$ 写 1、右移、进 q_{accept}

第 6 步：停机接受。反向验证：二进制 $1011 = 8 + 0 + 2 + 1 = 11$ ，11 是奇数 ✓；机器接受，正确。

输入 1010（二进制 10，偶数，应拒绝），完整格局序列：

步	状态	纸带	依据
初始	q_0	<u>1</u> 0 1 0 □	初始格局
1	q_0	1 <u>0</u> 1 0 □	读 1 → 右移
2	q_0	1 0 <u>1</u> 0 □	读 0 → 右移
3	q_0	1 0 1 <u>0</u> □	读 1 → 右移
4	q_0	1 0 1 0 <u>□</u>	读 0 → 右移
5	q_1	1 0 1 0 <u>□</u>	读 □ → 左移、进 q_1
6	q_{reject}	1 0 1 0 <u>□</u>	读 0（个位）→ 进 q_{reject}

第 6 步：停机拒绝。反向验证：二进制 $1010 = 8 + 0 + 2 + 0 = 10$ ，10 是偶数 ✓；机器拒绝，正确。

核算记录：① 两例均在第 5 步“读到 □ 左移”进入 q_1 ，头落在最右输入符号（个位）上——用输入长度 4 手工核对，4 个符号扫描后第 5 格是 □，左移即回到第 4 格（个位），正确；② 1011 个位 1 → 接受， 1010 个位 0 → 拒绝，均与“末尾 1 奇、0 偶”的奇偶判定一致；③ 转移表每个（状态,符号）恰好一个动作，符合定理 1 的确定性。**反向验证通过。**

反例/边界：形式定义 vs 直觉的差距

- 图灵机可无限跑不停：**形式定义允许机器“永远不停机”（无休止移动/改写）。**直觉“算法必然结束”在这里不成立**——是否存在“对每个输入都停机”的机器，恰恰是“可判定”与“不可判定”的分界（下一章的核心）。
- 纸带“无穷”是理想化：**真实计算机内存有限，图灵机纸带无限。**图灵机是“如果内存无限够用，能算到什么程度”的极限模型**——它刻画的是“算法上能不能”，不是“工程上内存够不够”。
- 转移函数必须定义在非停机状态上：**若把 $q_{\text{accept}}/q_{\text{reject}}$ 也放进 δ 定义域，机器就不会“停”而是继续跑——“停机”是靠“停机状态无转移”约定出来的，不是机器的物理属性。
- “可判定”要求对每个输入都停机且正确：**一台只在“接受”时停机、在“拒绝”时永不停止的机器，**不算判定该语言**（只是半判定）。**“总是停机”是“可判定”里最硬的一条。**
- 变体等价不等于变体没用：**非确定图灵机“能算的集合”与确定性相同，但**可能快得多**——“算得出”和“算得快”是两回事，这正是第 8 章 P vs NP 的分界。

知识点总结

知识点（本章推出的核心概念）

- **图灵机（七元组）**： $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ 。四个零件：纸带（无限内存）+ 读写头（存取单元）+ 有限状态 + 转移函数（规则表=程序）。**每步"读→写→移→变状态"**。
- **转移函数** $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ ：对每个（非停机状态, 符号）给**恰好一个**结果——**确定性**的来源。
- **格局（configuration）**： $(q, \text{纸带}, \text{头位置})$ ，一次转移=一个格局变到下一个；**运行轨迹 = 格局序列**。
- **语言**： Σ^* 的子集； $L(M) = M$ 接受的串的集合。
- **可判定（decidable）**：存在一台图灵机，**对每个输入都停机**且正确回答"属于 L 否"。**"可判定"="能算"**。
- **可计算函数**：存在图灵机对每个输入停机时输出对应值。
- **Church–Turing 论题**：直觉可计算 = 图灵可计算。**是"论题"不是定理**（主语无法形式定义），从未被推翻，作为地基被接受。
- **变体等价**：多纸带、双向、非确定图灵机与原版**接受的语言集合相同**。

历史结论（本章涉及的史料）

- **图灵（1936）**：论文《论可计算数及其在判定问题上的应用》，构造图灵机刻画"可计算"。
- **丘奇（同期）**： λ 演算独立刻画同一件事；**二人独立发现、殊途同归**。
- **Church–Turing 论题**：命名与确立是共同体后续共识；**从未被证明，也从未被推翻**。
- **谨慎归因**：图灵**没有"发明计算机"**——图灵机是数学概念，电子计算机是工程与数学多年共同推进的产物；二战破译 Enigma 是工程应用，与图灵机发明是两回事。

工程实践结论（本章知识在真实工程里怎么用）

- **图灵机 = 最简指令集 CPU 的思想**：内存（纸带）+ 存取单元（读写头）+ 状态（寄存器）+ 规则表（指令集）。
- **"图灵完备"是分水岭**：语言/系统能力等价于图灵机就能写任意算法；不够图灵完备则有明确的表达上限。**判断工具"够不够强"就问"图灵完备吗"**。
- **格局 = 调试现场**：寄存器+内存+程序计数器，单步执行就是逐个格局走。
- **判定问题 = 布尔函数**；**"不可判定" = "没有任何算法能对所有输入给答案"**——这是远比"难/慢"严重的结论。

计算训练场

位置说明：对话负责"为什么需要图灵机、它为什么长这样"，这里负责"怎么亲手造机器、怎么走轨迹"。下面三题**必须动笔**，每题 10 分钟以上，先自己写再对答案。**难度递进：易→中→难**。重点：
① 设计状态/转移表；② 亲手走完整格局序列；③ 体会"什么不是图灵机可算"的直觉。

题 1（易）：设计判定"串含 1"的图灵机——完整状态表

题干：老谟让造物主设计一台图灵机，判定输入二进制串是否**至少含一个 '1'**。写出：状态集、纸带字母表、完整转移表，并解释每个规则的作用。然后亲手走输入 0010 的格局序列。

完整解答：

- 状态集： $Q = \{q_0, q_{\text{accept}}, q_{\text{reject}}\}$ ；输入字母表 $\Sigma = \{0, 1\}$ ；纸带字母表 $\Gamma = \{0, 1, \square\}$ ；起始 q_0 。
- 转移表：

当前状态	读到的符号	写	移动	下一状态	作用
q_0	0	0	R	q_0	不是 1，继续右扫
q_0	1	1	R	q_{accept}	找到 1，接受
q_0	$\square$	$\square$	R	q_{reject}	走到底没 1，拒绝

- 逻辑：**从最左向右扫，碰到 1 立即接受；碰到空白（输入结束）拒绝。

输入 0010 的格局序列：

步	状态	纸带	依据
初始	q_0	<u>0</u> 010□	初始格局
1	q_0	0 <u>0</u> 10□	读 0→右移
2	q_0	00 <u>1</u> 0□	读 0→右移
3	q_{accept}	001 <u>0</u> □	读 1→进接受

第 3 步停机接受（0010 含 1 ✓）。

常见错误：① 状态设太多/太少（一个扫描状态足够，接受/拒绝各一）；② 忘记处理空白 $\square$ （否则扫描到输入末尾不知怎么办）；③ 移动方向写反（应从左向右，R）。

题 2（中）：二进制奇数判定机——完整运行轨迹

题干：使用严格化走廊里设计的"二进制奇数判定机"（转移表见走廊），亲手走输入 1011 与 1010 的完整格局序列，每步标注依据；并核对最终接受/拒绝是否符合奇偶性。

完整解答（转移表同走廊： q_0 右扫，遇 $\sqcup$ 左移进 q_1 ， q_1 看个位 1 接受、0 拒绝）：

输入 1011（=11，奇数）：

步	状态	纸带	依据
初始	q_0	<u>1</u> 0 1 1 $\square$	初始格局
1	q_0	1 <u>0</u> 1 1 $\square$	读 1→R
2	q_0	1 0 <u>1</u> 1 $\square$	读 0→R
3	q_0	1 0 1 <u>1</u> $\square$	读 1→R
4	q_0	1 0 1 1 <u>$\square$</u>	读 1→R
5	q_1	1 0 1 <u>1</u> $\square$	读 $\sqcup$ →L、进 q_1
6	q_{accept}	1 0 1 1 <u>$\square$</u>	读 1(个位)→接受

接受（11 是奇数 ✓）。

输入 1010（=10，偶数）：

步	状态	纸带	依据
初始	q_0	<u>1</u> 0 1 0 $\square$	初始格局
1	q_0	1 <u>0</u> 1 0 $\square$	读 1→R
2	q_0	1 0 <u>1</u> 0 $\square$	读 0→R
3	q_0	1 0 1 <u>0</u> $\square$	读 1→R
4	q_0	1 0 1 0 <u>$\square$</u>	读 0→R
5	q_1	1 0 1 <u>0</u> $\square$	读 $\sqcup$ →L、进 q_1
6	q_{reject}	1 0 1 0 <u>$\square$</u>	读 0(个位)→拒绝

拒绝（10 是偶数 ✓）。

反向核算：两个例子的二进制值已手算核实（1011=11 奇、1010=10 偶）；机器第 5 步"读到 $\sqcup$ 左移"恰好落在最右输入符号（个位）上，个位 1→接受、0→拒绝，逻辑与结果一致。

常见错误：① 奇数机第 5 步**必须左移**（若右移会读到第二个 $\sqcup$ 而出错）；② 把"输入结束"（ $\sqcup$ ）当成个位判断——必须先左移回到个位再判断；③ 个位判断反了（0 是偶数应拒绝，1 是奇数应接

受)。

题 3 (难)：什么"不是"图灵机可计算的——直觉辨析

题干：老谟列出三样东西，问造物主哪个"不是图灵机可计算（可判定）的"：(a) 判定一个二进制串是不是奇数；(b) 判定一个**任意的**图灵机，在给定输入上是否**最终停机**（即"停机问题"）；(c) 判定一个自然数是不是质数。请逐一分析，并说出 (b) 为什么和 (a)、(c) 有本质区别。

完整解答：

- **(a) 可计算：**本章已构造出奇数判定机，对每个输入都停机并正确接受/拒绝。所以"判断二进制奇数"是**可判定的**。
- **(c) 可计算：**判断质数有确定的算法（如试除 $\sqrt{n}$ 内的所有整数），可以写成图灵机，对每个输入都停机。所以"判断质数"**可判定**。
- **(b) 需要深入想：**"给定一台图灵机 M 和一个输入 w ，问 M 在 w 上最终是否停机"——这个问题是否可判定？**本章还不能下结论**（下一章将证明它**不可判定**）。这里的**直觉关键在于**：它的问题对象是"图灵机本身"——你要判定的是"这台机器会不会停下"，而机器可能**无限运行、永远不停**。这和 (a)、(c)（输入是有限的数据串，判定的过程总能扫完输入）有本质区别：停机问题的困难在于，**你无法通过"跑一遍看看"得到答案**，因为你不知道它什么时候才"算完"（它可能永远不算完）。

答：(b) 是最可疑、最可能"不是图灵机可计算"的——它正是下一章要证明的**停机问题不可判定**。本章先种下直觉："图灵机可算"只覆盖"总能停机并给出答案"的问题；而有些问题，本质上是"问机器自己会不会停"——这类问题把图灵机当成输入，会引出深刻的悖论。

常见错误：① 误以为 (b) 也可计算，理由是"跑一遍看它停不停"——但"跑一遍"没法处理"永不停止"的情况（你永远等不到它停）；② 把"很难算"和"不可计算"混为一谈——质数判定很慢但可计算，(b) 是**原则上**算不出来；③ 没意识到 (b) 的输入是"图灵机+输入"这种**高阶对象**，这正是它特殊之处。

向导便签

这一章咱们干了什么：把“算”这个你天天用的词，从一团直觉钉成了一件能机械执行的东西——**图灵机**。你认识了它的七元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ 、四个零件（纸带+读写头+状态+规则表）、每步“读→写→移→变状态”的节奏；亲手造了“含 1 机”，亲手走了“奇数判定机”的完整格局序列；还摸到了那个地基级的信念——**Church-Turing 论题**。

钉死一句话：“能算” = “图灵机能算”（Church-Turing 论题）；“可判定” = “存在一台对每个输入都停机且正确回答的图灵机”。变体（多带/双向/非确定）都等价，算不出来的墙只有一面。

记住：这一章最大的转折，是问题对象从“有限的数据”跳到“机器本身”。我们已经能问“这台机器会不会停”这种把机器当输入的问题了——它既是“可计算”的边界，也是下一章那枚炸弹的引信。下一章，**可判定性与停机问题**——我们要亲手用反证法证明：**有一类问题，任何图灵机都算不出来**。那将是全册第一次直面“算的边界”。

造物主手记

我曾踩的坑：

我第一次想“计算的模型”，以为要造一台“更快更强的超级计算机”，被老谟点醒：**图灵机要的不是快，是最朴素到“什么都能算”的地基**。设计奇数机时我差点在第 5 步写成“右移”，结果会撞上第二个空白符——**必须先左移回到个位再判断**，这步最易错。走 1010 的轨迹时我一度以为“机器跑完了”就是接受，忘了要看**停在哪个状态**——1010 跑完停在 q_{reject} ，是**拒绝**不是接受，接受/拒绝取决于最终状态，不是“跑没跑完”。最要命的是理解 Church-Turing 论题：我一开始问“这怎么不证明一下”，才明白**“直觉可计算”没法放进形式系统，所以它是论题不是定理**——它靠“从未被推翻”的地基地位站着。

顿悟时刻：

真正开窍是亲手走完 1011 的格局序列那一刻——看着那一串“读→写→移→换状态”的格子，我忽然懂了：“算”**不是一个神秘的黑箱，它就是“按规则表一步步挪读写头”这个笨到不能再笨的过程**。而这台笨机器，却能算质数判定、最短路径——因为任何算法都能被翻译成一张足够长的规则表。而 Church-Turing 论题那句“直觉可计算 = 图灵可计算”，让“算不出来”第一次有了可下手的形式定义。**我手里第一次握住了一把能问“什么算不出来”的钥匙——下一章，就是用它开门的时候了。**

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 设计"串首为 1"的判定机

设计一台图灵机, 判定输入二进制串的**第一个符号是否为 '1'** (是则接受, 否则拒绝)。写状态集、转移表, 并走输入 101 的格局序列。

完整解答: 只需一个扫描状态: 起始 q_0 , 头对准首字符时: 读 1 → 接受; 读 0 → 拒绝。状态集 $Q = \{q_0, q_{\text{accept}}, q_{\text{reject}}\}$ 。转移表: q_0 读 1 → 写 1、R、进 q_{accept} ; q_0 读 0 → 写 0、R、进 q_{reject} 。(注意: 这只判断首字符, 无需扫描整串。) 走 101: 初始 q_0 头对准首字符 '1' → 读 1 → 进 q_{accept} , **第 1 步即接受** ✓ (首字符是 1)。

题 B (计算, 中) : 奇数机走 101 与 100

用本章奇数判定机的转移表 (q_0 右扫→遇 $\sqcup$ 左移→ q_1 看个位), 完整走输入 101 与 100 的格局序列, 并核对奇偶性。

完整解答:

- 101 (=5, 奇数, 应接受): 初始 q_0 头对准 '1' → 步1 读 1→R; 步2 读 0→R; 步3 读 1→R; 步4 读到 $\sqcup$ →L 进 q_1 , 头落第 3 格 '1'; 步5 q_1 读 1 (个位) →**接受**。5 是奇数 ✓。
- 100 (=4, 偶数, 应拒绝): 初始 q_0 头对准 '1' → 步1 读 1→R; 步2 读 0→R; 步3 读 0→R; 步4 读到 $\sqcup$ →L 进 q_1 , 头落第 3 格 '0'; 步5 q_1 读 0 (个位) →**拒绝**。4 是偶数 ✓。

题 C (概念, 检查理解)

(a) 为什么 Church–Turing 论题是"论题"而不是"定理"? (b) 什么是"可判定"? 为什么它要求"对每个输入都停机"? (c) 图灵机的变体 (多带、非确定) 能算的东西比原版多吗? 为什么? (d) 为什么说"判断二进制奇数"可计算, 而"判断某图灵机在给定输入上是否停机"在下一章将被证明不可计算? 两者的直觉差别在哪?

完整解答: (a) 因为"直觉上可计算"没有形式定义, 无法放进形式系统内证明; 它靠"从未被推翻"的累积证据作为可计算性理论的地基被接受。(b) "可判定" = 存在一台图灵机, 对每个输入都停机, 且属于 L 就接受、不属于就拒绝。要求"对每个输入都停机"是因为: 若某输入上机器永不停机, 就永远无法给出"否"的完整答案, 只能算"半判定"。(c) 不多。多纸带、非确定等变体都能被原版图灵机模拟, **接受的语言集合相同**; 变体可能更快/更方便, 但"能算的集合"没变 (这正是 Church–Turing 论题坚固性的佐证)。(d) "判断二进制奇数"的输入是**有限数据串**, 判定过程总能扫完输入后停机; 而"判断图灵机在给定输入上是否停机"的问题是问"机器会不会无限运行", 你无法通过"跑一遍"得到答案 (它可能永不停止) ——它把**机器本身**当输入, 引向自指与悖论, 这正是下一章停机问题不可判定的根源。

预告

这一章，你亲手把“算”从直觉钉成了图灵机——七元组、格局、Church-Turing 论题，还有一台能自己走的奇数判定机。你手里第一次有了一把能问“什么算得出来”的钥匙，也已经在题 3 里隐隐碰到那堵墙：“判定某台图灵机在给定输入上会不会停机”——这个问题，到底算不算得出来？

下一章，**可判定性与停机问题**——我们要用一个比全书任何证明都震撼的反证法，证明一个让整个数学界震动的结果：**存在一个问题，是任何图灵机都算不出来的**。它叫**停机问题（Halting Problem）**，而证明它的钥匙，你已经在前几章一步步备好了：把机器自己当输入、让机器问自己会不会停、构造一个“恰好让你矛盾”的 D。从“什么算得出来”到“什么注定算不出来”，这一步跨出去，就是可计算性理论的**心脏**。你准备好走进那扇门了吗？

第 07 章 · 可判定性与停机问题——算不出来的东西

本章从哪个问题出发：上一章我们亲手造出了图灵机，还认了 Church-Turing 论题——“直觉可计算”就是“图灵可计算”。可你心里一直压着一个上一章留下的问题：“判定某台图灵机在给定输入上会不会停机”——这个问题，到底算不算得出来？我们造过判定“二进制奇数”的机器，它总能给出答案；可“问一台机器会不会停”这种问题，连“跑一遍看看”都靠不住——因为它可能永远不停，你永远等不到它给出答案。有没有可能，有些问题是**数学上注定算不出来的**？推导到哪个结论：你要亲手完成全册最震撼的一段证明——**停机问题不可判定定理**。你会看到人类第一次“捉住”了一个**任何图灵机都算不出来的**问题，用一个巧到极致的反证法：先假设“能判定停机”的机器 H 存在，再用 H 造出一台会“吃自己”的机器 D ，最后让 D 输入它自己——两个分支都撞上矛盾。你还会看清“可判定”与“可识别”的差别：可判定的语言都可得识别，但有些语言**只能被识别、不能被判定**。

核心认知：**存在问题在数学上不可能算出来的——这不是“还没人想出算法”，而是“证明不存在任何算法”**。停机问题不可判定，是图灵 1936 年那篇开创性论文里最重要的一个结果。这一章是全册的皇冠：它不仅回答了“有没有算不出来的东西”（有），还教给你一个可复写一整生的证明工具——**反证法 + 自指**。下一章我们要去问另一个问题：“算得出”之外，还有“算得快不快”——那就是复杂度理论（P vs NP）的世界了。

引子

老谟书房里，白板上贴着我上一章写的三行字：

"判定二进制奇数——可计算。判定质数——可计算。判定某台图灵机在给定输入上会不会停机——？？？"

我把第三行圈起来，问老谟："这一行，到底填'可计算'还是'不可计算'？"

老谟盯着那行字，忽然说："你先把'不可计算'这四个字，逼到墙角。你说'不可计算'，是什么意思？"

".....就是没有任何算法能判定它？"

"那你怎么证明'没有任何算法'？'**我没想出算法**'，只说明'**我笨**'，不说明'**没有算法**'。你打算怎么证明一件'不存在'的东西？"

我愣住了。上一章的机器我都造得出来——奇数机、质数机，都能对每个输入停下给答案。可这一行，我连"从哪下手"都不知道。

"更麻烦的是，"老谟接着说，"你看这个问题——'**这台机器会不会停**'。它的输入里，装着一台机器。那.....如果我把一台机器，喂给它自己呢？"

我低头看着那行字，第一次感到一种说不出的不安。仿佛那扇写着“算不出来”的门，就等着我亲手推开。

对话引擎

第 1 轮：可判定的定义——衔接上一章

老谟：“先把上一章的地基钉牢。我们说一个语言（一个问题）**可判定（decidable）**，是什么意思？”

造物主：“.....就是存在一台图灵机，对**每个输入都停机**，输入属于这个语言就接受、不属于就拒绝。判断二进制奇数就是这么干的——不管给什么串，机器都能停下给出答案。”

老谟：“好。这里最硬的一条是——**对每个输入都停机**。为什么这一条这么要命？”

造物主：“.....因为如果某个输入上机器永远不停，你就永远拿不到‘否’这个答案。可能它是‘算得很慢’，也可能是‘根本没答案’——你分不清。”

老谟：“你抓到了命门。**可判定的本质，是‘每个输入都给你一个确定的‘是/否’**”。但我要提醒你：还有一类机器，它对属于语言的输入**总是停机接受**，但对不属于的输入，**可能停机拒绝、也可能永远不停**。这类机器不‘判定’，它只‘识别’——我们把它叫**可识别（recognizable）**。这一章你会看到，这两个词的差别，藏着个大秘密。”

知识锚点：可判定 vs 可识别——一个字的差别，天壤之别

【概念深挖】设 L 是字母表 Σ 上的语言。

- **可判定 (decidable / recursive)**：存在一台图灵机 M ，对每个输入都停机，且 $w \in L \iff M$ 接受 w (否则拒绝)。
- **可识别 (recognizable / recursively enumerable)**：存在一台图灵机 M ，使得 $w \in L \iff M$ 接受 w ；但对 $w \notin L$ ， M 可能停机拒绝，也可能永远不停机。

关键差别：可判定要求“每个输入都能在有限时间内给出‘是/否’”；可识别只要求“属于语言的，一定在有限时间内接受”——对不属于的，允许它“赖着不停”。

关系：可判定 $\subset$ 可识别 (凡可判定必可识别，因为判定机本身就是一台“总停机的识别机”；但反过来不成立——本章就要造一个“可识别却不可判定”的样板)。

【工程实践】“可判定 vs 可识别”在工程里的对应：**判定 = 保证返回结果的算法** (如 API 校验一定返回合法/非法)；**识别 = 可能无限等待的算法** (如“等一个进程结束”——如果它永不结束，你就永远得不到答案)。“半判定/可识别”是工程里“可能挂起”的抽象：你给了算法，却无法保证它在每个输入上都给答案。

第 2 轮：停机问题 HALT——把“会不会停”变成一个判定问题

老谟：“现在正式定义那个问题。设 $\langle M \rangle$ 表示图灵机 M 的编码 (把一台机器的七元组写成一个字符串，就像把程序源码存成文本)。再给一个输入串 w 。定义：

停机问题 HALT：给定 $(\langle M \rangle, w)$ ，问—— M 在输入 w 上是否最终停机？

它要你答的，是一个“是/否”。所以它形式上一个判定问题，可以被问“它可判定吗”。

造物主：“……所以停机问题，就是把‘这台机器会不会停’，变成‘这个输入串属不属于某个语言’。可它的输入是一个编码好的机器—— $\langle M \rangle$ ——这感觉和以前的‘二进制串’完全不是一个东西了。”

老谟：“你说的‘不是一个东西’，恰恰是关键。这台机器不只是‘数据’，它是‘程序’——可它又被当成了‘数据’喂给别人。程序和数据在别的地方是分开的；在停机问题里，它们合体了：一个编码 $\langle M \rangle$ 既是‘一个输入串’，又‘代表一台能跑的机器’。你记住这个‘合体’，它是这一章一切的种子。”

知识锚点：机器编码 $\langle M \rangle$ 与停机问题——程序即数据

【概念深挖】**机器编码 $\langle M \rangle$** ：图灵机是**有限对象**（有限状态集、有限字母表、有限转移表），所以可以按一套固定规则，把它编码成一个**有限字符串 $\langle M \rangle$** （比如用 0/1 或某个字母表）。这就像：任何程序（有限源码）都能存成文本文件。**"程序即数据"**：一台机器既能被"运行"，又能被"当作输入串传给别的机器"。

停机问题 HALT：语言（问题）

$$\text{HALT} = \{ (\langle M \rangle, w) \mid \text{图灵机 } M \text{ 在输入 } w \text{ 上最终停机} \}.$$

问题：HALT 是否**可判定**？即——是否存在一台图灵机 H ，对**每个** $(\langle M \rangle, w)$ 都停机，且正确回答" M 在 w 上停机吗"。

为什么不能"跑一遍看看"：若 M 在 w 上不停机，你"跑一遍"就永远等不到它停——**你无法区分"它很慢"和"它永不停"**。所以"跑一遍"不是判定算法。

【工程实践】**"程序即数据" = 解释器/编译器/反病毒**：Python 解释器把 `.py` 文本（数据）当程序跑；编译器把源码（数据）编译成可执行文件（程序）；杀毒软件把程序（数据）当字符串扫描。**"把程序当数据处理"是工程的核心能力**——而停机问题正是在问：这种"把程序当数据来判定的能力"，有没有极限？

第 3 轮：输入可以是"机器自己"——自指的预演

老谟："在动手之前，我们先做一件热身：**让一台机器吃它自己**。你还记得上一章我们造的'判定二进制奇数'的机器吧？它的输入是二进制串。那如果有人给它一个'它自己的编码' $\langle \text{奇数机} \rangle$ ——虽然那串 0/1 看起来像乱码，但机器还是会按规则去扫、去判——**它根本不懂'这串代表什么，它只是机械地走规则'。**"

造物主：".....所以'机器吃自己'不是魔法，就是把 $\langle M \rangle$ **当普通输入喂给 M** ？就像把一段 Python 源码 `print("hello")` 当作字符串打印出来，Python 也能处理——只是这次它打印的是'自己'。"

老谟："对，完全对。**机器对输入没有'敬畏心'**——它不知道这串是不是'它自己'，只是机械地扫描、转移、写符号。**正因为如此，'机器吃自己'是完全合法的计算。**这一章的证明，就要用这个'吃自己'当发动机。你先把' $\langle D \rangle$ 能被 D 自己读'这个事实咽下去——它不玄，它就是'程序当数据'而已。"

知识锚点：自指为何合法——机器无敬畏

【概念深挖】**自指 (self-reference) 的合法性**：图灵机对输入没有“语义理解”，它只做机械的“读→写→移→换状态”。因此，把机器 M 的编码 $\langle M \rangle$ 作为输入喂给 M 自己（甚至喂给任意机器），是**完全合法、可执行的**。这不同于自然语言的“这句话是假的”那种可疑的语义自指——在图灵机世界里，自指是**纯语法的、机械的**：一个有限字符串被当作输入，而已。

为什么这是地基：停机问题不可判定定理，正是建立在这个“自指合法”之上的。如果自指不合法，整个证明就会崩。所以你要先确认：“喂自己吃自己”没有任何障碍。

【工程实践】自指在工程里处处都是：**自举编译器**（编译器用自己编译自己）、**病毒/自解压包**（程序内嵌自己的副本）、**Quine**（打印自身源码的程序）、**自动更新**（程序检查并重写自身）。“程序能处理自己的表示”不是 bug，是能力——停机问题告诉我们，这种能力也有算不出来的上限。

第 4 轮：假设 H 存在——先给“能判定停机”一个机会

老谟：“现在正式开场。我们要证 $HALT$ 不可判定。用的武器是**反证法**：先假设存在一台图灵机 H ，它能判定 $HALT$ ——也就是说，对每个 $(\langle M \rangle, w)$ ， H 都停机，且正确回答‘ M 在 w 上停机吗’。然后我们用它造一个东西，让这个假设**自相矛盾**。”

造物主：“……反证法我懂：假设 P ，推出矛盾，所以 P 错。可这里 H 已经‘每个输入都给出正确答案’了，它看起来完美无缺，我从哪找它的漏洞？”

老谟：“记住一句话： H 的假设是‘**对一切输入都正确**’——一个都漏不掉。所以我不必去找它‘错在哪’，我只要**精心设计一个输入，让 H 对它的回答必然出错**。只要有一个输入让 H 错，就击穿了‘对一切输入正确’这个假设。这个‘精心设计的输入’，就是下一轮我们要造的机器 D 。”

造物主：“……所以不是‘找漏洞’，是‘造一个它必错的输入’？这就和‘构造一个反例’一样——我只要证明存在一个反例，就推翻‘全都正确’。”

老谟：“正是。**反证法的核心，不是证明‘所有输入都错’，而是证明‘存在一个输入让假设必错’**。一个反例就够。现在，我们把 D 造出来。”

知识锚点：反证法的力量——一个反例击穿“一切”

【概念深挖】为什么一个反例就够：假设 H “对每个 $(\langle M \rangle, w)$ 都正确回答”。这个命题是“ $\forall$ 输入， H 正确”。它的否定是“ $\exists$ 输入， H 错误”——只要找到一个输入让 H 错，就否定了“ $\forall$ 正确”这个假设。所以：

- 不需要证明 H “处处错”（那太难）；
- 只需要证明**存在一个输入， H 必然错**（这是可操作的）。

反证法的结构：假设 P (H 存在且全对) $\rightarrow$ 由 P 构造 $D \rightarrow$ 考察 D 的一个特定输入 $\langle D \rangle \rightarrow$ 发现 H 在这个输入上**必然自相矛盾** $\rightarrow$ 由矛盾律， P 假 $\rightarrow$ 不存在这样的 H 。

【工程实践】反证法 = “**构造性反例**”：你想反驳“这个系统对所有输入都对”（比如“我的校验逻辑万无一失”），不必穷举所有输入，只需**造出一个让系统必错的输入**。测试里这叫“反例构造”；图灵机不可判定证明，就是数学史上最强的一次‘反例构造’。

第 5 轮：构造 D ——用 H 造一台“吃自己”的机器

老谟：“现在我们手里有 H （假设它能判定停机）。用它造一台新机器 D 。 D 的输入是一个编码 $\langle M \rangle$ ，它这么做：

D 拿到 $\langle M \rangle \rightarrow$ 运行 $H(\langle M \rangle, \langle M \rangle) \rightarrow$ 如果 H 说‘ M 在 $\langle M \rangle$ 上停机’， D 就进入一个死循环，永远不停；如果 H 说‘ M 在 $\langle M \rangle$ 上不停机’， D 就停机。”

造物主：“……所以 D 就是‘ H 说停、我偏不停机； H 说不停、我偏停机’？ D 是在‘反转’ $M(\langle M \rangle)$ 的停机行为—— M 要是停， $D(\langle M \rangle)$ 就偏不停机； M 要是不停， $D(\langle M \rangle)$ 就偏停机。”

老谟：“别慌，理清楚： $D(\langle M \rangle)$ 停机 $\Leftrightarrow M$ 在 $\langle M \rangle$ 上不停机。也就是说， D 把它读到的机器 M 在‘吃自己’时的停机行为，**反过来了**。这是关键构造——我们造了一台‘和自己的判断反着干’的机器。”

造物主：“那……现在该把 D 喂给它自己了？让它吃 $\langle D \rangle$ ？”

老谟：“你闻到了。下一轮，我们让 D 吃 $\langle D \rangle$ ——这是整个证明的引爆点。”

知识锚点：机器 D 的定义——反转“吃自己”的行为

【概念深挖】机器 D （由假设存在的 H 构造）：

- 输入： $\langle M \rangle$ （任意图灵机的编码）；
- 步骤 1：运行 H 于输入 $(\langle M \rangle, \langle M \rangle)$ （问： M 在 $\langle M \rangle$ 上停机吗）；
- 步骤 2（ H 说“是”）： D 进入**死循环**（永不停机）；
- 步骤 3（ H 说“否”）： D **停机**。

因此 D 满足关键性质：

$$D(\langle M \rangle) \text{ 停机} \iff M \text{ 在 } \langle M \rangle \text{ 上不停机.}$$

（左边 = “ H 说否”，右边 = “ H 说否”；因为 H 被假设为正确，二者同步。）等价地， $D(\langle M \rangle)$ 停机 $\iff M(\langle M \rangle)$ 不停机。 D 是“反停机”的机器。

注意：构造 D 是合法的—— D 的“死循环”是一个标准图灵机构造（比如一个永远右移、永不进入接受/拒绝状态的状态），完全可以在图灵机里实现。 D 的存在**只依赖** H ：没有 H ，就没有 D 。所以一旦我们证明“ D 导致矛盾”，矛盾的根源就是 H 。

第 6 轮：D 吃自己——两个分支都撞墙

老谟：“现在，把 D 喂给它自己——考察 $D(\langle D \rangle)$ 。因为 $D(\langle D \rangle)$ 要么停机、要么不停机，只有这两种可能（排中律）。我们**两个都看**。”

分支一：假如 $D(\langle D \rangle)$ 停机。那 $H(\langle D \rangle, \langle D \rangle)$ 必须说‘是’（因为 H 假设正确， D 在 $\langle D \rangle$ 上停， H 就答‘停’）。可 D 的规则是： H 说‘停’， D 就死循环——**那 $D(\langle D \rangle)$ 就不该停**。这和‘ $D(\langle D \rangle)$ 停机’矛盾。

分支二：假如 $D(\langle D \rangle)$ 不停机。那 $H(\langle D \rangle, \langle D \rangle)$ 必须说‘否’（ D 不停， H 答‘不停’）。可 D 的规则是： H 说‘不停’， D 就停机——**那 $D(\langle D \rangle)$ 就该停**。这和‘ $D(\langle D \rangle)$ 不停机’矛盾。”

造物主：“……两个分支都矛盾！ $D(\langle D \rangle)$ 停也矛盾、不停也矛盾。可排中律说它**必须停或不停**——**所以从一开始‘存在 H ’这个假设就是错的！**”

老谟：“**对了。这就是全部。** H 不存在，HALT 不可判定。你刚刚，亲手捉住了一个任何图灵机都算不出来的问题。”

知识锚点：停机问题不可判定——直觉全览

【概念深挖】**停机问题不可判定定理（直觉版）**：不存在图灵机 H ，能对每个 $(\langle M \rangle, w)$ 都停机并正确回答" M 在 w 上停机吗"。

- **证明思路（直觉）**：假设 H 存在 $\rightarrow$ 用它造 D （反转"吃自己"的停机行为） $\rightarrow$ 考察 $D(\langle D \rangle)$ $\rightarrow$ 无论它停不停，都违背 D 的定义 $\rightarrow$ 矛盾 $\rightarrow H$ 不存在。
- **为什么不可绕**：任何"判定停机"的方案 H ，都会被这个 $D(\langle D \rangle)$ 的构造击中。**这是数学上不可能的，不是"还没人想到"。**

这条证明的三大支点（缺一不可）：

1. **机器可编码** ($\langle M \rangle$ 是合法输入) —— 否则无法把"程序当数据"；
2. **自指合法** (机器可吃自己) —— 否则无法构造 $D(\langle D \rangle)$ ；
3. **反证法 + 排中律** ($D(\langle D \rangle)$ 必停或必不停，且两者都矛盾)。

【历史溯源】图灵在 1936 年论文《论可计算数及其在判定问题上的应用》中，用同一套"自指 + 对角线"论证，证明了停机类问题的不可判定性，并由此推出判定问题 (Entscheidungsproblem, 即判定一阶逻辑公式是否恒真) 不可判定。要准确表述、不夸大：图灵证明的核心是"可计算数"与判定问题不可判定；"停机问题 (Halting Problem)"作为独立命名与标准形式，是后人（图灵本人论文里已含这一思想）整理的。但"停机问题不可判定"的思想确实出自图灵 1936 那篇论文，它和 Church 的结果 (λ 演算的不可判定性) 一起，宣告了"希尔伯特判定问题"的否定答案。

第 7 轮：不可判定不是"还没想出"——是"数学上不可能"；预告 P vs NP

老谟："现在把'不可判定'这句话，钉死。当我说 HALT 不可判定，我不是说'程序员还没写出来'，我是说——**已经证明了不存在这样的算法**。这是两种完全不同的失败：'算法还没找到'和'算法不存在'。前者是工程问题，后者是数学事实。"

造物主：".....就像'证明费马大定理'和'证明 $1+1=3$ '的区别？前者是'太难没找到'，后者是'根本不存在'。"

老谟："形象。不可判定是' $1+1=3$ '那种：**不是找不到，是逻辑上不可能存在**。而且你注意到没有——我们证的是'HALT 不可判定'，也就是'判定停机'这个问题**不在可判定类里**。但 HALT 其实是**可识别**的（它属于可识别类）。所以我们就有了：**可判定 $\subset$ 可识别，且真包含**——可识别里藏着不可判定的东西。这正是下一轮定义要落的地方。"

造物主："那.....算得出来和算得快，是另一回事了吧？"

老谟："你又提前闻到了。这一章证明'有没有算不出来的'（有）；下一章我们要问'算得出来的里面，有没有算得快/算得慢的区别'——那就是复杂度理论，是 P vs NP 的世界。 D 这台机器虽然是图灵机，但我们能不能在多项式时间内判定某些东西？那是另一个、至今未解的问题。先把这一章的反证法消化透。"

知识锚点：不可判定的分量 + 预告复杂度

【概念深挖】"不可判定"的精确含义：一个问题（语言）不可判定 = 已被证明不存在图灵机能判定它。这不是"当前技术不足"，而是"逻辑上不可能"。与"尚未找到算法"有本质区别：后者随时可能被一个巧妙算法推翻，前者永远站得住（证明是数学事实，不随技术进步改变）。

可判定 $\subset$ 可识别（真包含）：凡可判定必可识别（定理 2，见走廊）；而 $HALT$ 可识别但不可判定，故二者是真包含。这给出了一条明确的"边界线"：可识别类里，有的可判定，有的不可判定。

【工程实践】"不可判定"的工程含义：一旦你证明某个问题不可判定，就应停止寻找完美算法，转向可行方案——启发式、限制子问题、近似、牺牲完备性。例如程序分析里的某些性质（如"这个程序会不会死锁"、"这段代码是否永不返回"）若不可判定，工程上就用超时、静态近似、运行时监控来"够用"。知道"这堵墙存在"，本身就指导你怎么在墙这边高效工作。

【历史溯源】后续： $HALT$ 不可判定之后，人们发现一大批问题都不可判定（可归约为停机问题），包括"某程序是否打印某个字符"、"两个程序是否等价"等。而复杂度理论（ P vs NP ）是另一条线——它不问"能不能算"，问"算得快不快"。下一章我们就走进这条线。

严格化走廊

位置说明：对话负责"直觉"——为什么停机问题可疑、为什么要假设 H 、为什么 D 吃自己会矛盾。本走廊负责"严谨"——把可判定/可识别/ $HALT$ 钉成形式定义，给出本册皇冠定理——定理 1（停机问题不可判定）的完整反证法，每步标依据、把假设 H 用足；再证定理 2（可判定 $\Rightarrow$ 可识别）。红线 1 的硬要求：每个证明标"证明类型 + 每一步依据"，禁"显然/易证/显见"。

定义：可判定、可识别、停机问题——精确表述

定义（可判定）： 设 $L \subseteq \Sigma^*$ 。称 L **可判定**，若存在图灵机 M ，使得对**每个** $w \in \Sigma^*$ ， M 在输入 w 上都**停机**，且 $w \in L \iff M$ 停在 q_{accept} （否则停在 q_{reject} ）。称这样的 M 为 L 的**判定机**。

定义（可识别）： 称 L **可识别**，若存在图灵机 M ，使得对每个 $w \in \Sigma^*$ ， $w \in L \iff M$ 在输入 w 上停在 q_{accept} 。（对 $w \notin L$ ， M 可停机拒绝，也可**永不停机**。）

定义（停机问题）： 给定 $\langle M \rangle$ （图灵机 M 的编码）与输入串 w ，定义语言

$$\text{HALT} = \{ (\langle M \rangle, w) \mid M \text{ 在输入 } w \text{ 上最终停机} \}.$$

停机问题即问：HALT 是否可判定？

证明类型标注： 本章皇冠证明——**定理 1（停机问题不可判定）用反证法 + 自指构造（对角线法）：** 假设 H 存在，构造 D ，考察 $D(\langle D \rangle)$ ，两个分支都矛盾，由排中律与矛盾律推出 H 不存在。**定理 2（可判定 $\Rightarrow$ 可识别）用直接证明：** 同一台判定机本身即识别机。

关于对话： 对话第 5–6 轮是读者**第一次**接触"构造 $D \rightarrow D$ 吃自己"的完整反证链，做了"引导式推导"（教学示范——为什么假设 H 、为什么 D 反转、为什么看 $D(D)$ 两分支）。**正式的可复写证明（每一步标依据）在本走廊定理 1。** 对话负责"为什么会想到这步"，走廊负责"怎么把它写标准"——与册 06 及前六章口径一致。

定理 1（停机问题不可判定）：HALT 不可判定

前提： 无（定理本身无额外前提，只依赖"图灵机可编码""自指合法"这两个已确认事实）。

结论： 不存在图灵机 H ，使得 H 对每个 $(\langle M \rangle, w)$ 都停机，且 $H(\langle M \rangle, w)$ 接受 $\iff M$ 在 w 上停机。即：HALT 不可判定。

证明类型： 反证法（构造 D + 考察 $D(\langle D \rangle)$ 的两分支矛盾）。

证明：

- 第 1 步：**假设存在判定机 H ：** 假设存在图灵机 H ，对**每个** $(\langle M \rangle, w)$ 都停机，且

$$H(\langle M \rangle, w) \text{ 接受 } \iff M \text{ 在 } w \text{ 上停机.}$$

（依据：反证法的假设——我们要证明它不成立，先假设它成立。）

- 第 2 步：**用 H 构造机器 D ：** 定义 D 输入 $\langle M \rangle$ 后：
 1. 运行 H 于 $(\langle M \rangle, \langle M \rangle)$;

2. 若 H 接受 (即" M 在 $\langle M \rangle$ 上停机"), 则 D 进入死循环 (永不停机);
3. 若 H 拒绝 (即" M 在 $\langle M \rangle$ 上不停机"), 则 D 停机 (进入接受状态)。(依据: 构造合法性—— D 由 H 明确构造; "死循环"是标准图灵机构造, 可在有限状态内实现。 D 的存在完全依赖 H 。)

- 第 3 步: **刻画 D 的行为**: 由第 2 步的构造, $D(\langle M \rangle)$ 停机 $\Leftrightarrow H(\langle M \rangle, \langle M \rangle)$ 拒绝。再由第 1 步 H 的正确性, $H(\langle M \rangle, \langle M \rangle)$ 拒绝 $\Leftrightarrow M$ 在 $\langle M \rangle$ 上不停机。故

$$D(\langle M \rangle) \text{ 停机} \iff M \text{ 在 } \langle M \rangle \text{ 上不停机.}$$

(依据: 第 2 步的构造定义 + 第 1 步 H 的正确性——把"拒绝"与"不停机"对应起来。)

- 第 4 步: **令 $M = D$, 考察 $D(\langle D \rangle)$** : 将 $M = D$ 代入第 3 步:

$$D(\langle D \rangle) \text{ 停机} \iff D \text{ 在 } \langle D \rangle \text{ 上不停机,}$$

即 $D(\langle D \rangle)$ 停机 $\Leftrightarrow D(\langle D \rangle)$ 不停机。(依据: 代入 $M = D$, 左边 $M(\langle M \rangle) = D(\langle D \rangle)$, 右边同理。)

- 第 5 步: **导出矛盾**: 由排中律, $D(\langle D \rangle)$ 要么停机、要么不停机 (且必居其一)。
 - 若 $D(\langle D \rangle)$ 停机, 由第 4 步 $\Leftrightarrow$ 知 $D(\langle D \rangle)$ 不停机, **矛盾**;
 - 若 $D(\langle D \rangle)$ 不停机, 由第 4 步 $\Leftrightarrow$ 知 $D(\langle D \rangle)$ 停机, **矛盾**。两个分支都矛盾。(依据: 排中律 + 第 4 步的双条件 + 矛盾律——一个命题不能同时为真又为假。)
- 第 6 步: **否定假设**: $D(\langle D \rangle)$ 停机、不停机均矛盾, 故假设"存在 H "导致矛盾。由矛盾律, 假设为假——不存在这样的 H 。故 **HALT 不可判定**。■

依据标注: 步 1 用"反证法假设" (把待否定的命题当作前提); 步 2 用"构造" (显式给出 D , 其合法性依赖"死循环可构造"); 步 3 用"构造定义 + H 正确性" (把 D 的行为翻译成与 H 的关系); 步 4 用"代入 $M = D$ " (自指的关键一步: 让机器吃自己); 步 5 用"排中律 + 矛盾律" (两个分支都撞出矛盾); 步 6 用"反证法收尾" (假设导致矛盾, 故假设假)。**读法**: 这条证明的发动机是**步 4 的代入 (自指)**—— $D(\langle D \rangle)$ 停机 $\Leftrightarrow D(\langle D \rangle)$ 不停机, 这个" $\Leftrightarrow$ "把 D 的行为拧成了不可能。**注意**: 证明没有"手动模拟" $D(\langle D \rangle)$ 到底停不停——它不需要知道, 只要 H 必须给一个答案 (是/否), 而两个答案都会让 D 反着来, 从而矛盾。**这正是 H "对一切输入正确"这个假设被一个输入 $\langle D \rangle$ 击穿的方式**。若没有"自指合法" (步 4), 或没有"机器可编码" (步 2 的 $\langle M \rangle$), 整个证明就不成立——三大支点缺一不可。

定理 2 (可判定 $\Rightarrow$ 可识别): 凡可判定的语言都可识别

前提: $L \subseteq \Sigma^*$ 可判定, 即存在判定机 M , 对每个 w 都停机, 且 $w \in L \iff M$ 接受 w 。

结论： L 可识别。

证明类型：直接证明（同一台机器既是判定机又是识别机）。

证明：

- 第 1 步：**回忆可判定定义**：存在图灵机 M ，对每个 w 都停机，且 $w \in L \iff M$ 停在 q_{accept} 。（依据：可判定的定义——前提。）
- 第 2 步：**验证识别条件的第一半**：对 $w \in L$ ，由前提 M 停在 q_{accept} ——即 M **接受** w 。（依据：可判定的定义—— $w \in L \iff M$ 接受。）
- 第 3 步：**验证识别条件"允许永不停机"这一半**：对 $w \notin L$ ， M 停在 q_{reject} （拒绝）——它**不会**接受 w 。识别条件对"不属于语言的输入"只要求"不接受"（允许拒绝或永不停机），而 M 已满足"不接受"。（依据：可识别的定义——只要求"属于 $\iff$ 接受"，对不属于的允许拒绝/永不停机。）
- 第 4 步：**结论**： M 同时满足可识别的两条要求（属于就接受、不属于就不接受），故 M 是 L 的识别机， L 可识别。■

依据标注：步 1 用"可判定定义"（前提）；步 2、3 用"可识别定义的放宽条件"（识别只要求"属于 $\iff$ 接受"，不要求总停机）；步 4 收束。**读法：**这个证明说明"可判定比可识别更硬"——判定机因为"总停机"，自动满足识别机的要求；反之识别机不保证总停机，故可识别不一定可判定。**配合定理 1**（HALT 可识别但不可判定），得到"可判定 $\subset$ 可识别"是真包含。

反例/边界：证明的边界与常见误解

1. **"跑一遍看它停不停"不是算法**：若 M 在 w 上不停机，"跑一遍"永远得不到答案——无法区分"很慢"和"永不停"。这正是为什么"停机"不能靠模拟 M 来判定。
2. **D 的构造依赖 H** ：没有 H 就没有 D 。一旦证出 H 不存在， D 也无从构造——**这不是矛盾，反而印证了结论**：我们证明的是"假设 H 存在会推出 D ，而 D 导致矛盾"，故 H 必须不存在。
3. **证明不依赖"手动模拟 $D(D)$ "**：证明只用了" $D(D)$ 停机 $\iff D(D)$ 不停机"这个由构造推出的双条件 + 排中律。它**不试图预测 $D(D)$ 实际停不停**，因为那样会陷入无限模拟；它直接指出"停与不停都矛盾"。
4. **对角线法不是诡辩**：这里不是"谎言悖论"那种语义自指，而是**纯语法的机械自指**—— $\langle D \rangle$ 就是普通输入串，机器机械地处理它。唯一"特殊"的是这个串恰好编码了处理它的那台机器。**自指合法**是经过确认的事实，不是漏洞。
5. **编码 $\langle M \rangle$ 的存在性**：图灵机是有限对象（有限状态、有限字母表、有限转移表），必然可编码成有限串。**若机器无限，编码就不存在，证明就崩**——但图灵机是有限的，故 $\langle M \rangle$ 恒存在。
6. **"不可判定"与"不可计算"区别**：不可判定（undecidable）指判定问题（是/否）没有算法；不可计算（uncomputable）指计算函数没有算法。停机问题**不可判定**（同时作为函数也不可计算）。本

书聚焦"不可判定"。

知识点总结

知识点（本章推出的核心概念）

- **可判定 (decidable)**：存在图灵机，对**每个输入都停机**且正确"接受/拒绝"。"**每个输入都给是/否**"是可判定的命门。
- **可识别 (recognizable)**：存在图灵机，属于语言就**接受**；不属于的，可拒绝**或永不停机**。
- **可判定 $\subset$ 可识别 (真包含)**：凡可判定必可识别（定理 2）；但 HALT 可识别却不可判定，故真包含。
- **机器编码 $\langle M \rangle$** ：图灵机是有限对象，可编码成有限串——"**程序即数据**"。
- **自指合法**：机器可机械地吃自己的编码，纯语法、无语义玄学。
- **停机问题 HALT**： $\{(\langle M \rangle, w) \mid M \text{ 在 } w \text{ 上停机}\}$ 。**不可判定**。
- **停机问题不可判定 (定理 1)**：反证法——假设 $H \rightarrow$ 造 D （反转吃自己） $\rightarrow$ 考察 $D(\langle D \rangle) \rightarrow$ 两分支都矛盾 $\rightarrow H$ 不存在。

历史结论（本章涉及的史料）

- **图灵 (1936)**：论文《论可计算数及其在判定问题上的应用》用"自指 + 对角线"证明停机类问题不可判定，并推出**判定问题 (Entscheidungsproblem) 不可判定**——否定了希尔伯特"判定问题可解"的期望。
- **准确表述**：停机问题作为**独立命名/标准形式**是后人整理，但**"停机不可判定"的思想确出自图灵 1936 论文**；丘奇同时用 λ 演算得出同类结果。**不夸大、不混淆**：图灵证明的核心是判定问题不可判定（停机问题不可判定是其关键步骤）。

工程实践结论（本章知识在真实工程里怎么用）

- **不可判定的工程应对**：证明某问题不可判定后，**停止寻找完美算法**，转向启发式/近似/限子问题/牺牲完备性（如静态分析的超时与近似）。
- **"程序即数据"**：解释器、编译器、杀毒、自解压包、Quine——都是"把程序当数据"的能力，停机问题问它的上限。
- **可判定 vs 可识别的工程对应**：判定 = 保证返回（校验 API）；识别 = 可能挂起（等进程结束）。识别类问题工程上常需"超时"兜底。

计算训练场

位置说明：对话负责"为什么停机问题不可判定"，这里负责"怎么把逻辑链走通、怎么亲手复写反证法"。三题**必须动笔**，每题 10 分钟以上。**难度递进：易→中→难**。重点：① 复写反证法的每一步依据；② 把"自指"和"反例击穿"落到具体。

题 1（易→中）：补全反证法的逻辑链——填空推理

题干：老谟给出反证法的骨架，让造物主补全括号里的内容并说明每处依据。假设 H 判定 HALT。

(1) 用 H 构造 D ： $D(\langle M \rangle)$ 运行 $H(\langle M \rangle, \langle M \rangle)$ ；若 H 接受， D 就 (A)；若 H 拒绝， D 就 (B)。(2) 由构造， $D(\langle M \rangle)$ 停机 $\Leftrightarrow H$ 拒绝 $\Leftrightarrow$ (C)。(3) 令 $M = D$ ，得 $D(\langle D \rangle)$ 停机 $\Leftrightarrow$ (D)。(4) 由排中律与矛盾律，假设 (E) 不成立。

完整解答：

- (A) **进入死循环（永不停机）**。依据： D 的构造定义——反转"停"的行为。
- (B) **停机**。依据： D 的构造定义——反转"不停"的行为。
- (C) M **在 $\langle M \rangle$ 上不停机**。依据： H 被假设正确， H 拒绝 $\Leftrightarrow M$ 不停机。
- (D) $D(\langle D \rangle)$ **不停机**。依据：把 $M = D$ 代入 (C)（自指）。
- (E) **"存在判定机 H "**。依据： $D(\langle D \rangle)$ 停/不停都矛盾，由排中律 $D(\langle D \rangle)$ 必居其一，故假设假。
- **答：**A=死循环、B=停机、C= M 在 $\langle M \rangle$ 上不停机、D= $D(\langle D \rangle)$ 不停机、E=存在 H 。

常见错误：① 把 (C) 写成" M 停机"（弄反了反转方向）；② 在 (D) 处不代入自指，卡在" $D(D)$ 到底停不停"——不必知道，只靠双条件+排中律；③ 忘了 (E) 是反证法的假设本身。

题 2（中→难）：完整复写停机问题反证法（分步标依据）

题干：不看书，从零完整复写"HALT 不可判定"的证明，**每一步标注依据**（与走廊定理 1 一致），并特别说明"自指"、"反例击穿"、"排中律"三个关键点各在哪里起作用。

完整解答（可复写卡）：

- **第 1 步：**假设存在判定机 H ，对每个 $(\langle M \rangle, w)$ 都停机且正确回答。（依据：反证法假设。）
- **第 2 步：**构造 D ：输入 $\langle M \rangle \rightarrow$ 运行 $H(\langle M \rangle, \langle M \rangle) \rightarrow H$ 接受则死循环、拒绝则停机。（依据：构造合法性——死循环可构造， D 由 H 定义。）
- **第 3 步：** $D(\langle M \rangle)$ 停机 $\Leftrightarrow M$ 在 $\langle M \rangle$ 上不停机。（依据：第 2 步构造 + H 的正确性。）
- **第 4 步：**令 $M = D$ ，得 $D(\langle D \rangle)$ 停机 $\Leftrightarrow D(\langle D \rangle)$ 不停机。（依据：代入 $M = D$ ——**自指**在这里。）

- **第 5 步**：由排中律， $D(\langle D \rangle)$ 停机或不停机；两种情况都由第 4 步导出矛盾。（依据：排中律 + 矛盾律。）
- **第 6 步**：假设导致矛盾，故 H 不存在，HALT 不可判定。（依据：反证法收尾。）
- **关键点定位**：自指 = 第 4 步 ($M = D$)；反例击穿 = H 对输入 ($\langle D \rangle, \langle D \rangle$) 的回答必错；排中律 = 第 5 步（保证两个分支穷尽所有可能）。

常见错误：① 第 3 步把"停机"和"不停机"写反；② 第 4 步忘了代 $M = D$ （把 $\langle D \rangle$ 当普通输入，没意识到它是自指）；③ 第 5 步想"手动模拟 $D(D)$ "来定停不停，而不是靠"双条件+排中律"（手动模拟会陷入无限，证明不依赖它）。

题 3 (难)：辨析"可判定 / 可识别 / 不可判定"——一个反例击穿一切

题干：老谟给出三句论断，让造物主逐一判断正误并说明理由：(a) "可判定的语言，一定可识别。" (b) "可识别的语言，一定可判定。" (c) "如果一个问题不可判定，那是因为还没人想出算法。" 并针对 (b) 举出反例，针对 (c) 解释错在哪。

完整解答：

- **(a) 对**：判定机"总停机且正确接受/拒绝"，本身就是一台识别机（定理 2）。可判定 $\subset$ 可识别。
- **(b) 错**：可识别不一定可判定。**反例就是 HALT**：它可识别（有识别机：跑 M ，若 M 停就接受；对不停的输入可永不停），但不可判定（定理 1）。故可判定是可真子集。
- **(c) 错**："不可判定"是**已被数学证明不存在算法**，不是"还没想出"。这与"尚未找到算法"有本质区别：后者随时可能被新算法推翻，前者是逻辑上不可能、永远成立。HALT 就是"证明不存在算法"的样板。

常见错误：① 误以为"可识别"包含"可判定"（方向反了，是"可判定 $\subset$ 可识别"）；② 以为 HALT 既不可判定就不算可识别（它可识别，只是不可判定——这正是"真包含"的样板）；③ 把"还没想到"和"数学上不可能"混为一谈。

向导便签

这一章咱们干了什么：你亲手捉住了一个**任何图灵机都算不出来的问题——停机问题**。你理解了为什么"跑一遍看看"不行（可能永不停机、无法区分"慢"与"永不"），看懂了"程序即数据"（ $\langle M \rangle$ 把机器编码成串），学会了"机器吃自己"是合法自指，并**完整复写了全册皇冠证明**——假设 $H \rightarrow$ 造 D （反转吃自己） $\rightarrow$ 让 D 吃 $\langle D \rangle \rightarrow$ 两个分支都矛盾 $\rightarrow H$ 不存在。

钉死一句话：**"不可判定"不是"还没想到算法"，是"数学上证明不存在算法"——一个反例（ $\langle D \rangle$ ）就击穿"对一切输入正确"的假设。可判定 $\subset$ 可识别，且真包含；HALT 可识别却不可判定。**

记住：**这一章的证明工具——反证法 + 自指 + 一个反例击穿"一切"——是你在可计算性理论里最锋利的一把刀**。下一章，我们要从"算得出/算不出"这条线，跨到另一条线："算得出来"之后，还有**"算得快不快"**——那就是复杂度理论，是 P vs NP 的世界。你已经会用反证法了，下一章那扇门，就在眼前。

造物主手记

我曾踩的坑：

我第一次想证明停机问题不可判定，卡在"不知道怎么找 H 的漏洞"——因为 H 看起来对每个输入都对。老谟点醒我：**不是找 H 错在哪，是造一个让它必错的输入 $\langle D \rangle$** ，一个反例就击穿" $\forall$ 正确"。我差点把 D 的构造写反——以为" H 说停、 D 就停"（那是跟随不是反转），正确是" H 说停、 D 就死循环； H 说不停、 D 就停"。最险的是第 4 步 $M = D$ ：我一度想"手动模拟 $D(D)$ 到底停不停"，结果陷入无限模拟——老谟说**证明不靠模拟，只靠"停机 $\Leftrightarrow$ 不停机"这个双条件 + 排中律**，两个分支都矛盾就够了。还有一次我差点把"不可判定"当成"还没人想出来"，被纠正：那是**已被证明不存在算法**，是数学事实，不是工程困难。

顿悟时刻：

真正开窍是看着" $D(\langle D \rangle)$ 停机 $\Leftrightarrow D(\langle D \rangle)$ 不停机"这个式子落地那一刻。**一个由构造推出的双条件，竟能让一台机器的行为自相矛盾**——原来数学可以"捉住"不可能。更震撼的是"一个反例击穿一切"： H 号称对每个输入都正确，可只要我造出 $\langle D \rangle$ 这一个输入，它就必错。**"全对"的假设，被"一个精心构造的输入"击穿**——这让我第一次懂了反证法的力量不是证明"处处错"，而是证明"存在一错"。停机问题的墙，我亲手摸到了。下一章，我要去另一面墙：算得快不快——P vs NP。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (概念, 易→中) : 可判定与可识别

(a) 什么是"可判定"? 它最硬的条件是什么? (b) 什么是"可识别"? 它和可判定差在哪? (c) 可判定和可识别是什么包含关系? 请用 HALT 说明。

完整解答: (a) 存在图灵机对每个输入都停机, 且正确"接受/拒绝"。最硬条件: **对每个输入都停机** (否则拿不到"否"的完整答案)。(b) 存在图灵机, 属于语言就接受; 不属于的可拒绝或永不停机。差别在: 可识别不要求"对每个输入都停机"。(c) 可判定 $\subset$ 可识别 (真包含)。HALT 可识别 (跑 M , 停则接受) 但不可判定 (定理 1), 证明二者真包含。

题 B (计算/逻辑, 中→难) : 复写反证法核心两分支

不看书, 独立写出停机问题不可判定反证法中" $D(\langle D \rangle)$ 的两个分支", 并说明每一分支如何用" H 正确 + D 反转"导出矛盾。

完整解答:

- **分支一 (假设 $D(\langle D \rangle)$ 停机)**: 因 H 假设正确, $H(\langle D \rangle, \langle D \rangle)$ 必须"接受" (D 停则 H 答"停")。但 D 的规则是 H 接受→死循环, 故 $D(\langle D \rangle)$ 不停机。与"停机"矛盾。
- **分支二 (假设 $D(\langle D \rangle)$ 不停机)**: 因 H 假设正确, $H(\langle D \rangle, \langle D \rangle)$ 必须"拒绝" (D 不停则 H 答"不停")。但 D 的规则是 H 拒绝→停机, 故 $D(\langle D \rangle)$ 停机。与"不停机"矛盾。
- **结论**: 两分支都矛盾, 故 H 不存在。

题 C (概念, 检查理解)

(a) 为什么"跑一遍看看 M 停不停"不能作为停机问题的判定算法? (b) "不可判定"和"还没想出算法"有什么区别? (c) 为什么 D 的构造依赖 H ? 证出 H 不存在后, D 还能存在吗? (d) 为什么说这里用"自指"不是诡辩?

完整解答: (a) 若 M 在 w 上不停机, "跑一遍"永远得不到答案, 无法区分"很慢"和"永不停", 所以不是"每个输入都给答案"的算法。(b) "不可判定"是**已被证明不存在算法** (数学事实, 永远成立); "还没想出"是工程状态, 随时可能被新算法推翻。(c) D 的第 1 步"运行 H "依赖 H 存在; 没有 H , D 就无从构造。证出 H 不存在后, D 也不存在——但这不矛盾, 反证法正是用"假设 H →造 D →矛盾"来否定 H 。(d) 这里不是"这句话是假的"那种语义自指, 而是**纯语法的机械自指**: $\langle D \rangle$ 是普通输入串, 机器机械处理; 特殊只是它编码了处理它的机器。自指合法是经确认的机械事实, 不是漏洞。

题 D (综合, 中→难) : 一条逻辑链的终点

老谟让造物主把"可判定 $\subset$ 可识别"这条链, 从定义到结论完整推一遍: 定义可判定→定义可识别→证可判定⇒可识别→证 HALT 可识别→证 HALT 不可判定→得出结论"可判定是真子集"。

完整解答:

1. **可判定**: 存在总停机的图灵机, 正确接受/拒绝 (定义)。

2. **可识别**：存在图灵机，属于即接受，不属于可拒绝/永不停（定义）。
3. **可判定 $\Rightarrow$ 可识别**（定理 2）：总停机的判定机本身就是识别机。
4. **HALT 可识别**：识别机“跑 M ， M 停则接受”——对不停机的输入允许永不停，故 HALT 可识别。
5. **HALT 不可判定**（定理 1）：反证法（假设 $H \rightarrow$ 造 $D \rightarrow D(\langle D \rangle)$ 两分支矛盾 $\rightarrow H$ 不存在）。
6. **结论**：存在“可识别但不可判定”的语言（HALT），故“可判定 $\subset$ 可识别”是**真包含**（可判定是可真子集）。

预告

这一章，你亲手捉住了一个任何图灵机都算不出来的问题——**停机问题不可判定**，用了一个可复写一整生的证明：假设 $H \rightarrow$ 造反转的 $D \rightarrow$ 让 D 吃自己 $\rightarrow$ 两分支都矛盾。你看到了可计算性的**第一堵墙**：有些东西，数学上注定算不出来。可你心里一定冒出了一个新问题：“**算得出来**”的那些，是不是也有‘**算得快慢**’之分？判断一个语言可判定是一回事，**在合理时间内**判出来是另一回事——有的算法跑几万年，有的瞬间完成。

下一章，**复杂度理论：P、NP 与 NP-完全**——我们要问那个至今悬而未决的世纪难题：“**容易验证**”和“**容易求解**”是一回事吗？你会在那里看到“规约”这个新工具，看到 Cook-Levin 定理把一大类问题捆在一起，看到 P 与 NP 到底在问什么。从“**算不算得出来**”到“**算得快不快**”，你正站在计算理论的另一扇大门前。

第 08 章 · P 与 NP——算得快不快，与算不算得出来

本章从哪个问题出发：上一章我们证明了有些问题**不可判定**——任何图灵机都算不出来。可你心里一定还压着一个更实际的问题：“**算得出来**”的那些问题里，有没有‘算得快’和‘算得慢’之分？同样是一个可判定的问题，有的算法瞬间完成，有的要跑几万年。判断“这个数是不是质数”在数学上完全可判定，可它到底要跑多久？有没有可能：**某些问题的“答案容易验证”，但“答案本身却极难找到”**？这正是复杂度理论要问的问题。推导到哪个结论：你要看清“可判定”之外，还有一个全新的维度——**时间复杂度**。你会认识两个划时代的类：**P**（多项式时间内可判定）和 **NP**（多项式时间内可验证），并亲手证明 $P \subseteq NP$ 。你会理解**归约**这个工具，看到 Cook-Levin 定理如何把一大批问题捆成“NP-完全”这个家族，最终来到那个克雷数学研究所悬赏 100 万美元、至今未解的世纪难题——**P 与 NP 是否相等**。

核心认知：“**算得出来**”（可判定）和“**算得快**”（复杂度）是两回事；**P vs NP** 问的正是——“**答案能否被快速验证**”和“**答案能否被快速找到**”是不是同一回事。这一章是全册的收尾：我们从图论的具体算法出发，走到了图灵机（什么能算），走到了停机问题（什么注定算不出来），现在抵达最后一个问题——**在算得出来的世界里，困难到哪种程度，人类至今测不出它的底**。这既是一册的终点，也是“什么能算”这个问题的边界线。

引子

老谟书房的墙上，贴着这一册的“地图”：图论（第 1–5 章）→ 图灵机（第 6 章）→ 停机问题（第 7 章）。我在地图最右边，用红笔画了一个大问号。

“老谟，”我指着问号问，“上一章我们证明了有些东西**算不出来**。可我想知道——那些**算得出来的**东西，是不是也有差别？同样是可判定的问题，为什么有的‘唰’一下就出来，有的要等一辈子？”

老谟在白板上写了两个问题，说：“那我考你。**第一个：判断一个二进制串是不是奇数**。你上一章亲手造过这台机器——它几步就给出答案？”

“.....输入多长，就扫几步，加上看个位——线性，输入越长它越慢，但慢得很‘温和’。”

“好。**第二个：给一张有 n 个顶点的图，判断它有没有一条经过每个顶点恰好一次的回路（哈密顿回路）**。这个也是可判定的（暴力枚举所有排列），可它要跑多久？”

我想了想：“.....要试所有顶点的排列，大约 $(n-1)!/2$ 种。 $n=20$ 时，那就是.....大概 6×10^{16} 种排列——**就算每 1 纳秒试一种，也要约 1.9 年**。可 n 每加 1，工作量就再乘 n 倍： $n=29$ 时排列数暴涨到约 1.5×10^{29} 种，每纳秒试一种也要约 4800 亿年——这才叫**指数爆炸**。”

“那你发现没有——”老谟盯着我，“这两个问题**都能算**，可**难度差了天和地**。第一个‘线性’，第二个‘指数爆炸’。**我们给这种差别起个名字，就够出一章书了**。”

我看着那个大问号，第一次意识到：算得出来，只是开始；算得快不快，才是日常揪着你的那个问题。

对话引擎

第 1 轮：从“可判定”到“可判定但太难”——复杂度登场

老谟：“先把概念钉清楚。上一章的‘可判定’，只回答一个问题：**有没有算法？** 有，就是可判定；没有，就是不可判定。它**不关心算法快不快**——一个判定机跑 10 步和跑 10^{100} 步，只要都‘停下来给答案’，在‘可判定’眼里就是平等的。”

造物主：“.....所以‘可判定’是个‘有没有’的问题，不问‘快不快’。可实际生活里，我天天在意‘快不快’。”

老谟：“对。**复杂度理论 (complexity theory)** 就是把‘快不快’也拿进来：在‘能判定’的基础上，再问‘要用多少步’。注意它和上一章的界限：停机问题是问‘有没有算法’（可判定性），P vs NP 是问‘在‘算得出来’的前提下，代价多大’（复杂度）。**这两个问题不在一个轴上**——这就是为什么上一章讲完停机问题，我们还要开这一章。”

造物主：“.....所以这一章的坐标系，是‘算得出来的问题，各自要跑多久’。”

老谟：“正是。**可判定性是‘能不能’，复杂度是‘多快’**。这一章，我们把‘多快’这个轴，用图灵机的步数精确地钉出来。”

知识锚点：从可判定到复杂度——换一个轴

【概念深挖】可判定性（上一章）与复杂度（本章）是**两个不同的轴**：

- **可判定性：**问“是否存在算法”（有没有）。答案二值：可判定 / 不可判定。
- **复杂度：**在“存在算法”的前提下，问“算法要多少资源（这里是时间/步数）”（快不快）。答案是函数：随输入规模 n 增长，步数如何增长。

关键：一个**可判定**的问题，其算法可能有**多项式时间**（步数 $\approx n^k$ ，温和增长）或**指数时间**（步数 $\approx 2^n$ ，爆炸增长）。**两者都是“算得出来”，但“难度”天差地别**。复杂度理论正是研究这种“难度分级”。

【工程实践】你天天在“可判定但很难”的问题里做工程取舍：排序是可判定的，可你会选 $O(n \log n)$ 的快排而不是 $O(n^2)$ 的冒泡；找最短路径可判定，可你不会枚举所有路径而是用 Dijkstra。“**能算**”是底线，“**算得快**”是日常——复杂度就是给“算得快”一个精确的标尺。

第 2 轮：时间复杂度——用步数做标尺，多项式 vs 指数

老谟："现在，把'快不快'钉成数字。一台图灵机 M 在输入长度 n 上，用了多少步才停机？我们把这个步数，当作'它花了多少时间'。为什么用步数而不是秒？"

造物主：".....因为秒依赖机器快慢、依赖实现，不客观；步数是图灵机这个模型里、不依赖具体硬件的'时间'。第 6 章我们统一了计算模型，正好用它当计时器。"

老谟："对。一个判定机 M 的时间复杂度，是一个函数 $t(n)$ ：对长度 n 的输入， M 最坏要跑 $t(n)$ 步。然后我们只关心它增长得多快——是像 n^2 、 n^3 这样'多项式地'长，还是像 2^n 、 $n!$ 这样'指数地'炸开。"

造物主：".....所以关键是'随 n 变大的趋势'，不是某个具体输入跑几步。多项式是'温和的长'，指数是'疯狂的长'。"

老谟："记住这个分水岭：多项式时间被当作'实际可行'，指数时间被当作'实际不可行'。原因很简单： n 一变大， 2^n 瞬间盖过任何 n^k 。你上一章造的奇数机是 $O(n)$ ，哈密顿暴搜是 $(n-1)!/2$ 这种爆炸——这就是'能算'和'算得动'的分界。

知识锚点：大 O 记号——只看"增长趋势"

【概念深挖】大 O 记号 (Big-O)：称 $f(n) = O(g(n))$ ，若存在常数 $c > 0$ 与 $n_0 \geq 0$ ，使对所有 $n \geq n_0$ ，有 $0 \leq f(n) \leq c \cdot g(n)$ 。直觉： f 从某个 n_0 之后，被 $c \cdot g$ 压在下面——它不会比 g 的某个常数倍"长得更快"。

- $O(1)$ ：常数时间（不随 n 变长）。
- $O(n)$ ：线性（如扫描一遍输入）。
- $O(n^2), O(n^3), \dots$ ：多项式。
- $O(2^n), O(n!)$ ：指数/阶乘（爆炸）。

为什么多项式是分水岭：对任何常数 k 和任何 $c > 0$ ， $n^k = O(2^n)$ 成立（指数终将压过多项式）；但反过来 $2^n \neq O(n^k)$ 。所以"多项式 vs 指数"是一条不可跨越的鸿沟，而非"快一点"的区别。

【工程实践】大 O 是工程的第一语言：数据库索引 ($O(\log n)$)、哈希 ($O(1)$)、快排 ($O(n \log n)$) vs 冒泡 ($O(n^2)$)、暴力搜索（指数）。选算法先看大 O；指数算法通常意味着"换个思路"（近似、启发式、限定规模）。大 O 忽略常数和低阶项，只抓主导趋势——这正是我们想要的"标尺"。

第 3 轮：P 类——多项式时间可判定

老谟："现在定义第一个类。 P (**polynomial time, 多项式时间**)：一个语言 $L \in P$ ，如果**存在**一台图灵机 M 判定 L ，且 M 的时间复杂度是某个多项式 $O(n^k)$ 。简单说：**能在多项式时间内判定的问题，就是 P 里的问题。**"

造物主："..... P 就是'能在合理时间内算出答案'的问题集合？判断二进制奇数、判断质数、最短路径.....这些都在 P 里？"

老谟："对，这些都是 P 。 P 代表'实际可解'——现实中你能跑完的算法，绝大多数落在 P 里。注意： P 里的机器必须**总停机**（是判定机），而且**多项式时间**——这两条缺一不可。你上一章所有'总停机'的机器，如果还能证明步数是多项式的，就属于 P 。"

造物主：".....那问题来了：是不是**所有**可判定的问题，都能放进 P ？有没有可判定、但不可能多项式时间判定的？"

老谟："你问到点子上了。**这就是 P 之外、指数级难题的世界**——它们可判定，却可能不在 P 里。而 NP ，就是那扇还没撬开的门。"

知识锚点：P 类的精确定义

【概念深挖】设 $\text{TIME}(t(n)) = \{L \mid \exists \text{ 图灵机 } M \text{ 判定 } L, M \text{ 时间} \leq t(n) \text{ 对长度 } n \text{ 的输入}\}$ 。则

$$P = \bigcup_{k \geq 1} \text{TIME}(n^k).$$

直觉： P = 存在一个多项式时间的判定机的语言集合。

两个要点：① P 里的机器必须**总停机**（是判定机）；② 且步数 $\leq$ 某多项式（不是任何算法都行，必须是多项式时间）。

例子：判断二进制奇数 ($O(n)$)、判断质数 (AKS 算法，多项式时间)、单源最短路径 (Dijkstra, $O(n^2)$) 都在 P 里。

【工程实践】 P 基本 = "工程能接受"：绝大多数日常算法（排序、查找、最短路、正则匹配、图遍历）都在 P 。当工程师说"这个问题可解"，往往隐含"在多项式时间内可解"；遇到非 P （疑似指数）的问题，就要换策略。

第 4 轮：NP 类——多项式时间可验证（直觉）

老谟："现在看第二个类。 NP 的直觉，我先不绕弯子，直接给你一个**日常场景**：

判断一个数是不是合数，和验证一个数是不是合数——哪个难？

合数就是能写成两个更小整数之积的数（不是质数）。要**找**它的因子很难（所以密码学靠它）；但**验证**它——如果有人塞给你一个因子，比如 $15 = 3 \times 5$ ，你检查 $3 \times 5 = 15$ 是不是成立——**一瞬间**。

造物主：".....找因子难，验证因子容易！**别人给个'证书'，我验一下它是不是真的。**"

老谟："这就是 NP 的直觉：问题难在'找答案'，但答案（证书）一旦给你，验证它是'多项式时间'、很容易。形式化说： $L \in NP$ ，如果存在一个**多项式时间的验证器** V 和多项式 $p(n)$ ，使得——对属于 L 的每个输入 w ，存在一个长度 $\leq p(|w|)$ 的'证书' c ，让 $V(w, c)$ 接受；对不属于 L 的 w ，无论给什么 c ， $V(w, c)$ 都拒绝。"

造物主：".....所以 NP 是'答案能快速验证'的问题。那它和 P 是什么关系？"

老谟："你猜？——下一轮，我们证明 $P \subseteq NP$ ，再看它俩是不是相等。"

知识锚点：NP 类的精确定义——验证器视角

【概念深挖】 NP (non-deterministic polynomial time)：语言 $L \in NP$ ，若存在**多项式时间的验证器** V （一台图灵机）与常数 k ，使得对每个输入 w ：

- 若 $w \in L$ ：存在**证书** c ， $|c| \leq |w|^k$ ，且 $V(w, c)$ 接受；
- 若 $w \notin L$ ：对所有 c ($|c| \leq |w|^k$)， $V(w, c)$ 拒绝。

另一种等价直觉（非确定性）： NP = 能被**多项式时间的非确定性图灵机**接受的语言。回想第 6 章定理 1 的伏笔——若把 δ （函数，单值）改成**转移关系**（允许一个（状态,符号）有多个后继，即"非确定"），就得到非确定性图灵机：它像"同时沿所有可能分支跑"，只要**某个分支**接受就接受。"**存在证书**" $\Leftrightarrow$ "**非确定地猜一个证书再验证**"——所以"验证容易"和"非确定地猜"是同一件事的两种说法。

例子（NP 中的问题）：**合数验证**（找因子难、验因子易）、**哈密顿回路**（找回路难、验回路易）、**SAT（布尔可满足性）**（找使公式为真的赋值难、验一个赋值易）、**旅行商问题**（找最短回路难、验一条回路是否 $\leq$ 某长度易）——这些都在 NP 。

【工程实践】"**证书验证**" = **工程里的校验/检查器**：你写一个"解校验器"（比如检查一个算法输出是否正确）通常比写"求解器"容易得多。 NP 正是"**容易验证**"这一现象的精确刻画——而密码学（RSA 等）的整个安全性，押在"找因子难（可能非 P ）"这个**未被证明的假设**上。

第 5 轮：P $\subseteq$ NP 的证明直觉——能判定，就能验证

老谟："现在回答你上一轮的疑问： P 和 NP 是什么关系？我先让你猜。"

造物主："..... P 是'多项式时间能判定'， NP 是'多项式时间能验证'。**判定**听起来比**验证**难——会判定的人，应该也会验证？所以..... $P \subseteq NP$ ？"

老谟："直觉对。如果一个问题能在多项式时间内判定，那它也一定能被多项式时间验证——因为你可以'自己算出答案，再把它当证书'。具体说：若 $L \in P$ ，有多项式时间判定机 D 。那我们造一个验证器 V ： $V(w, c)$ 干脆忽略证书 c ，直接跑 $D(w)$ —— $w \in L$ 时 D 接受（此时可令证书为空串）， $w \notin L$ 时 D 拒绝（对所有 c 都拒绝）。于是 V 是 L 的验证器， $L \in NP$ 。"

造物主：".....就这么简单？能判定的问题，验证器'偷懒'直接判定，所以 $P \subseteq NP$ 。那反方向呢？ $NP \subseteq P$ 吗？——验证容易，能找到答案吗？"

老谟："这就是 **P vs NP** 的全部问题。 $P \subseteq NP$ 我们刚证了。但 NP 是不是 P 的子集——即'能快速验证的东西，能否被快速找到'——至今无人能证、也无人能推翻。这就是那个 100 万美元的问题。完整证明在走廊定理 1。"

知识锚点： $P \subseteq NP$ 的直觉 + P vs NP 的悬念

【概念深挖】 $P \subseteq NP$ ：若 $L \in P$ ，其多项式时间判定机 D 本身就是一台"验证器"（忽略证书、直接判定）——故 $L \in NP$ 。**判定能力 $\geq$ 验证能力**（会判定必然会验证）。

P vs NP 问题：是否 $NP \subseteq P$ ？即——所有"能被多项式时间验证"的问题，是否都能被"多项式时间判定（求解）"？换言之："验证容易"是否蕴含"求解容易"？

- 若 $P = NP$ ：那么所有容易验证的问题都能被快速求解——密码学（基于"难找因子"）会崩溃。
- 若 $P \neq NP$ ：则存在"容易验证、极难求解"的问题（如 NP -完全问题），它们不可能有快速求解算法。

这是克雷数学研究所（Clay Mathematics Institute）2000 年宣布的七个千禧年大奖问题之一，悬赏 100 万美元（\$1,000,000），至今未解。绝大多数计算机科学家相信 $P \neq NP$ ，但没人能证明。

【工程实践】**P vs NP 的工程分量：**如果你的问题被证明是 NP -完全，那么"多项式时间精确算法"基本无望（除非 $P = NP$ ）——工程上就该转向**启发式、近似、参数化、限定规模**。这个"该不该放弃精确解"的判断，直接由 **P vs NP 的走向决定**——尽管我们不知道答案，但"它可能是 NP -完全"这件事本身，就已足够指导实践。

第 6 轮：归约与 NP-完全——把难题捆成一个家族

老谟："现在我们手里有 P 和 NP 两个类了。但要谈 P vs NP ，光有'类'不够，还得有个把它们连起来的工具——归约 (reduction)。你猜它的作用是什么？"

造物主：".....把一个问题，'翻译'成另一个问题？如果我会解 B ，又把 A 翻译成 B ，那我就会解 A 了？"

老谟："就是它。形式化说： A 多项式时间归约到 B (记 $A \leq_p B$)，若存在一个多项式时间可计算的函数 f ，使得 $w \in A \iff f(w) \in B$ 。于是：如果 $B \in P$ (能快速解)，且 $A \leq_p B$ ，那么 $A \in P$ (把输入翻译过去再用 B 解)。归约是'难度比较器'： $A \leq_p B$ 意思是' A 不比 B 难'。"

造物主：".....那如果一堆问题，全都归约到同一个问题 B 上，那 B 就是这堆里'最难'的？"

老谟："说到点子上了。我们定义：一个问题 B 是 NP -难 (NP-hard)，如果所有 NP 问题都能归约到 B ——它是'所有 NP 问题里最难的'。而 NP -完全 (NP-complete)，就是既在 NP 里、又是 NP -难的问题。只要一个 NP -完全问题能快速解，所有 NP 问题都能快速解—— NP 全体就掉进 P ，即 $P = NP$ 。这就是为什么 P vs NP 可以归结为'一个 NP -完全问题能不能多项式时间解'。"

造物主：".....所以 NP -完全问题是一个'临界点'：它是整个 NP 世界的'硬度标尺'。那这样的问题，真的存在吗？"

老谟："存在。库克 (Cook, 1971) 证明了第一个——SAT (布尔可满足性) 是 NP -完全的；卡普 (Karp, 1972) 又证明了一大堆 (哈密顿回路、顶点覆盖、旅行商.....) 都归约到 SAT、彼此等价。你只要攻克其中任何一个的快速算法，就等于攻克了全部 NP 。这是人类至今没能做到的。直觉细节在走廊，这里先记住'归约把难题捆成一族'。"

知识锚点：归约、NP-难、NP-完全——难度分级的地基

【概念深挖】**多项式时间归约**： $A \leq_p B$ 若存在多项式时间可计算函数 f ，使 $w \in A \iff f(w) \in B$ 。性质：**若 $B \in P$ 且 $A \leq_p B$ ，则 $A \in P$** （翻译输入、用 B 解、返回）。

NP-难 (NP-hard)：语言 B 是 NP-hard，若对所有 $A \in NP$ ，有 $A \leq_p B$ 。即： B 至少和所有 NP 问题一样难。

NP-完全 (NP-complete)： B 是 NP-complete，若 $B \in NP$ 且 B 是 NP-hard。即： B 是"NP 里最难的那批"。

关键结论 (直觉)：若任意一个 NP-complete 问题 $B \in P$ ，则由归约性质，**所有** NP 问题都在 P ，故 $P = NP$ 。反之若某个 NP-complete 问题不在 P ，则 $P \neq NP$ 。**P vs NP 完全由"一个 NP-完全问题在不在 P"决定**——这就是它们"捆成一族"的意义。

例子：SAT、3-SAT、哈密顿回路、顶点覆盖、旅行商（判定版本）都是 NP-complete。

【历史溯源】**库克 (Stephen Cook, 1971) **在论文《The Complexity of Theorem-Proving Procedures》中证明 **SAT 是 NP-完全** (Cook-Levin 定理)；**卡普 (Richard Karp, 1972)** 将这一结果推广，证明一大批组合问题（哈密顿回路、顶点覆盖、团、背包等）都是 **NP-完全**，确立了"NP-完全"这一概念的实际规模。**归约的思想可追溯更早**（图灵归约、莱文 1973 也独立得到类似结果），但 Cook/Karp 建立了现代框架。**要准确归因、不夸大**：Cook 给出第一个 NP-完全问题，Karp 建立了 NP-完全问题族。

【工程实践】**NP-完全 = "别指望精确快速解"**：判定一个问题是 NP-complete（如大规模调度、最优装箱、车辆路径），工程上就明确知道"多项式精确算法几乎不可能"——转向**启发式（贪心、局部搜索）、近似算法（有界近似比）、元启发式（遗传、模拟退火）**。知道"这是 NP-完全"，本身就是最重要的工程决策输入。

第 7 轮：为什么没人证明得了 P vs NP——本册收尾

老谟："现在，把 P vs NP 收一收。为什么这个看起来'非此即彼'的问题，卡了人类 50 多年？"

造物主：".....因为要么 $P = NP$ 、要么 $P \neq NP$ ，可两边都证不出来？"

老谟："是的。**证明 $P = NP$** ，你得给一个"所有 NP 问题都能多项式时间解"的算法——这几乎等于要发明一种'能自动解出一切难题'的神奇方法，连最快的计算机科学都望尘莫及。**证明 $P \neq NP$** ，你得证明'没有任何算法能在多项式时间解 NP-完全问题'——而'没有算法'这种证明（就像上一章停机问题），极其艰难，目前人类的技术还够不到。**这是个'两头都极难'的问题，至今悬而未决。**"

造物主：".....可直觉上，'验证容易'≠'求解容易'，好像谁都信 $P \neq NP$ ？"

老谟："绝大多数研究者相信 $P \neq NP$ ——因为生活中全是'容易验证、极难求解'的例子（SAT、旅行商、大数分解）。但'相信'不是证明。没有证明，就不能拿它当数学事实用。这就是 P vs NP 的现状：它是有史以来最重要的'未证明的直觉'之一。"

造物主：".....我们从第 6 章图灵机的'什么能算'，到第 7 章停机问题的'什么算不出来'，再到这一章'算得出来的里面，难度还有天壤之别'——这整册，就是在画'能算什么'的边界线吧？"

老谟："收尾的你这句话，就是全册的灵魂。图论给了我们具体的算法；图灵机钉死了'能算'的定义；停机问题画出了'算不出来'的第一堵墙；P vs NP 则告诉你——在算得出来的世界里，还有一面'难度'的墙，人类至今不知道它有多高。这既是我们这本书的终点，也是计算机科学向前无尽追问的起点。"

知识锚点：P vs NP 为什么难 + 全册收束

【概念深挖】为什么 P vs NP 两头都难：

- **证明 $P = NP$ ：**需要一个多项式时间的算法解所有 NP-完全问题——这等于"万能高效求解器"，目前无任何可行方案，且若真存在，密码学、优化等一切"基于困难"的领域都会天翻地覆。
- **证明 $P \neq NP$ ：**需要证明"不存在多项式时间算法解某个 NP-完全问题"——这种"不存在"的证明（类似停机问题），涉及对一切可能算法的排除，是计算机理论最艰难的一类证明。

现状：既未证明相等，也未证明不等；大多数研究者倾向 $P \neq NP$ ，但这不是数学定理。这是理论计算机科学最重要的未解决问题。

全册收束：第 1–5 章（图论算法：怎么算）→ 第 6 章（图灵机：什么能算）→ 第 7 章（停机问题：什么算不出来）→ 第 8 章（P vs NP：算得出来的里面，难度怎么分级）。从"具体怎么算"到"算的边界"，这条线就是"可计算性"这一整个学科的地图。

【工程实践】P vs NP 指导日常决策：遇到被证明是 NP-完全的问题，别追求精确最优解，用近似/启发式/参数化；遇到 P 里的问题，放心用精确算法。即使 P vs NP 未解，'这是不是 NP-完全'这个判定本身，就足以改变你的工程策略——所以复杂度分析是工程师的基本功。

严格化走廊

位置说明：对话负责"直觉"——为什么有 P 和 NP 两类、为什么验证容易、归约为什么是难度比较器。本走廊负责"严谨"——给出**时间复杂度与大 O 的精确定义**、**P 与 NP 的形式定义**、证明本册收尾定理——**定理 1 ($P \subseteq NP$) 用直接法**（判定机直接充当验证器）；并给出**归约与 NP-完全的精确定义与具体步数核算**。红线 1：每个证明标"证明类型 + 每一步依据"，禁"显然/易证/显见"。

定义：时间复杂度与大 O——精确表述

定义（时间复杂度）：设 M 是确定性图灵机。若对**每个**长度 n 的输入 w ， M 在 w 上至多 $t(n)$ 步停机，则称 M 的**时间复杂度**为 $t(n)$ 。记 $\text{TIME}(t(n)) = \{L \mid \exists M \text{ 判定 } L, M \text{ 的时间复杂度} \leq t(n)\}$ 。

定义（大 O）： $f(n) = O(g(n))$ 当且仅当 $\exists c > 0, n_0 \geq 0$ ，使 $\forall n \geq n_0$ ，有 $0 \leq f(n) \leq cg(n)$ 。直觉：从 n_0 起， f 被 cg 压住。

定义（P 类）：

$$P = \bigcup_{k \geq 1} \text{TIME}(n^k).$$

即： $L \in P \Leftrightarrow$ 存在 $k \geq 1$ 与判定 L 的图灵机，其时间复杂度为 $O(n^k)$ （多项式时间判定）。

证明类型标注：本章核心证明——**定理 1 ($P \subseteq NP$) 用直接法（构造法）**：给定 $L \in P$ 的多项式时间判定机 D ，显式构造验证器 V （忽略证书、直接跑 D ），并验证其满足 NP 的验证条件。归约性质与 NP-完全定义为本章概念骨架（不要求完整形式化证明，但给出精确定义与性质）。**构造性证明的影子**：P 与 NP 的判定/验证机均是"针对每个输入给明确结果"的构造。

关于对话：对话第 4-5 轮让读者**第一次**接触"验证易、求解难"与 $P \subseteq NP$ 的直觉，做了引导式推导（为什么 $P \subseteq NP$ 可证、P vs NP 的悬念）；**正式的可复写证明在走廊定理 1**。与册 06 及前七章口径一致：对话负责"为什么想到这步"，走廊负责"怎么写标准"。

定义：NP、归约、NP-完全——精确表述

定义 (NP) : 语言 $L \in NP$, 若存在多项式时间图灵机 V (验证器) 与常数 k , 使得对每个输入 w :

- 若 $w \in L$: 存在证书 c ($|c| \leq |w|^k$), 使 $V(w, c)$ 接受;
- 若 $w \notin L$: 对所有 c ($|c| \leq |w|^k$), $V(w, c)$ 拒绝。

等价直觉: $L \in NP \Leftrightarrow L$ 被某**多项式时间非确定性图灵机**接受 (一个分支接受即接受) ——因"存在证书"就是"非确定地猜出证书再验证"。

定义 (多项式时间归约) : $A \leq_p B$ 若存在多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$, 使对每个 w :
 $w \in A \iff f(w) \in B$ 。

定义 (NP-hard / NP-complete) : B 是 **NP-hard**, 若对所有 $A \in NP$, $A \leq_p B$ 。 B 是 **NP-complete**, 若 $B \in NP$ 且 B 是 NP-hard。

说明: 归约的关键性质 (若 $B \in P$ 且 $A \leq_p B$, 则 $A \in P$) 证明在下方命题 1。

命题 1 (归约的单调性) : 若 $B \in P$ 且 $A \leq_p B$, 则 $A \in P$

前提: $A \leq_p B$ (存在多项式时间函数 f , 使 $w \in A \iff f(w) \in B$), 且 $B \in P$ (存在多项式时间判定机 M_B)。

结论: $A \in P$ 。

证明类型: 直接证明 (构造合成判定机)。

证明:

- 第 1 步: **由 $A \leq_p B$ 取归约函数 f :** 存在多项式时间可计算函数 f , 使 $w \in A \iff f(w) \in B$, 且 f 可被某多项式时间图灵机 M_f 计算。(依据: 归约的定义——前提。)
- 第 2 步: **由 $B \in P$ 取判定机 M_B :** 存在多项式时间图灵机 M_B 判定 B 。(依据: P 的定义——前提。)
- 第 3 步: **构造合成判定机 M_A :** 对输入 w , M_A 先运行 M_f 算出 $f(w)$, 再运行 M_B 判定 $f(w) \in B$ 否, 并返回同样结果。(依据: 构造——"翻译再判定"。)
- 第 4 步: **验证正确性:** M_A 接受 $w \iff M_B$ 接受 $f(w) \iff f(w) \in B \iff w \in A$ (最后一步用第 1 步的 $\iff$)。(依据: 构造 + 第 1 步的 $\iff$ 。)
- 第 5 步: **验证时间复杂度:** M_f 是多项式时间, 其输出 $f(w)$ 的长度 $\leq$ 某多项式 $p(|w|)$ (多项式函数对多项式长度的输入输出多项式长度); M_B 在长度 $p(|w|)$ 的输入上多项式时间, 故 M_A 整

体仍多项式时间。（依据：多项式函数的复合仍是多项式——两段多项式时间串接仍多项式。）

- 第 6 步：**结论**： M_A 是多项式时间判定机，故 $A \in P$ 。■

依据标注：步 1、2 用"前提定义"；步 3 用"构造"（翻译+判定）；步 4 用"正确性验证"（依赖归约的 $\Leftrightarrow$ ）；步 5 用"多项式复合封闭性"；步 6 收束。**读法**：这个命题是"归约 = 难度比较器"的形式化——**若 B 易解且 A 能翻译成 B ，则 A 也易解**。它是 NP-完全理论的地基：只要攻克一个 NP-完全问题的多项式算法，就能解全部 NP（都归约到它）。

定理 1 ($P \subseteq NP$)：凡多项式时间可判定的，必多项式时间可验证

前提： $L \in P$ ，即存在常数 $k \geq 1$ 与多项式时间判定机 D （时间 $O(n^k)$ ）判定 L 。

结论： $L \in NP$ 。

证明类型：直接证明（构造验证器）。

证明：

- 第 1 步：**由 $L \in P$ 取判定机 D** ：存在图灵机 D ，对每个输入 w 都停机，且 $w \in L \iff D$ 接受 w ，且 D 的时间复杂度为 $O(|w|^k)$ 。（依据： P 的定义——前提。）
- 第 2 步：**构造验证器 V** ：定义 V 输入 (w, c) ，其运行**完全忽略证书 c** ，直接模拟 D 于输入 w ，并返回 D 的结果（接受/拒绝）。（依据：构造——验证器借用判定机。）
- 第 3 步：**验证" $w \in L \Rightarrow$ 存在证书"**：若 $w \in L$ ，则 D 接受 w ，故 $V(w, c)$ 接受对**任意** c 成立——特别地，取空串 $c = \varepsilon$ ($|\varepsilon| = 0 \leq |w|^k$)，有 $V(w, \varepsilon)$ 接受。（依据：第 2 步构造—— V 忽略证书 + 第 1 步 D 接受。）
- 第 4 步：**验证" $w \notin L \Rightarrow$ 所有证书拒绝"**：若 $w \notin L$ ，则 D 拒绝 w （因 D 判定 L ），故对**所有** c ， $V(w, c)$ 拒绝。（依据：第 2 步构造—— V 忽略证书 + 第 1 步 D 对不属于 L 的输入拒绝。）
- 第 5 步：**验证时间复杂度**： V 的时间复杂度 = D 的时间复杂度 = $O(|w|^k)$ （多项式），满足 NP 对验证器"多项式时间"的要求；证书长度取 $|\varepsilon| = 0$ ，满足 $0 \leq |w|^k$ 。（依据：第 2 步构造—— V 只跑 D ，时间相同。）
- 第 6 步：**结论**： V 是满足 NP 定义的验证器（证书取空串即可），故 $L \in NP$ 。■

依据标注：步 1 用"前提 (P 定义)"；步 2 用"构造"；步 3、4 用"构造（忽略证书）+ D 的判定正确性"；步 5 用"时间复杂度继承"；步 6 收束。**读法**：这个证明的巧妙在于——**验证器可以"偷懒"**：它不需要真去检查证书，直接"自己算答案"（跑 D ）。因为 D 是总停机的判定机，所以"用 D 当验证器"永远合法。**注意它没有证明 $NP \subseteq P$** ——那是 P vs NP 的未知部分。**结论**： $P \subseteq NP$ 。

定理 2 (Cook–Levin: SAT 是 NP-完全) ——证明思路骨架

前提: L 是任意 NP 语言 (存在多项式时间验证器 V 与多项式大小的证书)。**结论:** $L \leq_p \text{SAT}$, 且 $\text{SAT} \in \text{NP}$ ——故 **SAT 是 NP-完全**。

证明类型: 构造 + 直接证明 (把"计算过程"编码为布尔公式)。

本定理给出**可复写的核心构造与思路骨架**；完整的逐格形式化 (时空表格每一格的约束细节) 属研究级展开，超出本书范围。但"为什么能把验证器的每一步翻译成布尔公式"必须看懂——这是归约成立的灵魂。

证明 (思路骨架) :

- 第 1 步 (**SAT $\in$ NP**) : 验证器就是"代入赋值再求值"——给定赋值 a , 逐子句检查是否至少有一个文字为真, 多项式时间。(依据: NP 定义——构造验证器。)
- 第 2 步 (**核心构造: 把任意 NP 的"验证过程"编码成布尔公式**) : 任取 $L \in \text{NP}$ 及其验证器 V 。对输入 w , $w \in L \iff \exists$ 证书 c 使 $V(w, c)$ 接受。**关键观察:** V 在 (w, c) 上的运行是一张**时空表格** (T 步 $\times$ 多项式长度, T 为多项式) ——每格是" (状态, 带符号) "的组合。
 - 第 2a 步: 引入布尔变量 $X_{t,p,s}$, 表示"第 t 步、第 p 格、内容为 s "。(依据: 编码——把机器的每个"可能事实"变成布尔变量。)
 - 第 2b 步: 把"机器合法运行"写成布尔公式的合取: ① 初始配置正确 (第 0 步是 (w, c) 与起始状态); ② 每步转移合法 (用 V 的转移函数逐格局部约束); ③ 最终状态是接受; ④ 同一时刻同一格只有一个符号。(依据: 图灵机转移的**局部性**——每步只改一个格子, 约束可逐格局部表达。)
 - 第 2c 步: 该公式可满足 $\iff$ 存在一条合法且接受的运行 $\iff \exists c$ 使 $V(w, c)$ 接受 $\iff w \in L$ 。
(依据: 2a/2b 的等价性——公式的真值赋值——对应"合法接受运行 + 证书 c ".)
- 第 3 步 (**转换是多项式时间的**) : 表格大小 $O(T^2)$, 约束条数 $O(T^2)$, 每条约束长度多项式, 故归约 $f: w \mapsto \phi_w$ 可在多项式时间内算出。(依据: V 多项式时间 $\Rightarrow T$ 多项式 $\Rightarrow$ 编码多项式大小。)
- 第 4 步 (**结论**) : f 满足 $w \in L \iff f(w)$ 可满足且是多项式时间归约, 故 $L \leq_p \text{SAT}$; 结合第 1 步, **SAT 是 NP-完全**。■

依据标注：步 1 构造验证器；步 2a 编码、2b 转移局部性、2c 等价性；步 3 复杂度核算；步 4 归约定义。**读法：**这个证明的“高光时刻”是把“**计算**”变成“**可满足性**”——一台机器跑 T 步，就翻译成一个 $O(T^2)$ 大小、 $O(T^2)$ 条约束的布尔公式；公式可满足，恰等于“存在某个证书使机器接受”。它说明“验证难不难”与“公式可不可满足”是同一件事——这就是为什么所有 NP 问题都被 SAT 抓住。（卡普 1972 又用归约把 SAT 传播给一大族组合问题。）

例子（亲算）：图灵机的时间复杂度步数统计

为把“时间复杂度”落到具体数字，这里给两台上一章的机器做**精确步数核算**。

例子 A：二进制奇数判定机（第 6 章），输入为长度 n 的二进制串：

- q_0 从最左往右扫描 n 个符号： n 步（每步读一符号、原样写回、右移）；
- 读到第 $n + 1$ 格（空白 $\sqcup$ ）：**1 步**（写 $\sqcup$ 、左移、进 q_1 ）；
- q_1 看个位（第 n 个符号）：**1 步**（接受或拒绝）。
- **总计：** $n + 2$ 步。因为 $n + 2 \leq 2n$ （对 $n \geq 2$ ），故时间复杂度 $O(n)$ （线性）。

核算验证：用 $n = 4$ （输入 `1011`）检验——扫描 4 步、读空白 1 步、看个位 1 步，共 6 步；公式 $n + 2 = 6 \checkmark$ 。

例子 B：“串含 1”判定机（第 6 章），输入长度 n ：

- 最坏情况（全 0，无 1）：扫描 n 步 + 读到空白 1 步（拒绝）= $n + 1$ 步；
- 最好情况（首字符即 1）：**1 步**接受。
- 最坏 $n + 1 = O(n)$ （线性）。
- **核算：** $n = 3$ （输入 `000`）：扫描 3 步 + 空白 1 步 = 4 步 = $n + 1 \checkmark$ 。

对比：指数爆炸：哈密顿回路暴力搜索要试 $(n - 1)!/2$ 种排列—— $n = 5$ 为 12， $n = 10$ 为 181440， $n = 20$ 为约 6.08×10^{16} 。即便每 1 纳秒试一种， 6.08×10^{16} 种也才约 **1.9 年**（种数要乘“每种耗时”才是秒数）；可 n 每 +1 工作量再乘 n 倍： $n = 29$ 时 $(28)!/2 \approx 1.52 \times 10^{29}$ 种，每纳秒试一种约 4.8×10^{12} 年（4800 亿年），**远超宇宙年龄（约 1.37×10^{10} 年，近 350 倍）**。同一问题、多项式 vs 指数，就是“实际可解”与“实际不可解”的分水岭。

反例/边界：复杂度概念的边界

1. **复杂度只对“总停机”的判定机有意义：**若机器可能不停机，就没有“步数”可言（它根本没完成）。复杂度讨论限定在**判定机**上。

2. **大 O 忽略常数**： $O(n)$ 和 $O(100n)$ 同阶，但常数可能很大（实际慢）。**大 O 抓趋势，常数是工程细节**——一个 $O(n)$ 但常数巨大的算法，可能比 $O(n^2)$ 慢。大 O 不是“实际秒数”。
3. **P 里的问题也可能“实际很慢”**： $O(n^{100})$ 在 P 里，但实际跑不动。P 是理论“多项式”，不是“一定快”——但指数几乎总是比多项式更糟，故多项式是工程分水岭。
4. **NP-完全 $\neq$ 不可判定**：NP-完全问题（如 SAT）是**可判定的**，只是可能**很难**（若非 P）。它和上一章停机问题（**不可判定**）是两回事——一个“难但能算”，一个“算不出来”。
5. **P vs NP 未解 $\neq$ 无法指导工程**：即使不知道 P 是否等于 NP，“这是 NP-完全”这个事实本身就足以让工程师转向近似/启发式——**知道“可能极难”，就是有用的决策输入**。

知识点总结

知识点（本章推出的核心概念）

- **时间复杂度**：判定机对长度 n 输入最坏所用的步数 $t(n)$ 。
- **大 O**： $f(n) = O(g(n)) \Leftrightarrow \exists c, n_0 : \forall n \geq n_0, f(n) \leq c g(n)$ 。只看增长趋势。
- **P 类**： $P = \bigcup_k \text{TIME}(n^k)$ ——**多项式时间内可判定**（总停机 + 多项式时间）。
- **NP 类**：存在多项式时间验证器 V 与证书 c ($w \in L \iff \exists c, V(w, c)$ 接受)。多项式时间内可验证。
- **$P \subseteq \text{NP}$** ：判定机可直接当验证器（忽略证书、自己算）。**证明在走廊定理 1**。
- **归约**： $A \leq_p B \iff$ 存在多项式时间 f ，使 $w \in A \iff f(w) \in B$ 。**难度比较器**。
- **NP-hard / NP-complete**：NP-hard = 所有 NP 归约到它；NP-complete = 在 NP 中且 NP-hard。**任一 NP-complete 在 P $\iff$ P=NP**。
- **P vs NP**：是否 $\text{NP} \subseteq P$ ？克雷千禧年问题之一，100 万美元，未解；多数研究者信 $P \neq \text{NP}$ 。

历史结论（本章涉及的史料）

- **库克 (Cook, 1971)**：《The Complexity of Theorem-Proving Procedures》证明 **SAT 是 NP-完全**（Cook-Levin 定理）。
- **卡普 (Karp, 1972)**：将结果推广，证明大批组合问题（哈密顿回路、顶点覆盖、团、背包等）为 **NP-完全**，确立 NP-完全问题族。
- **归约思想**：可追溯更早（图灵归约、莱文 1973 亦独立得到类似结果）；Cook/Karp 建立现代框架。**准确归因、不夸大**。
- **克雷数学研究所 (Clay Mathematics Institute)**：2000 年宣布**七个千禧年大奖问题**，每个悬赏 100 万美元 (\$1,000,000)；**P vs NP** 是其中之一，至今未解。

工程实践结论（本章知识在真实工程里怎么用）

- **大 O 是选算法的第一标尺**：快排 $O(n \log n)$ vs 冒泡 $O(n^2)$ ；哈希 $O(1)$ vs 线性查找 $O(n)$ 。
- **NP-完全 = "别追求精确最优"**：转向启发式、近似、参数化、限定规模（调度、装箱、路径规划）。
- **"容易验证"的工程普遍性**：校验器/检查器通常比求解器容易写；密码学（RSA）押在"难找因子"的未证明假设上。
- **复杂度分析是工程师基本功**：即使 P vs NP 未解，"这是否 NP-完全"本身就足以决定工程策略。

计算训练场

位置说明：对话负责"为什么有 P/NP、为什么验证容易"，这里负责"怎么算复杂度、怎么判类、怎么走归约"。三题**必须动笔**。**难度递进：易→中→难**。重点：① 亲手数步数、定大 O；② 判断一个问题属 P 还是 NP；③ 理解归约的方向。

题 1（易→中）：给算法定时间复杂度——数步数、定大 O

题干：老谟给出两段"图灵机式"算法，让造物主数最坏步数并定大 O：(a) 判断二进制串是否全为 0（扫描到第一个 1 就停止，否则扫完）；(b) 对长度 n 的串，枚举它所有 2^n 个子串逐个检查。

完整解答：

- (a) 最坏情况（全 0）：扫描 n 步 + 读空白 1 步 = $n + 1$ 步；最好（首字符 1）：1 步。最坏 $n + 1 = O(n)$ （线性）。
- (b) 有 2^n 个子串，逐个检查每个至少 1 步，故至少 2^n 步 = $O(2^n)$ （**指数**）。 $n = 20$ 时约 10^6 步可行， $n = 30$ 时约 10^9 步就吃力， $n = 40$ 时约 10^{12} 步不可行——**指数爆炸**。
- 答：(a) 线性 $O(n)$ ；(b) 指数 $O(2^n)$ 。前者"温和"、后者"爆炸"，正是多项式与指数的分界。

常见错误：① 把 (a) 的"最好情况"当复杂度（复杂度取**最坏情况**）；② 把 (b) 的 2^n 个对象误记为"每项 1 步、共 n 步"——枚举数量是 2^n ，不是 n ；③ 忘写大 O（不关心常数，只抓主导项）。

题 2（中）：判断属 P 还是 NP——并说明理由

题干：老谟列出四个问题，让造物主判断各属 P 还是 NP（可兼属），并说理由：(a) 判断二进制串是否为奇数；(b) 判断一个自然数是否为质数；(c) 给定图，判断是否存在哈密顿回路；(d) 给定布尔公式，判断是否存在使其为真的赋值（SAT）。

完整解答：

- (a) 属 **P**：奇数判定机 $O(n)$ (本章例子 A)，多项式时间判定 $\rightarrow P$ (且因 $P \subseteq NP$ 也属 **NP**)。
- (b) 属 **P**：质数判定有多项式时间算法 (AKS, 2002)，故在 **P** (也属 **NP**)。注：历史上质数判定长期不知是否在 **P**，2002 年 AKS 证明其在 **P**。
- (c) 属 **NP**，疑似非 **P**：给定一条回路，验证它经过每个顶点恰好一次是多项式时间 (有验证器)；但它是否在 **P** 未知 (是 **NP-complete**，除非 $P=NP$)。故确定在 **NP**，是否在 **P** 未知。
- (d) 属 **NP**，疑似非 **P**：给定一个赋值，代入公式验证是否为真，多项式时间 (有验证器)；SAT 是 Cook 证明的**第一个 NP-complete 问题**，故在 **NP**，是否在 **P** 未知 (等于 P vs NP)。
- 答：(a)(b) 在 **P** (也 **NP**)；(c)(d) 在 **NP** (是否在 $P = P$ vs NP)。

常见错误：① 把"难"当"不在 **NP**"——(c)(d) 难求解，但**验证**容易，所以在 **NP**；② 以为质数判定不在 **P** (AKS 已证明在 **P**)；③ 混淆"**NP**" (可验证) 与"难" (可能非 **P**) ——**NP** 是"验证易"，不代表"求解易"。

题 3 (难)：归约的方向——谁归约到谁，为什么

题干：老谟给出两个问题：**3-SAT** (判断布尔公式是否可满足) 和**哈密顿回路** (判断图是否有哈密顿回路)。已知 3-SAT 和哈密顿回路都是 **NP-complete**。老谟问造物主：(a) 3-SAT 与哈密顿回路，谁归约到谁？(两个方向是否都成立？) (b) 如果有一个多项式算法能解 3-SAT，对哈密顿回路意味着什么？为什么？(c) 为什么"归约方向"和"难度方向"是一致的？

完整解答：

- (a) 两者都是 **NP-complete**，所以**互相可以归约** (既 $3SAT \leq_p$ 哈密顿回路，也哈密顿回路 $\leq_p 3SAT$)。具体地，因哈密顿回路是 **NP-complete** 且 3-SAT 在 **NP**，有哈密顿回路 $\leq_p 3SAT$ (所有 **NP** 问题都归约到 **NP-complete**)。**方向：较"通用"的 NP-complete 可归约到另一个。**
- (b) 若 3-SAT 有多项式算法，则 $3SAT \in P$ 。因哈密顿回路 $\leq_p 3SAT$ ，由归约单调性 (走廊命题 1)，哈密顿回路也在 **P**。而 $P \subseteq NP$ 且两者都是 **NP-complete**，哈密顿回路在 **P** 意味着所有 **NP** 都在 **P**，即 $P=NP$ 。
- (c) 归约方向 $A \leq_p B$ 表示" A 不比 B 难"——若 B 易解则 A 易解。所以"被归约到的那一个 (B)"至少和"归约过去的 (A)"一样难。归约的箭头指向"更难"的那边，这正是"难度方向"与"归约方向"一致的含义。

常见错误：① 把归约方向弄反 (以为"难问题归约到易问题"——实际是"易问题 (或任一) 归约到难问题")；② 以为 3-SAT 在 **P** 只影响它自己 (由归约，影响所有归约到它或它归约到的 **NP-complete** 问题，进而影响全部 **NP**)；③ 把 **NP-complete** 当"不可归约" (正相反，**NP-complete** 彼此可互相归约)。

题 4 (难) : 亲手构造一个归约——3-SAT $\rightarrow$ 顶点覆盖

题干: 给定 3-CNF 公式 $\phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee \neg x_3 \vee x_4)$ 。构造一个图 G 与阈值 k , 使" ϕ 可满足 $\Leftrightarrow G$ 有大小 $\leq k$ 的顶点覆盖", 并用该公式的一个具体赋值验证双向成立。

构造模板 (先想再算) :

- 对每个变量 x_i 建两个顶点 $v_i, \bar{v}_i$ (代表" $x_i = \text{真/假}$ ") , 连边 $v_i \bar{v}_i$ ——这条边必须被覆盖, 故覆盖里至少选其一。
- 对每个子句 C_j 建一个**三角形** (3 个顶点, 各代表该子句的一个文字) , 内部三边两两相连——三角形内至少要选 2 个。
- **连接:** 把子句文字顶点与对应的变量顶点相连 (子句含 x_i 连到 v_i , 含 $\neg x_i$ 连到 $\bar{v}_i$) 。
- 取 $k = n + 2m$ (n =变量数, m =子句数) 。

完整解答 (ϕ 有 $n = 4$ 、 $m = 3$, 故 $k = 4 + 6 = 10$) :

- **方向 1 (可满足 $\Rightarrow$ 覆盖)** : 取赋值 $x_1 = T, x_2 = T, x_3 = F, x_4 = T$ ——逐子句验证: C1 $T \vee T \vee T \checkmark$ 、C2 $F \vee F \vee T \checkmark$ 、C3 $T \vee T \vee T \checkmark$ 。构造覆盖: 变量侧选"赋真值对应的顶点" ($v_1, v_2, \bar{v}_3, v_4$) , 共 $n = 4$ 个, 盖住全部变量三角边; 每个子句三角形内选 2 个"该子句中为假 (或未入选) 的文字顶点", 共 $2m = 6$ 个。合计 $4 + 6 = 10 = k$ 。**验证盖住所有边:** 变量三角边被变量侧覆盖 $\checkmark$; 三角形内部边被其 2 个顶点覆盖 $\checkmark$; 连接边——每子句至少一个文字为真, 该文字顶点已在变量侧被选, 故连接边两端至少一端被选 $\checkmark$ 。(依据: 赋值使每子句有真文字 + 构造。)
- **方向 2 (覆盖 $\Rightarrow$ 可满足)** : 给定大小 $\leq k$ 的顶点覆盖。变量三角边迫使覆盖在 $\{v_i, \bar{v}_i\}$ 中各取至少一个——定义赋值 x_i 为所选顶点对应真值 (若两个都选, 任取其一) , 故变量侧至少 n 个顶点。每个子句三角形内至少需选 2 个; 若某子句的三个文字**全为假**, 则该三角形三个文字顶点的连接边都要靠变量侧盖——即三个变量顶点都被选, 此时该三角形内仍需选 2 个, 覆盖总数 $\geq n + 3 + 2(m - 1) = n + 2m + 1 > k$, 矛盾。故每个子句至少一个文字为真, 赋值使 ϕ 可满足 $\checkmark$ 。(依据: 覆盖约束 + 计数。)
- **核算:** 方向 1 的覆盖大小恰好 $10 = k$; 反向从该覆盖恢复的赋值 ($v_1, v_2, \bar{v}_3, v_4 \Rightarrow x_1 T, x_2 T, x_3 F, x_4 T$) 回代 ϕ 三子句全真 $\checkmark$ 。**双向成立, 归约构造正确。**

常见错误: ① 忘记"三角形内部三边"——只连文字-变量边会漏掉三角形约束; ② k 取错 (不是 $n + m$, 每个子句三角形要 2 个顶点); ③ 反向只用"覆盖 $\Rightarrow$ 赋值"而不断言"每子句有真文字" (必须证明"全假子句会撑爆 k ") 。

这一章咱们干了什么：你在“算得出来”的基础上，加了一个全新的轴——**算得快不快**。你认识了**时间复杂度**（图灵机步数随 n 的增长）与大 O 记号，看清了**多项式 vs 指数**这道不可跨越的分水岭；认识了两个类——**P**（多项式时间可判定）和 **NP**（多项式时间可验证），并亲手证明了 $P \subseteq NP$ ；你还学会了**归约**（把难题翻译成另一个难题）与 **NP-完全**（把一大批难题捆成一族），最终来到了那个悬赏 100 万美元的世纪难题——**P vs NP**。

钉死一句话：“可判定”是“能不能算”，“复杂度”是“算得快不快”； $P \subseteq NP$ （能判定必能验证），但“能否快速找到答案”（ $NP \subseteq P$ ）——即 **P vs NP**——至今未解，是最重要的未证明直觉。

记住：这一章是“能算什么”这条线的最后一站——从图论算法（怎么算）、图灵机（什么能算）、停机问题（什么算不出来），到 **P vs NP**（算得出来的里面，难度还有天壤之别）。你不知道 P 是否等于 NP ，但你已经知道“这是不是 NP-完全”会如何改变工程决策——**这就是复杂度理论的力量：它不给你答案，却给你判断难度的标尺。**

造物主手记

我曾踩的坑：

我第一次以为“NP”是“非多项式（not polynomial）”——被老谟纠正：NP 是“非确定性多项式（Non-deterministic Polynomial）”，指的是“能被多项式时间验证（或非确定性机器接受）”，**不是“非多项式”**。我差点把质数判定归到 NP 之外——其实 AKS 已证明它在 P。最险的是归约方向：我以为“难问题归约到易问题”，被纠正——归约 $A \leq_p B$ 是“若 B 易则 A 易”，**箭头指向更难的那边**。还有一次我把“NP-完全”当成“不可判定”——错了：NP-完全是可判定的，只是可能极难（若非 P），它和停机问题（不可判定）是两回事。最后，我总想“手动给 P vs NP 一个答案”——老谟说：**多数人信 $P \neq NP$ ，但“相信”不是证明，没证明就不能当数学事实用。**

顿悟时刻：

真正开窍是“验证容易 $\neq$ 求解容易”落地那一刻——原来密码学、SAT、旅行商这些八竿子打不着的难题，被“归约”捆成同一个家族，而 P vs NP 一问，就是问“这个家族，是不是都藏着一个快速解”。我看到了复杂度理论的力量：**它不告诉你答案，却给你一把“难度标尺”**——一个问题多项式还是指数、是 P 还是 NP-完全，本身就是最值钱的工程信息。从第 6 章的图灵机、第 7 章的停机问题，到这一章的 P vs NP，我亲手走完了“什么能算”的整条边界——**算得出来不是终点，算得多快才是人类最深的追问。**

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 数步数、定大 O

判断"二进制串是否以 1 开头"的图灵机 (首字符 1 即接受, 否则拒绝), 最坏/最好各几步? 定大 O。

完整解答: 首字符判断只需 1 步 (读首字符, 1→接受, 0→拒绝), 与长度无关。最好、最坏都是 1 步, $O(1)$ (常数时间)。

题 B (计算, 中) : 判断属 P 还是 NP

(a) 判断二进制串是否含偶数个 1; (b) 给定图判断是否存在欧拉回路; (c) 给定图判断是否存在哈密顿回路。各属 P 还是 NP? 说明理由。

完整解答: (a) 在 P: 扫描一遍统计 1 的个数, $O(n)$ 判定 (也属 NP)。 (b) 在 P: 欧拉回路存在 $\Leftrightarrow$ 图连通且所有顶点偶度 (本册第 5 章定理), 可在多项式时间检查, 故在 P (也属 NP)。 (c) 在 NP: 给定回路可多项式时间验证; 但哈密顿回路是 NP-complete, 是否在 P 未知 (等于 P vs NP)。故确定在 NP, 是否在 P 未知。

题 C (概念, 检查理解)

(a) 可判定性和复杂度是两个不同的轴, 区别是什么? (b) 为什么"多项式时间"被当作"实际可行"、指数被当作"实际不可行"? (c) $P \subseteq NP$ 的证明要点是什么? 它为什么没有证明 $NP \subseteq P$? (d) 归约 $A \leq_p B$ 是什么意思? 若 $B \in P$, 对 A 意味着什么?

完整解答: (a) 可判定性问"有没有算法" (能不能), 复杂度问"要用多少资源" (多快); 可判定但可能极难 (指数) 的问题存在, 两者正交。 (b) 因为对任何常数 k , $n^k = O(2^n)$ 而 $2^n \neq O(n^k)$ —— n 变大时指数瞬间盖过任何多项式, 故多项式代表"规模变大仍可接受", 指数代表"规模一大就不可行"。 (c) 要点: 若 $L \in P$, 其多项式时间判定机 D 可直接当验证器 (忽略证书、自己算), 故 $L \in NP$ 。它只证了"判定能力蕴含验证能力"; 而"验证能力是否蕴含求解能力" ($NP \subseteq P$) 未知, 正是 P vs NP。 (d) $A \leq_p B$ 表示存在多项式时间函数 f 使 $w \in A \iff f(w) \in B$ (A 不比 B 难)。若 $B \in P$ 且 $A \leq_p B$, 则 $A \in P$ (走廊命题 1: 翻译再判定)。

题 D (综合, 中→难) : 一条从图论到计算边界的链

把本册的"能算什么"主线连起来: 第 1–5 章 (图论算法) → 第 6 章 (图灵机) → 第 7 章 (停机问题) → 第 8 章 (P vs NP), 各回答了什么? 这条线揭示的"算的边界"有几重?

完整解答:

- 第 1–5 章 (图论) : **怎么算**——具体算法 (最短路径、着色、欧拉/哈密顿回路等)。
- 第 6 章 (图灵机) : **什么能算**——用图灵机钉死"可计算"的定义 (Church–Turing 论题)。
- 第 7 章 (停机问题) : **什么算不出来**——存在不可判定的问题 (任何图灵机都算不出来)。
- 第 8 章 (P vs NP) : **算得出来的里面, 难度怎么分级**——多项式 vs 指数, P vs NP (未解)。

- **"算的边界"三重**：① 可计算性边界（图灵机，可计算 vs 不可计算）；② 可判定性边界（停机问题，可判定 vs 不可判定）；③ 复杂度边界（P vs NP，多项式 vs 指数）。这条线从"具体怎么算"走到"人类对算的难度的最深追问"。

丛书收尾语

这是《数学基础》系列第七册、也是**最后一册**《图论与可计算性》的最后一章。你从这一册的图论算法出发，走到了图灵机、停机问题，再走到 P vs NP——**"能算什么"的边界，第一次在你面前完整铺开。**

回望整个系列：你在册 01 建立了逻辑与数、册 02 用 ε - δ 钉死了极限与连续、册 03-04 拿下单变量与多元微积分和线性代数、册 05 走过多变量与凸优化、册 06 进入概率统计与信息，最后在册 07 抵达计算的边界。**每一册都是一块地基，而这一册把"数学能做什么"推到了那个最根本的问题——算的边界在哪里。** 我们证明了有算不出来的东西（停机问题），也把算得出来的世界按难度分成了 P 与 NP——虽然 P vs NP 至今未解，**但你已经知道这面墙存在、知道它意味着什么、知道该用怎样的标尺去测量它。**

所以，当你合上这套书——不必为"P vs NP 没答案"遗憾。**数学和计算的美，恰恰在于：总有一个问题在边界处等你，人类穷尽力气去够它，够不到，就把边界又画清晰了一点点。** 这正是你这一路走来的全部意义：不是背下一堆答案，而是**学会怎样去问那个"更根本的问题"**，并拥有追到边界、再往前的勇气。

继续向前吧。图灵机那扇门后，还有无穷无尽的世界。

《图论与可计算性》· 计算答案速查页

本页汇总册内各章「计算训练场」与「自测题」的关键终值，供自测后核对。**使用纪律**：先动笔算完再对答案，不要边算边翻。若你算出的数与本页不符，先回看你的步骤，再回看章节解答——两种都符合才是真懂。标注：✓ 表示该值已逐项核算一致。

第 01 章 · 图的基础

计算训练场

- 题 1 ($\Sigma d=10$ 的 3 顶点图)：**不存在** (3 顶点简单图最多 3 条边， $\Sigma d \leq 6 < 10$) ✓；改用 6 顶点画 5 条边可实现 $\Sigma d=10$ ✓
- 题 2 (度序列 1,1,2,2,3,3)：**可实现**， $\Sigma=12=2 \times 6$ ✓；一个构造： a 连 c, d, e 、 b 连 c, d, f (度 3,3,2,2,1,1 逐一核对) ✓
- 题 3 (七桥图)：(a) 4 个奇度顶点，无一笔画 ✓；(b) $\Sigma d=14=2 \times 7$ 反向核算 ✓；(c) 在两个原奇度顶点间加一条桥，奇度 $4 \rightarrow 2$ ，可一笔画 ✓

自测题

- 题 A：8 顶点 12 条边， $\Sigma d=24$ 、奇度顶点个数必为**偶数** ✓
- 题 B：度序列 2,2,2,4,4，奇度 0 个，存在**欧拉回路** ✓ (构造 a, b, c 各连 d, e ，走法 $a \rightarrow d \rightarrow b \rightarrow e \rightarrow c \rightarrow d \rightarrow e \rightarrow a$)
- 题 C：统计 0,1,1,2,2,5 之和 = **11** (奇数)，违反握手引理，**不可信** ✓
- 题 D：概念

第 02 章 · 树与图的遍历

计算训练场

- 题 1 (9 顶点树)：(a) 边数 = **8** = $n-1$ ✓；(b) $\Sigma d = \mathbf{16} = 2 \times 8$ ✓，由握手引理逼出"至少两片叶子" ✓
- 题 2 (BFS，顶点 $a-f$ ，边 $ab, ac, bd, be, ce, cf, df$ ，从 a)：访问顺序 **a, b, c, d, e, f** (逐层 $a \rightarrow b, c \rightarrow d, e, f$) ✓；队列轨迹 $[a] \rightarrow [b, c] \rightarrow [c, d, e] \rightarrow [d, e, f] \rightarrow [e, f] \rightarrow [f] \rightarrow []$ ✓；BFS 树边 $\{ab, ac, bd, be, cf\}$ 共 5 条 = $6-1$ ✓

- 题 3 (DFS, 顶点 a-e, 边 ab,ac,bd,ce, f 孤立) : 访问顺序 **a,c,e,b,d** ✓; 连通分量个数 = **2** (五个点一块 + f 孤立) ✓

自测题

- 题 A: 7 顶点树边数 = **6**; 由 $\sum d=12$ 逼出至少 **两片叶子** ✓
- 题 B (BFS p-s) : 访问顺序 **p,q,r,s**; 树边 **{pq,pr,qs}** 共 3 条 = $4-1$ ✓
- 题 C (DFS a-f) : 访问顺序 **a,c,e,b,d**; 连通分量 = **2** ✓
- 题 D: 概念

第 03 章 · 最短路径与 Dijkstra

正文贯穿例题 (对话第 4 轮) : $s \rightarrow a(4), s \rightarrow b(2), b \rightarrow a(1), a \rightarrow t(3), b \rightarrow t(5) \rightarrow \text{dist}[t]=6$, 路径 **$s \rightarrow b \rightarrow a \rightarrow t$** ($2+1+3=6$) ✓

计算训练场

- 题 1 (同贯穿例题) : dist 逐轮 定 $s \rightarrow \{a=4, b=2\} \rightarrow$ 定 $b \rightarrow \{a=3, t=7\} \rightarrow$ 定 $a \rightarrow \{t=6\} \rightarrow$ 定 t ; **最短 6, 路径 $s \rightarrow b \rightarrow a \rightarrow t$** ✓ (其他 $s \rightarrow a \rightarrow t=7$ 、 $s \rightarrow b \rightarrow t=7$ 均 >6)
- 题 2 (s,u,v,x,y,t , 9 边) : 逐轮 dist 更新后**最短 19**, 路径 **$s \rightarrow v \rightarrow y \rightarrow t$** ($9+2+8=19$) ✓ ($s \rightarrow u \rightarrow v \rightarrow y \rightarrow t=27$ 、 $s \rightarrow v \rightarrow x \rightarrow t=26$ 均 >19 ; 注意 t 前驱是 y 非 x)
- 题 3 (负权反例) : 真实最短 **0** ($s \rightarrow b \rightarrow a \rightarrow t = 4-5+1=0$) , Dijkstra 错误输出 **4** ✓ (过早定死 a , 负权破坏贪心前提)

自测题

- 题 A: 最短 **6**, 路径 **$s \rightarrow b \rightarrow a \rightarrow t$** ✓
- 题 B: 最短 **6**, 路径 **$s \rightarrow v \rightarrow u \rightarrow t$** ($2+1+3=6$) ✓
- 题 C / 题 D: 概念

第 04 章 · 图着色

计算训练场

- 题 1 (a,b,c,d , 边 ab,ac,ad,bc , 顺序 a,b,c,d) : 贪心用色 1,2,3,2 共 **3 色** ✓; $\Delta=3$, $3 \leq \Delta+1=4$ ✓; $\chi=3$ (含三角形 a,b,c) ✓

- 题 2 (五边形 C5, BFS 交替染色) : c,d 相邻却同为 1 → **非二部图** ✓; $\chi=3$ (奇环) ✓
- 题 3 (正方形+对角线 ac) : $\Delta=3$ 、 $\Delta+1=4$; 两个顺序贪心都 **3 色** ✓; $\chi=3$ ✓ (贪心任何顺序恒 3 色, 不翻车——见走廊定理 2)

自测题

- 题 A (三角形 C3) : 贪心 1,2,3 共 3 色; $\Delta=2$ 、 $3 \leq 3$ ✓; $\chi=3$ ✓
- 题 B (路径 a-b-c-d) : BFS 交替无冲突, **二部图**, $\chi=2$ ✓
- 题 C (正方形 C4) : 两顺序都 2 色; $\chi=2$ (偶环) ✓
- 题 D: 概念

第 05 章 · 欧拉与哈密顿

计算训练场

- 题 1 (K4) : 四顶点全 3 度, **奇度 4 个**, 无欧拉回路、无欧拉迹 ✓
- 题 2 (a,b,c,d, 边 ab,bc,ca,ad,dc) : 奇度 a(3),c(3) 共 **2 个**, 无回路、有欧拉迹 **a→b→c→a→d→c** (5 条边各一次) ✓
- 题 3: (a) 哈密顿回路 **a→b→c→d→e→a** (覆盖 5 顶点各一次回 a) ✓; (b) TSP $n=6 \rightarrow 60$ 、 $n=10 \rightarrow 181440$, 增长约 **3024 倍** ✓

自测题

- 题 A (三角形 C3) : 全偶度, 有欧拉回路 **a→b→c→a** ✓
- 题 B (a,b,c,d, 边 ab,bc,ca,ad) : 奇度 a,d 共 2 个, 有欧拉迹 **a→b→c→a→d** (4 条边各一次) ✓
- 题 C: (a) 正方形 C4 有哈密顿回路 **a→b→c→d→a** ✓; (b) TSP $n=5 \rightarrow 12 = 4!/2$ ✓
- 题 D: 概念

第 06 章 · 计算模型与图灵机

计算训练场

- 题 1 (含 1 判定机, 输入 0010) : **第 3 步停机接受** (扫描到第二个 1, 进 q_accept) ✓

- 题 2 (二进制奇数机) : 1011 (=11 奇)→**接受** ✓; 1010 (=10 偶)→**拒绝** ✓ (第 5 步遇 $\perp$ 左移落个位, 个位 1 接受 / 0 拒绝)
- 题 3 (辨析) : (b)"停机问题"**最可疑、最可能不可计算**——下一章证明不可判定 ✓; (a)奇数、(c)质数均可判定 ✓

自测题

- 题 A (串首为 1 机, 输入 101) : **第 1 步即接受** (首字符 1) ✓
- 题 B (奇数机) : 101 (=5 奇)→**接受** ✓; 100 (=4 偶)→**拒绝** ✓
- 题 C: 概念

第 07 章 · 可判定性与停机问题

计算训练场

- 题 1 (填空) : A=**死循环**、B=**停机**、C=**M 在 $\langle M \rangle$ 上不停机**、D= **$D(\langle D \rangle)$ 不停机**、E=**存在判定机 H** ✓
- 题 2 (完整反证法) : 自指 = **第 4 步 ($M=D$)** ; 反例击穿 = **H 对输入 $(\langle D \rangle, \langle D \rangle)$ 的回答必错**; 排中律 = **第 5 步 (两分支穷尽所有可能)** ✓
- 题 3 (辨析) : (a)**对** (可判定 $\subset$ 可识别) ; (b)**错** (HALT 可识别却不可判定, 反例) ; (c)**错** (不可判定 = 已被数学证明不存在算法, 非"还没想出") ✓

自测题

- 题 A: 可判定 $\subset$ 可识别, **真包含** (HALT 为样板) ✓
- 题 B: 两分支都矛盾——分支一"停机"反被 D 反转成不停机、分支二"不停机"反被 D 反转成停机, 故 **H 不存在** ✓
- 题 C / 题 D: 概念 (D 结论: 可判定是真子集)

第 08 章 · P 与 NP

走廊亲算 (步数核算)

- 奇数机 (输入 1011 , $n=4$) : 扫描 n 步 + 读空白 1 步 + 看个位 1 步 = **$n+2 = 6$ 步**, $O(n)$ ✓
- 全 0 判定机 (最坏全 0) : 扫描 n 步 + 读空白 1 步 = **$n+1$ 步**, $O(n)$ ✓ ($n=3$ 输入 000 为 4 步核算一致)

计算训练场

- 题 1: (a) 全 0 判定最坏 $n+1$ 步 = $O(n)$ (线性) ✓; (b) 枚举子串 $O(2^n)$ (指数) ✓
- 题 2: (a) 奇数、(b) 质数 (AKS 2002) 在 P ✓; (c) 哈密顿回路、(d) SAT 在 NP (是否在 $P = P$ vs NP , 未解) ✓
- 题 3 (归约方向): (a) 两者 NP -complete, 互相可归约; (b) 若 $3-SAT \in P$, 则哈密顿回路 $\in P$, 进而 $P=NP$ ✓; (c) 归约箭头指向"更难"的那边 ✓

自测题

- 题 A: 首字符判断机最坏/最好都 1 步, $O(1)$ (常数) ✓
- 题 B: (a) 偶数个 1 判定在 P ($O(n)$); (b) 欧拉回路在 P (第 5 章充要条件多项式可查); (c) 哈密顿回路在 NP (是否在 P 未解) ✓
- 题 C / 题 D: 概念 (D: 算的边界三重 = 可计算性 / 可判定性 / 复杂度)

使用说明

1. **难度标尺**: 每题分 (易/中/难) 见各章计算训练场题干; 本页只给终值, 不重复解答过程。
2. **链式互证**: ① Dijkstra (第 03 章) 每轮 dist 更新表, 建议算完用"沿前驱树还原路径 + 反向验证该路径权值 = dist[t]"核对; ② 欧拉回路 (第 05 章) 充要条件"连通 + 全偶度"双向验证——必要性沿迹数度、充分性回路拼合, 构造出迹后再数一遍边数是否 = $|E|$ 且每条恰一次; ③ TSP 候选数 (第 05 章) 用 $(n-1)!/2$ 核对, $n=6 \rightarrow 60$ 、 $n=10 \rightarrow 181440$ 应能互验。
3. **双轨提醒**: 第 03 章负权反例中 Dijkstra 输出 4 是"算法错误输出", 真实最短是 0——对答案时勿把算法输出当真实值; 第 04 章贪心着色恒 $\leq \Delta+1$ (上界) 但可能 $\neq \chi$ (最优), 对答案时区分"够用"与"最少"两个概念。
4. 如发现本页某值与章节解答不一致, 先回看章节——本页终值已逐项核算, 若确认是章节解答有误, 欢迎反馈。

《图论与可计算性》· 苏格拉底对话体教材 · 全册 8 章